

STANFORD ARTIFICIAL INTELLIGENCE LABORATORY
OPERATING NOTE 54.6

August, 1978

M O N I T O R C O M M A N D M A N U A L

M O N I T O R C O M M A N D M A N U A L
(Fourth Edition)

by

Brian Harvey

ABSTRACT

This document describes the monitor commands available to users of the WAITS timesharing system at the Stanford Artificial Intelligence Laboratory, and the use of the terminals connected to this system. The first section is an introduction to the system for new users. Several appendices are included containing documentation of commonly used system programs. This manual supersedes SAILON 54.5.

This work was supported by the Advanced Research Projects Agency of the Office of the Secretary of Defense under contract MDA903-76-C-0206.

ACKNOWLEDGMENTS

Several sections of this document were excerpted from program documentation written by others. Contributors include Mike Clancy, Mark Crispin, Bo Eross, Martin Frost, Ralph Gorin, Dick Helliwell, Andy Moorer, Jeff Rubin, Jim Stein, and Dan Swinehart. John McCarthy wrote the *Introduction to Timesharing* section, and Mark Crispin, Martin Frost, Ralph Gorin and Brian McCune edited and corrected the manual's draft versions. This document was produced on the Xerox Graphics Printer using the PUB program written by Larry Tesler and improved by Rich Johnsson, Pitts Jarvis, and Tovar. Thanks also to Martin Frost for editing the Third Edition while I was away, and to Mark Faust for help and moral support while I was editing this edition.

TABLE OF CONTENTS

SECTION	PAGE
1 INTRODUCTION FOR NEW USERS	1
1.1 Introduction to Timesharing	1
1.2 Timesharing at the AI Lab	2
1.3 Introduction to Terminals	4
1.4 Typing Commands to the Monitor	5
1.5 File Storage	7
1.6 LOGIN and KJOB Commands	8
1.7 Editing Commands	9
1.8 Commands for Compiling and Running Programs	10
1.9 Information Commands	11
1.10 Programs and Core Images	11
2 DISPLAY TERMINALS	13
2.1 DD and III Keyboards	13
2.2 The Line Editor	14
2.3 Line Editor Activation	17
2.4 ESCAPE and BREAK Commands	18
2.5 The WHO Line	20
2.6 Page Printer Control	23
2.7 Data Disc Control	24
2.8 III Control	29
2.9 Audio Switch Control	29
2.10 Datamedias	31
3 NON-DISPLAY TERMINALS	35
3.1 Non-display ESCAPE commands	36
3.2 The TTY Command	37
4 BASIC MONITOR COMMANDS	39
4.1 Running Programs	39
4.2 Manipulating Core Images	40
4.3 Starting Programs	41
4.4 Detached Jobs	42
4.5 Device Control	43

ii CONTENTS

4.6	System Information	45
4.7	Miscellaneous Commands	47
5	SYSTEM PROGRAM COMMANDS	48
5.1	LOGIN	48
5.2	LOGOUT	52
5.3	SNAIL	54
5.4	COPY and SPOOL	55
5.5	MAIL	56
5.6	DART	56
5.7	DO	57
5.8	ZERO	58
5.9	WHO	58
5.10	FIND	62
5.11	Other System Information Programs	65
5.12	Miscellaneous System Programs	66
6	PRIVILEGED COMMANDS	69

A P P E N D I C E S

A1	SNAIL	71
A1.1	EDIT-Class Commands	71
A1.2	COMPILE-Class Commands	73
A1.3	DOCUMENT-Type COMPILE-Class Commands	73
A1.4	LOADBUF-Type COMPILE-Class Commands	74
A1.5	COMPILE-Type COMPILE-Class Commands	75
A1.6	CREF Command	82
A1.7	Switch Function Summary	83
A1.8	LOADER Switches and LINK Equivalents	84
A1.9	Error Messages	85
A2	COPY	97
A3	SPOOL	100
A3.1	XSPOOL Special Features	100
A3.2	QSPool Command	102

A3.3	UNSPOOL Command	103
A4	MAIL	105
A4.1	Command Format	105
A4.2	Mail to Other ARPA Network Hosts	115
A4.3	The MAIL Command	117
A4.4	The SEND Command	120
A4.5	The REMIND Command	121
A4.6	The GRIPE Command	122
A4.7	The EVENT Command	122
A4.8	The PLAN Command	123
A4.9	The RETRY Command	123
A4.10	The LATER Command	123
A4.11	The BATCH Command	125
A4.12	The REENTER Error Recovery Facility	126
A4.13	Hand-Holding Mode	127
A4.14	Interfacing to MAIL from Other Programs	127
A4.15	The CANCEL Command	130
A4.16	The RCV Command	130
A5	DART	138
A6	SERVICE LEVEL SYSTEM (RSL)	142
A7	FIXIMLAC	143
A8	ARPA NETWORK	143
A8.1	The User TELNET	143
A8.2	Control Commands to TELNET, PTYJOB, and DIAL	146
A8.3	Datamedia Simulation	148
A8.4	SUPDUP	149
A8.5	The File Transfer Protocol	150
A9	CARE AND FEEDING OF DEVICES	161
A9.1	The Line Printer	161
A9.2	DECtapes	162
A9.3	Magnetic Tapes	162
A9.4	Xerox Graphics Printer	163
A9.5	User Disk Pack	165
A9.6	Core Storage	166

iv CONTENTS

A10 HOW TO START WAITS 169

A11 MONITOR ERROR MESSAGES 184

A12 BIBLIOGRAPHY 185

A13 STANFORD CHARACTER SET 186

SECTION 1

INTRODUCTION FOR NEW USERS

This manual describes the monitor commands used in the PDP-10 computer system at the Stanford Artificial Intelligence Laboratory. The first section is an introduction for new users, including brief explanations of terminals, file storage, and some of the essential monitor commands. The computer is operated exclusively as a timesharing system, and the section begins with an introduction to timesharing.

1.1 Introduction to Timesharing

A timesharing computer system includes the computer itself (in our case a PDP-10 computer made by the Digital Equipment Corporation), user terminals for input to and output from the computer (in our case about 60 keyboard and display terminals), and a file system that keeps programs, data for the programs, reports, and other data for the users.

The basic idea of timesharing is to allow each user to behave as though he had a computer to himself controlled from his terminal. This is accomplished by having the computer cycle its attention among the users wanting service in rotation. It does this fast enough so that small requests are serviced in a time that ranges from a small fraction of a second to a few seconds, depending on the number of people demanding service at that instant. If the desired computation is long, the computer will do a bit of it, then service other people, then return to it, and so on until the computation is done. The time taken will depend on the size of the computation and how busy the machine is.

At any time the user's terminal is interacting with some program. These programs include the timesharing monitor; editors used to prepare programs, documents, reports and other data; various utility programs; and the user's own programs.

When you first sit down at a terminal, you will be interacting with the monitor, and you can always get back to the monitor by pressing the CALL key on the terminal. The first thing you have to do is to *log in*, which tells the monitor who you are so it can connect you to the file directory that contains your files and can do its accounting. The last thing you do is *log out* so that the terminal and other resources you have been occupying can be used by someone else.

You will have a certain quota of disk space and will be able to keep files such as programs between uses of the computer. Therefore, you will not ordinarily need any external form of storage such as punched cards or magnetic tape. (Our computer doesn't even have punched card equipment.)

A user gets service from programs by typing commands to the monitor. Some monitor commands perform the service and leave you again talking to the monitor. Others cause a program to be started; thereafter, you have to interact with that program in the way prescribed by it. When you are done with it, you go back to the monitor.

Your program also gets service from the monitor by means of special instructions, called UUOs, used in the program. The monitor provides many different services as a sort of subroutine of your program; most importantly, it is responsible for allocating resources, like core storage and input/output devices, among different jobs. Therefore, input and output must be done through the monitor to avoid conflicts. These UUOs are documented in their own manual; this one is about the commands you type at your terminal.

A typical task is to write a program in some computer language such as SAIL (our dialect of ALGOL) or LISP and then debug the program by running it, checking the results, and changing it until it gives the right answers. This is accomplished in the following steps:

1. First you run an editor. (We have several, but the current favorite is called E.) The editor reads whatever you type and displays it; what you type becomes the text of the program you are writing. However, the editor gives you convenient facilities for going back in your text and making changes that you think of as you go along. Most people don't write their programs on paper, but make them up as they use the editor. When the program is ready to try, you exit from the editor back to the monitor, and your program is a file on the disk with a name you have given it. At any time, you can re-edit this file to make changes in it. You can also print it on the line printer to take a copy home, and when you don't need it any more you can delete it.
2. Normally, the next step is to call a compiler to translate the program from a source language into machine language, call the loader to load it into your *core image*, and finally ask the computer to run it. You can do these things as separate operations, but facilities are provided for initiating all of this with a single command.
3. When the computer runs your program, it may just produce answers and stop. If you like the answers, perhaps you are done. Otherwise, you go back to the editor to make changes in the *source file* and try again. On the other hand, you may have written an interactive program which expects you to type input on the terminal. It interprets your input and gives output, then asks for more input, etc. Your program may read files and write them. It may display pictures on your terminal. Also, it may interact with the physical world by reading pictures from one of the TV cameras attached to the computer and by waving the mechanical arm.
4. In order to debug a program, you may have to do more than just look at the output. Therefore, the timesharing system has facilities for examining and changing the contents of registers and for putting break-points in programs.

1.2 Timesharing at the AI Lab

Our timesharing system, called WAITS, runs on a Digital Equipment Corporation (DEC) PDP-10 computer. This is a 36-bit machine designed for timesharing applications. We currently have 917,504 words of core storage online and expect to have over 2 million words online in the not-too-distant future. Our timesharing system, derived from old DEC software, has been extensively modified for our special requirements.

Programming languages available here include SAIL, a version of ALGOL with added features for backtracking, list processing, associative processing, and coroutines; several versions of LISP; FAIL, our PDP-10 assembly language; and the MICRO-PLANNER problem solving language.

To use the computer, you must type a LOGIN command at your terminal. (See Section 1.6 for a description of this command.) The monitor assigns you a job number, which it uses internally to keep track of you. Associated with your job are various resources, of which the most important is the core image, i.e., the simulated core storage maintained for you by the monitor so that your program can use addresses starting from zero although it may really be located anywhere in core while running. Your job only has one core image; if you type a monitor command which causes a program to be run, that program replaces your old core image. There is no way for one job to run two programs at once.

As a user of the Stanford AI Lab system, you will be given a *programmer name* to identify you to the computer. The main reason for this is so that the monitor knows which files on the disk storage belong to you. This name will probably be your initials. When you log in, you must type in this name, along with a *project name* which allows you to divide your files into separate groups for the various projects you are working on. For example, if you are doing homework for a Computer Science course, the course number might be your project name. The combination is called a *project-programmer name* or *PPN* and is typed PRJ,PRG or sometimes [PRJ,PRG] when used in a file specification. It is possible to switch your attention from one project to another without having to log out and log in again by giving a monitor command to change your *alias* or *disk PPN*, the PPN used for file access if you do not specify a file directory explicitly. Therefore, many users always log in using a single “master” project name; project “1” is the conventional name for this purpose.

Ours is just about the most heavily used computer we know of. It's hard to get good service, especially weekday afternoons. Therefore, please be careful about leaving jobs logged in when you leave, and don't play Spacewar during the day! Even a job which is not running uses up scarce resources. The system will automatically log out a job which has not run a program for ten minutes (an hour at night), even if the job is waiting for the user to type something on the terminal. For example, if you are editing a file and have made changes which have not yet been written on the disk, your changes will be lost if you are logged out automatically for not running for a while. A warning message is typed five minutes before the actual logout. You can prevent autologout by typing the monitor command ENABLE LIV (see Section 6), but it is antisocial to do this routinely. If you are logged in at a Data Disc terminal, and there are no free Data Disc channels, and you would be eligible for logout but have enabled LIV, your job will be detached, so the job will stay around but your terminal will be freed for other users.

Sometimes the system stops working while you are using it. You will notice that characters you type no longer appear on your terminal. In many cases, the system fixes itself after a few seconds, so all that happens is that a few characters you typed are lost. Otherwise, someone has to go fix it manually. Sometimes it can be gotten to work again so that you can just continue with what you were doing. Other times, a new copy of the system is loaded and you have to start over again. This is called *reloading*. If this happens, you will see a message to that effect on your terminal. When editing, it is a good idea to save your file on the disk every so often in case the system crashes. If the system crashes and nobody is around to fix it, you may have to reload it yourself. See Appendix 10 to find out how.

Sometimes the system is stopped intentionally for software or hardware maintenance. This downtime is scheduled in advance, and part of the system messages you see when you log in is a maintenance schedule. Some time before the system is taken down for scheduled maintenance, you will see a warning on your terminal which counts down to the end. This warns you to save any files you are writing and stop what you are doing. Such maintenance is often scheduled between 5 and 7 weekday evenings.

This document, the *Monitor Command Manual*, describes the use of monitor commands typed in at terminals. Another manual describes the UUOs, instructions used by programs to communicate with the monitor. Other system information, including documentation of specific system programs, can be found in some printed manuals and on the disk; look at the [S,DOC] and [UP,DOC] file directories for program documentation. Some system programs, which are invoked by special monitor commands, are written up in appendices to this manual. Most of these appendices are derived from [UP,DOC] files. Some of those files are updated more frequently than this manual and some aren't. *Caveat emptor.*

1.3 Introduction to Terminals

A computer terminal is a typewriter-like device which is used to communicate with the computer. You talk to the computer through a keyboard like that of an ordinary typewriter. If you are using a *hardcopy terminal*, then the computer talks to you by printing on a sheet of paper, again like an ordinary typewriter. However, you may be using a *display terminal*, in which case a TV-like display screen is used for output instead of a sheet of paper. Almost all of the terminals within the laboratory building are display terminals. The display terminals at the lab differ from most terminals in that they use a special character set. That is, many special characters can be typed besides the usual letters, digits, and punctuation. *Hereafter, the term "display terminal" will be used only for these special SAIL terminals. Normal commercially manufactured terminals are considered "non-displays" whether or not they use display screens for output!* For use outside the lab, there is a specially modified version of the Datamedia model 2500 display terminal which allows the SAIL character set to be typed; these terminals are considered to be displays also. Non-displays are sometimes called Teletypes, for historical reasons, although the name is a trademark and should not be so used. Both "Teletype" and "console" are sometimes used to mean any terminal, although "console" is more properly used to refer only to the computer's operating console in the machine room and its associated terminal.

There are two kinds of display terminals in use at the lab, Data Discs and IIIs ("triple-I"). They use the same keyboards, but have different display screens. The Data Discs, which are more numerous (about 60 of them), use TV monitors. (The name Data Disc actually refers not to the terminals themselves, but to the machine in the computer room which controls them.) The six IIIs are vector plotting displays, used mainly for graphics display programs like Spacewar. Unless you are writing a display program, the difference is not too important except that the character set on the IIIs is less legible than that on the Data Discs.

A vacant terminal should say TAKE ME I'M YOURS! along with some cryptic information. On a III, the cryptic information is the *system WHO line*, described on page 21. On a Data Disc, besides the system WHO line, there will be a status display for all the jobs using the system. If it's completely blank, make sure it's turned on, and that the brightness and contrast (intensity on IIIs) are turned up. If it says NO DATA DISC CHANNELS LEFT, it means that the Data Disc, which can run only 31 terminals at once, is full. A terminal that is not in use is called an *available terminal*. Please remember to log out when you are done using a terminal so that it will become available.

Each of the printing character keys on our terminals has two characters printed on it (except the 0 key). The bottom one is usually a letter or digit, and the top one is some special character. There are two SHIFT keys and a SHIFT LOCK key, which make letters upper case. The keys which produce the special characters are labelled TOP. For example, hitting the "<D" key alone produces

lower case “d”; also holding down SHIFT produces capital “D”; and holding down the TOP key makes it produce “<” (SHIFT and TOP together is interpreted as TOP).

When you type something on an available terminal, the available message is replaced by whatever you typed. The characters you type are kept in a special buffer called your *line editor* until you *activate* them by typing RETURN or one of a few other special characters. The line editor allows you to change the contents of the line by using special editing command characters. The characters in the line editor are displayed on the screen with two *cursors*: underlines on Data Discs and triangles on IIs. The left cursor is underneath the first character in the line editor; the right cursor is under the place where the next character you type will go (generally the end of the line). When your line editor is empty, the cursors coincide. The line editor commands are explained in Section 2.2. Note: when you type RETURN, the monitor generally supplies a LINE character automatically. RETURN signifies a return to the left margin, and LINE an advance to the next line.

To get you started using display terminals before you read about all the line editor commands, all you need to remember is that the BS key will delete the last character, and the CLEAR key will delete the entire line. The CONTROL and META keys are used for other line editor functions.

A note on terminology: In the body of this manual, characters are referred to by the name printed on the keyboard, e.g., RETURN and FORM. In other documentation, including some of the appendices to this manual, different notations are used. Some of these are shown here:

function	key	other notation
break	BREAK	<BRK> [BRK]
escape	ESC	<ESC> [ESC] <ESCAPE>
call	CALL	<CALL> [CALL] ↑C
clear	CLEAR	[CLEAR] ↑U
tab	TAB	<TAB> [TAB] <HT>
form feed	FORM	<FORM> [FORM] <FF> ±
vertical tab	VT	<VT>
backspace	BS	<BS> [BS]
carriage return	RETURN	[RETURN] <CR> CR
line feed	LINE	<LF> LF <LINEFEED>
shift	SHIFT	<SHIFT>
top	TOP	<TOP>
alt mode	ALT	<ALT> <ALTMODE> \$
meta	META	<META> β ±
control	CONTROL	<CONTROL> <CTRL> α

The combination of RETURN and LINE (remember that LINE is supplied by the monitor) is sometimes referred to as CRLF.

1.4 Typing Commands to the Monitor

What you type may be read by different programs at different times, so you should be aware of what program you’re talking to. When you first type on a vacant terminal, you are talking to the

monitor's command decoder. You continue to talk to the monitor until you start running a program; then you're generally talking to that program. To distinguish these conditions, the state of the terminal is described as *monitor mode* or *user mode*. When the monitor is ready to read a command from your terminal, it types a period. Programs usually type some special character(s) to prompt you when they are ready for a command; many system programs prompt you with an asterisk.

It is possible to *type ahead* when the system is not waiting for typein. Characters you type which are not immediately read by either the monitor or a program are stored in your terminal's input buffer until they are needed. Programs can cause this buffer to be cleared. For example, some programs ask for confirmation of certain commands, e.g., when you specify for output a filename which already exists. Some such programs clear your input buffer first and others don't, so you can get confused by typing ahead to a program whose behavior you don't know. Your input buffer is also cleared when you type CALL.

Monitor commands consist of the command name, possibly followed by arguments separated by spaces or punctuation, and then a RETURN. The command may be typed using upper and lower case letters interchangeably. Monitor command names may be typed in completely, or they may be abbreviated to only as many letters as are necessary to make the name unique; for example, the LOAD command may be abbreviated LOA, but not LO because there is also a LOGIN command. A few common commands have single-letter abbreviations (L is accepted to mean LOGIN). Only the first six letters of a command name are relevant.

Some monitor commands are completely processed by the monitor, without running a program in your core image. For example, the DAYTIME command types the current date and time, without disturbing your core image. Most commands, however, require a program to be run. The monitor usually requires you to log in before you may run programs; there are a few exceptions, to permit system information programs like WHO, and of course the LOGIN program itself. If, when not logged in, you try to run a program which requires you to be logged in, the monitor automatically runs LOGIN, which will ask you for your project-programmer name. In this situation, when LOGIN finishes running, it will re-enter the command which you typed in the first place, so the command will be performed without making you retype it. Commands which require login but which refer to a pre-existing core image rather than running a new program (e.g., START) do not cause this automatic login, but give an error message.

When you first enter a command which runs a program, the system assigns you a job number and types a line like

```
Job 19    SU-AI WAITS 8.72/G  Assembled 08/02/78
```

which tells you what version of the monitor is running and when it was created, as well as your job number. If it types

```
Job capacity exceeded
```

too many jobs are already logged in. When a program like WHO is run without logging in, the job is killed when the program exits or if the user types CALL. The LOGIN program, however, leaves you logged in when it exits.

Some of the monitor commands are briefly described starting in Section 1.6. The full descriptions of all the commands are in Sections 4 and 5, and in the appendices. Appendix 14 is a summary of the commands for quick reference.

1.5 File Storage

A file is a stored collection of information, perhaps a program or data. Most files are stored on a permanently available disk unit. Files on the disk must be identified by name and by owner so that you can get at the one you want. Files on other devices may also be used; we have magnetic tape, DECtape (a small mag tape unit unique to Digital Equipment Corporation machines), and paper tape, as well as a line printer and stuff like that. Whenever a program does input or output, it has to specify the particular file it wants. A file identifier is typed in the form

```
DEV: FILNAM.EXT [PRJ, PRG]
```

some parts of which can be left out. DEV specifies the device you want. Generally, if you want the disk (DSK), you may leave out the device name. If you leave out the device name, leave out the colon too. Device names may include a number to specify a particular unit; thus, TTY is your own terminal (all terminals used to be Teletypes, and the name stuck), but TTY14 is the one next to the vending machine. Besides these *physical device names*, you can define *logical device names* to use in your program. The monitor command ASSIGN is used to make your logical name refer to some particular physical device.

Some devices, like the disk, require the use of filenames, and some, like the line printer, don't. If you specify a filename for a device which doesn't use one, the name is ignored. Devices which require file names are called *directory devices* because they include special files called directories which tell where the other files are. Files are specified by a filename, another name called the *extension*, and, on the disk, a PPN specifying the file's owner. Letters in the filename are upper case, although most programs will accept lower case letters. The filename is one to six letters or digits, and can say anything you want to identify the file. The extension, which is optional, is one to three characters. Although the monitor allows any SIXBIT characters (the ones with octal ASCII codes between 40 and 137—see Appendix 13) to be used in filenames and extensions, some programs which read filenames use some punctuation characters as punctuation characters and won't recognize them in filenames. It is best to stick to letters and digits. If you find that some program has written a file called @# \$!.X ? in your directory and you are having trouble dealing with it, you will find that several programs, including the COPY program for file manipulation, use the notation ↓...↓ for such files, allowing any SIXBIT characters between the downarrows as part of a filename. (You must, of course, say ↓FILNAM↓.↓EXT↓ if that's what you want, not ↓FILNAM.EXT↓.) The only directory devices in this system are the disk, UDPs (User Disk Packs), and DECtapes.

The extension of a file is generally one of a standard set used to indicate what sort of file it is, like SAI for a SAIL program, REL for a relocatable binary program as produced by a compiler, or DMP for a dumped core image. Many programs use these standard extensions as defaults, so you often can type the file name alone.

Disk files require a project-programmer name, which selects the directory to use for the file (see page 3). If you do not type in a PPN, you generally get the one you gave to LOGIN. The disk directory for each PPN is called a *UFD* (User File Directory); the files in a given directory are also said to comprise a *disk area*, although there is not really a physical area involved.

Files are divided into smaller units, called *records*. The precise meaning of a record depends on the particular device you are using, but often a record is the smallest unit of data which the device will transfer to or from the computer at a time. For example, files on the disk are divided into

200-word (octal) records. The disk hardware reads or writes one such record at a time. On magnetic tape, you can write any size records; however, when you read the tape, each record must be read all at once. The monitor provides *buffering* so that your program can deal with the data a character or a word at a time. The monitor gets a record and stores it for you.

The following devices are available on this system.

DSK	the public disk
SYS	same as DSK, but implies PPN [1,3] (system files)
UDP1,2	user disk pack, a disk unit for private disk packs
MTA0,1	magnetic tape drives
DTA1-4	DECtape drives
LPT	line printer
XGP	Xerox Graphics Printer
IMP	Interface Message Processor for ARPA network connections
DLN0-3	Dialnet ports
PTR	paper tape reader
PTP	paper tape punch
PLT	CALCOMP plotter
SIX	PDP-6 computer, used to control music devices:
ADC	analog-to-digital converter
DAC	digital-to-analog converter
SAM	Samson (Systems Concepts) digital synthesizer
FRM	Moore digital synthesizer
KIM	real-time console interface
RTJ	real-time job
ELF	PDP-11 computer, used to control robotics devices
TV	TV cameras
CAR	cart transmitter
TTYn	terminals and pseudo-teletypes (see below)
TTY	the user's own terminal
CTY	PDP-10 console terminal
NUL	null device, ignores output, gives end-of-file on input

A *pseudo-teletype* (PTY) is a simulated terminal-like device used to allow programs to "type" monitor commands. The program uses special UUOs to output text to the PTY, which is then seen by the monitor as if it were typed at another terminal, with its own TTY number. Similarly, text typed out to the PTY can be read by the controlling program. One user of PTYs is the program which allows people at other computers to use our system via the ARPA network (Appendix 8).

Disk files are copied to magnetic tape daily, using the DART program (see Appendix 5). Monitor commands which run DART can be used to retrieve files which have been clobbered by system failure (uncommon) or user error.

1.6 LOGIN and KJOB Commands

In this and the following sections, a few of the monitor commands used in creating and debugging a program are described briefly. More complete descriptions come later and in other manuals.

The first thing you have to do is log in. The command for this is called LOGIN, abbreviated L. Your project-programmer name is its argument. (See page 3 for an explanation of project-programmer names.) Most users use project name 1 when they log in; to do this you simply give your programmer name as the argument. If you are named V. I. Lenin, you might type

```
L VIL
```


which means the same thing as

```
L 1/VIL
```

(The slash means that you don't want to see any system messages which you've seen before. You could have used a comma to see all messages or a period to see none at all.) The monitor processes this command by assigning and typing out a job number, and then starting the LOGIN program in your core image.

If this is the first time you're logging in, LOGIN will type out

```
Are you sure?
```

to which you must type a Y. It will tell you that you are not known as an authorized user of the system, unless your initials have been added to the authorized users list. Then you get the messages, and the program exits. You are now logged in. Section 5.1 explains the LOGIN command more fully.

When you finish doing whatever you want, you should log out. The command for this is called KJOB, abbreviated K. No arguments are required, just K and RETURN. (As with all monitor commands, your terminal must be in monitor mode when the command is given.) This command runs the LOGOUT program, which types out how much computer time you used, etc., and kills your job. Your terminal will become available in a few seconds if you don't type anything else. If there are no files in your directory, the LOGOUT program deletes the directory. Section 5.2 gives more complete documentation of the logout procedure.

1.7 Editing Commands

To enter your program, or to correct it, you will use one of our text editors. There are three editors in common use here:

1. E. This is the one you'll generally use. It is a fast display editor with many diversified features. It is sometimes called ETV after the monitor command by which it is invoked.
2. SOS. This editor is designed for use at non-display terminals. Files created with SOS include a line number at the beginning of every line. Editing commands use the line numbers to specify what lines to change, delete, or whatever. Hopefully you will have no need to use this editor, since you'll be using display terminals.
3. TECO. This editor is the XTEC version of a standard editor supplied by DEC. It is extremely powerful, in that it allows you to write editing programs which can process a file systematically, and it is very flexible about strange characters in the file. However, for routine editing it is somewhat inconvenient to use because it does not use the display facility of the terminals.

Each of these editors is described in its own manual. A bibliography is included as an appendix to this manual. The point to be explained here is how to use monitor commands to invoke an editor; we'll assume here that you're using E.

Suppose you want to write a program called PROG in the SAIL programming language. The monitor command to create a new file with E is CETV. So you would type

```
CETV PROG.SAI
```

(SAI is the standard extension for SAIL programs.) This command tells the monitor to run E, and tells E to create a new file named PROG.SAI. E remembers the last editing command you typed, so later you'll be able to edit the file simply by entering an ETV command without having to type the file name again. (Note—LOGOUT makes E "forget" your commands, so you have to type the file name again when you next log in.)

The command for editing an already existing file with E is ETV. You can enter this command with no argument and it will remember the file from the last edit command, or you can specify a filename as with the CETV command above.

The other editors (SOS and TECO) do not read monitor commands themselves. Instead, the corresponding monitor commands for those editors are interpreted by the SNAIL program, which then starts the appropriate editor. Later in this manual the ETV and CETV commands will be referred to as "SNAIL commands" even though they do not actually run SNAIL, since they carry out SNAIL-like functions.

There is a file you can edit to learn how to use E. To do this, log in and enter the following command:

```
HELP ETEACH
```

Then just read what appears on your screen and follow instructions. This command creates a file which you may want to delete when you are done.

1.8 Commands for Compiling and Running Programs

Once you have typed in your program, you have to compile it, load it, and run it. Compilers like SAIL put out relocatable binary programs. This means that the program is in a form which contains enough information so that it can be loaded into any address in core. Addresses within the program are all considered relative to the program's first word, so the LOADER has to add the actual address of that word to all the relative addresses in the program. Your program can be loaded along with others, for example a debugging program; the actual location of any program in your core image will depend on what other programs are loaded with it. (Note that this relocatable binary program format has nothing to do with the hardware relocation discussed on page 3.) The core image produced by the LOADER can be saved in a DMP (dump) file, which can then be run directly by a monitor command without going through the LOADER again.

Suppose you have typed in your program, PROG.SAI. You want to compile, load, and run it. You could say

```
EXECUTE PROG
```

and all those things would happen automatically. SNAIL would generate a command file for SAIL

telling it to compile PROG.SAI and start up the loader, which also gets a command file. (SNAIL knows that it should use the SAIL compiler because of the SAI extension in your filename.) Again, SNAIL remembers the command, so next time you just have to say EXECUTE and it'll all happen. In fact, the editors all have commands which automatically re-do the last compile-type SNAIL command.

Suppose, to make this clearer, you find a bug in the program. To fix it, you can give the monitor command

ETV

and E will start editing PROG.SAI.

Then you use E commands to edit the file. When you're done, you use the exit-to-SNAIL command, which in E is CONTROL-X GO. This will automatically repeat the EXECUTE command you typed before.

There are many different versions of this compilation command: COMPILE will just compile the program, LOAD will compile and load, etc. The SNAIL commands are listed in Section 5.3 and fully described in Appendix 1.

1.9 Information Commands

You can sometimes find out how to use a system command by typing HELP followed by the command name. Typing HELP with no argument will list the kinds of help available this way. For instance, if you forget how to log in, you can type HELP LOGIN.

More complete program documentation is accessible through the READ command, which takes a name as its argument, like HELP, and starts the E editor reading a file with that name in the documentation directories, if any. This is most useful from a display terminal, since E is a display-oriented editor.

To get a list of the files in your directory, type the DIRECTORY command. You can get the list for another disk area by including a PPN in square brackets as an argument.

You can see the status of all the jobs on the system with the WHO command, abbreviated W. You can find out about jobs belonging to a particular user by typing WHERE prg (prg is the programmer name) or FINGER prg. These type out information about the given user's jobs; try them to find out more. All of these commands run programs in your core image, so you shouldn't run them if you need the program you have there. These information commands can be run without logging in first, except for READ.

1.10 Programs and Core Images

This section discusses the allocation and use of core storage for a job. The PDP-10 storage is

divided into *pages* of 512 words each. Currently, pages are always allocated in pairs; thus a job's core image is an even number of pages in size. A job's core image may be located in any set of actual core pages while the program is running.

Actually, a job may use two distinct sets of core pages, called the *lower* and *upper segments*. The job must have a lower segment in order to run. This segment belongs to that job alone. If the job has an upper segment, it may be shared with other jobs. One use of this is to allow several users to run the same system program with only one copy of the program in core; the data for the individual users is stored in each job's lower segment, and the common code is in the upper segment.

An upper segment may be write protected, so the program in it cannot be accidentally changed. This is usually done with shared upper segments. Programs can control the write protection of their upper segment by UWO. Upper segments are more fully explained in the *UWO Manual*.

Each upper segment has an entry in the system job table, including a job number, a segment name, and much of the status information associated with jobs. Some of the monitor commands described in later sections of this manual refer to upper segments.

The lowest locations in each lower segment, called the *Job Data Area*, are used by the monitor to store information about the job. Some of this information is of interest only to the monitor, while other parts can be used and modified by the job itself. In particular, some monitor commands use information from Job Data Area words, e.g., starting addresses for the program. The description of such commands will refer to the relevant word by its symbolic name, of the form JOBxxx, e.g., JOBSA is the word containing the program's normal starting address. The complete description of the Job Data Area is contained in the *UWO Manual*.

SECTION 2

DISPLAY TERMINALS

A display terminal is one which uses a TV-like screen, rather than a sheet of paper, as its output medium. However, in this document (and in general around SAIL), the term "display terminal" will be used to refer specifically to the special kind of display terminals for which special services are provided by the WAITS operating system. These include the Data Disc and III terminals at the laboratory, as well as a specially modified version of the Datamedia model 2500 terminal which can be used for remote communication with our system. These Datamedia terminals are discussed in Section 2.10; for now we consider only the terminals within the laboratory building.

There are two kinds of display terminals in use at the lab, Data Discs and IIIs ("triple-I"). They use the same keyboards, but have different display screens. The Data Discs, which are more numerous (about 60 of them), use TV monitors. (The name Data Disc actually refers not to the terminals themselves, but to the machine in the computer room which controls them.) The six IIIs are vector plotting displays, used mainly for graphics display programs like Spacewar.

2.1 DD and III Keyboards

Our display terminals use a character set called *Stanford ASCII*. ASCII is a standard computer character set; our version is similar to the standard but somewhat extended. In particular, the "control" codes in ASCII, which mean things like "end of text," are used here for printing characters like α and $\exists$ and $\cap$. There are seven bits in a character code; since the PDP-10 has 36-bit words, five characters fit in a word with one bit left over. The complete Stanford ASCII character set is listed in Appendix 13.

Each of the printing character keys on our terminals has two characters printed on it (except the 0 key). The bottom one is usually a letter or digit, and the top one is some special character. There are two SHIFT keys and a SHIFT LOCK key, which make letters upper case. The keys which produce the special characters are labelled TOP. For example, hitting the "<D" key alone produces lower case "d"; also holding down SHIFT produces capital "D"; and holding down the TOP key with it produces "<" (SHIFT and TOP together is interpreted as TOP).

There are several special character codes, generally not printing characters, which are not affected by SHIFT and TOP. One is the space bar, which spaces as on any typewriter. The TAB key sends a tab code, which prints by spacing out to the next tab stop; these are set eight spaces apart and cannot be changed. The RETURN key sends a carriage return code, and the monitor generally provides a line feed code after it automatically. The LINE key sends a line feed alone; this is not too often useful. The FORM key sends a form feed code, and the VT key a vertical tab. These two are used by some programs as control codes, but not often for their printing effect. The BS key is backspace (in real ASCII this code is called delete). This code is usually not sent to your program, but is taken by the line editor as a command to back up the cursor. The ALT key sends an *alt mode*; this code has no printing significance, but is used as a control code by programs.

Although ASCII is a seven-bit code, display terminals actually send nine-bit codes to programs. The two extra bits represent the CONTROL and META keys. These keys are like SHIFT and TOP in that they are held down along with some other key. The CONTROL and META bits are sometimes called “bucky bits”, and the two combined are called “double bucky”. The bits have two different functions. When the character they are used with is sent to a program, it can be used as a control command. Editing and debugging programs use such single-character commands heavily. For example, in the E editor, CONTROL-META-D means to delete a line. However, the CONTROL and META keys are also used as commands to the system line editor for inserting characters inside a line, etc. The use of the line editor is explained in Section 2.2.

There are four more keys to be described. They are unusual in that none of them send characters to your program or to the monitor’s command decoder; they are processed by the monitor’s keyboard scanner and the line editor. CLEAR is simply a line editor command to delete the entire line in the line editor. CALL is used to interrupt a program running in your core image and return your terminal to monitor mode. ESC (escape) and BREAK are used to signal that the characters which follow are a special command to the display service routines in the monitor; the many different functions of these commands will be explained in Section 2.4 and following sections. There should be a list of the ESC and BREAK commands attached to your keyboard; if not, see page 18 to find out how to print one.

Please note that the ESC and BREAK keys generate codes by themselves; they are not shift-like keys as CONTROL and META are. Thus, for example, CONTROL-A means “press the A key while holding down CONTROL,” whereas ESC A means “press ESC and then press A.” Also, the ASCII character which is represented on our keyboards as ALT is sometimes called “escape” (the name was changed a few years ago; another name for the same function is “prefix”) and is produced by a key marked ESC on some other computer terminals, but our ESC is not one of the ASCII codes and should not be typed when ALT is wanted.

2.2 The Line Editor

At a display, the characters you type are accumulated by a part of the monitor called the line editor. When you end a line by typing an *activation character*, the entire line is sent to your program (or to the monitor command decoder). Generally you will activate a line with the RETURN key, although others also work. The exact workings of activation are discussed in Section 2.3, but first, there are several editing command characters which can be used to change the text in the line editor.

There are two line editor *cursors* displayed on your screen. The left cursor is underneath the first character in the line editor and is called the *line cursor*. The right cursor is under the character position you are about to write into and is called the *character cursor*. The cursors are underlines on Data Discs, and triangles on IIIs. Also, on IIIs, the characters in the line editor are displayed somewhat below any other characters on the same line. If there is no text in the line editor, the two cursors coincide.

On Data Disc terminals, when there is no text in the line editor, you will sometimes see a “broken” cursor consisting of two very short lines. This means that the program you are running is waiting for a single character to be typed, rather than an entire line. The reason for this display is that many programs ask you yes-or-no questions, and some of them expect you to type, e.g., Y followed

by RETURN while others just wait for the Y. The broken cursor warns you that a RETURN is not expected. However, it is possible for programs to wait for a single character in sneaky ways which do not cause a broken cursor, so the warning is not perfectly reliable.

When you type a normal character into the line editor, it is displayed at the character-cursor position, replacing any other character which was there before. The CONTROL and META keys are used to change the function of characters typed in. Also, several characters have different functions depending on whether or not they are typed with the character cursor at the end of the line. For example, BS at the end of the line deletes the last character; inside the line, it moves the cursor left one position without deleting anything.

As explained below, some characters with CONTROL or META bits are normally interpreted as line editor commands. These commands may only be meaningful under certain conditions; for example, a command to space forward into the line is not meaningful if the cursor is already at the end of the line. CONTROL or META characters which are not meaningful line editor commands can be used as commands to user programs (in which case they are activation characters); they can be treated as if they had been typed without the CONTROL or META bit; or they can be ignored altogether. The choice is made by the user program, which can also disable some or all line editor commands. This flexibility makes it a bit complicated to describe exactly what happens in each case (we make an attempt in the next section), but in general programs are written so that the “right” thing happens. Roughly, in monitor mode, control characters which are not line editor commands are treated as if typed with META, i.e., inserted into the line; when typing to a user program, they generally activate. For the rest of this section, we ignore the question of CONTROL or META characters which are not taken as line editor commands, and simply deal with the ones which are.

META along with a printing character (including space and tab) inserts that character at the cursor position, in front of the character already there, rather than replacing the old character. META characters are considered meaningful line editor commands only when not at the end of the line.

BS at the end of the line deletes the last character. If it's not at the end, BS just backs up the cursor without deleting. People talk about *deleting backspace* and *non-deleting backspace* to distinguish these. CONTROL-BS is always non-deleting, and META-BS is always deleting.

CONTROL-SPACE is the opposite of CONTROL-BS: it moves the cursor forward without changing the text. SPACE alone and META-SPACE act like any printing character. At the end of the line, CONTROL-SPACE usually does nothing (it neither moves the cursor nor activates).

CONTROL-META-SPACE and CONTROL-META-BS are unlike all other characters typed in this system in that they are “repeating” commands: they move the cursor forward and backward, like CONTROL-SPACE and CONTROL-BS, but continue moving the cursor as long as you hold down the keys. No other CONTROL-META characters are line editor commands.

CONTROL-TAB moves the cursor to the end of the line, as if you typed CONTROL-SPACE many times. (Inside the line only.)

CONTROL-FORM is the opposite: it moves the cursor to the beginning of the line, so the two cursors coincide. (Inside or end of the line is ok for this one.)

CONTROL-D deletes the character above the character cursor. (Inside only.) Note that this is different from META-BS, which deletes the character to the left of the cursor.

CONTROL-S skips ahead from the current position of the character cursor to the next occurrence of the character you type next. If the next character you type does not occur to the right of the cursor, nothing happens. If the search character is a letter, the case must match. If the character you type next is RETURN, the cursor moves to the end of the line. (Inside only.)

CONTROL-K is like CONTROL-S but instead of moving the cursor it kills characters up to (not including) the one you select. Again, nothing happens if the character does not occur. If the character you type after CONTROL-K is RETURN, all the characters from the cursor to the end of the line are deleted. (Inside only.)

CONTROL-B searches for the character you type next, like CONTROL-S, but searches backwards (toward the beginning of the line). It is normally accepted as meaningful anywhere except on an empty line.

CONTROL-L kills backwards, like CONTROL-K, but toward the beginning of the line. Meaningful except on an empty line.

CONTROL-R repeats the last search or kill command given. It does not wait for you to type another character, since it remembers the argument from the previous search or kill command. This works even if the line editor has been activated since the last search or kill. This is a meaningful command except on an empty line.

CONTROL-I enters *insert mode*. In this mode, all characters are treated as if you were holding down the META key, until you leave insert mode by typing another CONTROL command or META-character. That is, after CONTROL-I, normal characters typed are inserted instead of overwriting old ones, and BS is a deleting backspace. (Inside only.)

CONTROL-number (that is, any decimal number typed while holding down the CONTROL key) will cause a CONTROL command immediately following to be repeated that many times. For example, CONTROL-5 CONTROL-BS will move the cursor left five positions. This can be used to repeat the BS command as well as CONTROL-BS, META-BS, CONTROL-SPACE, CONTROL-D, CONTROL-S (finds the nth occurrence), CONTROL-K, CONTROL-B, CONTROL-L, and CONTROL-R.

CLEAR will delete everything in the line editor.

CONTROL-RETURN will retrieve the last line which was edited, as long as you haven't typed anything since then. That is, after you activate or CLEAR a line, if the very next character you type is CONTROL-RETURN, the line is returned to the line editor. You can send the same line again by just typing RETURN, or you can edit it. This is very handy when a monitor command, for example, fails because of a spelling error; you can simply say CONTROL-RETURN, correct the error, and say RETURN to activate the corrected command. Typing CALL or an ESCAPE or BREAK command (see Section 2.4) will not prevent a later CONTROL-RETURN, but any character which is entered into the line editor will make it forget the previous line.

CONTROL-T transposes the two characters to the left of the cursor. This is often convenient for bad typists. It is normally considered meaningful except on an empty line, although it has no effect if there are not at least two characters to the left of the cursor.

The TAB character is treated in a special way by the line editor. Commands like CONTROL-SPACE treat a TAB as a single character, so a CONTROL-SPACE when the cursor

points to a TAB will move several spaces to the right. Typing a character without control bits, however, only replaces the first space of the TAB. There will still be a TAB in the buffer, unless the TAB was positioned so that it was only equivalent to one space, in which case it goes away completely when another character is typed over it.

2.3 Line Editor Activation

This section is about the workings of line editor activation: how the line editor decides to release characters to your terminal input buffer. Normally, the characters you type are accumulated in the line editor buffer until you type an *activation character*, which tells the monitor to send the line you have typed into your input buffer. The standard activation characters are just RETURN, LINE, and ALT. (Note: a RETURN typed by the user is automatically translated into a RETURN plus a LINE, and it is the LINE which is the real activation character.) User programs can specify additional activation characters by using the SETACT UUU.

Those CONTROL or META characters which are not meaningful line editor commands may be activation characters, at the option of the user program. Alternatively, they may be treated as ordinary characters to be inserted as if typed with META but without CONTROL. There are two main categories to consider, those control characters which are line editor commands but not meaningful as such in the present context, and those which are never line editor commands.

In monitor mode (while typing monitor commands), the rules are always the same: all line editor commands are enabled; a line editor command which is not meaningful in the present context (e.g., at the end of a line) has no effect (note that, e.g., CONTROL-S will “absorb” the following character even in this case); control characters which are not line editor commands are treated as if typed with META, i.e., they are inserted into the line.

The processing chosen by user programs will depend on whether or not they use control characters as commands. The default processing is to consider all control characters as activators unless they are line editor commands. All possible line editor commands are considered meaningful (i.e., don’t activate) except at the end of the line, where CONTROL-D, CONTROL-I, CONTROL-S, CONTROL-K, CONTROL-TAB, and META-anything activate. If the line is empty, CONTROL-B, CONTROL-L, CONTROL-T, and CONTROL-R also activate. (The phrase “control characters” include those with the META bit as well as those with the CONTROL bit.) The program may, however, suppress activation of some or all control characters, or suppress the effect of some or all line editor commands. User program documentation should indicate which control characters, if any, are used specially by that program. The SETACT UUU is used to control the processing of these characters. If the user program enables 7-bit characters other than RETURN, LINE, and ALT as line activators, then your terminal is said to be in *special activation mode*. Otherwise it is in *normal activation mode*. The program can also disable the CONTROL-RETURN function. Also, programs can choose to activate on any character at all.

The RETURN character is treated specially by the line editor: the cursor is moved to the end of the line before the RETURN is inserted. Therefore typing RETURN always activates the entire contents of the line editor. (Note: this is true only of a RETURN typed without control bits; also, a RETURN typed while in insert mode is treated like a META-RETURN and therefore is not moved to the end of the line.) This special handling of RETURN, the most common activator, makes it convenient to modify something inside the line and then activate it without worrying about getting to the end of the line first.

When you type an activation character other than a bare RETURN, the characters up to and including the activator are available for input to the user program or the monitor command scanner. Any characters following the activator remain in the line editor. (This behavior is also subject to user program control via the SETACT UUO.) The activator is inserted in the line; it does not overwrite the character which was at the cursor position before typing it.

If the activation character is a RETURN with control bits, what the program sees is not a RETURN with control bits, but a bare RETURN followed by a LINE with the corresponding bits. (The system always adds a LINE when you type RETURN; what is special about this case is that the control bits are moved to the LINE.) In particular, the code 612 (CONTROL-META-LINE) should not be interpreted by programs as an end-of-file code if it is preceded by a 015 (RETURN).

When you re-edit with CONTROL-RETURN a line which was activated by a character other than RETURN or LINE (either an ALT or a special activation character established by the user program), the original activation character appears in the line (with its control bits removed), and if the line is then activated by typing RETURN, the RETURN is not actually inserted in the buffer; the original activation character is re-used. Any control bits on the RETURN are put on the re-used original activation character.

2.4 ESCAPE and BREAK Commands

Certain special characters are used to control the status of your display terminal. These are processed directly by the monitor's keyboard scanner, whether or not you are in monitor mode. One example, mentioned above, is CALL. This key stops your program, if one is running, and returns you to monitor mode. You can also type CONTROL-CALL or META-CALL for a *deferred call*—the program is stopped when it tries to read the CALL character from your input buffer. Thus you can type some commands into a program, followed by a deferred call; when the program is finished with your commands and asks for more, it will be stopped.

There are several commands using the ESC and BREAK keys. Generally, the format is ESC or BREAK, then possibly a numeric argument, then a letter or punctuation character which specifies the command. The characters making up the command are not echoed and are not entered into the line editor; typing BS while entering an escape command aborts the entire command. A BREAK command is equivalent to an ESC command with a negative argument, e.g., BREAK 3 W is the same as ESC - 3 W. There should be a concise summary of these commands attached to your keyboard; if not, log in and type this:

```
HELP DDKEY
```

(or IIIKEY instead of DDKEY if you are using a III terminal) to print the list on the XGP.

In the descriptions which follow, upper case letters and punctuation characters represent keys to be typed, e.g., ESC is the ESC key, and characters like [and * are to be typed as indicated. Spaces in the manual separate the keys to be typed, and do not represent actual spaces to be typed, which would be indicated as SPACE. Thus, E S C represents three keystrokes. Lower case letters represent parameters to the commands, so ESC n W means to type the ESC key, then a number (one or more digits), and then the W key. Letters may be typed in upper or lower case. Parameters are almost always numbers before the command letter, as in the last example, but a few commands

allow a programmer name as a parameter, using the syntax ESC [prg CONTROL-letter where the final letter with CONTROL key indicates the command name.

ESC O will stop timeout from your program. Any characters already in your output buffer are deleted, and any attempted timeout from your program is ignored. The program continues running, however. Timeout is resumed if you type BREAK O, or if your program tries to read any characters from your terminal.

It is possible to suspend timeout from your program temporarily without losing output, as with ESC O. Typing CONTROL-BREAK starts this holding. The word HOLDING appears at the top of your screen, and the program is stopped if it tries to type more characters than fit in your output buffer. Holding is released by typing any line editor activation character, or CONTROL-CLEAR.

Display terminals are normally in *full character set mode*, which means you can type in upper and lower case letters. You can leave this mode by typing BREAK F, which has the same effect as pushing the SHIFT LOCK key and also making output appear in upper case. You type ESC F to return to full character set mode. The only use for this that I know of is that you can convert a line of text in a file to upper case in E by typing BREAK F, then something like CONTROL-SPACE to load the line into the line editor, then RETURN, then ESC F. Try it.

The system includes facilities which allow users to see what other users are doing by “spying” on other terminals. The ESC H command will *hide* your terminal, notifying other users that you do not want to be observed. Please note that it is still possible, although considered rude, to override this protection. BREAK H “unhides” your terminal.

There is a FILES command which can be typed when in monitor mode, to list the files in use by your job. So that this information can be seen without having to stop your user program, typing ESC V will type out this file information. ESC n V will type out the files in use by job number n, and BREAK n V will type out the same information for the job controlled by terminal number n. See page 46 for details on what is typed by the FILES command.

Some programs which read data from your terminal expect an “end of file” indication after the data. The thing to type to supply this signal is CONTROL-META-LINE.

Sometimes you may want to write a program which you can interrupt with a signal from your keyboard, without having to keep checking for terminal input in the program. For example, if you run a program which searches a large data base by command, you may want to be able to abort a search without stopping the program. Typing ESC I or BREAK I will generate an interrupt condition which your program may enable; the use of program interrupts is described in the *UUC Manual*. You may type a numeric argument with ESC I or BREAK I; the user program is given the number you type and is told whether you used ESC or BREAK.

If you want to repeat the same escape command several times, but possibly with a different numeric argument, the command ESC + will repeat the last escape command which you gave with a numeric argument, but add one to the argument. BREAK + will do the same thing but subtract one. (Note that there cannot be an ESC - command for this latter function because ESC - is equivalent to BREAK.) With the ESC + and BREAK + commands, what is incremented or decremented is actually the absolute value of the argument; for example, BREAK 3 V followed by ESC + does the equivalent of BREAK 4 V.

The command ESC * will repeat the last escape command which had a numeric argument, without

changing the numeric argument. `BREAK *` will repeat the command but replacing `ESC` by `BREAK` or vice versa. Note that none of `ESC *`, `ESC +`, etc. will repeat an escape command which was given without a numeric argument.

2.5 The WHO Line

The next set of commands have to do with the *WHO line*, a display of system status and the status of a particular job on the top two lines of your screen. Once you start a WHO line display, it is automatically updated until you turn it off explicitly. To start a WHO line for the job at your terminal, type `ESC W`. You can see the WHO line for another job, job number *n*, by typing `ESC n W`. Typing `BREAK n W` will display the WHO line for the job using terminal number *n*. To turn off your WHO line display, type `BREAK W`. (Note: this does not actually make the WHO line disappear on Data Discs, where anything you display stays on the screen until it is explicitly erased.) Typing `ESC W` causes your WHO line to be updated immediately, even if it was already being displayed. The command `ESC 0 W` causes whatever WHO line you are watching to be updated immediately.

One use of the WHO line is to keep track of the progress of a job of yours other than the one at the terminal you're using. To facilitate this, typing `ESC Q` will display the WHO line of the next job in the monitor's job table with your programmer name (or with the same name as the job whose WHO line you are already displaying, if any). Typing `ESC Q` repeatedly will cycle through all such jobs. `BREAK Q` does the same thing, except that it searches the job table backwards. `ESC` or `BREAK Q` simulates the typing of `ESC n W`, where *n* is the job number it finds, so that you can use `ESC *` to force updating of the display.

A final form of the escape command to display a WHO line is provided to find the WHO line for a particular user if you do not know his or her job number: `ESC [ prg CONTROL-W` where "prg" is the user's programmer name (not project name) and the square bracket is used to indicate that this form of argument will be used. This command will display the WHO line for the lowest numbered job logged in with the specified programmer name; if the user has more than one job, `ESC Q` can be used to find the others.

This is the format of the WHO line:

```
LOAD DM III DD JBS,TCOR R,RCOR UCOR NL% DSKQ DSKF DATE DAY TIME TEMPF TEMPC
JOB PPN TTY#! QUEUE JOBNAM SIZE RUNTIME-RP XTIME-XP DSKOPS SEGNAME (ALIAS)
```

The first line is the *system WHO line*:

LOAD	avg number of jobs in TQ or RUNQ over last 68 secs.
DM	number of high-speed phone lines in use (for Datamedias)
III	number of III terminals in use (maximum=6)
DD	number of Data Disc channels in use (maximum=31)
JBS	number of jobs logged in
TCOR	total core of all jobs (including those swapped out) in 512-word pages
R	number of jobs actually run in the last few seconds
RCOR	total core of the jobs counted in R
UCOR	amount of core available for running user jobs
NL	system null-time percentage over the last few seconds
DSKQ	number of jobs waiting to use the disk
DSKF	number of free tracks on the disk
DATE	today
DAY	day of the week
TIME	time of day (24-hr clock)
TEMPF	outside temperature in Fahrenheit
TEMPC	outside temperature in Celsius

The second line refers to a particular job:

JOB	the job number of the job being displayed
PPN	its logged-in project-programmer name
TTY	its terminal number (in octal) or DET for detached jobs
#	"#" is displayed if the WHO line is not for this TTY's job
!	!" is displayed for this TTY's job, if the TTY is hidden (see page 19).
QUEUE	the system queue that the job is in (see below)
JOBNAM	the program being run
SIZE	size of the job's core image
RUNTIME	total run time since login, in minutes and seconds
RP	RUNTIME expressed as a percentage of the job's total wait time (time the job was runnable but not running)
XTIME	incremental runtime, normally reset on terminal input but see below
XP	XTIME as a percentage of incremental wait time
DSKOPS	number of incremental disk operations (reset along with XTIME)
SEGNAM	name of upper segment, if any
ALIAS	disk PPN (see below), if any

Some of these need more explanation. The **QUEUE** is an indication of the immediate status of the job—is it runnable, waiting for input, stopped, etc. These are the possible queue names:

RUNQ	The job is runnable and not waiting for I/O, etc.
TQ	This is a high-priority run queue which jobs are in for a short time after leaving certain kinds of I/O wait.
STOP	The job is stopped because it exited, the user typed CALL , or there was an error in running the job.
NULQ	The job has no core image.
IOWQ	The job is waiting for some input/output operation, or for the completion of a SLEEP or JOBROD UUO . In the case input or output, you may see instead of IOWQ the name of the device being used and the direction of the transfer, e.g., TTYO for terminal output, MTAI for MTA input. (For devices IMP and DLN , the fourth letter is always W for Wait regardless of the direction of transfer.) Also, if the job is waiting for terminal input, you may see these special pseudo-queue names:
LINE	The terminal is waiting for an entire line of input.
CHAR	The terminal is waiting for a single character of input.
SNK	The program has done a SNEAKW UUO and will read the next single character of terminal input without removing it from the input buffer.
DIOW	The job is waiting for some I/O operation and will not be swapped out before it is completed.
INTW	The job has suspended itself waiting for an interrupt.
DWQ	The job is waiting for a device (e.g., line printer) to be free.
MTQ	The job is waiting for the magnetic tape controller.
DTQ	The job is waiting for the DECtape controller.
DCQ	The job is waiting for the data controller (used for mag and DEC tapes).
CMQ	The job is not in core and needs to be brought in quickly.

Some special characters may be displayed after the queue name:

-	The job is currently running.
*	The job is swapped out. A second * means the upper segment is swapped out.
@	The job has been locked in core by the LOCK UUO .
\	The job is running a spacewar module on the KL-10 .
/	The job is running a spacewar module on the KA-10 .
X	The job is running spacewar modules on both processors.

Some of the above may not be clear until you read the *UUO Manual*. The *alias* is a PPN used as the default directory for disk files instead of your logged-in PPN. There is an **ALIAS** monitor command to set the alias for your job.

XTIME is an incremental run time, measured in minutes, seconds, and 60ths of a second (a 60th of a second is called a *tick*). It is normally reset whenever the program is started by a monitor command other than **CONTINUE**, and also whenever the job is awakened by input from the terminal after having been in **IOWQ** waiting for such input. Note that if you type ahead, so that the job never enters **IOWQ**, terminal input will not reset **XTIME**. It is possible to inhibit resetting of **XTIME** with the **BREAK X** command (see below). If resetting is inhibited, the character separating seconds from ticks in the display will be " instead of ' as it usually is.

BREAK X inhibits resetting your job's **XTIME**. **ESC X** resumes the automatic resetting as described above. Neither of these changes the current value of your **XTIME**. Typing **ESC 1 X** will set your **XTIME** to zero; **BREAK 1 X** will set it equal to your **RTIME** (total run time). Both **ESC 1 X** and **BREAK 1 X** inhibit automatic resetting, so you have to type **ESC X** if you want to restore this.

When a job has not run for several minutes, the ticks part of the XTIME field, the XP field, and the DSKOPS field in its WHO line display are replaced with the words “Idle n mins” to indicate that the user may have left even though still logged in. When the job’s WHO line is displayed in response to an explicit ESC W or equivalent command, however, the full XTIME and DSKOPS are displayed once even if the job is idle.

User programs may add a third line to their WHO line display: a display of the status of a particular file in use by the job. What is displayed is the file name, extension, PPN, file length in records, USET pointer, and flags (R for read, W for write, RA for read-alter, U for UDP, E for end of file); this is the same information typed by the FILES command, page 46. This information is displayed to the right of the HOLDING message area on the third line of the screen. This display is started by the SHOWIT Uuo, or by ESC 2 X (see below).

The command ESC 2 X will start a file-status display of the first file in the system’s list of files in use by the job whose WHO line you are displaying, if it has no file-status display already, or the next file if it does. The command can therefore be repeated as necessary to find a particular file. The command BREAK 2 X will erase the file-status display. Since the file-status display is associated with the job, rather than just with the WHO line display on your own terminal, these commands are accepted only if the job’s programmer name is the same as yours, or if it is a detached job.

2.6 Page Printer Control

There are two ways of printing on display terminals. To exploit the full capabilities of the display for graphics, display programs can explicitly control the positioning of points, vectors, or characters. On III terminals, 16 such display programs can be written independently, and any combination of them *activated* or displayed at any time under program control. These display programs are called *pieces of glass*. On Data Disc and Datamedia terminals, a display program runs only once, and is displayed until erased explicitly. There are no pieces of glass as on IIIs. On a Data Disc, the effect of multiple pieces of glass can be gotten in a different way by displaying more than one channel on the screen (see Section 2.7, which describes the video switch).

The other way of printing on a display terminal is simply to output text to be printed in order as if on a piece of paper. The monitor automatically “scrolls” such output; when the area of the screen being used fills up, the text is moved up a few lines, with the top part disappearing. Sixteen of these *pieces of paper* can be displayed at once, on different parts of the screen. At any time, one of these pieces of paper must be selected by the program as the one to receive *page printer* output. The dimensions of each piece of paper are controlled by the program, but the currently selected one can also be controlled from the terminal with ESC commands.

The dimensions of the page printer include several factors. First, the vertical position of the top line of the page. This can be set by the command ESC n Y for line n. (Lines 1 and 2 are where the WHO line goes; the normal position is line 4.) ESC Y resets the vertical position to normal.

The scrolling function is controlled by two parameters: the number of lines typed before a *glitch* (the upward repositioning of the text), and the number of glitches on the page. These are set by ESC n L for lines/glitch and ESC n G for glitches/page. The defaults are 4 glitches of 9 lines for Data Disc terminals, 12 glitches of 2 lines for III terminals, and 7 glitches of 3 lines for Datamedias.

ESC L and ESC G with no argument restore the respective counts to normal. (Note: there are BREAK L and BREAK G commands, having to do with the video switch; those commands are entirely unrelated to ESC L and ESC G.)

It is possible to cause automatic holding, as with CONTROL-BREAK, every *n* lines or every *n* glitches. The commands for this are ESC *n* E for every *n* lines, and ESC *n* J for every *n* glitches. At each pause, the word HOLDING will appear on the third line of the display. CONTROL-CLEAR will resume output as usual. ESC E and ESC J disable the respective automatic pauses.

Typing ESC N will normalize your page printer geometry. BREAK N will clear your screen, select piece of paper 0, normalize the page printer, and display the text in the page at the normal position. To refresh the current page printer display without changing the page printer geometry, type ESC P. BREAK P will clear the screen and then do ESC P. These are useful for getting rid of any noise which may appear on your screen. There is a rarely-used ESC R to refresh just the display of your line editor buffer.

The command ESC FORM will erase the currently selected piece of paper. Then, if the terminal is in monitor mode (waiting for a monitor command), it will type a dot.

Note: Any RESET of your job, either by the RESET UUO or automatically by the system, eliminates all but piece of paper 0, selects that piece of paper for further output, and normalizes the page printer display. The *UUO Manual* explains the RESET function.

2.7 Data Disc Control

Some ESC and BREAK commands are specific to one of the three kinds of display terminal. This section is about the Data Disc ones.

The usual display on a Data Disc screen is green letters (hereafter called “white”) on a black background. It is also possible to display black on white, by typing BREAK C. To restore the normal white on black, type ESC C. Both of these clear the screen, so you have to type ESC P to refresh your page printer display. Black on white display requires increasing the contrast and brightness controls to be visible. This is not the world’s most useful feature; it is also possible, under program control, to display only part of the screen in black on white.

Virtual Terminals

The remaining Data Disc escape commands control the connection between your physical terminal and some *virtual terminal* defined by the system software. For all types of terminal other than Data Disc terminals (dialup, hardcopy, III, or pseudo-TTY), each physical terminal corresponds to a unique software TTY number. For example, the Teletype in the Prancing Pony is always associated with the software terminal line called TTY14. This unique correspondence does not exist for Data Disc terminals. Since there are 58 physical Data Disc terminals, and only 31 video channels available for users, not all of the physical terminals can be used at once; when a user first

types on a vacant Data Disc keyboard, the monitor assigns a virtual terminal line number and associates that number with the physical terminal. For example, the Data Disc terminal located near the PDP-10 console is TV-66, but it may or may not be associated with virtual TTY66.

Actually, there are two different correspondences to consider: what appears on a particular physical screen may not correspond to the virtual terminal controlled by the associated keyboard. That is, you may be seeing one thing on your screen while typing at something different. There are escape commands both to determine *keyboard mapping* (that is, which virtual line is being controlled by your keyboard) and to control the *video switch*. The video switch allows you to select, from among the 32 Data Disc channels, plus various TV cameras and video synthesizers, what you want to see on your screen. You can see more than one Data Disc channel at a time, but only one of the *analog* channels (the cameras). You can select an analog channel along with a Data Disc channel only if the two are synchronized; e.g., the lounge TV signal can't be combined with a Data Disc channel because the synchronization of the former is not under our control.

In the following discussion, the word "keyboard" refers to a physical Data Disc terminal keyboard, "screen" to a physical screen, "line" or "terminal line" to a virtual terminal, and "channel" to one of the video images generated by the Data Disc controller. A number like TV-102 refers to a keyboard and screen, TTY66 to a terminal line, and DD2 to a Data Disc channel. Note: all of these numbers are *octal*; there is no TTY68!

When a virtual terminal line is created (by typing at an available keyboard), it is given a Data Disc channel. This is the terminal's "main channel". The job running on that line may choose to select other channels for viewing in addition to (or instead of) its main channel. For example, a program which does video synthesis may choose to display the video synthesizer output on its controlling terminal line. The channels selected for a particular line constitute the line's "video map". Actually, each line has both a temporary and a permanent map; a program RESET cancels any changes to the temporary map and reverts to the permanent map. (The permanent map does not necessarily select the line's main channel, although that is the most common case; both temporary and permanent maps are controlled by the program.) A screen which is watching the video map of a particular line is said to be "tied" to that line. Normally, when a keyboard is mapped to a line, the associated screen is tied to the same line.

Just as a terminal line can be hidden to discourage spying (see page 19), Data Disc channels in use by a job may be *public* or *private*. Only public channels may be selected by other users. The main channel used by a terminal is normally public. The command ESC H *hides* (makes private) your terminal's main channel; BREAK H makes it public.

Keyboard Mapping

Although a keyboard can only be mapped to one line at a time, you may be using several lines from the same physical terminal and mapping back and forth. (Note: during the day, when there is a shortage of Data Disc channels, this use of more than one line is strongly discouraged!) Therefore, at any given moment a particular line may not have any keyboard mapped to it. However, the monitor remembers which keyboard is *responsible* for the line. This is normally the keyboard which most recently typed a character into the line. (In other words, if you map to a line and map away again without typing anything into it, you do not become responsible for it.) It is also possible to claim or disclaim responsibility for a line explicitly. The concept of responsibility is

important for two main reasons: audio beeps directed to a line (see Section 2.9) are heard at the responsible physical terminal, even if its keyboard is mapped elsewhere at the time; and the keyboard responsible for a line can map to it even if it is hidden. When a keyboard becomes responsible for a line for which some other keyboard was previously responsible, the system also remembers the previous responsible keyboard, so that if the new responsible keyboard disowns the line, responsibility for it is returned to the previously responsible keyboard. However, only one previously responsible keyboard is remembered.

A keyboard may be responsible for more than one line, but there is always one line which is the *home* line for that keyboard. This is normally the first line into which a previously unused keyboard types, but can be changed by explicit escape commands.

All of the keyboard mapping commands in this section also tie your screen to the same line. Therefore, you will normally see what you are typing at, unless you give a special video switch command to control the screen separately.

The command to map your keyboard to a terminal line is `ESC n M`, where *n* is the line number. The line must be vacant, your own (i.e., you must be responsible for it), or public (see `BREAK H` above) for this command to have any effect. Typing `ESC M` will map you back to your home line. Since you may not know the line number if you want to map to a particular user's terminal, there is an alternate form of the mapping command in which you specify the programmer name of some user, and the system finds some terminal line on which that user is logged in. The form of this command is `ESC [ prg CONTROL-M` or alternatively `ESC [ prg ]` (the closing bracket implies the `M` command).

The command `BREAK M` maps your keyboard to the line of another job with the same programmer name as the one to which you are currently mapped, if there is another such job on a Data Disc line. The line must be set public, or your keyboard must be responsible for it, as for `ESC n M`.

The command `BREAK R` maps your keyboard to another line for which it is responsible, if any. Repeated `BREAK R` commands will cycle through all such lines. (Note: there is an `ESC R` command which is unrelated to this one.)

The command `ESC n M`, when *n* is an unused line number, will map your keyboard to an available line, but when you initialize the line by typing into it, you may not be assigned to the specific number you gave in the command. Of course, if you want an unused line, it doesn't matter what line number you get, and you may not know the number of an available line anyway. Therefore, you can ask for an unused line with the command `ESC CALL` which maps you to an unused line, types a `CALL` into the line, and makes you responsible for it. If there are no available lines (i.e., no free Data Disc channels), your keyboard number will be entered into a queue of people waiting for channels, and you will be mapped to an "available" line into which you can't type anything. You can then, of course, type `ESC M` to return to your other line, if you have one, while you're waiting. When you are placed in the queue, you will hear a beep to notify you. If you don't get the beep, it means that the queue was too full. (This is very rare.) The first several entries in the queue are displayed on the available channel (the one seen by unused terminals). Each keyboard can only be in the queue once, even if you type `ESC CALL` again. You can leave the queue by typing `BREAK CALL`, which will beep you if and only if you were actually in the queue. If you are in the queue, when your turn comes and a channel becomes available, you are beeped again. You are made responsible for the channel, so you can map to it with the `BREAK R` command. The beeps associated with entering and leaving the Data Disc queue are not disabled by the `ESC B` command, which prevents other kinds of beeping.

Please note that these keyboard mapping commands apply to your physical terminal. Most escape commands, like ordinary characters or line editor commands, apply to a virtual terminal line; for example, a command like ESC W affects the line to which you are mapped, and claims responsibility for the line as a side effect. The mapping commands, however, do not affect any line, and do not make you responsible for the line to which you are mapped. The video switch commands and audio switch commands, to be described shortly, also apply to your physical terminal.

The command ESC CLEAR is used to set your home terminal line explicitly to the one to which you are currently mapped. This home terminal line is the one to which the ESC M command maps your keyboard. You also become responsible for the line, if you weren't before. The command is rarely needed, because the default home terminal line is the first one into which you typed from a previously unused keyboard. If you lose responsibility for your home line, the system will assign another of the lines you are responsible for as your home line. (If you are not responsible for any other lines, your keyboard is considered unused.) You can lose responsibility because someone else maps to the line and claims responsibility, or because the line "goes available" a few seconds after the job on that line logs out. You can also explicitly disclaim responsibility for a line by typing BREAK CLEAR while mapped to that line. This command should be used when you want to permanently dissociate your physical terminal from the line, e.g., when you move to another room.

Video Switch Commands

The keyboard mapping commands automatically also tie your screen to the line to which your keyboard is being mapped. To tie your screen to a different line, type BREAK n L (note that this is completely unrelated to the ESC n L command described earlier!), and to tie it back to the line to which your keyboard is mapped type BREAK L. As for ESC M, there is a form of this command which refers to a programmer name: ESC [prg CONTROL-L ties your screen to a line being used by that programmer. The command BREAK G ties your screen to another line in use by the same programmer name as the one you are already tied to. Thus, BREAK G is to BREAK n L as BREAK M is to ESC n M. The same privacy rules which affect keyboard mapping also apply to screen tying.

When a screen is tied to a line, the program running under control of that line may modify the video map of the line, and such changes affect any screen tied to the line. The following video switch commands select a video map for your screen which is independent of any program selection. After you give such a command, your screen is *untied* until you give a tying command like BREAK L.

Video switch commands are generally of the form ESC n x, where n is a channel number (Data Disc or analog channel) and x is the command letter. The form BREAK n x interprets n as a terminal line number if n is a valid Data Disc line number, and selects that line's main channel. (This is not the same as BREAK n L, which selects whatever channel(s) the specified line has in its map!) Thus, ESC 30 S selects Data Disc channel 30, but BREAK 70 S selects whatever channel is assigned to TTY70. ESC x with no number selects the main channel associated with the line to which your keyboard is mapped. BREAK n x where n is between 3 and 6 selects analog channel 40+n. The reason for this alternate notation is that these analog channels are TV receivers, and the BREAK n x command selects the sound from the corresponding TV set (see the audio switch

section below) as well as the picture. `BREAK x` with no number selects the *available channel*, the one which is seen by screens not in use. This channel contains a display of status information for all jobs logged in.

For the remainder of this subsection, the notation `{ ... }` means that the characters inside the braces are optional, e.g., `BREAK {n} S` in the next paragraph means that the paragraph applies both to `BREAK n S` and to `BREAK S` without an argument. The notation `[ ... | ... ]` means that any of the characters shown may be used, e.g., `[ESC|BREAK]` means that the paragraph applies to both the `ESC` and the `BREAK` form of a command. (The two forms are distinguished by the interpretation of the command argument, as explained in the previous paragraph.)

The commands `ESC n S` and `BREAK {n} S` untie your screen and set its untied map. (Note that `ESC S` with no argument is interpreted specially, as explained below.) The map will not be changed until you give another video switch command.

The `[ESC|BREAK] {n} T` commands (note that `ESC T` is included) are *temporary* mapping commands. In addition to the effect described above, they set a flag bit associated with your screen so that when the program on the line to which your keyboard is mapped does a video switch RESET operation (either explicitly or because of a RESET of the job as a whole) you are automatically re-tied to that terminal as if you had typed `BREAK L`. The flag is cleared by `[ESC|BREAK] {n} S` and by the tying commands (the keyboard mapping commands and `BREAK {n} L`).

The commands `[ESC|BREAK] {n} [A|D]` add and delete channels to and from the map which you are currently watching, without modifying your tied/untied status. If you are untied, your screen's untied map is modified. If you are tied, the command affects the map of the line to which you are tied; if that line has a program-selected temporary map (one which will last only until a program reset), only the temporary map is modified. In other words, if there is a temporary map in effect, the escape command will be temporary also. (Note: the temporary and permanent maps associated with a virtual terminal line are unrelated to the distinction between the temporary and permanent untying commands `S` and `T`, which affect physical screens.) Note that if you are untied, these commands affect only what you see, but if you are tied, they affect other users who are tied to the same line you are.

The `ESC S` command with no argument is interpreted differently from the other video switch commands. It ties you to the line to which your keyboard is mapped, like `BREAK L`, and also sets that line's temporary and permanent maps to its main channel (the one assigned to it when it was first typed on). Thus, `ESC S` undoes whatever video switch manipulation your program has done as well as tying you to it. It affects other users tied to the line you are mapped to. Note that only `ESC S` has the effect described in this paragraph, not, e.g., `ESC T`. `ESC S`, like other forms of video switch reset, re-ties anyone who is keyboard-mapped to your line but untied by `[ESC|BREAK] {n} T`.

Spying on Hidden Terminals

When someone hides his or her terminal (by typing `ESC H`), it is considered a severe violation both of common courtesy and of laboratory policy to override the hidden status and spy on the terminal, unless at the specific request of its owner. Occasionally, though, you may want to see your

own hidden terminal while you are in someone else's office, or someone may ask you to override hiding. For this reason, special escape commands are provided which allow you to map or tie to hidden lines. These commands are prefixed with the special character CONTROL-META-TOP-UNSHIFT-BREAK (That is, the SHIFT and SHIFT LOCK keys must *not* be held down!) This character is known as MAGIC and the commands which use it are *magic mapping* commands. The commands are equivalent to ordinary mapping commands, except that they ignore hiding:

<i>magic cmd</i>	<i>equivalent normal cmd</i>	<i>function</i>
MAGIC n M	ESC n M	kbd map to TTYn
MAGIC n L	BREAK n L	tie screen to TTYn
MAGIC G	BREAK G	tie to same prg
MAGIC n S	ESC n S	map screen to DD chan n
MAGIC n A	ESC n A	add DD chan n to map
MAGIC n T	ESC n T	temp map screen to DD chan n
MAGIC [prg]	ESC [prg]	kbd map to prg
MAGIC [prg CONTROL-M	ESC [prg CONTROL-M	kbd map to prg
MAGIC [prg CONTROL-L	ESC [prg CONTROL-L	tie screen to prg

When you type ESC H, anyone who is mapped or tied to your line will be mapped to his or her home terminal line, or tied to the line mapped to if not the same as yours, even if magic-mapped. But they can repeat the magic mapping command after you type ESC H.

2.8 III Control

At a III terminal, several different display programs, called *pieces of glass* ("pog"), may be running at once. You can choose which of these (zero or more at once) are actually displayed.

ESC n S selects piece of glass number n only. ESC n A adds piece of glass n, and ESC n D deletes pog n. If no numeric argument is used, these commands apply to all pogs, e.g., ESC D clears all pogs from the screen.

ESC C eliminates all pogs (your page printer is unaffected). It is different from ESC D in that the display programs are completely forgotten, so they cannot be reselected.

Note that the selection of display programs can also be controlled by a UUO in your program.

2.9 Audio Switch Control

Associated with each Data Disc or III terminal is a loudspeaker which can be connected to various sound sources under computer control, by an audio switch analogous to the video switch for Data Disc channels. There are 20 (octal) audio switch input channels, numbered 0 to 17. Only one channel can be connected to your speaker at a time.

For Data Disc terminals, the audio switch selection applies to a physical terminal, not to a virtual terminal line. Program-initiated audio switch selections affect the physical terminal which is responsible for the program's terminal line, if any.

One of the input channels is normally silent, but is connected to the telephone paging system when a page is in progress (i.e., someone has dialed 9 on the intercom line). When you are listening to another channel, you may choose to be switched to the paging channel automatically when a page is in progress and then returned to your original channel.

The command `ESC n U` will select audio channel `n` and allow paging interruption. `BREAK n U` selects channel `n` and does not allow paging. `ESC U` selects the silence/paging channel.

At a Data Disc, the `BREAK n [S|A|T]` commands, with `n` between 3 and 6 inclusive, select both picture and sound from a TV receiver. Specifically, video channel `40+n` and audio channel `10+n` are selected.

Your program can control the audio switch selection for your terminal by `UUO`. Although programs cannot in general change the audio switch setting for another terminal, there is a `BEEP UUO` which can be used to cause any terminal's loudspeaker to be connected to the beep generator for a short time (less than one second). This is used by, e.g., the `SEND` command to call the recipient's attention to a message on his screen. You can permit or refuse such beep connections by `UUO` or by escape commands; `BREAK B` refuses beeps, and `ESC B` permits them. Any change to your audio switch setting by other means (e.g., `ESC U`) also cancels the effect of `BREAK B`, so you must repeat the `BREAK B` command after changing your selection.

The `ESC Z` command enables *automatic beeping* for the job controlled by your terminal line (*virtual* terminal for Data Discs), so that you will be beeped whenever your program runs for 15 seconds or more of real time and then becomes ready for you to type something. More precisely, if you are enabled for this automatic beeping, then you will be beeped when your job enters `STOPQ`, or `IOWQ` for TTY input, or `INTWQ` for TTY interrupts only, provided that your job's incremental run time plus wait time is 15 seconds or greater. A job's incremental wait time is the amount of time spent waiting for the system since the last command was given that caused the program to leave TTY input `IOWQ` or `STOPQ`. This wait time corresponds precisely to the incremental run time displayed on the `WHO` line—both incremental times are reset at the same instant. Also, if you type `BREAK X` or a similar command which causes the incremental times never to be reset, then you will never be beeped automatically.

The command `ESC n B` can be used to make a temporary selection of the silence/paging channel for `n` minutes. This is useful when listening to the radio, to avoid commercials or obnoxious songs. `BREAK n B` is the same, but disallows beeps during the `n` minutes.

The `BREAK U` command will undo a temporary selection of the paging channel. More specifically, if you are listening to a telephone page, `BREAK U` will restore your regular audio selection, but will not prevent future paging interruptions; if there is no page in progress, but you are connected to the paging channel because of an `ESC n B` command, the effect of that command is cancelled.

As we go to press, the following audio switch inputs are connected:

0	silence/paging
1	digital-to-analog converter channel A
2	KDFC
3	Systems Concepts synthesizer
4	FRM synthesizer
5	beep generator
6	KJAZ
7	KPFA
10	KARA
11	KSJO
12	K101
13	old cart TV receiver
14	new cart TV receiver
15	lounge TV receiver
16	silent channel
17	KSAN

Audio inputs 13-16 correspond to video inputs 43-46.

2.10 Datamedias

The WAITS facilities which are designed for display terminals are available to users outside the laboratory through a specially modified version of the Datamedia Elite model 2500A terminal. Hereafter, in this manual, the word "Datamedia" (or "DM") will be used to mean one of these specially modified terminals. (Programmable terminals which can be made to simulate a Datamedia will, of course, also work.)

The most important modification required to a standard Datamedia to make it work as a WAITS display terminal is the *SUMEX keyboard*. This keyboard contains two keys near the space bar, labelled EDIT. These keys control the eighth (200 octal) bit of the characters sent to the computer; in a normal terminal this bit is unused, or used as a parity check for the transmitted characters. The EDIT key, in other words, is a shift-like key used along with some other key, like the CONTROL and META keys on local SAIL displays.

Please note that the control (CTRL) key on the Datamedia is not the same as the Stanford CONTROL key. Instead, CTRL is like the Stanford TOP key: it is used to generate the nonstandard Stanford graphic characters. The CTRL key subtracts 100 octal from the character code normally produced by a character key. For example, the letter T is octal code 124; CTRL-T is 024, which is V in the SAIL character set. The command `HELP DMKEY` will print on the XGP a list of the correspondences between CTRL characters and the SAIL equivalents, along with other information about the DMs.

Since Datamedias are not permanently connected to the computer, but are used through telephone or ARPAnet connections, you must tell the monitor that your terminal is a DM before it can be used as a display. This is done with the TTY command; this command is followed with a keyword specifying some property of your terminal. Most functions of the TTY command are meant for

non-display terminals, but a few are used for Datamedias. For example, the basic command to tell the monitor that your terminal is a Datamedia is TTY DM. The command TTY NO xxx will have the opposite effect from TTY xxx. Note: if you often use the system remotely from a DM, you can get the LOGIN program to give the necessary TTY commands automatically when you log in; see Section 5.1. Here are the TTY keywords concerning DMs:

DATA MEDIA n	Tells the system you are at a DM. The number n, if used, tells the system the number of lines your terminal can display on its screen; this is needed only for a programmable terminal simulating a DM. TTY NO DATA MEDIA waits for all already-queued output to be sent to your terminal, and then sends a message to let you know you have become a non-display, and puts the terminal in ROLL mode.
DM n	Tells the system you are at a DM. Just like DATA MEDIA.
DM128 n	Tells the system you are at a DM which can display the ASCII control characters (001 to 037 octal) when they are output preceded by an ASCII escape character (033).
BLINK	Tells the system to display the nonstandard graphics as blinking or bold letters. This is the default; TTY NO BLINK will display them as ordinary letters instead. Your terminal must already have been declared a DM when you give this command.
BOLD	Same as BLINK.
BS	Tells the system that your terminal can display a special graphic representation for the BS (octal 177) character. Your terminal must already have been declared a DM when you give this command.

You should also read the description of the TTY command as used for non-display terminals, in Section 3. Some non-display keywords are also meaningful for DMs.

The information which follows is taken from the file DM.ME[UP,DOC]; that file also contains important information about how to set up a telephone connection to the PDP-10 from your DM, and how to use a DM through the ARPA network.

Since the DM keyboard does not have keys for all of the Stanford characters, certain characters must be typed in their ASCII CTRL form; for example, partial-sign (ø) is ↑O and thus must be input as CTRL-O. Also, since the DM display does not have symbols for all the Stanford characters, those characters that must be typed in as CTRL-something will be displayed in a special form, namely as either BOLD-something or as BLINKING-something. On SU-AI DMs, CTRL-something will appear as BOLD-something; on other DMs that have not been modified to interchange the BLINKING and BOLD functions, CTRL-something will appear as BLINKING-something. For example, on SU-AI DMs partial-sign (ø or ↑O) will be displayed as BOLD-o; on non-modified DMs it will be BLINKING-o. If you have a non-modified DM and don't like blinking (you won't), then type TTY NO BLINK.

Some DMs have been modified to display the IMSSS character set, a slightly modified version of the SAIL character set. If you have one of these IMSSS modifications, the command TTY DM128 will tell the system to display the SAIL characters as they are shown locally (more or less).

Here is a summary of what you have to type to get certain effects:

To get ESC, type NUL.

To get BREAK, type NUL and then - (minus sign).

To get CONTROL-<char>, type EDIT-<char>; that is, hold down EDIT while typing <char>.

To get META-<char>, type EDIT-NUL and then <char>.

To get META-CONTROL-<char>, type EDIT-NUL and then EDIT-<char>.

To undo ESC or BREAK, type DEL.

To undo META, type NUL and then DEL.

To get CLEAR, type the bottom unlabelled key at keyboard left, or MR.

To get FORM, type ↑L.

To get BS, type DEL.

To get ALT, type ESC.

To get CALL, type CALL.

To get deferred CALL (CONTROL-CALL), type EDIT-CALL.

To suspend timeout, type HOLD; to resume timeout, type HOLD again.

The three characters β , $\equiv$, and $\vee$ are hidden under the keys CALL, CLEAR, and HOLD, respectively. To get one of the characters $\beta\equiv\vee$, you must quote it by preceding it with NUL. However, if you want META with the character, then the quoting is automatic with the add-META command EDIT-NUL. Thus you cannot type META-CALL, META-CLEAR, or META-HOLD because you would get META- β , META- $\equiv$, or META- $\vee$ instead.

The line editor works as on local display terminals, except that there is no line cursor, only a character cursor. However, the characters in the line editor are displayed bold or blinking. The escape commands which are not specific to Data Discs or to IIIs (those in Section 2.4 through Section 2.6) are available on DMs.

When you tell the system that your terminal is a Datamedia, it will erase the DM screen and then set the DM's tab stops while typing an initialization message at the top of the screen. If you have a DM terminal that lacks hardware tabs (all of the AI Lab's DMs have them), then you should type TTY NO TABS; in that mode each tab to be output will be converted to an equivalent number of spaces, which will of course usually take longer to output than just a tab. If you log in before telling the system you are on a DM, the system will assume that your DM does not have hardware tabs unless you say TTY TABS.

If you do not get the initialization message, then you should immediately do a BREAK P (i.e., the three characters NUL - P) to force it. If you don't type BREAK P in this case, there is no guarantee of consistent output as your terminal may have its cursor at some place other than where the system thinks it is. If BREAK P doesn't get you the initialization message but does erase and redraw the screen, then type the monitor command TTY TABS (unless your DM terminal doesn't have hardware tabs), which should get you the message at the top of the screen. If timeout is garbled after correct initialization, then either you are having phone line trouble or there is a bug in the DM display service—the former is the best guess. If it happens repeatedly on one particular phone line, you should report it with the GRIPE command.

While you are holding (even if the HOLDING message does not appear), your line editor will appear on the HOLDING message line (the third line of the screen).

Two escape commands are available to DM users for suppressing user program display output outside of a specified range of columns on the display. ESC m < (where m is decimal) suppresses all user display output to the left of column m, and ESC n > (n decimal) suppresses all user display output to the right of column n. The only type of output these commands affect is user program output generated by the UPGIOT UUO (i.e., display program output), and even then it does not affect output in insert/delete mode nor does it affect output of RETURNS and LINES.

Note that ESC 5 < (for example) will suppress output to the left of column 5 and therefore whatever text already appears in columns 1 to 4 will (normally) not be erased even when new text appears starting in column 5. However, after an ESC 40 > (for example) old text in columns 41 to 80 will usually be erased by new text at or before column 40 since output of actual text usually causes the rest of the line to be erased (except when this erase-to-end-of-line feature is inhibited). Also, if, for example, ESC 20 < and ESC 10 > are given, all output is suppressed except for RETURNS and LINEs and the line erases.

Thus these commands can be used to speed up displaying by E, WHO and other display programs by suppressing unneeded output.

ESC < resets the left DM margin to zero and ESC > resets the right DM margin to infinity. ESC N resets both margins, as does a RESET or DPYCLR UUO. BREAK > sets the right margin to 0, thus suppressing all text.

SECTION 3

NON-DISPLAY TERMINALS

Any terminal other than a Data Disc, III, or known Datamedia is considered a “non-display” whether or not it uses a display screen for its output. Non-displays are sometimes called “Teletypes” after a particular brand of terminal. They cannot use the special display facilities described in the preceding sections, such as the line editor and WHO lines. However, some of those facilities are approximated by special commands for non-display terminals. Connections over the ARPA network or by telephone are considered non-displays unless declared to be Datamedias by the TTY DM command.

The CTRL (CONTROL) key on a non-display does not produce the same codes as CONTROL on a display. Instead, it produces codes which, on display terminals, represent some of the TOP characters. Most programs which deal with non-displays, however, either cleverly use only characters which are the same on all terminals, or have special notations for use at non-displays. In particular, there is an *SOS representation* (named after a non-display editor) for printing characters not found on non-displays, namely a question mark followed by some other character. See Appendix 13. The CTRL key generally subtracts 100 octal from the corresponding upper case letter, e.g., CTRL-D is 004 octal, 100 less than D, octal 104. Non-display control characters are usually echoed as uparrow (or circumflex on some terminals) followed by the corresponding non-control character, e.g., ↑D for CTRL-D.

There are a few cases in which the same graphic character is represented by different codes in standard ASCII and in the SAIL extended character set. Since non-displays have standard ASCII characters, these codes are converted on input and output. Here are the converted codes:

char	Stanford	↔	external
~	32	↔	176
}	176	↔	175
ALT	175	↔	33
≠	33	→	33
EOF	612	←	32

To get the effect of CALL, type CONTROL-C twice. (Now you know why CALL on displays prints ↑C.) Deferred CALL is a single CONTROL-C.

Although there are several models of non-display terminal, some do not have lower case letters. The SHIFT key on these terminals, like the TOP key on our displays, produces non-alphanumeric printing characters. Most of these are labelled on the keys, but you may not find [(SHIFT-K), \ (SHIFT-L), or] (SHIFT-M) labelled on some models.

There is no line editor on non-displays. You can delete the last character on a line by typing RUBOUT, which prints a backslash and then the deleted character. Successive RUBOUTs will print the characters deleted, until a non-RUBOUT, which prints another backslash and then whatever you typed. CONTROL-U will delete the entire line, like CLEAR on a display.

If your terminal is actually a display, even though not a SAIL one, and is capable of deleting a

character from the screen in response to the output sequence backspace-space-backspace, you can get the system to use that method of indicating deletions, rather than the backslash notation, by using the TTY GLASS command described below. Such a terminal is called a *glass TTY*.

The end-of-file character for non-displays is CONTROL-Z. If your program does input by TTYUUC, CONTROL-Z is converted to 612 (the code for CONTROL-META-LINE). TAB is typed as CONTROL-I. FORM is CONTROL-L, and VT is CONTROL-K. RETURN, LINE, and ALT have corresponding non-display keys. (The key for ALT may be labelled ALT MODE, ESCAPE, or PREFIX depending on when the terminal was built; if none of these appears, type CONTROL-[to get ALT.)

Some escape commands for displays have non-display equivalents. CONTROL-O flushes typeout like ESC O. A second CONTROL-O resumes typeout like BREAK O. CONTROL-B starts and stops typeout holding, like CONTROL-BREAK and CONTROL-CLEAR on display terminals.

3.1 Non-display ESCAPE commands

The character $\uparrow\leftarrow$ (octal 37, ASCII control-underscore, or control-leftarrow or control-slash on some keyboards) is used as a prefix for special commands, like ESC on a display. The $\uparrow\leftarrow$ character is not echoed. The following commands are now defined:

$\uparrow\leftarrow\uparrow\mathbf{B}$	send an α (alpha, $\uparrow\mathbf{B}$) to input buffer
$\uparrow\leftarrow\uparrow\mathbf{C}$	send a β (beta, $\uparrow\mathbf{C}$) to input buffer
$\uparrow\leftarrow\uparrow\mathbf{O}$	send a ∂ (partial-sign, $\uparrow\mathbf{O}$) to input buffer
$\uparrow\leftarrow\uparrow\mathbf{U}$	send a $\exists$ (there-exists, $\uparrow\mathbf{U}$) to input buffer
$\uparrow\leftarrow\uparrow\mathbf{Z}$	send a $\neq$ (not-equal) to input buffer
$\uparrow\leftarrow\uparrow\leftarrow$	send an $\vee$ (or-sign, $\uparrow\leftarrow$) to input buffer
$\uparrow\leftarrow-$	negate arg for following command
$\uparrow\leftarrow\mathbf{0}:\uparrow\leftarrow\mathbf{9}$	decimal arg for following command
$\uparrow\leftarrow\uparrow\mathbf{W}$	delete the last word on the input line
$\uparrow\leftarrow\mathbf{C}$	add CONTROL (200) bit to the next character
$\uparrow\leftarrow\mathbf{D}$	add Double-bucky (CONTROL-META, 600) bits to the next character
$\uparrow\leftarrow\mathbf{F}$	full character set
$\uparrow\leftarrow\mathbf{H}$	hide terminal
$\uparrow\leftarrow\mathbf{I}$	interrupt program
$\uparrow\leftarrow\mathbf{L}$	delete back to next character typed
$\uparrow\leftarrow\mathbf{M}$	add META (400) bit to the next character
$\uparrow\leftarrow\mathbf{N}$	No-op, but undo pending bucky bits
$\uparrow\leftarrow\mathbf{R}$	redraw the current input line
$\uparrow\leftarrow\mathbf{V}$	force a FILES command
$\uparrow\leftarrow\mathbf{W}$	force a TTY WHO command
$\uparrow\leftarrow\mathbf{X}$	control XTIME, like ESC X on display
$\uparrow\leftarrow\mathbf{Z}$	enable automatic beeping, like ESC Z on display

Lower case letters are the same as upper case. Undefined commands are echoed as a bell and otherwise ignored.

The first set of special commands are used to "quote" characters which normally have special meanings on a non-display terminal, namely, $\uparrow\mathbf{B}$ (hold/unhold), $\uparrow\mathbf{C}$ (call), $\uparrow\mathbf{O}$ (suppress output), $\uparrow\mathbf{U}$ (clear input), $\uparrow\mathbf{Z}$ (end of file), and $\uparrow\leftarrow$ itself. The corresponding SAIL characters are α (002, $\uparrow\mathbf{B}$), β (003, $\uparrow\mathbf{C}$), ∂ (017, $\uparrow\mathbf{O}$), $\exists$ (025, $\uparrow\mathbf{U}$), $\neq$ (033, represented by $\uparrow\leftarrow\uparrow\mathbf{Z}$), and $\vee$ (037, $\uparrow\leftarrow$).

The digits and minus sign are used to create an argument to a following command, as for ESC on a display. Minus sign is like BREAK on a display. For example, $\uparrow\leftarrow$ F turns on full character set mode, and $\uparrow\leftarrow -$ F turns it off. The digits or minus sign are typed between the $\uparrow\leftarrow$ and the command character, e.g., $\uparrow\leftarrow - 27$ W.

The $\uparrow\leftarrow \uparrow$ W (word delete) command is like a succession of rubouts; it deletes characters until the first (rightmost) letter or digit, then deletes until the first character which is not a letter or digit, not including that character. The deletions will be echoed with the backslash notation or by erasing the characters deleted depending on the TTY GLASS bit. $\uparrow\leftarrow n \uparrow$ W deletes *n* words.

The commands $\uparrow\leftarrow$ C, $\uparrow\leftarrow$ D, and $\uparrow\leftarrow$ M can be used to type a character with Stanford CONTROL and META bits into your program. C stands for Control, M for Meta, and D for Double-bucky (CONTROL and META). You can also say $\uparrow\leftarrow$ C $\uparrow\leftarrow$ M instead of $\uparrow\leftarrow$ D. These commands echo as [C] [M] or [D]. The bits are applied to the next ordinary character you type. If you change your mind, $\uparrow\leftarrow$ N will restore the No bucky bit state.

The commands $\uparrow\leftarrow$ F, $\uparrow\leftarrow$ H, and $\uparrow\leftarrow$ I correspond to the ESC commands for those letters: F sets the full character set bit, H sets the hidden bit, and I interrupts your program. Type a minus sign after the $\uparrow\leftarrow$ and before the letter to invert the sense of the command, e.g., $\uparrow\leftarrow -$ H unhides your terminal.

The $\uparrow\leftarrow$ R command echoes a crlf and then retypes the text in the current input line; it is useful if the line is full of deletions and you have trouble reading it.

The $\uparrow\leftarrow$ W command types out your job's WHO line on the terminal. $\uparrow\leftarrow n$ W types job *n*'s WHO line, and $\uparrow\leftarrow - n$ W types the WHO line for the job using TTY*n*.

$\uparrow\leftarrow$ N (stands for No-op) does nothing, except that if bucky bits have been set by $\uparrow\leftarrow$ C, etc., it will echo as [N] and forget the bucky bits. It is also good for cancelling a $\uparrow\leftarrow$ command you've started typing.

$\uparrow\leftarrow n$ V does a FILES command for job *n* (your own job if no argument) without having to stop your program. See page 46. $\uparrow\leftarrow - n$ V does a FILES command for the job using TTY*n*.

$\uparrow\leftarrow$ L followed by a character deletes back to that character, like CONTROL-L on a display. $\uparrow\leftarrow n$ L char will do it *n* times, like CONTROL-*n* CONTROL-L char.

$\uparrow\leftarrow$ X and $\uparrow\leftarrow$ Z are like ESC X and ESC Z on displays (control XTIME and enable automatic beeping, respectively.)

3.2 The TTY Command

The TTY command provides some of the ESC facilities for non-displays along with some other control features. It takes as argument a keyword specifying the thing to change. As in typing monitor commands, only enough letters of the keyword as necessary to make it unique need be typed. The keyword may be preceded by NO or - to reset the corresponding switch, e.g., TTY NO TABS or TTY -TABS. The TTY command is legal from displays as well as non-displays, but the keywords ARROW, CONVERT, ECHO, FILL, GLASS, and WIDTH have no effect on

display terminals. The TABS keyword has no effect on displays other than Datamedias. The keyword WIDTH also has no effect on PTYs which are not ARPA PTYs. You need not be logged in to give the TTY command, except for the keywords UPDATE and TIME. (Additional TTY command keywords for Datamedia display terminals are described in Section 2.10.)

A command of the form TTY TTYn keyword will cause the effect of a TTY keyword command for TTYn instead of your own TTY. This command is allowed only if the target TTY is not in use, or is owned by your job, or if you have the DEV privilege (see Section 6).

Here are the non-display keywords and their meanings:

ARROW	Causes character codes less than 40 octal to be echoed as uparrow-letter (e.g., ↑A for octal 001) rather than as the code itself, except for BEL (7), TAB (11), LF (12), VT (13), FF (14), and CR (15).
BEEP	Enables automatic beeping after long computations, like ESC Z on a display.
BREAK n x	Same as typing BREAK n x (where x is a command letter) on a display. Works for these command letters: F, H, I, V, X, and Z.
CONVERT	Enables the SAIL-ASCII character conversions mentioned above. This is the default state except for non-ARPA PTYs, but can be turned off with TTY NO CONVERT.
ECHO	Tells the monitor to send back to the terminal the characters you type in.
ESCAPE n x	Same as typing ESC n x (where x is a command letter) on a display. Works for these command letters: F, H, I, V, X, and Z.
FILL	Tells the monitor to insert extra carriage returns when a return is typed out at the terminal, to give the carriage time to return to the left margin. This is used principally for 30 character per second terminals.
FULL	Declares that the terminal has lower case letters, and lower case should not be converted to upper case by the monitor. Equivalent to ESC F on a display; NO FULL is like BREAK F.
GAG	Don't allow messages from other users to be typed out. This is the default for non-ARPA PTYs.
GLASS	Echo rubout as backspace-space-backspace instead of using the backslash notation for deleted characters.
HANG	Hang up the phone line associated with the TTY. This is meant to be used for some TTY other than your own.
SPEED n	Set the speed of the terminal to n, which must be one of 110, 150, or 300. Works only for low-speed dialup lines.
TABS	Declares that the terminal has hardware tabs, so the monitor will not convert tabs to spaces on output.
TGAG	Prevent messages from other users, like GAG, but only until the next program reset.
TIME	Resets XTIME and inhibits automatic resetting, like ESC 1 X. NO TIME, like BREAK 1 X, sets XTIME to the total run time.
UPDATE	Enables automatic resetting of XTIME, like ESC X from a display. NO UPDATE is like BREAK X. See XTIME on page 22.
WHO	Types out your job's WHO line. "TTY WHO n" will type the WHO line for job n, "TTY WHO -n" the WHO line for the job using TTYn. "TTY WHO 0" or "TTY -WHO" will type the system WHO line.
WIDTH n	Sets the maximum number of columns for TTY output to the decimal value n, which should be between 16 and 256.

SECTION 4

BASIC MONITOR COMMANDS

Commands like LOGIN tell the monitor to run particular system programs in your core image. In this section, monitor commands which do not refer to a specific program are described. Some of them do not refer to programs in your core image at all, but are handled entirely within the monitor itself; others run programs, but allow you to specify the program as a command argument rather than implying a particular one like LOGIN.

4.1 Running Programs

The RUN command takes as arguments an optional device name, a file name, and an optional core size argument. If the first argument is not a device name, DSK is used. The only allowable devices are DSK, SYS, UDPn, MTAn, and DTAn. Running programs from magnetic tape or DECtape is not recommended. If the file name does not include an extension, DMP is assumed.

The core size argument, if any, should be a decimal number indicating the number of 512-word pages desired for your core image. This must be at least as much as the saved core image, but you can ask for more. Certain programs with variable buffer space will use as much as you initially allocate. (Programs can increase their core size dynamically by UUO.)

If you are running from magnetic tape, the core size must be specified because the monitor cannot determine the size of a file on magnetic tape before reading it.

The program is loaded into your core image, your job name is set to the name of the dump file, and the program is started at the starting address specified in the dump file.

The R command is used for running system programs. Note that R is a separate command from, and *not* an abbreviation for, RUN. The command

```
R FOO 10
```

is identical to

```
RUN SYS:FOO 10
```

The SYS device is actually the disk, but with the special system directory [1,3] implied. This directory is used to store system dump files. Any PPN given explicitly with the R command is ignored; the PPN used is [1,3] no matter what.

By using the GET command (abbreviated G), it is possible to load a dump file into your core image without starting it. This command takes the same arguments as RUN, but instead of starting the program, it types out a message indicating the size of your core image and leaves your terminal in monitor mode.

Each of the RUN, R, and GET commands causes a RESET of your job.

4.2 Manipulating Core Images

The following commands refer to words in the Job Data Area (see page 12) of your core image. These words are generally set up by the monitor or the LOADER. The *UVO Manual* has a complete explanation of the Job Data Area.

The CORE command is used to find out or change the size of your core image in 512-word pages. CORE may be abbreviated C. If you include an argument, the size of your core image will be set to that many pages. If you do not give an argument, three lines of information are typed out in one of these two forms:

NNN pages	JJJ+KKK pages
MMM pages maximum	MMM pages maximum
UUU pages user core	UUU pages user core

The first form is used if you have no upper segment, the second if you do have an upper segment. NNN is the current size of your job in pages (no upper); JJJ is the size of your lower segment and KKK is the size of your upper segment; MMM is the maximum size allowed for your job (lower segment plus upper segment, if any); and UUU is the amount of core currently available for use by user jobs (i.e., core that is not being used by the system). If you give the CORE command when you are not logged in, you do not get the first line.

The command CORE 0 destroys your core image. It also causes a system RESET of your job, releasing any I/O devices your program was using, etc.

The way to create core image dump files which can be loaded later is with the SAVE command. This takes device, file, and size arguments just like GET. This command also does a RESET, and it does not save your accumulators. Therefore, the program cannot be continued after a SAVE command, although it can be restarted.

Dump files on the disk can be copied (in binary mode), renamed, etc., like any other files and will still be runnable as programs. However, dump files on DECtape are written differently from other files, and cannot just be copied by normal file operations. Instead, to store a dump file on DECtape, you must load it into your core image and give a SAVE command to the monitor.

Unless your core image includes DDT or RAID (JOBDDT nonzero), only the locations up to the address in JOBFF are saved. (This does not apply if you use a core size argument.) JOBFF is set by the LOADER to one more than the highest address it loads into, so any core which was allocated dynamically by your program will not be saved unless the program updates JOBFF. I/O buffers allocated for you by the system are allocated above JOBFF, which the monitor then sets to the new first free address.

If your program includes an upper segment which you wish to save, you should use the command SSAVE. This is just like SAVE in other respects. SAVE never saves upper segments.

It is possible to examine and alter words in your core image individually by monitor commands.

The command E (examine) with an octal argument will type the contents of the specified address in octal. The DE (deposit) command takes three arguments in octal: the left and right halves of the word you want to deposit, and the address. DE without the address argument uses the address of the last E or DE command. E with no argument examines the location following the last one used by E or DE. (Note that the DDT and RAID debugging programs provide a much more versatile way to examine and modify your core image.)

If your core image includes an upper segment (see page 12), you can set or clear the write protection bit with the SETUWP or CLRUWP commands. You cannot clear write protection for a sharable upper segment if it is protected against you. (See the *UUO Manual* for a complete discussion of upper segment protection.) These commands also take an optional octal argument, which is used to set the protection code for the segment if you are permitted to change the code.

4.3 Starting Programs

The following commands refer to words in the Job Data Area (see page 12) of your core image. These words are generally set up by the monitor or the LOADER (except for JOBREN). The *UUO Manual* has a complete explanation of the Job Data Area.

The START command (abbreviated S) is generally used to start a program at its normal starting address in JOBSA. This command can also be used with an octal argument specifying some other address. If the octal argument is preceded by + or -, the number is added to or subtracted from the program's normal starting address to determine the desired starting address.

The REENTER command starts the program at the reenter address specified in JOBREN, if there is one. JOBREN must be set by the program if it wants to use this capability. Program documentation will explain the use of the REENTER command for a particular program. An octal argument preceded by + (optional in this case) or - will be added to or subtracted from the reenter address.

The DDT command starts the program at the address in JOBDDT, if any. This address is set by the LOADER if either DDT or RAID (the debugging programs for non-displays or displays) is loaded with your program.

When a program is stopped by typing CALL, by a UUO in the program, or by various other conditions, the address of the next instruction to be executed is stored in JOBPC. If the job is continuable (not stopped because of an irrecoverable error), the CONTINUE command will start it at that address. (The START, REENTER, and DDT commands will work even if the job is not continuable, except for certain errors which destroy the core image irretrievably.)

All of the above commands place your terminal in user mode, that is, anything you type will be processed under control of your program. It is also possible to start a program, but leave the terminal in monitor mode. In this case you can still enter monitor commands which do not affect your core image. Monitor commands which are illegal while your program is running will give the message PLEASE TYPE ^C FIRST.

The CSTART command is like START but leaves your terminal in monitor mode. Like START, it will take the desired start address as an argument or use the contents of JOBSA. The

CCONTINUE command is the monitor mode equivalent of CONTINUE. There is no such equivalent for REENTER or DDT. With the terminal in monitor mode, the program can still type output on it, but an attempt at reading from the terminal will make the job wait (in IOWQ) until the terminal is placed in user mode.

The HALT command stops your program. The monitor converts the CALL key into this command, so you should never need to type it explicitly. (The only time you can type it explicitly, of course, is if you started the program with CSTART or CCONTINUE.)

4.4 Detached Jobs

Normally, any user job is associated with a particular terminal. However, it is possible to *detach* the job so it can continue running while you do something else (another job) at your terminal. Also, the monitor sometimes starts up detached jobs itself to perform various system functions; such jobs are called *phantoms*.

Detached jobs can run, and do all the things attached jobs can do, except that if they try to output to device TTY, the output is lost; attempted input from TTY makes the job wait indefinitely (until it is attached). If a detached program executes an EXIT UUO, the job is killed.

The DETACH command detaches your job, leaving your terminal in monitor mode and not logged in. You can then log in again without affecting the old job. DETACH takes no arguments. Several commands described below combine the DETACH function with other useful things. Please remember not to leave jobs detached forever, but to log them out eventually!

The ATTACH command is used to attach your terminal to a detached job. If you are already logged in, your old job is detached. The ATTACH command takes two arguments, a job number and the PPN under which that job is logged in. (If you are already logged in with the same PPN, you need not give the second argument.) Note that if you were displaying a WHO line for your old job, it will still be displayed, indicating the job as detached. This is handy for re-attaching it when you want to, in that it shows the job number. (Typing ESC W again will, of course, display the WHO line for your new job.)

You can attach a phantom (not logged in) job only if you have the ATT privilege, in which case this command will also log in the phantom by turning on the JLOG bit in its job status word. You can, however, attach a phantom job if you are already logged in with the same PPN as the job you are trying to attach, and the programmer name is not SYS.

If you are at a non-local terminal, and are not logged in or are logged in with a PPN different from the one of the job you are trying to attach, the ATTACH command will run the LOGIN program, which will require you to type the password for the PPN to which you are attaching. A password is also required to attach a privileged job, even from a local terminal, unless you already have all its privileges.

You can attach a job which was already attached, thereby stealing it away from its original terminal, as long as the above PPN conditions are met. A message will be typed on the job's original terminal indicating that the job has been attached away, and by whom. If your PPN before typing the command was not the same as the attached job's PPN, the message will also be typed on the console terminal in the machine room.

If you give the ATTACH command with no argument, it will look for a detached job with your PPN to attach. Repeating the command will cycle through all such jobs. The command will tell you the job name and number of the job it finds.

The FORK command will detach your old job and log in a new one. It is faster than DETACH followed by LOGIN, because it does not run the LOGIN program. Instead, it merely copies the system information from your old job into the new job. In particular, if you have an alias in your old job, it is copied to the new job as well. If the system's job capacity is used up, so that it is impossible to start a new job, your terminal remains attached to the existing job.

Often you want your program to continue running in the old job while it is detached. You could say CCONTINUE and then DETACH, but the command CDETACH combines these functions. There is also a CFORK command which combines CCONTINUE and FORK.

4.5 Device Control

Certain I/O devices can only be used by one user at a time, such as a magnetic tape unit. In some cases, like the line printer, the system provides a facility for queueing print requests so that individual users need not actually control the device themselves. However, when a user does need to use such a device, the ASSIGN command is used to ensure that only one user at a time tries to use the device. The ASSIGN command (abbreviated A) can also be used to cause a *logical device name* to be associated with a particular device. Thereafter, any reference by your program to the logical name will be translated into the corresponding physical device.

The command takes two arguments, a physical device name, and an optional logical name. The physical name can be *generic*: the command

```
ASSIGN MTA
```

will select an available mag tape unit (MTA0 or MTA1) if there is one available. You can also select a particular unit explicitly.

The logical name feature is sometimes used by programs to allow you to control their I/O without having to type instructions to the program itself. For example, a program which produces a listing output might direct it to a particular filename on device name LST. Before running the program, you could type

```
ASSIGN DSK LST
```

to write the listing file on the disk, or

```
ASSIGN TTY LST
```

to type it out at your terminal. You can also change the meaning of what is normally a physical device name this way—the command

```
ASSIGN DSK LPT
```

will make a program which was written to write its output on the line printer use a disk file instead. You might do this if the line printer is not available when you want to run the program.

Non-local guest users (those who are not logged in) cannot use most non-sharable devices. Since there is no operator on this system, remote users have no way to mount tapes, collect listings, etc.; hence, the monitor does not allow such users to attempt I/O on those devices.

The ASSIGN command is used in a special way for User Disk Packs. A UDP may be used as a *public* or a *private* device. In the former case, other jobs may use the same UDP at the same time, as with the system file disks. A privately assigned UDP, however, is restricted to the one job which assigned it. (Note: a privately assigned UDP is still a *sharable* device, in the sense that the job which owns it can operate on several files simultaneously, unlike non-sharable devices like DECtape drives, on which only one operation can be in progress at any time.) A UDP is normally a file-structured device, with a directory structure just like the one described for the main file disk in Section 1.5. However, it is also possible to use a UDP as an unstructured device, in which allocation of storage on the disk pack is done explicitly by the user program. This mode of using the UDP is called *old* mode (because it used to be the only mode available). The format of the ASSIGN command for a UDP is

```
ASSIGN mode UDPn logical-name
```

The optional mode argument can be

PUBLIC	public access, file structured UDP
PRIVATE	private access, file structured UDP
OLD	private access, non-structured UDP

The default mode is PUBLIC. A later ASSIGN command for the same device can be used to change the mode from PRIVATE to PUBLIC, or from PUBLIC to PRIVATE if no other jobs are using the device. However, to change into or out of OLD mode, the device must first be deassigned by all jobs using it. This is because a particular pack either is or isn't formatted for file structured use, and the mode given in the ASSIGN command must agree with that of the pack itself. (A mismatch is not detected when you give the ASSIGN command, but will be caught when your program actually tries to use the device.) The only exception is that a file-structured pack may be read, but not written, in OLD mode.

Because of the need to distinguish between the two different UDP formats, your program will not be allowed to open the UDP unless it is already open or assigned. Thus, when you first mount a UDP you must ASSIGN it to specify the mode. When a UDP has been assigned in PUBLIC mode by one job, any job may use it without assigning it.

We currently have two drives for user disk packs: UDP1 and UDP2. The monitor treats UDP as a generic device name; the command ASSIGN UDP will select a free drive if there is one. An ASSIGN command which specifies a particular drive in PUBLIC mode, e.g., ASSIGN UDP2, will succeed if the particular drive is not in use or if it is in PUBLIC use. For more details on the use of user packs, see the *UVO Manual*.

Once you have assigned a non-sharable device with the ASSIGN command, no other job can use it until you release it with the DEASSIGN command. This command, abbreviated D, takes either a logical or a physical device name as argument. (Note—Assigning DSK or SYS does not prevent other users from using the disk.) It is possible for a program to use a device which is not assigned,

but the device is released in that case as soon as the program stops using it. The DEASSIGN command with no argument will deassign all devices assigned by your job.

Because UDPs can be assigned in a way which permits public access, the DEASSIGN command for a UDP types a message at your terminal indicating whether or not the pack should be dismounted. If you interrupt a program which is using a UDP and deassign the UDP, it will still be considered in use by your program until you reset it (for example, by the RESET command). You will then be told that you may dismount the pack. Also, there is a light mounted near the drives labelled "don't unload UDP1" which is on whenever UDP1 is assigned or initied by a program.

The job which has a device assigned may give it to another job with the REASSIGN command. This command takes two arguments, the device name and a job number. This can occasionally be useful if, for example, one job writes a tape which another should then read. The REASSIGN command prevents the possibility of another job grabbing the tape unit before the intended recipient does.

The FINISH command (abbreviated F) with a device name argument releases the device from your program. If the device is a directory device which your program had open for output, the file is closed so that as much data as the program wrote will be saved. (This applies in the case in which the program was stopped before its normal exit, either by CALL or by an error.) FINISH with no argument releases all devices.

The FINISH command may also be given with a job number and possible PPN argument, like the ATTACH command. In that case, the specified job will have all its devices released, as if a FINISH with no argument had been typed at its terminal. The target job is notified, as for the ATTACH command. Also, the same restrictions about FINISHing phantom jobs apply as for ATTACHing phantoms.

The FLUSH command is used to empty the input and output buffers of a terminal which is not in use. It takes a device name as argument; the device must be a terminal which is not in use, or your own terminal. This command may be used without logging in. A user with the UPG privilege can use this command on any TTY whether it is in use or not. (Any user may have the UPG privilege, but it must be enabled specifically before giving the FLUSH command. See the ENABLE command, page 69.)

4.6 System Information

Several commands exist to find out various things about your job or others. Some of these duplicate information available on the WHO line at display terminals. None of the commands in this section affect your core image.

The PJOB command types the job number of your job, if you give the command with no argument. If you use a device name as the argument, it types the WHO line of the job using the device. (If the device is not in use, it says so.) The argument can be a name of the form DDn, to find out who is using Data Disc channel n. You will be told the TTY line number, if the channel is in use as a terminal's main channel, as well as the owning job's WHO line. Also, if the argument is a UDP, the command may type out UDP1 PUBLIC if applicable, and will type a job

the number of the record currently being read or written; and the way the file is being used: R for read, W for write, or RA for read-alter, possibly followed by E for end-of-file seen. The letter U indicates that the file is on a UDP.

The HELLO command types the name of the current version of the monitor.

All of the above commands are legal when you are not logged in. The following one is not.

The SLEVEL command types out your current service level reservation. The service level system attempts to guarantee the reserved percentage of the computer's time to programmers who make such reservations. See the RSL command, in Appendix 6.

4.7 Miscellaneous Commands

The ALIAS command is used to set your job's *disk PPN* or alias. If you have an alias, all disk references by your job which do not include an explicit PPN will use the alias directory rather than your logged-in PPN directory. The argument can be PRJ, PRG to set the alias to that PPN, or just PRJ to use your login programmer name in the alias. An argument of PRJ, (i.e., with a comma but no prg) uses your old alias programmer name. No argument resets your alias to zero, so your logged-in PPN will be used for disk files. If you have an alias, your logged-in PPN is still checked for file protection purposes.

The TALK command can be used to talk to a user at another terminal. It takes a programmer name or device name as argument. The device must be a terminal. If a programmer name is used, the system will look for a terminal in use by that programmer; there must be exactly one such terminal which is available for talking. If there are several such terminals, they will be listed and you must select one by device name in another TALK command. A terminal is not available for talking if it is gagged (i.e., the user has used the TTY GAG command, or the program running at the terminal has gagged it by UUO), if it is open in image mode, if it is holding output, or if it is a display whose currently selected piece of paper is smaller than 15 lines high or is not activated. (Note to remote users: There is no operator on this system; TALK CTY is not a good way to get assistance!) If the command succeeds, all characters you type are typed out on the other terminal, and vice versa. If the terminal you specify was already in a talk ring, you are added to the ring; all characters typed at any terminal in the ring appear on all the others. You leave the talk ring by typing CALL. You need not be logged in to enter a talk ring.

If the argument to the TALK command contains the character @ or %, then it is taken as an ARPA network user at a remote host. In this case, a program is run to establish the talk connection, destroying your former core image. Local talking does not affect your core image.

The KILL command can be used to kill another job. It takes the job number and PPN (if different from your own) as arguments. The RESET command resets the specified job, or your own job if no argument is given. (The KILL command cannot be used to kill your own job; use the KJOB command for that.) Needless to say, both of these commands are dangerous and should not be used unless you are sure you want to affect another job's operation. They are included to control runaway jobs which cannot be stopped normally, e.g., because they are detached. These commands can be given for any job by any logged-in user. If the job's PPN does not match your own PPN, the command is logged on the console terminal. Note: unlike the FINISH command, these commands do not close (and preserve) files being written by the affected job.

SECTION 5

SYSTEM PROGRAM COMMANDS

The remaining monitor commands all run particular system programs in your core image. There are a few main groups of these commands and a few miscellaneous ones. These commands are documented along with the programs they run. The next two sections are about the LOGIN and LOGOUT programs and the commands which run them; these were briefly introduced earlier. The major command groups are then presented, and finally the miscellaneous system programs. Many system programs are not run by special monitor commands, but by the R command. Those programs are not documented here, but in separate manuals.

5.1 LOGIN

The LOGIN command is used to begin a session of using the computer. It runs the LOGIN program, which provides several optional services as well as setting up the necessary system tables for your job.

The LOGIN command may be abbreviated L. It takes one argument, a project-programmer name. Different characters used between the two parts of the name are used for different options:

PRJ,PRG	types all system messages and processes OPTION.TXT (see below)
PRJ/PRG	types system messages new since last login, processes OPTION.TXT
PRJ.PR	types no messages, ignores OPTION.TXT; for fast login
PRJ%PRG	sets new password, as explained below, then acts like PRJ/PRG
PRJ!PRG	requires password even if at local terminal (see below); like PRJ/PRG

The PRJ can be omitted, in which case project 1 is implied. If the delimiter before the PRG is also omitted, 1/PRG is implied.

System messages are notices for all users, sent by the MAIL * command and stored in the file NOTICE.TXT[2,2]. There may also be messages addressed specifically to you. The handling of your personal mail is explained below.

Before typing any other messages, and regardless of the option used, LOGIN will notify you if you have any other jobs logged in. If you have a detached job logged in, you will be beeped. LOGIN will list the job number, PPN, TTY, and job name of any job with your login programmer name.

It is possible to set a password and file protection codes to restrict access to a particular PPN. If the PPN you specify to LOGIN has a password, LOGIN types Password= and you must type the correct password to log in. Echoing of input characters is turned off so the password does not print. Passwords may be one to six letters or digits followed by RETURN. To protect files from unauthorized access, there is a protection code associated with each file. The code specifies the access allowed to the file for its owner (i.e., a user logged in under the file's PPN), for other authorized AI Lab users, and for guest users (those who are connected remotely, from the ARPA network, dial-up telephone lines, or campus terminals, and who are not logged in but are using the

monitor commands allowed without login). You can set the protection code for a file using the COPY program (see Appendix 2). However, you must use the LOGIN program to set the protection code of your file directory itself, and the default protection code which is assigned to new files created in your directory if the program writing the file does not specify a value explicitly. If you use % as the delimiter between the project and programmer names when you log in, LOGIN will give you the opportunity to change your password, the file directory protection, and the default protection for new files. If you ask to change your password, you will be asked to type the new password twice to guard against typing errors, since the password will not be echoed. The protection codes are entered as three-digit octal numbers, with the following meanings:

Bit	Meaning in directory protection code	Meaning in file protection code
400	password is for remote logins only (see below)	don't dump this file (see DART, Appendix 5)
200	unused	delete protect (see COPY, Appendix 2)
100	owner may not write files	owner may not overwrite this file
040	local users may not change protection of files	l.u. may not change protection of this file
020	local users may not read directory or any files	l.u. may not read this file
010	local users may not write any files	l.u. may not overwrite this file
004	guest users may not change protection of files	g.u. may not change protection of this file
002	guest users may not read directory or any files	g.u. may not read this file
001	guest users may not write any files	g.u. may not overwrite this file

Although there is a lot of complicated detail here about file protection, please bear in mind that the system is not meant to be totally secure, and that most users do not protect their files at all. The default directory protection is 005, meaning that all logged-in users have read and write access, and remote guest users have read access. Also, there are both official (see the UNPROTECT command, page 68) and unofficial ways to get around file protection. Protecting a file simply guards against unintentional access and notifies polite users that you wish to keep the file private.

When you log in with %, you can also add your directory to one of several *file access groups*. Membership in such a group means that you want to allow anyone with the corresponding *file access privilege* to have owner access to your files, as if he or she were logged in under your PPN. For example, there is a SYS group for system files; such directories as [SPL,SYS] (LPT and XGP spooler files) are in the SYS access group, and the people in charge of system programming have the SYS access privilege. This mechanism allows users who work together on some project to share one another's files without allowing other users access to them. File access privileges, like all privileges, are assigned by system programmers. A special case is the MAS (master) file access group. The MAS privilege is automatically assigned to all PPNs with project name 1, but is effective only for access to directories with the same programmer name. That is, if you have a directory named [XYZ,PRG] and you give it the MAS access group attribute, you will have owner access to it when logged in as [1,PRG] but user [1,ABC] will not have owner access to it. This facility allows you to organize your files into several directories, but still have owner access to all of them while logged in under project 1. Remember that, although your project 1 PPN automatically has the MAS privilege, you must explicitly enter your other directories in the MAS access group for this facility to operate.

If the 400 bit is on in the protection code for a file directory, the password associated with that area is used only for logins from a remote site; no password will be required for local login on that PPN. (Note: if you log in locally without a password, you will not be given any privileges to which you would otherwise be entitled, including file group access privileges. You can use the delimiter ! between your PRJ and PRG to force LOGIN to ask for a password, to get your privileges.) Remote login is not allowed on a PPN which does not have a password; thus, if you want to be able to log in remotely but do not want to have to type a password for local login, you can set the 400 bit in your directory's protection code.

Users with access to local terminals who sometimes need to log in remotely can set a password (with %) for their own PPNs; remote users who want to establish accounts here should communicate with Lester Earnest. (Type the command MAIL LES.)

Sometimes you may find the system is in *maintenance mode*, which means that the monitor is being debugged and the system is not available for normal use. If you try to log in at such a time you will be asked for the maintenance password.

The further details of LOGIN's operation are controlled by a file named OPTION.TXT which you may have in your directory. If there is one, it is searched for a line of the form

```
LOGIN:opt1,opt2,opt3;comments
```

where opt1...optn are the desired options. Spaces may be used in the obvious places. Only the first six letters of an option name are read. Upper or lower case is ok. Options may actually be divided over several lines; it is the (required) semicolon which ends the option list. Several other programs use this OPTION.TXT file, looking for their own option lines.

The LOGIN options are as follows:

NOMAIL	Does not ask any questions about message files. This is for people who like to type ahead while LOGIN is running. The exact effect depends on what other options are used; see below.
XMAIL	Prevents notification about a message file for you if the file was last written by someone with your programmer name, unless it was written by a program named MAIL. In other words, it does not tell you about a message file which you have already edited since the last new message for you.
MESSAG	Types your mail without asking; see below.
NONOTE	Prevents typing out system messages from NOTICE.TXT[2,2].
GRIPE	Types out new gripes just like new system messages.
LOGRUN	Runs the LOGRUN program, which executes monitor commands from a LOGRUN entry in OPTION.TXT; see below for details.
INIT	Runs the file INIT.DMP in your directory after login.
RUN=file	Like INIT, but runs a program named explicitly in the option, from default device SYS, e.g., RUN=LOGRUN is the same as LOGRUN, and RUN=DSK:INIT is exactly like INIT (since DSK is specified as the device, your own PPN is the default in this case).
BEEP	Enables the system feature of automatic beeping whenever your job finishes doing something that took more than 15 seconds of real time. Like ESC Z.
WHO	If you are at a display, starts a WHO line. Like ESC W.
AUDIO=n	Selects audio switch input channel n to your speaker if you are at a display terminal. "n" is an octal number. "AUDIO=-n" inhibits telephone paging interrupts, like BREAK n U. If n is greater than 1000 octal, channel n-1000 is selected with beep interruptions disabled.
GAG	Tells the system not to allow other users to send messages to your terminal or to link to you with the TALK command. Like TTY GAG.
NOGAG	Tells the system to allow other users to send messages to your terminal or to link to you with the TALK command. Like TTY GAG. This is the default.
RPGSAV	Restores your TMPCOR files for remembered SNAIL commands from your previous session using the computer. You must also have the RPGSAV option specified for LOGOUT. This option will be explained more fully in the LOGOUT section.
DM	If you are not on a III or DD display, tells the system that you are on a Datamedia (DM) display.
DM=n	Just like DM option except that this specifies that your DM has n lines (useful mostly on DM-simulating displays that provide some number of display lines other than 24).
FDM	Just like DM option if you are on a high-speed dialup line. Ignored otherwise.
FDM=n	Just like DM=n option if you are on a high-speed dialup line. Ignored otherwise.
DM128	Like DM option but also tells the system that your DM can display the control characters (octal 1 to 37) when they are output preceded by an ascii escape character (octal 33).
DM128=n	Like DM=n and DM128 options combined.
FDM128	Just like DM128 if you are on a high-speed dialup line. Ignored otherwise.
FDM128=n	Just like DM128=n if you are on a high-speed dialup line. Ignored otherwise.

BOLD	If you are on a DM display, tells the system to display your line editor in boldface, assuming your DM has bold and blinking interchanged like all SU-AI DMs. This is the default for DMs.
NOBOLD	If you are on a DM display, tells the system not to display your line editor in boldface/blinking (especially useful if your DM does not have bold and blinking interchanged). This option should occur after any DM, FDM, DM128, or FDM128 option, if present.
BS	If you are on a DM display, tells the system that your DM displays the 177 character.
NOBS	If you are on a DM display, tells the system that your DM does not display the 177 character. If you are in DM128 mode, the system will precede output of each 177 with an 033 to force the 177 to print. Otherwise, the system will convert echo or output of backspace (177) to appear as a bold left-arrow.
BOW	If you are at a Data Disc display, tells the system to display piece of paper output in black on white (actually green on white) instead of the reverse. Not very useful, since any special display output will still be green on black.
NOBOW	Display green on black. This is the default.
PORNO	Try it and see.
DIGEST	Tells you if there is a new Associated Press news digest that came in after the last time you logged in and asks you if it should type it out.
XDIGEST	Always types the A.P. news digest if there is a new one since you last logged in, without asking and independent of any other option setting.
QMAINT	Only type maintenance forecast if it has changed since last login. Otherwise the maintenance forecast is always typed unless . is used as the PPN delimiter.
ME	Tells you your fortune.
COOKIE	Alternate fortune cookie selection. See which you prefer.
HIDE	Makes your Data Disc channel private, like ESC H.
UNHIDE	Makes your Data Disc channel public, like BREAK H. This is the default.
WIDTH=n	If you are at a non-display, sets the maximum number of columns for TTY output to the decimal value n, which should be between 16 and 256. Like TTY WIDTH n.
FULL	If you are at a non-display, sets the full character set mode switch. Like TTY FULL.
NOFULL	If you are at a non-display, clears the full character set mode switch. Like TTY NO FULL.
TABS	If you are at a Datamedia or a non-display, clears the tab expand bit; tabs will not be converted to spaces on output. Like TTY TABS.
NOTABS	If you are at a Datamedia or a non-display, sets the tab expand bit; tabs will be converted to spaces on output. Like TTY NO TABS.
FTABS	If you are on a high speed dialup line, clears the tab expand bit; tabs will not be converted to spaces on output. Like TTY TABS.
FNOTABS	If you are on a high speed dialup line, sets the tab expand bit; tabs will be converted to spaces on output. Like TTY NO TABS.
CONVERT	If you are at a non-display, tells the system to convert between SAIL and ASCII character codes, like TTY CONVERT. This is the default.
NOCONVERT	If you are at a non-display, tells the system not to convert between SAIL and ASCII character codes, like TTY NO CONVERT.
GLASS	If you are at a non-display, tells the system to echo rubout as backspace-space-backspace, like TTY GLASS.
NOGLASS	If you are at a non-display, tells the system not to echo rubout as backspace-space-backspace, like TTY NO GLASS. This is the default.
FILL	If you are at a non-display, sets the fill switch to insert extra carriage returns on output to give the carriage time to get all the way back. Like TTY FILL.
NOFILL	If you are at a non-display, clears the fill switch thus suppressing insertion of extra carriage returns on output. Like TTY NO FILL.
ECHO	If you are at a non-display, tells the monitor to send back to your terminal the characters you type. Like TTY ECHO.
NOECHO	If you are at a non-display, tells the monitor not to send back to your terminal the characters you type. Like TTY NO ECHO.
ARROW	If you are at a non-display, tells the monitor to type out characters below octal 40 as uparrow-letter (e.g., 1A) rather than the character itself except for BEL (7), TAB (11), LF (12), VT (13), FF (14), and CR (15). Like TTY ARROW.
NOARROW	If you are at a non-display, tells the monitor to type out characters below octal 40 as themselves instead of as uparrow characters. Like TTY NO ARROW.
SYSTAT	Types out a line indicating how many jobs are logged in and how many are running, to give an idea of system load.

The options RUN, LOGRUN, INIT, PORNO, and COOKIE are mutually exclusive, since they each tell LOGIN to transfer control to another program after the login. If more than one appears in your OPTION.TXT, the one which is earliest in the above sentence wins, e.g., RUN has highest precedence.

If there is a message addressed to you, LOGIN normally types

```
Message for YOU last written by PRG using PROGRM on dd-mon-yy at hhmm
Read it now?
```

where YOU is your programmer name, PRG is the programmer name of the last user to write anything in your message file, PROGRM is the name of the program used to do it, and dd-mon-yy and hhmm are the date and time it was done.

If you type Y, the message file is printed, and you are allowed to delete it (the whole file). If you type N, the message file is not printed. You can also type R, which will treat your mail file like a system message file; i.e., header lines will not be typed, and, if you used a slash in the command, the only messages typed will be those which came since you last logged in. However, R may not find the recently received messages correctly if your mail file is in E format. If you type R, LOGIN does not give you a chance to delete the file.

If you have the MESSAG option but not the NOMAIL option, then personal mail is always typed out without asking first. If you have both MESSAG and NOMAIL, the mail is typed but you are not given the chance to delete it. If you have NOMAIL alone, you get the Message for YOU... line but not the mail itself; many people use the NOMAIL option for LOGIN and run other programs to read their mail. See the file E.ALS[UP,DOC] on the use of E as a good way to process your mail; remote users may prefer to use RCV—see page 130.

If you have DIGEST and NOMAIL, you are notified of a new A.P. digest but it is not typed out. (RCV will also do this for you.) It is also possible to get automatic notification of incoming news stories on particular topics from the NS program; these messages are treated like personal mail with respect to MESSAG and NOMAIL.

The LOGRUN program looks for a line in your OPTION.TXT file starting with LOGRUN:. It then takes that line, and all following lines until a semicolon is seen, and makes them be executed as if you typed them in. (The semicolon is required.)

5.2 LOGOUT

The command for terminating a session on the computer is KJOB (kill job), which runs the LOGOUT program. The command may be abbreviated K. It takes an optional switch argument to select various optional features. The OPTION.TXT file is also used to control logout options.

The LOGOUT program normally types out several lines of accounting information, e.g., computer time used. If you are not logged in elsewhere, it deletes any files in your directory with extension RPG, as well as any files of length zero with extension TMP; and if there are no files in your directory, it deletes the directory itself.

If you have the RPGSAV option (see below), LOGOUT will look for TMPCOR files (simulated files in core storage) used by SNAIL to remember COMPILE- and EDIT-type commands, and save them on the disk. These TMPCOR files may exist for different aliases you have used while logged in; all TMPCOR files with names CM or ED, for any PPN, will be merged into files QSVCM.TMP and QSVED.TMP in your login directory. If you also have the LOGIN:RPGSAV

option, these disk files will be copied back into TMPCOR when you next log in, so SNAIL will remember your commands from one session to another. Note that the /F switch for fast logout will prevent saving the TMPCOR files, and that fast login (LOGIN PRJ.PRG) will not restore the disk files to TMPCOR. Please bear in mind that if you often alias to different directories, and you use the RPGSAV option, you can easily accumulate a lot of saved TMPCOR files that you may no longer need. Since any job has a fixed quota of TMPCOR files, you may run out. To avoid this, you should edit your QSVCM.TMP and QSVED.TMP files from time to time, to eliminate unnecessary entries. (Then log out /F so that your edits will not be overwritten by LOGOUT!)

Note that if you have the RPGSAV option for LOGIN but not for LOGOUT, you can create the QSVCM.TMP and QSVED.TMP files explicitly (create them /N—no directory—if you use E) to have a permanent set of remembered commands. Each file must contain one line per alias, containing the PPN (without brackets), a tab, and then the compile- or edit-type command.

This normal processing can be affected by switches in the command itself or by options in the OPTION.TXT file. LOGOUT looks for a line in OPTION.TXT of the form

```
LOGOUT:RPGSAV,ME,FAST;
```

(Of course, you need not use all the options.) These options, if found, have the following effect:

RPGSAV	Don't delete .RPG files, and save TMPCOR files on disk.
ME	Type a friendly message (try it).
FAST	Just log out, don't delete anything, don't save TMPCOR, don't print statistics.

Switches are single letters preceded by a slash (e.g., KJOB/F). The switches are:

/K	Kill .RPG files (only necessary if you have the RPGSAV option).
/S	Save .RPG files.
/M	Type a message, like the ME option.
/F	Fast logout.
/Z	Zero the disk. Deletes all your files, after asking if you really mean it.

Certain functions of LOGOUT are performed regardless of options or switches used. You are told if you have another job logged in with the same PPN, and if you have assigned any private devices (mag tape, etc.), you are reminded to unload them. The "date last written" entry for your file directory is updated to reflect your most recent logout (this information is used by FINGER). If you are not logged in elsewhere, and you have a file named LOGOUT.MSG, it is typed and deleted.

There are two other monitor commands which run the LOGOUT program, for logging in with another PPN, and for attaching to an existing job (see the ATTACH command, Section 4.4.) They are

```
KLOG logout-switches ppn
KATTACH jobnum ppn logout-switches
```

The logout switches are as described above. The other arguments are in the required form for the LOGIN and ATTACH commands.

Finally, there is a LOGOUT command, which is exactly the same as KJOB but no arguments are allowed.

Any of these commands, when typed at a PTY which is not owned by a TELNET server (that is, not the virtual terminal of an ARPA network user), simply kills the job without running the LOGOUT program. This has approximately the effect of KJOB/F.

5.3 SNAIL

The SNAIL commands run the SNAIL program, which then runs various other programs. The commands are divided into three basic groups. The first, for text file editing, take a file specifier as argument:

CREATE	create a file with SOS
EDIT	edit a file with SOS
MAKE	create a file with TECO
TECO	edit a file with TECO

The other major editor, E, includes the necessary code to carry out the SNAIL functions itself, e.g., default extensions and remembering the commands. The following commands are, therefore, not strictly SNAIL commands but function equivalently:

CETV	create a file with E
ETV	edit a file with E
ERead	edit a file with E in read-only mode
READ	edit a system documentation file with E
BOOK	edit with E in book mode

The second group, for program compilation, take one or more program names or a command string, along with various option switches:

COMPILE	compile programs
LOAD	compile and load programs
EXECUTE	compile, load, and run programs
PREPARE	compile and load with a debugger
DEBUG	compile, load with a debugger, and start the debugger
TRY	compile, load with a debugger, and start the program
PUB	produce a document with the PUB document compiler
POX	produce a document with the POX document compiler
RER	run a system program and give it a command string
RERUN	run a user program and give it a command string
PROCESS	pass a command string to the monitor

Finally, one more command, used in conjunction with the compile-type commands, which takes no arguments:

CREF	produce cross-reference listings from compilations
------	--

The complete description of these commands, with the command syntax and the processing options provided, is included as Appendix 1.

5.4 COPY and SPOOL

The next major group of commands runs the COPY program. Their arguments are generally in the form “new-file ← old-file”:

COPY	copy a file
TRANSFER	copy and delete the original
RENAME	rename a file or change protection

The following COPY commands imply a destination, and take just one or more input file specifications (separated by commas if more than one):

DELETE	delete a file
TYPE	copy to the terminal
LIST	copy to the line printer (SPOOL, below, is the preferred technique)
PRINT	copy to the line printer with different formatting (see Appendix 2)
XGPLIST	copy to the Xerox Graphics Printer (XSP00L is preferred)
DIRECTORY	type file directory

The COPY manual explains these commands fully, along with various options specified in the argument list; see Appendix 2.

Since only one job can use the line printer or XGP at a time, if several people want to make listings at once there is a scheduling problem. To solve this, the system allows listing requests to be stored on disk, and processed one at a time by *spoolers* which actually control the LPT and XGP. The system commands for this function are:

SPOOL	request line printer listing of specified file(s)
XSP00L	request XGP listing of specified file(s)
UNSP00L	delete a spooler request
QSP00L	display spooler status and queues

The SPOOL commands also allow editing options to be specified in the argument list; see Appendix 3 for details.

The commands in this section all require that you be logged in, except for TYPE, DIRECTORY, and QSP00L.

5.5 MAIL

Several commands are provided for sending messages to other users. Messages can be filed on disk, so that the addressee will be notified about them by LOGIN, or they can be sent directly to the terminal of a logged-in user. There is also an automatic reminder system which allows messages to be sent or programs to be run at a later time. The commands used for these purposes are:

MAIL	send a message to one or more message files
SEND	send a message to the terminals of one or more users
GRIPE	send a message complaining about a system problem
REMIN	schedule a message to be sent at some later time
PLAN	create a file describing how to find you when not logged in
EVENT	send a message to all users about an event on a given date
BATCH	schedule the execution of a command string at some later time
LATER	schedule the execution of a given program at some later time
RETRY	send any messages which were queued by earlier failing operations
ETV @	edit E-format message file
RCV	edit non-E-format message file
CANCEL	delete REMIND, BATCH, or LATER requests or queued mail

The usual format of the MAIL and SEND commands is

```
MAIL user message
SEND user message
```

where user is a programmer name and message is the one-line text you want sent. If you do not include a message in the command, the MAIL program will ask you to type a message, which may then have more than one line. The complete description of these commands is in Appendix 4.

The commands in this section require that you be logged in, except for MAIL, SEND, GRIPE, and RCV.

5.6 DART

DART is a program to save disk files on magnetic tape and restore them as needed. It also handles mag tape positioning commands. The file dumping commands take disk file names as arguments:

DUMP	dump disk files onto tape
RESTORE	restore files from tape to disk
PUMPKIN	Ask the Great Pumpkin to restore your files later
LOCATE	find which tape has dumped copy of files
TLIST	list all files on a tape
TURKEY	tell who last used a given DART tape

The tape positioning commands are:

```
REWIND
ADVANCE
BACKSPACE
EOT
```

The complete documentation of these commands is in Appendix 5 of this manual.

5.7 DO

The DO program allows automatic execution of an arbitrary sequence of commands. By writing DO command files, you can make your own sequences of program execution analogous to the SNAIL compiler-loader-execution sequence. The DO command takes a file specifier as argument, with the default extension being .DO. The text in the file is simply transferred into your terminal's input buffer, as if you had typed it. Then the DO program exits, and the monitor processes the commands that came from the file. The DO program cannot be run by detached jobs. The amount of text allowed in the DO file is limited by the capacity of your input buffer (currently 95 characters, plus one extra line of up to 160 characters on display terminals). You can sometimes circumvent this limitation by the following trick: if the command string to be processed by DO ever leaves you at monitor command level, you can insert at that point in the string a DO command to process another file (or another *function* in the same file—see “|” character below); the DO command must appear as the last thing in the string—subsequent commands should then appear in the next DO file. This trick can be repeated any number of times—simply make the last command in each DO file be a DO command to reference the next file.

DO does some conversion of certain characters in the file, to allow things like CONTROL and META characters. The characters which are processed specially are:

RETURN	ignored
LINE	ignored
↕	translated to RETURN followed by LINE
↓	translated to LINE
≠	translated to ALT
λ	translated to deferred CALL (one ↑C)
VT	adds CONTROL bit to the following character
α	adds CONTROL bit to the following character
FORM	adds META bit to the following character
β	adds META bit to the following character
⊗	translated to ESC
⊗-	translated to BREAK
	separates different DO functions (see below)
≡	quote the next character (do not process it specially)
?	takes the next character (other than RETURN or LINE) as a variable name. Suppose the character is A (i.e., ?A). If this is the first occurrence of ?A in the file, DO types out "A= " and waits for you to type in a text string ending with RETURN. This string is substituted for every occurrence of ?A in the file.

To allow more than one DO function to be stored in the same file (because small files are inefficient in using disk space), the vertical bar (|) character can be used as a separator. The command

```
DO DOFILE(3)
```

will DO the commands between the second and third | characters in the file.

The DO command remembers its argument in a TMPCOR file, so you can type DO without an argument to repeat the same command file. Also, the DO command is remembered as if it were a COMPILE-type SNAIL command, so the SNAIL exit mode commands in the text editors will also repeat the DO command. Thus, users of languages like LISP which are not recognized by SNAIL can write DO files to run LISP and read in their program.

The DO program ignores E directory pages and FORM characters found in E-format files.

5.8 ZERO

The ZERO command is used to initialize file directories on disk or DECtape. The command takes a device name as argument; the device must be DSK, DTAn, or UDP (user disk pack). If you do not type the word ZERO in full, it will not accept the device name on the command line, and will ask for it later. This is a safety feature in case you mistype some other abbreviated command.

For DECtape, ZERO simply executes the UTPCLR UUO, which tells the monitor to initialize the file directory for the DECtape unit you specified. Make sure you don't get the wrong unit number and zero someone else's DECtape!

If you specify device DSK, you must confirm (by typing Y) that you want to delete all the files in your disk area. ZERO then attempts to delete each file in your area. For each file, it types # if successful, otherwise a message indicating the reason for the failure (write protected, etc.). You cannot use the ZERO DSK command if you are aliased.

The ZERO program is not used for new format (file structured) UDPs. It can be used to change the password of an old format UDP, and also allows the initialization on such a UDP of the file structure formerly used by the COPY program, although this feature is now obsolete. The UFD command (see page 67) is used to set passwords and create file directories on new format UDPs.

5.9 WHO

The WHO command (abbreviated W) runs the WHO program, which displays system status information at your terminal. If you are at a display terminal, the information is automatically updated as long as the program continues to run. If you are at a non-display, the information is only typed once. The WHO command does not require that you be logged in. WHO accepts various commands, described later, to control what information is presented.

The first part of the WHO display is a list of the jobs on the system, with various information about each job. This list is divided into two sections, for jobs belonging to users and for system *phantom* jobs. (More precisely, the second section contains jobs which are detached and have the JLOG bit off in the job status word, i.e., are not logged in.) Jobs attached to pseudo-teletypes are listed under the controlling job, with the line for the controlled job indented. The listing for each job has several parts:

JOB	job number
QUEUE	queue name, as in the WHO line, with possibly an extra character indicating one of several states (if more than one applies, the one which comes first in this list is displayed): ← the job needs to be swapped in, e.g., for a monitor command - the job is now actually running ⊗ the job is locked in core → the job's upper segment is next in line to swap in ↑ the job is next in line to be swapped out ↓ the job is next in line to be swapped in * the job is swapped out
PPN	the job's logged-in PPN ("100100" for not-logged-in WHOs, etc.)
LINE	the job's terminal line number, or DET for detached jobs
JOBNAM	the job name
SIZE	the job's core image size in 512-word pages
TIME	the job's total run time since login
PL	percentage of CPU time spent on this job recently
SL	service level reservation for this programmer
SEG	job number of this job's upper segment
SW1	number of ticks between startups of job's KL-10 spacewar module
SW2	number of ticks between startups of job's KA-10 spacewar module
ALIAS	the job's ALIAS PPN

If a job has an alias and is not running a spacewar module on either processor, a dot (.) will appear in the area used for indicating spacewar modules, to help the viewer line up the alias with the proper job line.

After the job display comes a display for upper segments, containing the job number (possibly followed by one of the above single-character markers), the letter W if the segment is not write protected, the job name, core size, and number of jobs using the segment.

The next part of the WHO display is a summary of overall system statistics. This includes the time since the system was last restarted (Up Time); short and long term time spent running the null job (Null Time); short and long term time spent running the null job when another job wanted to be run but wasn't in core (Wasted); the number of jobs waiting to access the disk (Disk Queue); total user core image sizes in core and swapped out (Core); the total available user core not used by locked-in jobs, and the total available user core (Usable); the number of jobs in RUNQ and TQ and their combined core image size (Running Jobs); and the total service level reservation for logged-in users (Total Slevel).

The final section of the WHO display is a list of all I/O devices in use by programs or assigned by jobs. For each such use of a device (most devices can only have one at a time), a line is displayed containing the logical name, if any; the physical name; the character "#" if the device was assigned by the ASSIGN command; the job number using the device; if a particular file is open, the file name, extension, and directory ppn (for disk files), the number of records in the file, the record currently being read or written, and the read/write status. This last will be W if the file is open for output, R for input, or RA for read/alter mode, followed by E if the end of file has been read. Note: if the file has been closed, there may be no mode flag, and a large number like 1101 in the current-record position. Record numbers are displayed in octal.

WHO commands

WHO accepts commands to determine what information is to be presented. These are single-character commands, and should not be followed by carriage return. However, some of the commands must be followed by an argument, which is ended with carriage return; ending such an argument with alt mode will abort the command. WHO commands may be included on the monitor command line which starts the WHO program running, or can be typed separately while the program is running (the latter on display terminals only). Only one WHO command which takes an argument can be included on the monitor command line, and it must be the last thing on the line.

On display terminals, only part of the information can fit on the screen at a given time. Certain commands provide "scrolling" of the display up or down; these commands are ignored on non-displays.

The available commands are listed in the table below. Other characters will be ignored in a monitor command line, but will make WHO clear the screen and exit if typed while WHO is running on a display. If no command is typed to a WHO running on a display for two minutes, WHO will exit, unless the ∞ command has been given. After WHO exits, typing CONTINUE to the monitor will tell WHO to continue presenting the same information it had been presenting when it exited.

The following commands apply only to display terminals:

$\uparrow$ (or ;)	scroll up 1/2 screen
$\downarrow$ (or :)	scroll down 1/2 screen
FORM	(formfeed) scroll up whole screen
VT	(vertical tab) scroll down whole screen
8-9	repeat argument for $\uparrow$; $\downarrow$: FORM VT
∞	run forever (otherwise WHO exits after nothing has been typed in for two minutes)
U	Update screen. Used only for Datamedia terminals, for which WHO normally rewrites only the information which has changed since the last display. This command forces a complete redrawing of the display.
E	Exit, leaving information displayed on the screen and with the page printer set up to avoid erasing it

The following commands apply both to displays and to non-displays:

R	only display jobs which have recently been Running (not in NULQ, STOP, IOWQ, or INTW)
N	restore Normal display (all jobs)
T	Type out the job information only (not system statistics or file information). On a display, exit after typing the information once.
F	Like T, but type File information also.
Q	Type once only a Quick display of jobs which are not detached, typing only their PPNs and TTY numbers.
M V * " + - @	These seven commands allow you to specify which jobs should be displayed. These are explained below.

WHO keeps two special tables of job specifications for deciding which jobs it will display; these tables are called the PLUS table and the MINUS table. Jobs specified by the MINUS table will not be displayed (regardless of other conditions). The PLUS table is ignored if empty; but if it is non-empty, then only jobs specified by the PLUS table are displayed; however, jobs specified by both tables will NOT be displayed, since the MINUS table has priority. Both tables are initially empty, and the R and N commands reset both tables to empty. Jobs can be specified in these tables by project name, programmer name, project-programmer name, job number, and job name. Each

of the commands * + - " @ takes an argument which should be ended with a carriage return; if an argument is ended with an altmode, the command will be aborted. WHO will beep you if an argument is not accepted (illegal argument or aborted with altmode). The commands M and V take no argument. The argument to @ is a filename, the argument to " is a job name, and the argument to each of the commands * + - is a job specification which can be any of the following forms:

```

PRG          programmer name (letters and digits only)
,PRG         programmer name (letters and digits only)
PRJ,         project name (letters and digits only)
PRJ,PRG      project-programmer name (letters & digits only)
NUMBER       a decimal job number less than 64 (digits only)
"NAME        a job name (any sixbit characters after ")

```

A job will match specifications of the first four types if either its logged-in PPN or its ALIAS PPN matches the specification. Any job running on a PTY owned by a job being displayed will also be displayed, unless the job on the PTY matches a specification in the MINUS table. If a job running on a PTY is displayed and its controlling job is not displayed, then the job on the PTY will be indented using dots (.) instead of spaces (which would be used if the controlling job were being displayed).

The seven individual commands are described below.

```

M          only display jobs matching your logged-in programmer name or your ALIAS programmer name
           (clears both tables and then puts your logged-in programmer name and your ALIAS programmer
           name into PLUS table)
V          only display jobs belonging to volleyball players (simulates an @VB.DIS[P,DOC]--see @
           below)
*<spec>    only display jobs matching the specification <spec> (clears both tables and then puts
           <spec> into PLUS table)
+<spec>    also display jobs matching <spec> except those matching entries in MINUS table (adds
           <spec> to PLUS table and removes <spec> from MINUS table); leave running-jobs-only mode.
-<spec>    don't display jobs matching <spec> (adds <spec> to MINUS table and removes it from PLUS
           table)
"<name>    only display jobs with the job name <name> (clears both tables and then puts <name> into
           PLUS table)
@<file>    clear PLUS and MINUS tables and then put job specifications from the file <file> into the
           tables. E directory pages and SOS line numbers are permitted and ignored. The default
           extension .DIS is used if no extension is typed. If no PPN is typed, the disk area
           [P,DOC] is tried first, then your own disk area, to find the file.

```

While reading from a file, spaces are ignored (except in a job name). A semicolon not in a job name causes the rest of the line to be ignored. Non-SIXBIT characters (except lower-case letters, which are converted to upper case) are treated like CRLFs.

An entry in the file can be any of the forms listed below, and should be terminated by a comma or a CRLF. Note however that a comma (or any other sixbit character including semicolon and space) appearing as one of the first 6 characters of a job name will be taken as part of the job name.

```

+<specx>   Add the specification <specx> to PLUS table.
<specx>    Add the specification <specx> to PLUS table.
-<specx>    Add the specification <specx> to MINUS table.

```

Unless all entries in the file are of the form -<specx>, WHO leaves running-jobs-only (R) mode. A job specification in a file, <specx>, can be any of the following forms (note that these are slightly different from the forms for <spec> that you can type in):

PRG	programmer name
[PRG]	programmer name
[,PRG]	programmer name
[PRJ,]	project name
[PRJ,PRG]	project-programmer name
"NAME	job name. After the " the first 6 characters (or less if terminated by a CRLF) are taken as a job name. Then characters are skipped until a comma or CRLF is seen.

Note that you cannot specify a job number from a file.

If WHO is started with the REENTER command, it makes a listing of its information display on the line printer. The information printed will be controlled by the WHO commands in effect when WHO last exited, if any.

Finally, the command WHO user%host (user is optional) types once only a WHO-like status report on the specified user, or all users if none specified, at the given ARPA network host. The exact format of the report depends on the host selected.

5.10 FIND

The FIND command is used to locate information in a file by searching for a keyword and typing out the "paragraph" in the file containing the key. The main use of this program is to look people up in the AI Lab telephone directory. FIND may be run without logging in first.

```
FIND [EXACTLY] [WITHIN <delim>] [EXACTLY] <key> [[IN] <file>]
```

where square brackets [] indicate optional elements, <key> is the string of characters to be found in the file <file>, and <delim> is either a single character or one of the following words: MSG LINE PAGE PARAGRAPH GRAF.

If <key> contains any spaces or tabs, it must be separated from the filename <file> (if present) with the word IN, or delimited by double quotes (") or left and right single quotes (' and '). The left and right quote characters (' and ') may appear in the key when double quotes surround the key, and double quote and left single quote characters (" and ') may appear in the key when left and right single quotes surround the key. If the number of spaces and tabs in the key is significant (in EXACTLY mode), the quoted form must be used; note however that quotes do not imply EXACTLY.

The EXACTLY qualification (which, as indicated by the syntax above, may occur either before or after any WITHIN phrase) causes FIND to search for an *exact match*. Otherwise FIND performs a *template match*. A template match recognizes two special symbols in the key: *space* (one or more consecutive spaces or tabs in the key) and *ellipsis* (three or more consecutive dots). A space in the key matches zero or more spaces, tabs, and CRLFs in the unit being searched. An ellipsis in the key matches zero or more arbitrary characters in the unit being searched. Furthermore, in a template match, upper and lower case letters are equated. The EXACTLY search is somewhat quicker and requires that the unit searched (see the WITHIN phrase below) contain the key exactly as given.

The WITHIN <delim> phrase allows specification of the delimiter that determines the amount of

text searched as a unit and typed out when a key is found. The default typeout is a PARAGRAPH except when the partial-sign mail filename specifier is used, in which case MSG is the default. To specify some other quantity, use the WITHIN phrase in the command (see the syntax above). The text typed out will be that delimited by <delim> at the beginning and at the end. The delimiter <delim> can be specified as a single character or it can be specified by one of the following exact names:

MSG LINE PAGE PARAGRAPH GRAF

where MSG means that partial sign (∂) is the delimiter (designed for use with mail files), LINE means that the end of a line is the delimiter (i.e., only the line on which the key is found will be typed), PAGE means that formfeed (a pagemark) is the delimiter, and PARAGRAPH or GRAF means that a blank line is the delimiter.

The delimiter character will be treated as a delimiter only if it occurs as the first character on a line; a line starting with the delimiter character is considered the first line in a new text unit, and the previous line is the last line in the previous text unit.

Each separate text unit containing the key is a *hit* and is typed out, with each line of text preceded by an asterisk (*) except that the line in which the key occurs is preceded by a greater-than sign (>). Also, the hits are counted and the count is printed after the whole file has been searched.

Within a single delimited text unit, up to about 25 lines can occur both before and after the line in which the key is found. If more than about 25 lines occur before and/or after the key but within the delimited text area, an ellipsis (. . .) will be typed out before the first line typed out and/or after the last line typed out.

The default file to be searched is the A.I. Lab phone directory. However, if you have an OPTION.TXT file in your alias directory with a line beginning FIND:, then the default file searched will be that named after the FIND:. The default file is used unless you specify a filename in your command, in which case the specified file is searched. If you have a FIND:<file> entry in your OPTION.TXT file, you can still have the phone directory searched without typing its exact name by using the filename PHONE in your command.

The FIND program tries to be as clever as possible about deciding what is or is not a filename. For example, the command

FIND LARRY TESLER

will look for the string "LARRY" in the file TESLER if you have such a file in your directory, but will look for "LARRY TESLER" in the default file if you do not have a file TESLER. However, this rule is applied only if the key ("LARRY") is only one word; otherwise, the key must be quoted or delimited with IN if a file is also specified.

The filename given can use the partial sign (∂) to specify a mail file (prg.MSG[2,2]). The partial sign can optionally be followed by any combination of a programmer name to specify one other than your own; an asterisk (*) to specify the system message mail file NOTICE.TXT; an extension to specify one other than .MSG; and a PPN to specify one other than [2,2]. When a partial-sign filename specification is used, the default amount of text to be searched and typed out as a unit (see the WITHIN phrase above) is one message, that is, all text between line-starting partial signs in the searched file; this can be overridden using the WITHIN <delim> phrase.

For instance, the command

```
FIND garply baz in  
```

will search your own mail file for "garply baz" and type out the entire message(s) it occurs in. The command

```
FIND EXACTLY " ME" in  *
```

(that's "<tab>ME") will find all system messages (in NOTICE.TXT[2,2]) from ME. The command

```
FIND EXACTLY " ME" in  bh
```

(again that's "<tab>ME") will find all messages sent to BH by ME, and the command

```
FIND president in  .nap
```

will search your own News Service notification file on [2,2] for all notifications containing "president".

More example commands:

```
FIND EXACTLY JMC
FIND MCCARTHY
FIND TARGET IN COMLIN.FAI
FIND TARGET COMLIN.FAI
FIND "TEMPLATE ... TARGET" COMLIN.FAI
FIND TARGET BYTE IN COMLIN.FAI
FIND 'LOMA VERDE'
FIND EVENT IN  *
FIND EXACTLY " E " IN  *
FIND WITHIN GRAF EXACTLY Help IN  
FIND WITHIN   HAND-EYE IN SAVED.MSG
FIND WITHIN MSG PDP-11 IN OUTGO.MSG
FIND WITHIN PAGE UDPUD IN MONCOM.BHIS,DOC
FIND EXACTLY WITHIN LINE Foo IN FILE1
FIND john...lathrop
```

Example output for the last example command above:

```
*McCarthy, John (Prof. John)      207  74430 45    P[FR*/VB,JMC] 2 1600
>      846 Lathrop Dr., Stanford,94305, (9) 321-7580        Sep 4

1 HIT ON KEY = john...lathrop
```

The EFIND command runs the EFIND program, which is similar to FIND in purpose, but works only on E-formatted files (i.e., files prepared with the editor E) in which the entries are sorted and are WITHIN PAGE. If your file meets these format restrictions, it runs faster than FIND. The program is documented in the file EFIND.RSP[UP,DOC].

5.11 Other System Information Programs

This section describes other commands which run information programs. All of them except DSKSIZ are allowed to be run without logging in.

The WHERE command types out information similar to WHO for jobs belonging to a particular user. It takes a PPN or programmer name as argument, and for each job whose login or alias PPN matches the argument types out the job number, PPN, job name, core size, queue, total run time, idle time (real time since last run), terminal, and alias if any. If the job is logged in over the ARPA network, the host from which the connection was made is also shown. (The terminal is listed by type and unit number, e.g., TTY5, III63, DD66, PTY161. Note that these are not actual physical device names recognized by the system, except for TTYn.) If no argument is given, all jobs will be listed.

If you are logged in and run WHERE, you can give the REENTER command and type in a job name. WHERE will then list all jobs with that job name.

The FINGER command also lists jobs belonging to a particular programmer, but provides different information: the job number and job name, the programmer name, the real name of that programmer if known to the system, the number of minutes since the job was last run (followed by a dot if the job is stopped), the terminal number (and type, as in WHERE), and the actual location of the terminal (room number, if in the lab, and whose office it is). Unlike WHERE, for a Data Disc terminal FINGER shows the number of the physical terminal responsible for the line, not the TTY line number. The number will be shown in the form TV-n if the physical terminal screen is looking at the line belonging to that job, or as tv-n (lower case letters) if not. In other words, if the same physical terminal is responsible for more than one line, only the one which it's watching at the moment FINGER runs will be in upper case. If another number is shown, following the idle time, it is the number of additional Data Disc channels used by the job.

If more than one physical terminal is tied to the same Data Disc line, FINGER will show all of them; the normal output for the job controlled by that line will be followed by one or more lines containing only the terminal information (no job name, number, etc.); if the line is hidden, and someone has tied to it by overriding the hiding, then the line typed for physical terminals other than the responsible one will say *** SPY *** SPY *** SPY ***.

FINGER with no arguments displays all jobs which are logged in. If the programmer specified as the argument to FINGER is not logged in, the program types the date and time of his last logout. Also, if he has created a plan file (with the PLAN command, Appendix 4) it is typed. Typing -LOGOUT or -PLAN (abbreviated -L or -P) will eliminate the logout time or the plan file. FINGER will accept human being names as arguments instead of programmer names, if desired.

If more than one name is given as argument (separated by spaces or commas), FINGER will report on all the selected users. In this case, the default is not to give the logout time or plan file; /LOGOUT or /PLAN may be used to ask for it. An argument of the form &file will look for names in the specified file, with default extension null or .DIS and PPN your own or [P,DOC]; FINGER &VB will look for VB[your alias], VB.DIS[your alias], and VB.DIS[P,DOC] in that order.

If the argument to FINGER is of the form user@host or user%host, then FINGER will show similar information about the specified user (or all logged-in users if none is given) at the specified ARPA network host. Not all hosts support this FINGER protocol. The form @* will show the FINGER display for all cooperating hosts.

The WHEN command tells you the time of last logout for one or more users, and whether or not they are now logged in. Arguments can be programmer names or PPNs (without brackets), and are separated by semicolons. For example, WHEN DON;S,SYS;*,JMC will report on the last logout for any of DON's PPNs, for the specific PPN [S,SYS], and for each of JMC's PPNs separately.

The HELP command provides information about system programs and facilities. It takes as its argument the name of some program or concept, and provides some short description of that facility, if such a description exists. The data for HELP comes from the [3,2] directory; this data consists mostly of short text files which are typed by HELP, but may also include programs or monitor commands which HELP will run for you. If no HELP is available, but there is a longer document available in one of the standard documentation file directories, HELP will so inform you. If you type HELP with no argument, the program will list the topics on which HELP is available. (Some of the HELPers which run programs rather than just typing text at you require that you be logged in. HELP LOGIN is cleverly not among those.)

The DSKSIZ command reports how much disk you are using and what your disk allocation is. The command takes a programmer name as argument; no argument means your login PRG. The programmer name may optionally be preceded by a device name. The program types your allocation and the size of all your file directories and the size of the files you are using on the area [2,2]. Finally, it tells you whether you are under allocation or not and by how much.

The switch /F will suppress most of the typeout. The /V switch (Verbose mode) will start by asking if you would like to record the information provided in a disk file PURGE.DAT for later reference, and if the user is over quota, offers to write in a file named PURDAT a list of the files the purger would choose to purge.

Each line of DSKSIZ output is in the following format:

```
[PRJ,PRG]  n Files.  t T.  w DW.  p%  Date
```

This means that directory PRJ,PRG has n files occupying t disk tracks and w disk words (each track is 2334. words) with a storage efficiency of p%. Date is the date of the most recently written file in the area.

The SYSTAT command runs the WHO program (see Section 5.9). The old SYSTAT program is no longer supported on this system, but the command name was retained for compatibility with other systems.

5.12 Miscellaneous System Programs

The commands in this section run miscellaneous system programs. Except as noted below, they require that you be logged in.

RSL runs the RSL program, which is used to reserve service level (i.e., better service). The command takes no arguments. The meaning of service level and the use of RSL are described in Appendix 6.

The LISP command runs the MACLSP interpreter. It takes no arguments.

The FIXIMLAC command is used to initialize IMLAC terminals. (These are terminals which are treated as Datamedias by the monitor, but are actually small computers with display screens.) With any luck, you will never see an IMLAC. Just in case, the command is explained in Appendix 7. This command may be given when not logged in.

The TELNET and TN commands (TN is used to abbreviate TELNET) run the TELNET program, for talking to another computer on the ARPA network. The host name of the computer you want to use is the argument. The OTN command is used for a TELNET connection using the obsolescent "old" protocol. The DTN command runs a version of TELNET which simulates a Datamedia terminal.

The SUPDUP command (abbreviated SD) provides a TELNET-like connection to the ITS systems at MIT, but using special protocols for effective text display, as though the user were at a local ITS display terminal.

The FTP command takes an ARPA host name like TELNET, but runs the FTP program to carry out the File Transfer Protocol for copying files between computers.

Further details on TELNET, SUPDUP, FTP, and the ARPA net in general is contained in Appendix 8 of this manual.

It is possible to establish TELNET-like connections to remote computers by using dial-up connections via the dataphones normally used for incoming dial-up lines. The dataphone on TTY11 includes an automatic dialing circuit. The DIAL command will establish an outgoing dataphone connection and thereafter behave like a TELNET. The dial command takes as its argument either the device name of a TTY (the word TIP means TTY24, which is a line connected as a port of the SU-TIP), in which case you must dial the number you want manually on the associated dataphone, or a telephone number in the form

```
{(ddd)}ddd{-}dddd
```

where d represents a digit and braces enclose optional characters. In the latter case, the telephone number is dialed automatically on TTY11's dataphone. In either case, you may add the switches /110, /150, or /300 (abbreviated /1, /5, or /3) to specify the speed of the connection. The default is 300 baud.

The NS command runs the News Service program, which allows access to news stories which have been received from the Associated Press and New York Times news wires. The HOT command runs the HOT program, which types AP and NYT stories as they come in. These commands are restricted to local users at the request of the Associated Press. Full documentation on the news service is contained in the file NS.ME[S,DOC].

The UFD command and the UDPUFD command run the UFD program, which is used to create or delete user file directories on the system disk or on file-structured user disk packs. The difference between the two commands is that UFD has default device DSK, whereas UDPUFD will use whichever UDP is available to your job, if unique. The command format is UFD DEV:PRJ,PRG. To create a new directory, you must know the pack password; to change or delete a directory you must know its password. (You can create a new directory on the system disk with the same programmer name you are logged in under; this is the only way for remote users to create new directories.) You can change the directory's password or file protection codes with UFD as you can for the main file disk with LOGIN (see Section 5.1). You can delete a directory only if it is empty.

The UNPROTECT command allows any logged-in user to change the protection of any file on the disk to 005, which allows any logged-in user access to the file. The use of the command is logged on the console terminal and in the file UNPROT.LOG[1,2], which is otherwise not writeable and cannot itself be unprotected. The command is intended as a last-resort emergency technique for legitimate access to protected files, and its misuse is a serious offense. User file directories cannot be unprotected, and a protected directory will prevent access to any of the files in it. The argument to the command is the file to be unprotected.

The UNDELETE command can be used to try to recover a file which has been deleted accidentally. The format is UNDELETE newfile←oldfile where oldfile is the one which was deleted and newfile is the name under which the recovered version will be written. The file can be recovered only if the disk space it occupied has not yet been reassigned to another file, so if you accidentally delete a file you should try this right away. A file which can't be recovered this way may be recoverable from the backup tapes which are made daily, using the DART program (Appendix 5).

The DRD command (not related to the Department of Redundancy Department) runs the DIREDD program, for displaying and manipulating file directories at a display terminal. See the file DIREDD.SGK[UP,DOC] for instructions.

The @ command runs the @ (ATSIGN) program, which can be used to produce cross-reference program listings for XGP output. It also allows for incremental listings when a program is modified. The documentation for @ is in the file ATSIGN.RMS[UP,DOC].

The DECIDE command can be used to help make difficult programming decisions. For example:

```
DECIDE SHOULD I GO TO LOUIE'S FOR DINNER TONIGHT?
```

The TEST command runs the system program TEST, if any, which is used for testing new system features and programs.

SECTION 6

PRIVILEGED COMMANDS

Certain command functions can only be used by privileged users. (There are several privileges which may be associated with a PPN.) Most privileged functions are carried out by programs and are documented with those programs, but a few monitor commands themselves take on special meanings when used in a privileged way. Most users do not require any privileges; if you think you need to be assigned any, see a system programmer, who will disagree.

The DETACH command, if given with a device name argument by a user with the DEV privilege, makes the specified device unavailable to users. This is intended to be used for hardware maintenance. The ATTACH command with device name argument, also privileged, assigns a detached device to the job giving the command; it becomes generally available when the job deassigns it. These commands also accept pseudo-device names of the form DDn, to make Data Disc channel n unavailable. (A detached Data Disc channel can still be used if a program requests it specifically by number, or if no other channels are available.)

The DEV privilege also allows the TTY command to be directed to any terminal, even if in use by another job. Also, it allows TTY SPEED commands for any line, even if not a low-speed modem, and allows any speed to be specified.

A user with the ATT privilege can use the ATTACH command to attach any detached job, including specifically a phantom job (i.e., a detached job with the JLOG bit off in its job status word). In this case, ATTACH will log the job in by turning on its JLOG bit.

A few privileges are available to any user, but must be specifically enabled by giving the ENABLE command with the privilege name as argument. These privileges can be disabled by the DISABLE command, also with the name as argument. These privileges are UPG, LIV, and LUP. The UPG privilege must be enabled before giving a FLUSH command for a terminal which is in use. (See page 45 for the use of this command.) The FLUSH command disables the privilege, so you must enable it every time you use it. The LIV privilege prevents the automatic logout which normally happens when a job has been idle for a certain length of time (varying with the time of day). You should not leave idle jobs logged in without a good reason. If you have enabled LIV, and your job would otherwise be logged out, and you are at a Data Disc terminal, and there are no free Data Disc channels, then your job will be detached, so that the terminal will be freed for other users while still preserving your job. The LUP privilege means local user; it is set for all users on login, and distinguishes logged-in (and therefore authorized) users from guests who use the commands allowed without login. Various programs restrict the capabilities available to non-local users, and they cannot write most files. The privilege can be disabled primarily for the benefit of people debugging system programs which must distinguish local from guest users, and once disabled, it can be re-enabled only if you are at a terminal physically within the laboratory building.

Also, a few system programs intended for general use require special privileges. Such programs are run by special commands (not R or RUN), and any attempt to modify your core image while running such a program will disable the temporary privileges involved.

The EDDT command can be given only from the KL-10 console terminal. It stops timesharing and starts Executive DDT to allow debugging of the monitor.

The BEEP command can also be given only from the console terminal. It beeps all terminals on the system, and is used to signal the restoration of service after a system crash.

APPENDIX 1

SNAIL

SNAIL is a program which interprets certain monitor commands for editing files, compiling and loading programs, and loading the line editor buffer. It simplifies these tasks when necessary by expanding concise user commands into the different forms used by editors, language processors, and the loaders. Without user intervention, SNAIL will run the correct language processors to translate the user's files, and will run a loader to create an executable core image containing the user's program. SNAIL also provides a convenient means of communication between editors and other processors.

SNAIL knows about certain standard processors. These are the editors: SOS and TECO; the compilers: SAIL, FAIL, F40 (FORTRAN), MACRO, PALX, BLISS, AL, MIDAS, PASCAL, and FASM; the document compilers: PUB and POX; the debuggers: BAIL, RAID, and DDT; and the loaders: LOADER and LINK.

The E editor includes within it the SNAIL functions, so its "SNAIL" commands run E directly rather than via the SNAIL program. Nevertheless, the E commands are included here because they are operationally similar.

The SNAIL commands are divided into three classes: EDIT, COMPILE, and CREF.

A1.1 EDIT-Class Commands

There are six EDIT-class commands, two for each of the three commonly used editors:

Program	Command Name	
	Create File	Edit File
E	CETV	ETV
SOS	CREATE	EDIT
TECO	MAKE	TECO

CREATE and MAKE

These commands take a file name as the argument. SNAIL will call the editor and tell it to initialize the file of that name. For CREATE, SOS is started in line insertion mode. TECO (the MAKE command) will be initialized to write on the specified file.

EDIT and TECO

These commands take a file name (with optional project-programmer name) as the argument. SNAIL remembers the name of the last file that was edited in a TMPCOR file named ED. (A TMPCOR file is a simulated file kept in core by the monitor until the job is logged out.) If an

EDIT-class command is given without an argument, SNAIL will use the argument that it remembered.

Normally, the user's SNAIL commands are forgotten when the job is logged out. However, if both LOGIN:RPGSAV and LOGOUT:RPGSAV are included in the user's OPTION.TXT file, then the TMPCOR files containing the commands will be stored on disk between sessions. See Section 5.2.

If no extension is specified in the argument and no file with the given name and blank extension can be found, then the user's directory is searched for any file with the given name and one of several standard extensions. The standard extensions are searched in the order:

```
FAI S1 SAI FOR F4 LSP PUB POX TEX MAC MID VLI LAP PAL MIC WRU NSA OSA PAS LST CMD TXT
```

If none of these is found, the first file listed in the directory with the correct name and any extension is edited. However, the following extensions are never selected for editing:

```
REL DMP XGP FNT CFT DRW WD PC WPC PLT PCP PLX WL WLS LDI RIM FAS
```

If a user specifies a disk area other than his own current (alias) area, then the edited file will be put on his disk area and the source (from the other area) will not be changed.

The editors have a read-only mode in which files may be examined by using editor commands, but not changed. To get read-only mode type /R after the file name in the edit command (e.g., ETV M.SAI/R). Read-only mode in SOS prohibits any commands that would change the file. In the E editor, the mode may be changed from read-only to read-write at any time.

CETV, ETV, and EREAD

These are the create and edit commands for the E editor. They do not run the SNAIL program; instead, E itself simulates the SNAIL functions. There are some differences between these and the other editor commands:

1. The command can include /nP and /nL switches for page and line numbers and the /N switch for no-directory mode as well as /R for read-only. E saves its position in the file when it exits, so a later ETV command with no argument will continue the edit from the same point in the file. An E command may also contain a list of remembered files, pages, and lines for ease of switching around. There are also other possible switches. A complete explanation is in the file E.ALS[UP,DOC].
2. If you specify a PPN with the filename given, E will let you edit the file on that disk area instead of copying the file to your current area as the other editors do.
3. The EREAD command is equivalent to the ETV command with the /R switch, i.e., it edits the file in read-only mode.

READ

The READ command starts E in read-only mode, like EREAD, but looks for the file to be edited in the system documentation areas [*,DOC] and [3,2]. The argument to READ should be the name of a system program or concept for which you would like documentation.

BOOK

The BOOK command starts E in a special mode, called *book mode*, used to read long files over an extended time period. In this mode, E does several things differently from the normal editing procedure:

1. If there is a file with the name in the BOOK command, the extension BKP, and a PPN matching either that given in the command, your ALIAS PPN, or your login PPN (tried in that order), it is used as a SNAIL file to determine where to start editing the specified file. This “bookmark” file is not deleted by LOGOUT, so you can continue reading at a later session. The file, if found, is updated when you exit from E; if there was no BKP file, one is created on your (alias) area when you exit. However, if you exit from E while on the last page of the file, the BKP file will be deleted (E assumes you have finished reading the book in this case).
2. A BOOK command with no argument uses TMPCOR files like the ETV command does, but the TMPCOR filename used is different, so you can alternate ETV and BOOK commands for different files with no confusion.
3. E does not allow you to modify the file when using book mode.

A1.2 COMPILE-Class Commands

There are three types of COMPILE-class commands: COMPILE-type commands, which abbreviate a series of commands to compilers (and other language processors) and loaders; DOCUMENT-type commands, which process a document; and LOADBUF-type commands, which load the line editor buffer with a sequence of commands much like the DO processor (see Section 5.7). Each COMPILE-class command, if given with arguments, remembers both the command and the arguments in a TMPCOR file named CM. This is useful because all the editors allow an exit-and-go command (in SOS, the G command; in E, the CONTROL-X GO command; in TECO, the EG command). The exit-and-go feature runs SNAIL in a special way which makes SNAIL re-execute the command that it remembered.

A1.3 DOCUMENT-Type COMPILE-Class Commands

The DOCUMENT-type COMPILE-class commands are the easiest to describe. These commands, PUB and POX, run the document processors PUB and POX. Both commands take a file name as an argument, with optional switches. If an extension is specified with the file name, that file is processed by the appropriate document processor. If, on the other hand, no extension is specified with the file name, the user’s directory is searched first for the file with the given name and extension .PUB or .POX, depending on the command, then for the file with the given name and no extension. Whichever of those files is found first is the one which will be processed by the document processor.

Switches to the document processor may be specified in parentheses immediately following the file name.

Examples of use:

```
POX F00.POX
PUB F00.PUB (LH)
PUB XXX
```

The last of these will run PUB on the file XXX.PUB if it exists, otherwise it will run PUB on XXX.

A1.4 LOADBUF-Type COMPILE-Class Commands

The second type of COMPILE-class commands are the LOADBUF-type commands. There are three such commands: RER, RERUN, and PROCESS. These commands incorporate some of the features of the DO processor, allowing the user to take advantage of the exit-and-go feature of the editors for an arbitrary sequence of commands without paying the penalty of running an extra processor.

The RER and RERUN commands are similar in form. Both are followed by a processor name, an optional core size for the processor, a double-arrow, and a string (called the object string) terminated by a line feed. SNAIL swaps control to the designated processor after loading the object string into the line editor buffer. The only difference is that the processor in the RER command is retrieved from the SYS: disk area, while the processor in the RERUN command is retrieved from the user's own disk area, or one that he specifies explicitly. This distinction is the same one made between the R and RUN monitor commands.

The PROCESS command is followed by a double-arrow and an object string, terminated by a line feed. After loading the object string into the line editor buffer, SNAIL exits.

The command PROCESS @ will load into the line editor buffer the text of the CM TMPCOR file, i.e., the remembered COMPILE-class command of any type.

SNAIL preprocesses the object string for all three commands by replacing occurrences of the character double-arrow ("↔") by a carriage return-line feed. The DO processor also does this. However, the other character-replacement features of DO are not currently implemented.

The RER and RERUN commands are compatible in the following way. If a RER command with its arguments has been previously given by the user and saved by SNAIL in a TMPCOR file, and the user subsequently types RERUN without any arguments, the effect will be that of typing RERUN with the arguments of the previous RER command. That is, the processor will be retrieved from the user's disk area instead of from the system disk area. Analogously, if a RER command with no arguments is given and the command file contains a RERUN command, the designated processor is retrieved from the system disk area instead of the user-specified disk area. In either case, the core size and object string are processed as described above.

If a PROCESS command is given with no arguments, the command TMPCOR file must contain a PROCESS command. If a RER or RERUN command is given with no arguments, the command TMPCOR file must contain a RER or RERUN command. Otherwise an error message will be printed and the line editor buffer will not be loaded.

Examples of use:

```
RER IL↔(DSKIN FOO)↔
PROCESS↔R IL↔(DSKIN FOO)↔
RERUN IL [UCI, SYS]↔(DSKIN FOO)↔
```

The first two examples will do the same thing. The third example could be used to allow alternating use of RER and RERUN to compare the current version of ILISP with a new version.

A1.5 COMPILE-Type COMPILE-Class Commands

The last type of COMPILE-class command to be described is the COMPILE-type command. There are six COMPILE-type commands: COMPILE, LOAD, PREPARE, DEBUG, EXECUTE, and TRY. Following the command, the user lists files which are to be compiled or loaded.

All COMPILE-class commands normally cause compilation of the specified source files. All but the COMPILE command may cause execution of a loader (LOADER or LINK) to create a core image, possibly including a debugger. The table below summarizes the actions of the COMPILE-type commands.

	REL file	Core Image	Debugger	Execution
COMPILE	yes			
LOAD	yes	yes		
PREPARE	yes	yes	yes	
EXECUTE	yes	yes		starts the user program
TRY	yes	yes	yes	starts the user program
DEBUG	yes	yes	yes	starts the debugger

The COMPILE command takes as arguments one or more file descriptors, separated by commas. SNAIL will cause the appropriate translator to be used for each of the files. The COMPILE command does not produce a core image; to get an executable core image one of the other COMPILE-type commands must be used.

LOAD does a COMPILE if necessary and then starts a loader (LOADER or LINK). The loader takes the .REL files that were generated by the compilers and combines them into an executable core image. When the loader is done, it exits to the monitor. This command makes a core image but does not start it. If any SAIL program is loaded, the loader will be instructed to request the current SAIL segment.

PREPARE is like LOAD, but a debugger (BAIL for SAIL programs, RAID for other programs when the user is at a display, and otherwise DDT) is loaded with the user's program.

EXECUTE is like LOAD, but instead of exiting, the loader starts the program it loaded at its specified starting address.

The TRY command is like EXECUTE, but it loads a debugger (BAIL, RAID, or DDT, as in the PREPARE command) with the user program. Execution starts at the program's starting address.

DEBUG is like TRY except that rather than starting the user's program it starts the debugger

(RAID or DDT—if the debugger is BAIL this command is treated exactly the same as TRY). The user may give commands to the debugger to start his program, to plant breakpoints, etc.

SNAIL writes TMPCOR files for each processor involved in compiling or loading the files specified in the user's command string. SNAIL also attempts to minimize unnecessary compilations by checking the creation dates for the files the user wants compiled; for example, if the command was

```
EXECUTE X1.SAI,X2.SAI
```

and the file X1.REL has a later creation date than the file X1.SAI, SNAIL assumes that X1.SAI has not been changed since it was last compiled, and omits X1.SAI when it tells SAIL which files to compile. If the REL file and the source file have the same time written, then the file is compiled again for safety, in case the source has been changed.

The user can override SNAIL's date-checking by specifying certain switches. A SNAIL switch is selected by including a slash ("/") followed by the appropriate switch name. Switches which affect the date-checking actions of SNAIL are the following. A switch of the form /-XYZ turns off a previously set /XYZ.

/COMPILE	Force recompilation without checking the dates of .REL files.
/REL	Force loading from the .REL file if it exists.
/NOLOAD	Don't issue a loading request for this file.
/LONLY	(Implies /COMPILE, /NOLOAD, and /NODMP.) A listing should be made for this file, but no load request for it should be issued.
/CONLY	(Implies /COMPILE, /NOLOAD, and /NODMP.) A cross-reference listing should be made for this file, but no load request for it should be issued.
/LIBRARY	(Implies /REL.) Force the loader to search this file as a library.
/NODMP	Don't include the .DMP file in date-checking or deciding which file to give control to first. This switch has no inverse.

SNAIL remembers the last COMPILE-type command with arguments it was given, just as it does for DOCUMENT-type commands and LOADBUF-type commands, in a TMPCOR file named CM. This "remembering" may be turned off for COMPILE-type commands by using the /NOCMFI switch; if this switch appears in a command, the CM file will not be written for that command. This switch is useful when using either a LOADBUF-type command or the DO processor, and including within the command string another SNAIL COMPILE-class command; if /NOCMFI were not specified, the second command would be saved in the CM file instead of the first.

The simplest way to translate several programs is to create them with file extensions which define the processor to be used; then give a single COMPILE command with all these names separated by commas. (If a standard extension is specified for a file, SNAIL cannot be convinced to use some other processor.) Extensions which SNAIL knows about are the following:

.FAI	signals a FAIL program
.SAI	signals a SAIL program
.MAC	signals a MACRO program
.F4	signals a FORTRAN program
.PAL	signals a PALX program
.BLI	signals a BLISS program
.AL	signals an AL program
.MID	signals a MIDAS program
.PAS	signals a PASCAL program
.S1	signals a FASM program

SNAIL does not require file extensions to be typed explicitly. For the command COMPILE FOO,

SNAIL will look for any file named FOO and check its extension. If a file FOO is found that does have one of the standard extensions, SNAIL will select the corresponding translator. If a file does not have a standard extension, SNAIL can be told which translator to use by giving that information in the command string. Switches which tell SNAIL which translator to select are the following (/PROC undoes the effect of a previously seen /PROC):

/FAIL	Process this file with the FAIL assembler.
/SAIL	Process this file with the SAIL compiler.
/MACRO	Process this file with the MACRO assembler.
/F4	Process this file with the FORTRAN compiler.
/PALX	Process this file with the PALX assembler.
/BLISS	Process this file with the BLISS compiler.
/AL	Process this file with the AL compiler.
/MIDAS	Process this file with the MIDAS assembler.
/PASCAL	Process this file with the PASCAL compiler.
/S1	Process this file with the FASM assembler.

More on File Names

A project-programmer name (PPN) may be included with each file name in the command for COMPILE-type commands, just like for other SNAIL commands. The PPN specifies the disk area from which the file is to be read. The PPN, enclosed in square brackets ([and]), usually follows a file name; in this case the PPN applies only to the preceding term. If a PPN appears before a file name, then that PPN is sticky (i.e., it applies to all the terms that follow) for the rest of the command until another sticky PPN is seen.

A name followed by a colon (:) is a device name (which is sticky). Some devices are non-directory devices (for instance, TTY: and MTA:); a file name need not follow specification of such a device. For example, the command

```
COMPILE TTY:/SAIL
```

compiles, using the SAIL compiler, input from the user's terminal.

More on Switches

Switches are allowed in COMPILE-class commands by including a slash (/) and the switch name, e.g., /LIST. Switches allow the user to select options. In general, a switch placed before a file name is sticky. Switches placed after a file name are not sticky; they apply only to the preceding term. Specifying a minus ("-") before a switch turns off that switch. For example, the command

```
COMPILE/FAIL A,B,C/-FAIL
```

will compile the files A and B using the FAIL assembler, and will compile C using some other translator depending on extensions of files with file name C.

Concatenation of Source Files

It is often desirable to break up a program into two (or more) separate files which have only one END statement at the end of the last file. One reason for breaking up a program is because it is very long, and it would be inconvenient to keep and edit such a large file. Another reason for separating a program is to allow a portion of the program text to be shared. Several files can be processed together as one big translation by separating their names with plus signs (+) rather than with commas. For example,

```
COMPILE DATA+PROGRM
```

treats the files DATA and PROGRM as if they were one large file and produces one relocatable binary file, PROGRM.REL. SNAIL will not allow concatenation of source files that specify inconsistent processors.

Normally, the REL file will have the name of the last file in the command string; it can be given some other name by putting the desired name and an equals sign (=) at the front of the argument list, e.g.,

```
COMPILE FOO=SYMS+PROGRM+IOPACK
```

would do one translation as if SYMS, PROGRM, and IOPACK were one large file and produce relocatable binary output on the file FOO.REL.

Occasionally, a header file must be assembled with several other files, e.g.,

```
COMPILE ACUMS+SINE,ACUMS+COSINE,ACUMS+TANGNT
```

The brokets (< and >) feature allows this to be done more simply:

```
COMPILE ACUMS+<SINE,COSINE,TANGNT>
```

Brokets may not be nested. Also, a plus sign may not follow a right broket; e.g., the following command is illegal:

```
COMPILE <A,B,C>+ENDS
```

Producing Symbolic Listings

Assembly listings have the original code as it appears in the file plus the octal values that the code represents with the relative locations that the octal values go in. Assembly listings can be generated by means of a /LIST, /LONLY, /CREF, or /CONLY switch in the command.

The /LIST and /LONLY switches will produce a file with the same name as the REL file and extension LST. Such a file may be spooled or typed. The two switches differ in that the /LONLY switch implies /COMPILE, /NOLOAD, and /NODUMP; i.e., it means that the only thing to do with the file is make a listing for it, and no REL file will be generated by the compiler for the source file.

Note that COMPILE FOO/LIST will not compile anything if a current FOO.REL exists (this is another instance where the /COMPILE switch is useful). The /LIST may be used as a sticky switch or it may be applied to particular terms. The inverse switches are /-LIST and /-LONLY. (Processors other than the assemblers may also produce listings.)

If either the /CREF switch or the /CONLY switch appears, instead of /LIST, it generates an expanded listing that includes a list of all the symbols (identifiers) used in the program and the line numbers on which each symbol appears. This is called a cross-reference listing. These listing files cannot be printed directly. Instead, they must be processed by the cross-reference lister program, CREF. This is done by the CREF command (see page 82). The /CONLY switch differs from the /CREF switch in that the source file will always be compiled, and no binary file will be generated by the compiler for the source file. The distinction between the /CREF and /CONLY switches is the same as that for the /LIST and /LONLY switches described above. Note: not all the processors can produce a cross-reference listing. Consult the appropriate manual to find out if a given processor is capable of generating one.

Generating Core Images and DMP Files

The commands EXECUTE, DEBUG, LOAD, PREPARE, and TRY each generate an executable core image. Core images are made by translating source files (as in the COMPILE command) and then running a loader (either LINK or LOADER), which takes the REL files and produces a core image from them. For the DEBUG, PREPARE, and TRY commands, the loader is instructed to load a copy of RAID (or DDT if the user is at a non-display) and the program symbol table with the core image.

Several SNAIL switches are relevant to the selection of a loader, and to the actions of the loader once it starts loading. These switches are listed below. They can be negated with /-SWITCH.

/LINK	The LINK loader will be used to create the user's core image.
/LOADER	The LOADER loader will be used to create the user's core image.
/BAIL	If the command specified a debugger (i.e., if the command was DEBUG, TRY, or PREPARE), BAIL will be loaded.
/RAID	If the command specified a debugger, RAID will be loaded.
/DDT	If the command specified a debugger, DDT will be loaded.
/MAP	The absolute values of all the global symbols will be listed in a legible way on a file named MAP.MAP; this may not work for LOADER.
/NOSAISEG	The Sail segment will not be loaded.
/SAVE	A disk dump will be made of the core image. The dump file will have the same name as the first REL file and extension .DMP; it can be given some other name by putting the desired name and a left arrow (←) in the command string. /SAVE is unnecessary if the left-arrow construct is used.

The command

```
TRY DUMP←SINE,COSINE/SAVE
```

translates the programs SINE and COSINE, producing SINE.REL and COSINE.REL. The LOADER is run. Because the TRY command requests a debugger, RAID.REL (or DDT.REL) is loaded from the system area. SINE.REL and COSINE.REL are loaded next. Then the LOADER makes a dump copy of the core image in the file DUMP.DMP and starts the program.

If there already exists a DMP file of the right name and if it has a creation date more recent than those of the source files, no loading will be done. Instead, the dump copy itself will be called in. Reloading may be forced by the /NODMP or /NODUMP switch in the command line. Both an equals sign and a left arrow may be included in a command string. For example:

COMMAND	REL	DMP
TRY SINE+COS/SAVE	COS.REL	COS.DMP
TRY DUMP←SINE+COS/SAVE	COS.REL	DUMP.DMP
TRY DUMP=SINE+COS/SAVE	DUMP.REL	DUMP.DMP
TRY FOO←DUMP=SINE+COS/SAVE	DUMP.REL	FOO.DMP

Library Searches

Several REL files may be combined into a library. A library is a special REL file from which routines may be independently selected. Library files are made by combining REL files together using the FUDGE2 program (which is described in DEC documentation). For example, sine, cosine, and tangent routines might be combined into a trig library, TRIG.REL. Then, if a particular program used just the sine routine, instead of loading the entire trig package it would be able to select only the sine routine. The loader will search library files and load only the programs that are needed. The /LIBRARY switch after the name of a library file tells SNAIL to request this loader feature. The /LIBRARY switch may also be used as a sticky switch; its inverse is /-LIBRARY. Suppose there is a program called NLT which uses the sine routine that was included in TRIG.REL. Then the command LOAD NLT,TRIG/LIBRARY will cause NLT to be loaded and TRIG to be searched. The LOADER notices that NLT requires SINE.REL and searches TRIG.REL for it. Suppose another program, XLT, requires routines from two library files, TRIG.REL and IOPACK.REL, and must be loaded with another program LT. Then any one of the following commands could do this:

```
LOAD XLT,LT,/LIBRARY TRIG,IOPACK
LOAD /LIBRARY XLT/-LIBRARY,LT/-LIBRARY,TRIG,IOPACK
LOAD XLT,LT,TRIG/LIBRARY,IOPACK/LIBRARY
```

Programs which use library files must be placed before the library file names in the command. This is because the loader cannot know what routines to extract from each library until it has loaded the programs that require library routines.

Alternate Processor Features

Processors other than SAIL, FAIL, FORTRAN, PALX, or MACRO may be specified by using the alternate processor feature. The switch /PROCESSOR DEV:NAME specifies an alternate processor. In this switch, DEV (assumed to be DSK, if absent) is the device name on which to find NAME.DMP, the core image file of the alternate processor. Once the /PROCESSOR switch is used, SNAIL will recognize the extension NAM (i.e., the first three letters of the processor name) and the switch /NAME (the processor name) in the command line. A project-programmer name or device name may be used in specifying the alternate processor.

For example, for a user to translate the program SINE using a processor named PROC on his disk area, he could use the command:

```
COMPILE SINE/PROCESSOR PROC
```

When a standard processor is used, it is known that it will take a text source file and produce a binary REL file, and possibly a text LST file. With alternate processors there is no such guarantee. An alternate processor may process a text file into a new text file which becomes input to a standard processor. It is possible to specify all these functions in a single COMPILE-type command.

If an alternate processor generates a text file instead of a REL file, loading must be inhibited. The /NOLOAD switch accomplishes this. The /NOLOAD switch must be used for each term that does not produce a REL file (except in the COMPILE command, which does no loading anyway). The inverse switch is /-NOLOAD.

Suppose the preprocessor, PRE, translates the file BAZ into a file FOO which is a FAIL program. FOO must be translated when PRE is done. Also, MAIN and SINE must be translated and loaded with FOO. The command:

```
LOAD FOO=BAZ/PROCESSOR PRE/NOLOAD,MAIN,SINE,FOO/FORWARD
```

will do all of this. The /FORWARD switch is necessary because the file FOO does not exist at the time the command is given. Including the /FORWARD switch prevents SNAIL from looking for FOO; SNAIL will assume that FOO magically appears before FAIL is called. The inverse switch is /-FORWARD.

The processor DO (see Section 5.7) is another example of an alternate processor. The DO program writes the CM TMPCOR file to force the exit-and-go command from the editors to run DO again.

Translator Switches

Switch names enclosed in parentheses "(" and ")" are passed directly to the translator. The documentation for each of the translators explains what switches are available. The switches passed in this way appear on the source file specification. Sometimes it is necessary to put switches on the binary or listing term. For this purpose, there are two other switch constructions.

The construction (#1,#2), where #1 and #2 are any switch strings, passes #1 to the binary term and #2 to the source term. The construction (#1,#2,#3) passes #1 to the binary term, #2 to the listing term, and #3 to the source term. For example,

```
COMPILE/LIST FOO(AX,BY,CZ)
```

will pass the switch string "AX" to the binary term, "BY" to the listing term, and "CZ" to the source term.

Loader Switches

Switches to the loader may be specified in a COMPILE-class command. Switches to LOADER appear as a percent sign (%) followed by a letter (or a number and a letter) in the command string of one of the LOAD, TRY, EXECUTE, PREPARE, or DEBUG commands; switches to LINK appear as a percent sign followed by some character not appearing in the switch string, followed by the switch string, followed by the character that appeared after the percent sign. In either case, the switch string is directly specified to the appropriate loader. Examples:

```
EXECUTE F001,F002,F003%M
TRY BIGPRG.SAI%100K
TRY/LINK BIGPRG.SAI%"RUNCOR:100"
```

LOADER and LINK switches are listed at the end of this appendix. For a full description of what the other switches above mean, consult the DEC documentation for LOADER and LINK. The LOADER has been modified for Stanford, so the DEC documentation does not accurately reflect the state of the software. Consult a wizard if necessary.

Command Files

COMPILE-class commands are sometimes too long to fit on one line, or are too complex to be typed correctly every time. To solve this problem, arguments can be read from a file. The text contained in such a file is read by SNAIL as if it had been typed as the arguments to a command. The command file may contain any features available in a COMPILE-class command. Any text between a semicolon and the next line-feed is ignored; carriage-returns and line-feeds are also ignored, and hence may not be used to separate file names or other identifiers.

To use a term as a command file write an at sign (@) before the command file name, e.g., LOAD @SYS. Upon scanning the at-sign followed by a file name, SNAIL looks for a command file with that name; if no extension was specified, SNAIL checks first for the file with extension .CMD, then for the file with no extension and the specified name. The command file SYS might look like the following file which is used to assemble the entire operating system:

```
%S%B%? WAITS=HEAD (XLR)+OUTER+JOB DAT+ALLDAT+IMPDOB+DLNDOB;
+LOWCOR+PARSER+SYSINI+MPXSER+KL SER+COMCSS+CLKSER+PAGSER;
+UUOCON+SPWSER+SCHEDU+SWP SER+CORE;
+DPYSER+TTYSER+LINED+DLN SER;
+DSKSER+DTC SER+MTC SER+TVSER+LPTSER+ELF SER+CARSER;
+XGP SER+PTP SER+PTRSER+SIXSER+IMP SER+NULSER;
+PATCH+DDT+ONCE
```

A1.6 CREF Command

This command causes a cross-reference listing to be made. All the files made from COMPILE-class commands which used the /CREF switch are processed; for each such file, a cross-reference listing is

generated in a file with extension LST and the name of the file being processed. (LOGOUT makes SNAIL forget which listing files to process.) The CREF command activates SNAIL which in turn activates the CREF program in a special way. The CREF program when activated in this way reads a file that SNAIL wrote which contains the names of the files to process.

A1.7 Switch Function Summary

Switch	Abbrev	Meaning
/AL	/A	Use AL for this term.
/BAIL	/BA	Use BAIL if a debugger is required.
/BLISS	/BL	Use BLISS for this term.
/COMPILE	/COM	Force re-compilation without checking the dates of REL files.
/CONLY	/CON	Force re-compilation and produce only an extended listing to be input to the CREF processor.
/CREF	/C	Request a cross-reference listing.
/DDT	/D	Use DDT if a debugger is required.
/F4	/F4	Use F40 (FORTRAN) for this term.
/FAIL	/F	Use FAIL for this term.
/FORTRAN	/FORT	Use F40 (FORTRAN) for this term.
/FORWARD	/FORW	Prevent SNAIL from looking for this file to decide if the REL file is up to date.
/LIBRARY	/LIB	Force the loader to search this term as a library.
/LINK	/LIN	Use the LINK loader to create the core image.
/LIST	/L	Request a listing file.
/LOADER	/LOA	Use the LOADER loader to create the core image.
/ONLY	/LON	Force re-compilation and produce only a listing file.
/MACRO	/M	Use MACRO to translate this term.
/MAP	/MAP	Request a loader map of global symbols.
/MIDAS	/MI	Use MIDAS for this term.
/NOCMFI	/NOC	Don't generate a CM command file for this command.
/NODMP	/NOD	Force re-loading without checking the dates of DMP files.
/NOLOAD	/NOL	Prevent a loading request for this term.
/NOSAISEG	/NOS	Load SAIL programs with the SAIL library instead of the SAIL segment.
/PALX	/PAL	Use PALX to translate this term (produces .BIN file).
/PASCAL	/PAS	Use PASCAL for this term.
/PROCESSOR	/PR	Specify an alternate processor.
/RAID	/RA	Use RAID if a debugger is required.
/REL	/RE	Force loading from the REL file if it exists.
/S1	/S1	Use FASM to translate this term (produces .LDI file).
/SAIL	/S	Use SAIL to translate this term.
/SAVE	/SAV	Force the LOADER to make a DMP file of the core image.

A1.8 LOADER Switches and LINK Equivalents

LOADER Switch	LINK Equivalent	Meaning
%A	%"CONTENT:ZERO"	Causes a listing of all global symbols to be printed.
%B	%"SYMSEG:LOW"	BLT symbols down when done loading.
%1B		BLT symbols to upper segment.
%C		Chain beginning with Common.
%D	%"TEST:DDT"	Load DDT.
%E	%"EXECUTE"	Start up program when done loading.
%F	%"SYSLIB"	Enter library search mode; search all default libraries.
%G	%"GO"	Finish loading, make final links, and exit.
%H	%"DEBUG:RAID"	Load and start RAID.
%I	%"NOSTART"	Ignore starting address of this program.
%J	%"START"	Use starting address of this program.
%nK	%"RUNCOR:n"	Adjust to n K of core (2n pages) when done loading.
%L	%"SEARCH"	Search this file as a library.
%M	%"MAP:END"	Print storage map.
%N	%"NOSEARCH"	Leave library search mode.
%n0	%"SET:.LOW.:n"	Set program origin to n, absolute.
%P	%"NOSYSLIB"	Inhibit automatic library search for undefined globals.
%Q	%"SYSLIB"	Allow automatic library search.
%R		Chain beginning with resident module.
%S	%"LOCALS"	Load symbol table also.
%T	%"DEBUG:DDT"	Load and start DDT.
%U	%"UNDEFINED"	Print current list of undefined globals.
%V	%"TRY:RAID"	Load RAID.
%W	%"NOLOCALS"	Load without symbols.
%X	%"CONTENT:NOZERO"	Do not list all globals.
%Y		Load SAIL0W for 2 segment SAIL programs.
%Z	%"RUN:LINK"	Restart LOADER.
%<		Disallow loading into upper segment.
%1<		Allow loading into upper segment and load there next.
%n<		(n≥400000) Set upper segment relocation to n.
%-<		Allow loading into upper segment and load in lower next.
%?		Sort symbols.

A1.9 Error Messages

Obvious errors

No command file.

Incompatible commands.

You gave a command with no arguments, and either no command had been saved or the two commands (the one you typed and the one which had been saved) aren't compatible.

Unrecognizable file name.

No such device.

Unrecognizable processor name.

You mistyped a file or processor specifier.

Not a display.

You're trying to load the line-editor buffer and you're not using a display.

Missing double-arrow or invalid core size.

Missing double-arrow.

The LOADBUF-type commands must always be followed by a double-arrow.

Too many files.

Currently the PUB and POX document compilers can only process one document at a time.

File not found.

Either some file named in the command does not exist, or it doesn't exist with the correct extension. You'll get this message if you try to use the /FAIL switch with FOO.SAI, for instance.

Unknown command.

Command error.

Illegal character.

Something is wrong with what you typed, and SNAIL didn't find in the command what it expected to find. For example, a file name didn't follow an at-sign.

Nested brokets illegal.

Unbalanced brokets.

Can't use "=" with brokets.

You're not using brokets correctly. Each of the files within the brokets will produce a different REL file, so the equal-sign construct can't be used.

Unknown switch.

Negation of this switch not allowed.

Invalid switch string.

Illegal LOADER switch.

Illegal LINK switch.

Subtle errors

Empty command file.

Error in command file.

Tmpcor file too big.

The CM command file which SNAIL is attempting to read is messed up somehow.

Storage overflow.

Your command included too much of something (file names, device names, alternate processor specifications, translator or loader switches), or there are too many files in your directory that have to be date-checked.

Horrible errors (a wizard should be consulted)

SNAIL bug.

SNAIL allocation error.

Typing CONTINUE followed by a carriage return when the latter happens might work.

Can't open channel.

This is probably a bug in SNAIL.

Couldn't create file.

A disk file could not be written correctly. Possibly a file is write protected or in use by some other job. Otherwise you may need help.

UFD missing.

You are aliased to a non-existent disk area. Otherwise, you need a wizard.

APPENDIX 2

COPY

This appendix is taken from COPY.RPH[S,DOC], SAILON 61.1, the COPY manual. The manual first describes the basic COPY command with all possible options, and then lists the other monitor commands which run COPY and describes their special functions.

Syntax

In this section the following rules are observed. Anything in curly brackets is optional. Anything enclosed in brackets (e.g., <.....>) has a syntactic definition of its own and is described elsewhere. Upper and lower case letters are legal anywhere. A break character is something which is not a letter, a number, or a “*”. This will become important when you try to figure out some of the error messages. In this manual the term *sticky* means that the sticky item is applicable until changed or until the end of the current command.

The basic syntax for all COPY commands is:

```
COPY {{<destination term>} {,<list term>}<←>}<source term>
```

The character “=” may be substituted for “←”.

```
<destination term>::=      <switch term>
<source term>::=          <switch term>{,<source term>}
<switch term>::=          {<switch list>space}<term>{<switch list>}
<list term>::=            <term>
<term>::=                 {dev:}{filnam{.ext}}{[prj,prg]}
                           | {<switch list>}{dev:}{[prj,prg]}{<switch list>}
                           @<spec term>
                           | \<filehack selector>
                           | @{ <prg> | * }{.ext}{[prj,prg]}
<spec term>::=            {<spec switch list> }{dev:}{filnam{.ext}}
                           {[prj,prg]}{<spec switch list>}
<switch list>::=          {/<switch>}{(<page list>)}{<switch list>}
<spec switch list>::=     {/<switch>}{<spec switch list>}
<page list>::=            <page term>{,<page list>}
<page term>::=            N{:M}
```

General Operation

COPY reads data from the logical file(s) indicated by the *source term*, performs any operations requested by the *switch term(s)*, and writes the data on the logical file indicated by the *destination term*. Any listing information goes to the *list term*.

Dev and ppn are sticky in the source; that is, if they are absent, the last one specified is used. The default device is DSK and the default ppn is blank (those of the current job).

A single * may be substituted for any of the following:

```
filnam
ext
prj
prg
```

A * in the source term means “all.” A * in the destination term means “same as in source file being transferred.”

If filnam.ext is absent in the destination term, *.* is assumed. If the destination filnam.ext are specified (no *’s), all source files will be concatenated into one big file with that name.

For example:

```
COPY DING<FOO,BAZ
```

would cause a file named DING to be created on your disk area comprised of the files FOO and BAZ from your area. If filnam.* or *.ext is used in the destination term, the source files will be copied onto the destination one by one with the new filename or extension. <destination term> may be left out and DSK:*. *< will be assumed. For source terms with directory devices you must specify the filnam.ext.

If you try to write on top of a file which already exists, you will be given the option of deleting it or not. If any of the source files you have specified do not exist, you will be informed and given the proper option.

If a string of source descriptors is specified, the string is executed from left to right.

You cannot write a file in two different modes (e.g. FIL1<TTY:;DSK:FIL2). However FIL1<TTY:;DSK:FIL2/ASCII is quite acceptable (/ASCII causes the data mode to be 0). Data modes are explained in the *UUO Manual*.

The <spec term> construct causes the scanner to continue scanning in the file specified by the term. The term must indicate a unique file. When end of file occurs the scanner again reads from the terminal. No spec terms may appear in the file. Switches preceding the “@” are treated as sticky switches. Switches following the “@” are treated as sticky switches only while reading from the indirect file. In a sense the indirect file is a “program block.” Upon leaving the “block” all defaults are reset to those in force just before the “@”. A device name and ppn may precede the “@”, in which case the defaults are changed to those, but no operation is done. The device and ppn used in the spec term do not affect the current defaults.

The \<filehack selector> construct for a term is a special shorthand feature to allow commonly referenced files which would normally require a large amount of typing to be entered quickly. The following is a list of legal filehack selectors and what they are short for (<prg> stands for the current programmer name right justified). Only enough letters to uniquely identify the name among those in this list need be typed.

Filehack		
Selector	File selected	What for?
MSG	<prg>.MSG[2,2]	Current user's mail file
MAIL	<prg>.MSG[2,2]	Current user's mail file
PLAN	<prg>.PLN[2,2]	Current user's plan file
PLN	<prg>.PLN[2,2]	Current user's plan file
NAP	<prg>.NAP[2,2]	Current user's NS notification file
NS	<prg>.NAP[2,2]	Current user's NS notification file
RPG	*.RPG[*,<prg>]	All of current user's RPG files
OPTION	OPTION.TXT[*,<prg>]	All of current user's option files
DOWN	MAINT.TXT[2,2]	System downtime forecast
DIGEST	DIGEST[2,2]	Latest A.P. news digest
NOTICE	NOTICE.TXT[2,2]	System message file
GRIPES	GRIPES.TXT[2,2]	System gripe file
GOLD	GRIPES.OLD[2,2]	System gripe archive file
DAY	DAY.TXT[2,2]	Daily birthday or holiday message
FORWARD	FORWD.TXT[MAI,SYS]	Mail forwarding list

The filehacks which implicitly use the current programmer name to select a file can also be given an explicit prg in the form \HACK:<prg>.

The $\partial\{\text{<prg>|*}\}$ construct is another form of shorthand for referencing MAIL files (see Appendix 4). If no argument follows the ∂ then the current user's mail file is indicated. If <prg> is used, it means that user's mail file. And lastly, if * is used the file NOTICE.TXT[2,2] is selected (this is the public mail file). The default extension (.MSG) and PPN ([2,2]) implied by ∂ can be explicitly overridden, e.g., ∂ PRG.PLN will get user PRG's plan file.

There is a page counter in COPY which monitors ASCII output for form feeds. When a form feed is encountered, the counter is incremented. If a page list is used in a term, N is taken as the starting page of this output and M (or N if M is absent) as the ending page. Output is only active when the counter is within the range of the page list (e.g., LIST COPY(21) would list page 21 of the file COPY). When the end of the first term is exhausted, control is transferred to the next term, and so on until the right parenthesis is reached, at which point output ceases. If more than one file is indicated, the page list is re-scanned for each one. Page lists are never sticky. An asterisk (*) may be used in a page list instead of the final page number, to denote the last page, e.g., LIST COPY(3:*) would list all but the first two pages of the file COPY.

Switches will be described in a section of their own. See /LIST for an explanation of "list term."

Logical device names may be used, but all switches apply to the physical device. The following are legal physical devices for the copy command:

DSK	UDPn	DTAn	MTAn	TTYn
LPT	XGP	PTR	PTP	TTY

When the special character $\downarrow$ ($\uparrow$ A on non-displays) appears, the scanner will read the line character-by-character without interpretation until the next $\downarrow$ (or $\uparrow$ A). This is useful for allowing non-alphanumeric characters to be included in filenames, e.g., editor or spooler temporary files.

For numeric input there is always a default radix (either decimal or octal). Sometimes, however, it is desirable to force one or the other. A number preceded by a single quote is always interpreted in octal. A number preceded by a double quote is always interpreted in decimal.

Device Peculiarities

Non-Directory Devices:

If the source device doesn't have a directory and the filename and/or extension is specified by a *, then a generated filename and/or extension is used. The generated filename starts out as AAAAAA and the generated extension starts as COP. Either of these may be changed by specifying a filnam.ext in the source term. If a star (*) is used, the old name is retained; otherwise it is replaced by the new one. At the beginning of each term the filename is either changed to that which was specified or incremented by one letter (i.e., AAAAAZ.COP is followed by AAAABA.COP). The following example may clear things up:

```
COPY DSK:*. *←MTA0:,,FOO.*,,*.BAZ
```

This would read 5 files from MTA0 and give them the following names:

```
AAAAAA.COP
AAAAAB.COP
FOO.COP
FOOA.COP
FOOB.BAZ
```

Disk:

You may delete files belonging to other users (please exercise extreme care). However, if you attempt to delete a file on a user file directory whose programmer name is different from the one you used when logging in, you will be asked if you are sure. All normal file protection applies to deletions.

User Disk Pack:

COPY treats new format (file structured) UDPs like the main file disk. The file-structure simulation which COPY provided for old format UDPs is no longer available.

DECtapes:

DECtapes are assumed to be in PDP-6 format (which is the current format our system uses). See the *UUO Manual* for an explanation of DECtape formats. If the directory becomes full (more than 30 files), you will be told which was the first file not transferred. If the DECtape becomes full, the file currently being written may be incomplete. An attempt is made to detect this condition before the transfer is initiated, but this is not always possible. The total file capacity of a DECtape is about 71.4K words. In order to write on a DECtape the OFF-WRITELOCK-WRITE switch must be in the WRITE position. If not, you will be told to set it.

Magnetic tapes:

Mag tapes may be made to look like directory devices by use of the /SAVE switch which is described later. All mag tape switches are separate for input and output (see rules about switches in switch section). This means that mag tape switches in the destination term don't apply to the source terms.

Paper tape:

Paper tape I/O is done in binary non-checksummed mode unless otherwise specified (see /ASCII). You should turn off the paper tape reader when you are done. A paper tape with a jagged end may cause spurious data to be read. It is best to tear it evenly at a fold.

Terminals:

On display terminals, COPY displays the number of the page being read (if >1) if reading in mode 0, i.e., the number of form feeds seen +1.

Spooling pseudo-devices:

COPY recognizes two pseudo-device names, TPL: and PGX:, to allow spooling of the output of some copy function to the LPT or XGP, respectively. These device names may be used only in output terms, and may not be used with file names. COPY creates a file on the [SPL,SYS] directory containing the desired output, and enters a spooling request for the appropriate device. The file is deleted after spooling. For a standard COPY command, the effect is essentially equivalent to the recommended SPOOL/RECOPY (see page 98); the main advantage of this capability is for direct spooling of list terms, e.g.,

```
DIRECTORY TPL:←[* ,SYS]
```

to spool a list of the [* ,SYS] file directories. (Note: It is claimed that these pseudo-device names stand for The Pseudo Lpt and Pseudo Graphic Xerox-machine respectively.)

Switches

The term *sticky* means "applies until the end of the line or until changed temporarily or permanently by some means." Most switches appearing in the destination term are carried over as sticky switches in the source. Those switches which are separate for destination and source terms are indicated by a star (*) preceding the switch name. Switches appearing in the source term are sticky only if they precede the term with which they appear. Otherwise, they apply only to the current term and must follow it. Only the first six characters of a switch are significant. You are only required to use enough characters to uniquely specify the switch.

The following switches are now available:

SWITCH	ABBREV	MEANING
ALL	AL	This switch includes all of the input file in the output even if the file uses the offset feature on the disk to 'hide' some data at the beginning. See the disk offset feature in the UUO Manual.
ASCII	A	This forces transfers to be in ASCII. Beware! This switch will cause SOS line numbers to become part of the text.
ASK	ASK	This causes dev:filnam.ext[prj,prg] to be printed for each file under consideration followed by a question mark. If you answer "Y" the operation will proceed, if you answer "G" the operation will proceed but you will not be asked the for the rest of the term, otherwise the file will be skipped.

BINARY	B	This causes the data mode to be 13 and forces word by word transfer of data.
BLOCKED	BL	This causes records to be ended on output whenever they are ended on input.
CONVERT	C	This causes copy to treat the character following a line feed and the first character of a file as a FORTRAN control character. Be careful, this happens on output.
*DENSITY=n	DE	This sets the density in bpi to n where n equals 800, 556, or 200. This switch is ignored for devices other than mag tapes. The system default density is 556.
*DSPPOOL	DS	This switch, when used with destination or list terms, causes the output file to be spooled with the /DELETE switch. Output device must be disk. This switch is illegal in source terms.
DUMPED	DU	This causes each 36 bit word from the input device to be converted to the ASCII representation of the octal number it represents and to then be sent to the output device. The following format is used. A "word number" is sent to the output device which indicates which word in the file is the first word in this row. Then 8 data words are transferred. Then a <cr><lf> is inserted. If desired, a "page list" may be used for /DUMP; however, the values are those of word addresses in the file instead of pages. However, if the /SEARCH switch appears before this switch (or the DIRECTORY monitor command was typed), this switch has a different meaning: it includes in the directory listing the date on which the file was last included in a system DART dump, and the number of the tape on which it was dumped. If the file has been permanently dumped twice or more, the character ">" appears after the tape number.
*EVEN	EV	This sets mag tape transfers to even parity.
EXTRA=n	EX	Whenever a line feed is seen n extra line feeds are put out; n must be an unsigned decimal integer. If n=0 line feeds are converted to a 177 followed by a 21; this inhibits form ejection at page boundaries on the line printer.
EXTRA=n	EX	(Note "=" not "=.") This switch, used with the XGP only, sets the inter-line spacing to n raster units.
FAST	F	All the names specified by the source term are listed (no size or other info). This switch is therefore much faster than /SEARCH.
FONT#n=file	FON	This switch is used when copying to the XGP to select a font. Font number n (default is 0) is taken from the specified file. The file specification can include any of filename, extension, and PPN; the defaults are FIX25.FNT[XGP, SYS].
FOO	FOO	This switch is used to produce a file directory listing meant to be read by various programs rather than by a human being. The switch name derives from the fact that the switch was created for the benefit of layout and drawing programs used in the design of the late, lamented SUPER FOONLY computer.
FULL	FU	If /SEARCH is specified, this switch provides more information than is usually given in the directory listing; specifically, it implies /UFDPRO, /TIME, /PROTECTION, /WRITER, /REFERENCED, and /DUMPED. This switch does not automatically imply /SEARCH.
GTOTAL	G	At the beginning of the execution of a term the term is printed. At the end of the term, a line is typed of the form TOTAL= x.x n BLK pp% where x.x is the size of the file in K and tenths (if the file is smaller than 1K, the size in words is typed instead), n is the number of disk blocks allocated to the file, and pp is the percent of the allocated space which is actually used. This switch implies /SEARCH.
HEADER	H	This causes a header to be put out at what would be the top of each page on the line printer. The header looks like this: DATE TIME FILNAM.EXT PPN PAGE N-M The current date and time are used, N is the logical page number (number of form feeds seen plus one), and M is the physical page of that logical page.
IGNI	IGNI	This causes input errors to be ignored (no message).
IGNO	IGNO	This causes output errors to be ignored.
KILL	K	The input tape is deleted after the transfer is finished, even if the

		output filename is the same as the input filename. If the 200 bit in the file's protection code is on, COPY asks before deleting the file.
LIST	L	This causes the names of the files transferred during that term to be listed. If the input device is the disk the ppn is also listed. Non-directory devices will obviously list nothing. The output will go to the "list term". If the list term is absent, the output will go to your terminal.
*MLENGTH=nnnn	M	This allows the user to specify nonstandard length records for mag tape I/O, where nnnn is a 1 to 4 digit number interpreted in octal which specifies the number of data words per record.
NONUMBERS	N	This deletes line numbers from files that have them. This forces an ASCII mode transfer; in fact, /NONUMBERS can be used on any ASCII transfer. Do not use this switch on anything but text files or you will be sorry.
NOFF	NOF	This suppresses output of formfeed characters. It implies /ASCII, and can be applied to single source files.
NOSPACES	NOS	This switch eliminates spaces at the end of a line.
ODD	OD	This sets mag tape transfers to odd parity. This mode is standard and is the default option.
OFFSET	OF	This switch implies /SEARCH, and causes the file offset (in decimal) to be included in the directory listing. The offset, if any, is the number of "hidden" records at the beginning of the file which will not normally be seen when the file is read. See the UUO Manual for more information about offsets.
OPTIMIZE	OP	This forces COPY to treat the file as one produced by SOS, and to pack it by allowing lines to span a record break as long as line numbers are not the last word of a record. SOS will read an OPTIMIZED file, but will not write a record ending with a partial line. A file which cannot be read by SOS because of line numbers or page marks at the ends of records will be "fixed" by this switch.
PAUSE	PA	This switch is effective only for output to the terminal. It causes output to pause before each element of a page list if one is specified, e.g., for the command TYPE FILE(2,3;7,14) before pages 2, 3, and 14, or before every page if no page list is used. It also pauses after the last page, before typing EXIT. You must type RETURN for the output to continue. The switch is intended for typing documents on a hardcopy terminal.
PROTECTION	PR	If this switch is preceded by /SEARCH (or used with the DIRECTORY monitor command), it causes the protection of the listed files to be included in the listing and implies the /UFDPRO switch to print the UFD protection and the UFD's default file protection. Otherwise, it must be given with a protection code and has a different meaning, as in the following paragraph.
PROTECTION=nnn	PR	This causes the output file to have the protection nnn, where n is a digit from 0 to 7. All three n's must be present. This switch applies only to disk output. If no protection switch is specified, the protection of the input file is used (for non-disk input 000 is used). When concatenating files, the protection applicable to the first file transferred is used.
QUIET	Q	This switch prevents you from getting the message "FILE ALREADY EXISTS...." If the output file already exists, it is deleted without a sound. Also, if this switch appears with /SEARCH, then only the ppn and the size on that ppn are printed. This switch also prevents being asked for confirmation when trying to delete a file with the 200 bit on in its protection code.
REFERENCED	REF	This switch is like /SEARCH below, but includes in the directory listing an extra column containing the date on which each file was last read (referenced by LOOKUP).
RENAME	REN	A rename is done on the input file with the output file information. /QUIET applies when the output filnam.ext already exists.
*SAVE	SA	If the output device is a mag tape, four words of directory information are written. If the input device is a mag tape, four words of directory information are read and compared with the source name specified. If they don't match, that file is skipped and the next examined until a match is found or the end of tape is reached.

SEARCH	SE	This switch overrides all other switches. It gives you more information than /LIST and suppresses any transfers. Added to the /LIST information, you get the size of the file in 1K blocks (or in words if less than 1K) and the date it was last written. If you are at a Data Disc or III display terminal, it also lists the time when the file was written. The information given can be further extended by the switches /FULL, /TIME, /PROTECTION, /WRITER, /REFERENCED, /DUMPED, and /OFFSET. (The offset will always be listed if nonzero for a particular file.) If a list term is specified, all of the possible information is listed. The output goes to the same place as with /LIST, with the following variation: If the "list term" is omitted, the listing goes to your terminal and the information and heading for disk input will be truncated to FILNAM.EXT, [PRJ,PRG], and SIZE. If /QUIET is used with this switch and no list term was specified, only the ppn and the total size of each disk area is printed.
*SPOOL	SP	This is the same as /DSPPOOL except that the /DELETE switch is not used in spooling the file.
TIME	TIM	This switch implies /SEARCH, and causes the directory listing to include the time as well as the date when the file was last written.
TITLE	TIT	This causes a "title page" to be put out in large block letters followed by a form feed. The title is of this form: FILNAM.EXT PPN TIME DATE The date and time are when the file was last written. For non-directory devices, <device name>: is printed in place of filnam.ext and ppn.
UFDPRO	U	This switch implies /SEARCH but also causes the UFD protection and the UFD's default file protection to be printed for each PPN for which a header is listed. These two protection codes are output on the PPN's header line in the following form: [PRJ,PRG] UFD PRO=405; DEF PRO=000 The /UFDPRO switch is implied by /FULL and by /PROTECTION (in a DIRECTORY command or following /SEARCH). To get just these protections typed out (without listing any files), use DIRECTORY [PRJ,PRG]/Q/F/U. Note that if the PPN header is not printed, the UFD protection will not be listed.
WAIT	WA	This switch will automatically wait for a busy device (e.g., XGP) without requiring confirmation by the user.
WRITER	WR	This switch includes the PPN and job name of the last writer in the directory listing for disk files. It implies /SEARCH.

Monitor Commands

The following is a list of the monitor commands which use COPY and how they work. The syntax is the same as for COPY.

COMMAND	EFFECT
DELETE	This deletes all the files specified by the source term, and types the space reclaimed as with the /GTOTAL switch. If the 200 bit is on in the file protection key, you will be asked if you really want to delete the file (/QUIET overrides this feature). The destination term is interpreted as the list term. The list term is therefore illegal.
DIRECTORY	This gives you all or part of the file information for the source term(s) (or *.* if no source term is used). If /L appears anywhere in the command string the default list device will be set to the line printer. The destination term is treated the same as for DELETE.
LIST	This sets the default output device to the line printer and turns on the /HEADER switch.
PRINT	This sets the default output device to the line printer and turns on the /NONUMBERS and /TITLE switches.
RENAME	This allows you to change the filename, extension, PPN, and protection of a file. The format is newname-oldname. Use /PROTECTION=nnn to change protection.
TRANSFER	This copies files from one place to another and deletes the source files.
TYPE	This sets the default output device to your terminal.
XGPLIST	This sets the default output device to the XGP.

Examples

```
COPY DTA4:←*.*
```

This would cause all the files on your disk area to be written on DTA4.

```
COPY ←DTA4: *.*
```

This would cause all the files on DTA4 to be written on your disk area.

```
COPY <filnam.ext>/NONUMBERS
```

This would delete sequence numbers (if any) from the file indicated.

```
COPY [2,RPH]←*.*[1,RPH]
```

This would cause all the files on 1,RPH to be copied onto 2,RPH no matter who you are logged in under (subject to file protection failure).

```
TYPE <filnam.ext>
```

This would cause the specified file to be listed on the terminal you are logged in on.

```
LIST <filnam.ext>(2)/NONUMBERS
```

This would cause page 2 of the specified file to be listed on the line printer with headers and without line numbers.

```
DIR LPT:←*.TXT[*,*]
```

This would cause the filnam.ext, ppn, size, etc. for all files in the world with the extension TXT to be listed on the line printer.

```
COPY FOO←TTY: ,FOO/ASCII
```

This would allow you to add text at the beginning of the file FOO (/ASCII is mandatory).

```
RENAME FOO/PROTECTION=077
```

This would change the protection of the file FOO to 077 without doing a transfer.

```
PRINT FOO
```

This would cause the file called FOO to be listed on the line printer without line numbers and with a title page.

```
DIR[*,RPH]/QUIET
```

This will tell RPH the size of each of his areas without printing the filenames and other information.

```
COPY TEMP←↓$ED$09↓.TMP
```

This copies the file \$ED\$09.TMP into TEMP so that you can edit it. (These names are created by SOS.)

```
DELETE @DEL
```

This would use the file DEL as a list of files to be deleted. This method is very good for “cleaning” your area.

```
DIR FOO/WRITER
```

This would list the standard directory information for file FOO along with the PPN and job name of the job that wrote the file.

APPENDIX 3

SPOOL

This description of the SPOOL commands is taken from the SPOOL manual, SPOOL.REG[UP,DOC].

Spooling provides a method of producing line printer listings of files without the usual disadvantage of tying up a user's job while the listing is actually being done.

To use the spooler, a user must create commands for it. A command is the name of a file to list, and control switches. Spooler commands are created by the monitor commands SPOOL and XSPOOL.

As an example of spooling, the monitor command

```
SPOOL SPOOL.REG [UP,DOC]
```

will cause the LPT spooler to list the file SPOOL.REG[UP,DOC].

Spooled listings are preceded by a title page (except see /NOTITLE below), used to make it easy to separate different listings. This title includes the file identifier, the name of the requestor and his project-programmer name, the current date and time, and the date and time when the file was last written. Programmer names are read from the file FACT.TXT[SPL,SYS]. Project names (of which there are few) are read from the file XFACT.TXT[SPL,SYS].

In the discussion below, "SPOOL" is used to mean either the command SPOOL or the command XSPOOL, except as noted. SPOOL makes requests for the LPT spooler; XSPOOL makes requests for the XGP spooler.

To have the spooler make a listing the user must tell the spooler what file(s) to print. This easiest way to tell the spooler what to do is by the monitor command SPOOL. This command consists of the word SPOOL followed by the name(s) of the file(s) to be listed. File names are separated by commas. Each file name may be followed by switches, explained below. The program SPOOL will send commands to the spooler.

Files to be spooled are specified in the conventional way, by file name, extension, and optionally project-programmer name. If no extension is specified, and the given file name does not exist with no extension, then .LPT and .XGP are assumed for SPOOL and XSPOOL.

Asterisk, *, may be used to allow matching of any file name, extension, project name, or programmer name. (RPG, DMP, and REL files will not be matched by * in the extension).

Switches are allowed either preceding or following the file name. Switches that precede the name are sticky and will be applied to subsequent names. A switch consists of a slash, the switch name, and sometimes, a parameter. For example, /REPEAT=5.

The following switches are available. Note that most switches occur in two senses (e.g., /DELETE and /NODELETE). This allows an explicit switch in some term to override a sticky switch.

Switch	Action
DELETE NODELETE	DELETE this file after printing. The file protection of this file must be low enough so that the spooler can delete it.
NONUMBER NUMBER	Omit SOS line numbers from the listing. "N" is an accepted abbreviation for NONUMBER.
FORTTRAN NOFORTTRAN	The first character of each line is interpreted as a FORTRAN carriage control character.
LPT100 LPT0	The spooler will open the LPT in mode 100, instead of the usual mode 0. This changes some character codes.
HEADING NOHEADING	Print a page heading at the top of each page. "H" is accepted as HEADING. HEADING is the default for all files, except: LST or DOC extensions, any file from any area with programmer = DOC. The heading includes date, time, filename, and page numbers.
TITLE NOTITLE	Precede the listing with a title page. TITLE is the default for all files.
NARROW NONARROW	Center the title page to fit an 8.5 inch wide page.
RECOPY NOCOPY	Copy the file to [SPL,SYS] and list the copy. This allows the file to be edited while the listing is in progress.
REPEAT=n EXTRA=n	Make n copies of the listing. (REPEAT is NOT sticky.) Insert n extra line feeds in the listing after every line feed in the file. (EXTRA is NOT sticky).
DUMP NODUMP	Octal listing. Data will be printed as octal full words. (OCTAL is an equivalent switch.)
NOFF FF	Each line feed in the file is translated to '177&'21 which causes single spacing with no form-feed between pages.
NOWARN WARN	Suppress "spooling done" typeout after listing.
ASK	Before making the spool command, SPOOL will type the file name and wait for the user to answer with "Y" to spool the file. (This is useful with *.* terms).
HOLD=hhmm	The listing will be held in the queue until the time specified by hh hours, mm minutes. (24-hour time is assumed.) If hhmm is prior to the current time, hhmm tomorrow is assumed.
MODE=n	Open the file in mode n (n is octal).
QSPPOOL	Type a listing of the LPT or XGP spooler's status and queue after entering the request.

File specifications may be read from an auxiliary file by including in the command line the name of the auxiliary file preceded by @. This file may include file names and switches.

Delayed Listing

Usually requests for spooled listings will be processed as rapidly as possible. Under some circumstances this may not be desired. For example, a long listing impacts the normal operation of the system by making the line printer unavailable during its duration. The spooler allows users to delay such listings until a time of minimum loading (e.g., 0500) when the unavailability of the line printer is less likely to interfere with other users.

If the construction /HOLD=hhmm appears in a command line then hhmm is interpreted as the (24 hour) time of day when the listing should actually be sent to the line printer. In the example below, 1730 will be interpreted as 5:30 PM.

Example:

```
SPOOL SYSTEM.LST [S, SYS] /HOLD=1730
```

If the time specified is smaller than the current time, it is interpreted to mean, "list the file tomorrow at the time specified." That is, if at 10PM you give the command:

```
SPOOL BAZ.FAI /HOLD=400
```

then the listing will occur in six hours.

Page Selection

You may follow a file name by a list of ranges of pages that you want listed. The list must be parenthesized. For example:

```
SPOOL FOO.SAI (1,7,5:8)
```

will cause page 1, page 7, and pages 5 through 8 inclusive to be listed. Page ranges are never sticky.

In octal dump mode, this page range list is used to specify a word range to list.

Multiple Copies of a File

The REPEAT switch (/REPEAT=n) is accepted by SPOOL. The effect is to interpret n as a decimal number and to list that many copies of the file. Multiple copies are spooled as usual, except that the two pages of heading that normally precede a file (except if the /NOTITLE switch is used) are suppressed on the second and subsequent copies of the file.

A3.1 XSPOOL Special Features

The XSPOOL command is quite similar to the SPOOL command. The XGP has greater flexibility than the line printer, so more parameters can be specified in the XSPOOL command.

The following switches are legal only in the XSPOOL command:

FONT#n=f	n is a number from 0 to 127 decimal. f is a file name. The XGP spooler will use the font named f as font n when spooling. If #n is omitted, n=0 is assumed. Up to 16 font select switches are permitted. The defaults for filename, extension, and PPN are GACS25.FNT[XGP, SYS]. See partial font loading below.
LMAR=n	Set left margin to column n. Columns in the XGP are numbered from 0 to 1699 (approximately). The left margin is the column which the carriage return character selects.
RMAR=n	Set right margin to column n. If the XGP is going to write a character that exceeds this margin, a new line will be started.
TMAR=n	Set the number of blank scan lines between the top edge of the page and the first line of text.
BMAR=n	Set the number of blank scan lines between the bottom of the text and the bottom edge of the paper. (The monitor interprets BMAR=0 to mean no paper cutting.)
PMAR=n	Set the number of scan lines in the page body. Text is written inside this area. (The monitor interprets PMAR=0 to mean that the end of a page body is signalled only by a form feed in the file, for variable-length pages. This is true only if BMAR≠0.)
XGP	The first page of the file is taken as font and margin commands, preceded by slashes as in the command line. Only the special XGP spooler switches (the ones in this table) are allowed. Carriage returns are ignored. If the first page of the file is an E directory, the second page is used for the commands. The command page is not printed. A file extension of .XGP also invokes this feature.
NOXGP	Turns off /XGP.
XLINE=n	Set the minimum interline spacing to n scan lines.
NTNODE=n	Set the number of text nodes allocated by the monitor (see below).
NVNODE=n	Set the number of vector nodes allocated.

Text nodes and *vector nodes* are units of core storage allocation used by the monitor's XGP service routines. Generally, you need not worry about them, but if you have trouble printing a complex document the allocation control switches may be able to help you. See a system wizard to find out.

The /XGP switch, when present, turns off the usual default assumptions about other switches; in particular, /HEADING is not assumed unless it is explicitly used. Spooling a file with extension .XGP implies a default of /NONUMBER as well as /XGP.

More peculiarities:

If any FONT select switch specifies some font number other than 0, no /HEADING switch will be assumed unless it is explicit.

Page headings will occur in font#0. Whatever font was in effect before the heading line will be reinvoked before any further processing. The title page, if present, will be made in font#0.

The title page will contain any page specifications and format-affecting switches (e.g., /HEADING but not /REPEAT) which came from the XSPOOL command.

Partial font loading: A font select switch of the form /FONT#n=file=chars is permitted in a /XGP file header to tell the system font compiler to load only the characters in the string chars from this font. The string is terminated by the first character which appears twice in a row (including null). The terminating character is included in the ones to be loaded. This switch is useful if a document

needs just a few characters from many different fonts, and the complete fonts would be too big for the amount of core storage available to the font compiler.

An additional switch, /USETI=*n*, is also allowed only inside a disk file when /XGP is in effect. It indicates that no more switches are on the current page (which is the first page of the file, or the second if there is an E directory), but that more switches can be found starting at record number *n* (decimal) of the file. No text in or after that record will be printed. The switch is meant for one-pass document compilers which want to use the partial font loading feature but don't know until after writing the output file what characters will be needed.

Suspended XGP listings: The XGP spooler sometimes breaks a long listing into two or more parts in order to let shorter listings through without their having to wait until the long listing is completely finished. Thus it is not necessary (or desirable) for users to break up long XGP spooler requests into pieces.

A listing being made with BMAR=0 (no paper cuts) will never be suspended. For all other listings, the spooler uses the following rules to decide whether or not to suspend the current listing:

The spooler only considers suspending a listing when a formfeed (pagemark) is encountered in the input file. To avoid suspending a listing that was about to finish anyway, the spooler will not consider breaking a listing that is within two records of the end of the file.

When a formfeed is encountered with more than two records left in the file, the spooler checks to see if the "guaranteed unbreakable" minimum amount of paper has been output for the current listing. This minimum is currently 25 normal-size (11-inch) pages. Note that the spooler actually checks the amount of paper spewed out, not the number of pages. If less than the minimum paper has been output, then no break is considered and the listing continues.

Then the XGP spooler checks to see if anyone is in the device-wait queue (DWQ) waiting for the XGP. If so, the current listing is suspended and the spooler releases the XGP.

If no job is waiting for the XGP, the spooler checks to see if 50 formfeeds have been output since the last time it checked the priorities of spooler requests. (The spooler of course checks priorities before starting a listing.) If not, the listing continues. If so, then the spooler re-checks the priorities of all the XGP spooler requests. In making this check, the priority of the request in progress is figured based on the amount of the input file that is left, not on the total size of that file. If the current listing still has the top priority, then it is continued, although a page or two of blank paper may have come out of the XGP while the spooler was thinking. If the current listing no longer has the highest priority, then it is suspended and the new top priority task is started.

When a listing is suspended, the requestor's TTY will get a message and the last page of XGP output will be a short piece of paper indicating that the listing was suspended and giving the page number at which the suspension came. When the listing is resumed, another TTY message is sent.

When the XGP stops working for some hardware reason (out of paper, KA-10 down, etc.) the current listing is suspended, and will be continued when the XGP is ready again.

A3.2 QSPOOL Command

The QSPOOL command starts the SPOOL program in a special way. QSPOOL reads the directory of [SPL,SYS] and interprets the contents of the SPX and XSP (spooler command) files. Essentially, it displays the queue of files to be printed.

The commands QSPOOL/LPT and QSPOOL/XGP will list spool requests only for the indicated device.

The queue is based on a priority scheme. The priority of a spool request is determined by:

$$(\text{time in minutes since spool request} + 1) / (\text{file size in records})$$

Requests are processed in the order of decreasing priority. (The priority of a suspended XGP listing is based on the number of records remaining in the file, rather than its total length.)

The QSPOOL command may be given without logging in.

QSPOOL will display the queue status repeatedly (every 15 seconds) for 3 minutes or until exited by typing RETURN if run on a display terminal. On a non-display it just types the information once. The command QSPOOL/TYPE will type the information once even if at a display.

While the queue display is being updated every 15 seconds, the updating will continue for about 3 minutes after any command before automatically stopping. Typing BREAK I will make the updating continue for about 30 minutes, and typing ESCAPE I will make it stop in 3 minutes.

The commands you can give while the queue(s) are being displayed are:

ALT	Update the display immediately without waiting 15 secs.
RETURN	Terminate the displaying (exit if came from monitor cmd).
LINE	Same as RETURN
αβE	Same as RETURN
<switches>RETURN	Select particular mode(s) and continue. One or more switches can be included. When any switches are given while displaying the queue(s), the default is to resume outputting both queues (hence the significance of the /DISPLAY switch below). Also, the normal 15 second delay is suppressed when any switches are given; the display will be updated immediately (unless /TYPE was given). Here are the available switches:
/DISPLAY	Display the queues; given alone, this switch displays both queues.
/TYPE	Type the queues; given alone, this switch types both queues.
/XGP	Output only the XGP queue.
/LPT	Output only the LPT queue.
/ALL	Output the [SPL,SYS] command file name for each queue entry.

Any switch may be abbreviated to just one letter after the slash, but giving an illegal switch will terminate the display.

Note that it takes only half as long to evaluate a single queue being displayed as it does to evaluate both queues. This means that the display will appear sooner if you are asking for only one queue (e.g., /XGP).

A3.3 UNSPOOL Command

UNSPPOOL is a monitor level command that invokes the program SPOOL. If you run UNSPOOL, it will display all the spooler requests that you (your PPN) have requested, and it will allow you to delete any of them. If you delete a request which hasn't been seen by the spooler, then nothing special happens since the spooler is not yet aware of the command, so it won't be missed. If it happens that the spooler is already processing your commands, then the listing in progress will be purged.

If you give the command UNSPOOL/ALL, or if you give the command REENTER after UNSPOOL exits, then not only are you allowed to purge any command file that is either in progress or pending but you also can change the spooler behavior in the following ways:

1. You can cause the present listing to be purged.
2. You can cause the present listing to be started over.
3. You can cause the spooler to hold the queue and release the LPT or XGP immediately before starting the next file.
4. You can release the queue from the state induced by holding it.
5. You can cause the XGP spooler to consider suspending the current listing at the next page mark, even if its guaranteed minimum page count has not been met.

The commands UNSPOOL/LPT and UNSPOOL/XGP will list and delete spool requests only for the indicated device.

APPENDIX 4

MAIL

The information in this appendix is taken from the files MAIL.BH[UP,DOC] and RCV.BH[UP,DOC], the manuals for the MAIL and RCV programs. See also E.ALS[UP,DOC] on the use of E in reading mail files and sending messages.

The MAIL program

The MAIL program is used to send messages to users and sometimes to arbitrary disk files for special purposes. The program is invoked by one of several monitor commands, depending on the function desired:

MAIL	send a message to one or more message files
SEND	send a message to the terminals of one or more users
GRIPE	send a message complaining about a system problem
REMIND	schedule a message to be sent at some later time
PLAN	create a file describing how to find you when not logged in
EVENT	send a message to all users about an event on a given date
BATCH	schedule the execution of a command string at some later time
LATER	schedule the execution of a given program at some later time
RETRY	send any messages which were queued by earlier failing operations

The commands MAIL, SEND, and GRIPE may be given when not logged in, so that people who are not authorized users of our system can use them to communicate with us. A user who is not logged in will be asked for his name, which will be included in the header of the message. However, users who are not logged in may not use wildcard (send to all users) or ARPA network destinations.

The commands MAIL, SEND, and REMIND require a list of destinations (recipients of the message). All of the commands except LATER and RETRY require some message text and also can be modified by the use of certain optional switches, either applied globally (all destinations) or just to a particular destination. The commands REMIND, PLAN, EVENT, and BATCH all require a date and/or time which can be included by itself or in a global /DATE or /TIME switch. (Any date given must appear before any time given, whether in switches or not. See the section below on dates and times.)

GRIPE takes a single-word gripe topic, e.g., GRIPE MAIL to report a problem with the MAIL program. This will be more fully explained on page 122.

LATER takes up to four specific arguments which describe the program to be run and when it is to be run; these will be described on page 123.

RETRY takes no arguments at all; it merely looks for queued mail originating from the current user and tries once again to send each such message (this will be done automatically if RETRY is not used).

The normal command syntax includes all necessary information on one or more lines without prompting. There is an alternative format, called *hand-holding mode*, in which each piece of information required is prompted for separately. Hand-holding mode is more verbose and time-consuming to use, but may be less confusing to new users. It is invoked by typing just the command of interest without any arguments, and will be described in a separate section. What follows here assumes that the normal syntax is being used.

Before explaining the complete range of command options, here are a few sample commands:

```
SEND BH Want to have dinner?
```

This is a very simple example of a command with one destination and no option switches. It types the message "Want to have dinner?" on BH's terminal if he is logged in, along with a header indicating who sent the message. If BH is not logged in, MAIL asks if the message should be mailed instead of sent.

```
MAIL/DIST @NL Language group meeting Tuesday at 3pm.
```

This command mails the message "Language group ..." to the mail files of the users listed in the file NL.DIS or NL or NL.DIS[P,DOC] (the [P,DOC] file directory contains lists of users by project group). The first file in that list which exists is used. The message will include the standard header indicating who sent the message and when, and also (because of the /DIST switch) a line indicating to whom the message is being sent.

```
REMIND/DATE=10-*-* . Rent due tomorrow
```

This command reminds the user who issued the command (because of the "." destination meaning self) to pay the rent on the tenth day of each month of every year. The reminder is both mailed and sent each time, and expires after 50 times. (These are default conditions modifiable by switches.)

A4.1 Command Format

The command format for all MAIL commands except LATER and RETRY is shown below. The formats for the LATER and RETRY commands are explained in the sections on those commands beginning on page 123.

```
<command & global switches> <destinations & local switches> <message>
```

The <destinations & local switches> part only applies to the MAIL, SEND, and REMIND command. Destinations and destination lists will be described in the next section. There are no destinations for the BATCH command. EVENT and PLAN have implicit destinations, and GRIPE uses a *gripe topic* as a form of destination, but not in the format of MAIL, etc.

The message text can be provided in any of three ways: (1) a one-line message can be included after the destinations on the command line and ended with <CR>; (2) if no message specification is given on the command line (or the same line as the last destination for a multi-line command), MAIL asks for a message to be entered from the terminal, ending with the end-of-file character

(<CONTROL><META><LF> from a Stanford display terminal or local PTY, <control>Z otherwise); (3) if the notation

```
@file.ext[prj,prg]
```

is used on the command line after the last destination, the message text is taken from the specified file. If no extension is typed, the extension .TXT is tried first, and then a blank extension. To force the use of a filename with no extension, type the dot but no extension, for example:

```
@file. [prj,prg]
```

The file may contain an E directory or SOS line numbers, which will not be included in the message.

If something not starting with “@” follows the destination list on the command line, it is taken as the message text. This makes single-line messages more convenient to enter. If you start to enter a single-line message, and decide you need more than one line after all, end the command line with <LF> instead of <CR> and you will be allowed to continue the message on later lines.

When entering a message, the usual line editing facilities are available. In addition, if the <bs> key is typed when the input line is empty (i.e., at the beginning of a new line), the last line typed is loaded into the line editor (Stanford displays) or input buffer (non-displays) and the cursor is positioned at the end of that line. The text of that line is deleted from the accumulated message. The effect is that a carriage return can be deleted by <bs> like an ordinary character. It is possible to back up another line by doing another <bs> after clearing the newly reloaded line. This procedure destroys the lines that followed the one to which you back up, and the backup is only possible for a small (about 30) number of lines.

When entering a message from a Stanford display terminal, a better facility is available for correcting errors in the command or in the message more than one line before the current line. Typing <CONTROL><META>E at the *end* of a line of message text will cause a file called MAIL\$E.TXT to be written on your login disk area (not alias) with the command, switches, and destination list that you gave on page 1, and the message text up to the <CONTROL><META>E on page 2. Then E is run automatically on this file to allow you to edit the text and/or command. E will start up pointing at page 2 of the file, but you can modify the command or destinations by editing page 1 (there will be no directory page). When you are finished editing the message and command, type to E the command <CONTROL>XRUN to restart the MAIL program automatically and send the message as edited. The MAIL\$E.TXT file is automatically deleted after the message is sent. Note: if you accidentally or purposefully exit from E without saying <CONTROL>XRUN, you can still send the message from MAIL\$E.TXT by simply editing that file with E (using /R if you don't need to further edit the message or command) and then giving E the command <CONTROL>XRSYS MAIL, which will run MAIL to send the message and delete the file.

The character <CONTROL><META>E cannot be typed on the MAIL command line itself, because control characters are not allowed in monitor commands. Therefore, on the command line only, the character <altmode> will start running E.

The character <altmode> cannot appear in a message (because it is a line editor activator and would confuse some mail-reading programs). If an altmode is seen in the message text as input (other than on the monitor command line), it is changed into a dollar sign (\$). Also, the character

<formfeed> cannot appear in a message (because it would mess up the E-format of mail files). If a formfeed is seen in the message text as input, it is changed into an exclamation point (!). Finally, the character <bs> cannot be part of a message text. This is not a problem if the message is entered from the terminal, because <bs> is interpreted as a line editor command, but if a <bs> is seen in a message text file, it is changed into a lambda (λ).

If the /SUBJECT switch is used to include a subject line in the message header, the subject is taken from the first text line entered: for a file, the first line of the file is the subject; what would otherwise be a single-line message is taken as the subject line for a multi-line message to follow. If no message is provided on the command line, you are prompted for a subject line and then for the message text.

Destinations and Destination Lists

The commands MAIL, SEND, and REMIND require one or more recipients, or destinations, specified in a list called the destination list. The destinations should be separated by commas. The list may extend over more than one line if the lines before the last end with comma. (Any characters from a semicolon to the following linefeed are ignored.) The following types of destinations may be used:

prg	programmer name of local user
.	yourself
*	notice for all users
name	real name (or leading substring) of local user
name % host	user at another ARPA network host
% host	sticky ARPA network host to be used for following names
#fil.ext[pj,pg]	(MAIL only) arbitrary file to receive message
TTYn	(SEND only) terminal to receive message
n	(SEND only) job number to receive message
@fil.ext[pj,pg]	file containing destination list
ARPA*	(SEND only) all users logged in via ARPA network

The form "prg" in the above list must be one to three letters and digits, the first of which is a letter (upper and lower case letters are equivalent). The forms "name" in the above list must consist of either any number of letters, digits, and hyphens, starting with a letter, or of arbitrary characters enclosed in quotation marks ("...") or downarrows ($\downarrow$... $\downarrow$). In the SEND command, the TTYn form may optionally be followed by a colon.

ARPA network hosts are specified by host name (letters, digits, and hyphens starting with a letter) or by decimal host number. If the host name is used it can be abbreviated by enough characters to identify it uniquely. Many hosts have short *nicknames* recognized by the network programs at Stanford. Mail will only be sent to a host specified by name if the host is listed in our host table as a server (that is, a host which accepts incoming connections).

A host may be specified for a single destination, using the "name%host" notation, or a *sticky* host may be specified by just "%host". A sticky host will apply (until another sticky host is specified) to all following destinations that start with a letter or a quote character and which do not have a host specification themselves. A host name of SU-AI (or SAIL) will override a sticky host but will not actually send the message via the ARPA network.

There is an ARPA network protocol for implementing the SEND command to a remote host.

However, most hosts do not implement it. Most notably, it works to the ITS systems at MIT and the TENEX systems at SRI. Certain other hosts also accept this protocol, but not very many. The MAIL program will attempt it to any server host, but it generally won't work.

More information about ARPA network mail will be given in a later section.

In the `#file` construction, if no extension is given the default extension of `.MSG` is used. To mail to a file with no extension, you must type the filename in the form `"#file."` with no extension after the dot.

In the `@file` construction, if no extension is given the default extension of `.DIS` (distribution list) is tried first. If that file does not exist, then the null extension is tried next (both of these on your current alias disk area unless you gave an explicit PPN). If neither of these exists, then the extension and disk area `.DIS[P,DOC]` is tried (unless you gave an explicit PPN). The `[P,DOC]` directory contains several distribution lists derived from the laboratory personnel directory. For example,

```
SEND @VB Time for volleyball!
```

will send that message to all known volleyball players as listed in the file `VB.DIS[P,DOC]` (see LES to be added to the list). Distribution list files may contain E directories or SOS line numbers. Only the first page (second page if in E format) is read. Everything on that page will be interpreted as destinations even if not properly separated by commas. (The semicolon format for comments will work.)

For the MAIL command, if there is a file called `OUTGO.MSG` in your login (not alias) directory, it is automatically included as a destination for the message. This provides a simple and automatic method for saving a copy of each message you MAIL to anyone. The `/-OUTGO` switch can be used to prevent sending a copy to your `OUTGO.MSG` file.

MAIL cannot be sent to local users who do not have file directories. However, if you must MAIL a message to a programmer name that has no file directories, you can do

```
MAIL #"    prg".MSG [2,2]
```

to mail explicitly to the message file of programmer name "prg" (prg should be right-justified in six quoted spaces).

The check for syntax validity and existence of local users is made as soon as the destination list is read. If some but not all of the destinations are valid, MAIL asks whether or not it should proceed using the valid ones. If you want to correct a mistake in a destination (or in a switch or in some part of the message already typed) before sending the message, answer *no* to the query and then use the monitor command `REENTER`, as explained on page 126.

For the SEND command with a multi-line message (so the destination list is read before the message has been entered), a warning message is given if any programmer name recipient is not logged in. This warning does not require reconfirmation before the command is executed.

For people who want to send mail to people listed in a file designed for use by `SENDMSG` at a TENEX site, which requires a different syntax, the `/ARPA` switch can be used to cause distribution list files to be parsed differently, including the use of `"@"` to indicate a network host name.

Obviously nesting of distribution list files is not possible within such a file. See the description of the /ARPA switch below for more details.

Message Formats

Messages are usually sent preceded with a special header which tells who sent the message and when it was sent. This header takes three forms: one for messages being typed out on terminals (SEND command), another for local (SU-AI) mail files, and a third for messages sent out over the ARPA network. The header may be followed by an optional distribution list (explained below). After the distribution list (if present), the message text is sent followed by a blank line (unless the message already ends with a blank line). SEND leaves off the extra blank line when typing the message on terminals.

The header used for local files starts with a partial sign to facilitate detection of the beginning of each message by various message editing programs. To ensure that trouble is not caused when a partial sign actually occurs in a message, any partial sign that occurs as the first character in any line of a message is indented by one space, whether or not the message is going only to local files.

Here are examples of the first two header formats:

```
;; Message from ME    at TTY46  0032    Records
023-Dec-75  0032      ME    Records
```

The SEND header (first above) contains the sender's programmer name, terminal number, the 24-hour time of sending, and then the subject (if any). If the sender is at a Data Disc terminal, the header contains the number of the responsible physical terminal as well as the TTY number.

The local file header (second above) contains a partial sign followed by the date and the time of day, then the sender's programmer name and the subject (if any). The local file header can be suppressed by use of the /-HEADER switch (see the section on switches below). However, this switch is not recommended for general use when mailing to users' message files since it causes the message to be sent without any identification about who sent it or when it was sent; in fact, it effectively causes the message to be appended to some other message in the file. This switch is useful, however, when mailing text to a special file which is simply to contain message text.

In both the SEND header and the local file header, if the sender was logged in via the ARPA network at the time of sending, the phrase "via host" will follow the sender's programmer name, where "host" is the name of the host from which the sender was connected to Stanford. The name preceding the "via ..." is, nevertheless, a Stanford programmer name.

If the sender is not logged in, his programmer name is 100 and he is asked to supply his real name, which is included in the header.

If the message was sent to all users by a SEND * command, a "to *" phrase is added to the header line to make it look like this:

```
;;Message to * from ME    at TTY46  0032 Records
```

Finally, if the message is being sent as a reminder (see the REMIND command), then the programmer name in the header will be followed by an asterisk (*) as in these examples:

```
;; Message from ME* at TTY46 0032      Records
023-Dec-75 0032      ME*      Records
```

Now, here is a sample header for messages sent out over the ARPA network:

```
Date: 23 Dec 1975 0032-PST
From: ME at SU-AI (Martin Frost)
Subject: Records
To:   rmf at MIT-ML
```

The network header contains the date and time, the sender's real name as well as his programmer name, the subject (if any), and the distribution list (list of recipients of the message, starting "To:"). There is a new mail header standard in the works, which calls for lots of useless garbage like a supposedly-guaranteed-unique message ID line to help the bureaucrats or the CIA or someone like that keep track of the message; we don't supply that.

The distribution list can be omitted from network mail by use of the /NODIST switch (see the section on switches below) and can be included in local messages (MAIL or SEND) by use of the /DIST switch. Note that /NODIST applies only to network messages and /DIST only to local messages. For messages MAILED to two or more destinations, a global /DIST switch is assumed unless a global /-DIST switch is given. The distribution list may use more than one line if lots of destinations were specified. Also, some destinations may be listed on lines beginning "CC:" instead of "To:"; such destinations are considered secondary recipients of the message and are specified by use of the /CC switch.

Normally, if a distribution list file was used to specify the recipients of the message, a distribution list included with the message will contain only the name of that file, not the actual recipients listed in it, to avoid very long distribution lists. The /LIST switch (see the section on switches below) will list in the message the individual recipients named in the file.

Message File Formats

When mail is sent to a file which already exists, the beginning of the file is read to see if it contains an E directory. If so, the message is added at the end of the file preceded by a formfeed so that the message will be on a new page. The directory is not updated to reflect this new page, but the formfeed will be at a record boundary, so that when the file is next edited with E, the directory can be updated without reformatting the file. If the file does not already contain an E directory, the message is added at the beginning of the file without any formfeed. In this case, record boundaries are not necessarily preserved. The /APPEND switch will cause the message to be added at the end of the file along with a formfeed as if there had been a directory. The /-HEADER switch which omits the local mail header also omits the formfeed which would have just preceded the header in E format files.

When a message is sent to a file which did not formerly exist, the file is created with an initial E

directory so that subsequent messages will be added at the end of the file. The switch `/-E` can be used to omit this directory normally inserted for a new file. However, this switch and its inverse `/E` will be ignored for user message files (`*.MSG[2,2]`) and for user plan files (`*.PLN[2,2]`). New user message files always get an `E` directory; user plan files never do.

The command `MAIL #LOGOUT` can be used to send a message to the file `LOGOUT.MSG`, which will be typed and deleted automatically when you log out (and have no other jobs still logged in). This file is created `/-E` automatically.

Command Switches

Various switches can be specified in the command line to modify the operation of the `MAIL` program. Some apply only globally; these may appear either just after the command name or after an individual destination. Other switches can be applied either globally (by appearing just after the command name) or to a particular destination (by appearing just after that destination). A switch applied to a particular destination overrides a global switch. Switches may appear at the beginning of a distribution list file, in which case they apply globally to all destinations within that file. Switches appearing with the `@file` destination format override file-global switches at the beginning of the file.

A switch name may be abbreviated by enough characters to identify the switch uniquely. All switches are available in positive and negative senses (e.g., `/LIST` and `/-LIST`) except for the following switches which are *not* available in the negative sense: `/CC`, `/DATE`, `/TIME`, `/COUNT`, and `/ARPA`; also, the switch `/-OUTGO` is available only in the negative sense. The default value for most switches with a negative sense is the negative sense; the `/HEADER`, `/E`, and `/QUEUE` switches are the primary exceptions, but in addition the `/SUBJECT` and `/DIST` switches can be implied by the command.

Here are the available switches:

Switch	Meaning of positive switch sense
<code>/NOMAIL</code>	(SEND only) Do not mail the message to a destination programmer name if the user is not logged in. Suppresses the question about such mailing.
<code>/YESMAIL</code>	(SEND only) Do mail the message to a destination programmer name if the user is not logged in. Suppresses the question about such mailing.
<code>/MAIL</code>	(SEND only) Mail the message to a destination programmer name as well as sending even if the user is logged in. Suppresses the question about mailing of SEND messages which would be asked if a destination programmer name were not logged in. (Note that this switch has a different meaning for the REMIND command--see below.)
<code>/MAIL</code>	(REMIND only) Do not send the reminder to the destination programmer name when the time comes, only mail it. Normally reminders are both mailed and sent. (Note that this switch has a different meaning for the SEND command--see above.)
<code>/SEND</code>	(REMIND only) Do not mail the reminder to the destination programmer name when the time comes, only send it. Normally reminders are both mailed and sent.

/DIST	Include the list of destinations in the text of the message for local recipients (see message format in a previous section). A global /DIST is implied by the MAIL command when two or more destinations are given unless a global /-DIST is specified. Compare /NODIST below.
/NODIST	Do not include the list of destinations in the text of the message for ARPA net recipients (see message format in a previous section). This switch is not the inverse of /DIST!
/NO	This is equivalent to /NOMAIL for the SEND command and to /NODIST for the MAIL command, so that /N can be used to abbreviate the appropriate switch.
/CC	List this destination and all after it as secondary (CC) recipients (see message format in a previous section). This switch implies /DIST unless a global /-DIST is given.
/SUBJECT	Include a subject line in the message. The effect of this switch is always global. /SUBJECT is implied by the MAIL and GRIPE commands unless some message text or the name of an indirect message file or the /-SUBJECT switch appears on the command line. However, typing just <CR> to the "Subject:" prompt will omit the subject from the message. Subjects are useful in local mail files because the first line of each page (usually the header line of a message) appears on the directory page in an E-format file, and for local files the subject is included in the header line and thus in the directory.
/WHERE	(MAIL and SEND only) Type the WHO line of each job that is logged in under a programmer name included in the destinations.
/QUEUE	Queue ARPA network mail for later delivery without first trying to send it immediately. The first attempt (by the remind phantom) to send such queued mail will happen right away. This switch defaults on, but affects only the MAIL command; network SENDs are done while you wait.
/APPEND	(MAIL only) Put the message at the end of the file instead of the beginning even if it has no directory. A formfeed will precede the message (unless /-HEADER was given).
/LIST	List the destinations specified within a distribution list file as well as the file itself in any distribution list included with the message. For REMIND, /LIST implies /EXPAND.
/HEADER	Include the header line in messages mailed to local files. See message formats in a previous section. Unlike most switches, the default for this switch is on.
/E	Include an E directory if creating a new message file. The default for this switch, like that for /HEADER, is on. See the section above on message file formats.
/DATE	(REMIND, PLAN, EVENT, and BATCH only) This switch is used in the form /DATE=<date> and sets either (1) the delivery date for a reminder, (2) the expiration date for a plan, (3) the occurrence date of an event, or (4) the execution date for a batched command sequence. Reminders, events, and batch executions can also be given a time (see /TIME switch below), but any date must be specified before any time is. For the <date> formats accepted, see the section below on dates and times.
/TIME	(REMIND, EVENT, and BATCH only) This switch is used in the form /TIME=<time> and sets either (1) the delivery time for a reminder, (2) the occurrence time of an event, or (3) the execution time for a batched command sequence. Reminders and batched executions can also be given a date, and events must be given a date; see the /DATE switch above. If /DATE and /TIME are both used, /DATE must come first. For the <time> formats accepted, see the section below on dates and times.
/COUNT	(REMIND and BATCH only) This switch is used in the form /COUNT=<number> and sets the expiration count for a reminder or a batched command sequence. This count is only relevant if the command contains a wildcard date. See the section below on dates and times.

/EXPAND (REMIND only) This switch applies only to destinations of the form @file. It causes such a distribution list file to be expanded now instead of when the reminder is actually sent. This means that there will actually be a separately stored reminder for each destination now contained in the distribution list file rather than one reminder that actually references the file when the reminder is being sent. This switch applies only to the file itself (not to any @file destinations within the file), unless the switch is given globally or file-globally, in which case it propagates downward. /-EXPAND implies /-LIST.

/ARPA This switch may only be used with a destination of the form @file or as a file-global switch in such a distribution list file. It changes the syntax of the destination processor so that the character "@" precedes an ARPA network host name instead of a distribution list file name, and all SIXBIT characters except space, comma, colon, quote ("), and at-sign (@) are allowed in names without using quotes. A name ending with a colon (distribution list name for SNDMSG) is ignored. The switch is intended for files from systems like TENEX with less sophisticated mail programs.

/-OUTGO This switch is meaningful only in the negative sense. It prevents mailing the message to your OUTGO.MSG file.

Dates and Times

The commands REMIND, PLAN, EVENT, BATCH, and LATER all accept dates and (except for PLAN) times. The date and/or time given for one of these commands specifies when a particular thing should happen: REMIND causes a reminder message to be mailed and or sent to one or more users at a given instant; PLAN creates a plan file for the user giving the command and specifies the date on which the plan file should automatically be deleted; EVENT puts out a system message announcing an event of general interest which will take place at the specified time and date; BATCH causes a sequence of commands to be executed at the specified time and date; and LATER causes a given program to be run at the specified time and date. For all of these commands except LATER, there are two ways to provide the date and time information.

The less confusing manner of giving dates and times is to use one or both of the switches /DATE=date and /TIME=time after the command name (if both are used, /DATE must precede /TIME). These switches may only occur immediately after the command name. (Switches are not permitted with the LATER command, which takes a fixed one-line format that will be explained on page 123. Dates and times in the LATER command must use the non-switch format.) If neither switch is used in a command that needs a date or time, then a date and/or a time will be expected after the destination list (which is only present for REMIND) and before the message text.

Dates may be given in any of the following formats (whether in the switch form or not):

4/28/75	28-APR-75	APR 28, 75
4/28/1975	28-APR-1975	APR 28, 1975
4/28	28-APR	APR 28
4/28/*	28-APR-*	APR 28, *
/28/	28-*-*	+3
WEDNESDAY	WED	W
WEDNESDAY*	WED*	W*

(Visitors from Europe please note that the slash format is month/day/year, not day/month/year!)

If a specific month and day are given but no year, the current year will be used unless the date has already passed, in which case next year is used. Today's date is considered as already having

passed except in the EVENT command. Months and days of the week may be indicated by as many letters as are required to identify the name uniquely. For days of the week, the one-letter abbreviations M T W R F S D are provided. The format "+3" is used to indicate a date of 3 days from today. The asterisks represent wildcard dates which are explained in the section below on wildcard dates and expiration counts.

Times may be given in any of the following formats (whether in the switch form or not):

1423	223pm	223p	02:23p
14:23	2:23PM	2:23P	02:23PM
0223	223am	223a	223
02:23	2:23AM	2:23A	2:23
2	2AM	2PM	14
+2:23	+2:	+23	+14:
2*	2AM*	2:23*	2A*

If an explicit AM or PM is given, it must follow the time number with no intervening spaces! This requirement is necessary to distinguish the AM or PM from the possible beginning of the message text or of a destination name. Times with three or four digits and no colon normally imply AM if less than 1200 and PM otherwise (the allowable range is 0000-2359); it is possible to specify AM or PM explicitly, however, if the number is in the range 0100-1259. If no explicit date is given in the command, times with one or two digits, or with a colon, and in the 1:00-12:59 range without an explicit AM or PM, will represent the nearest such 12-hour time which has not yet passed (which might be this morning, this afternoon, or tomorrow morning). If a date is given as well as a time, times in the 0:00-11:59 range will normally be considered AM and 12:00-23:59 will be considered PM as for the four-digit times. Note: A time of 12:00am is midnight, and 12:00pm is noon; the mnemonic is that 12:00 is a minute before 12:01.

If a time is given but no date, today's date is used unless the given time has already passed, in which case tomorrow's date is used. If a date is given but no time, the time used is midnight at the beginning of the given date.

The relative time format (e.g., +2:23) specifies the length of time before the command is to be executed (in this example, 2 hours and 23 minutes); this format may only be used if no date is given and, except in the switch form, must contain a colon so that +2 days can be distinguished from +2: hours.

A time followed by an asterisk represents the given time and a wildcard date; see the section below on wildcard dates and expiration counts.

Wildcard Dates and Expiration Counts

As demonstrated in the example dates and times above, an asterisk (*) can be used as a wildcard in place of the month or year, or following a day of the week or a time. Such dates and times cause the command to be executed repeatedly until exhaustion of the expiration count, which is explained below. (Wildcard dates are not permitted in the PLAN and EVENT commands.) An asterisk used in place of the month will cause the command to be repeated once every month, and an asterisk used in place of the year will cause yearly repetition; asterisks used for both the month and the year will cause monthly repetition every year on the given day of the month. An asterisk

following a day of the week will cause weekly repetition on the given day of the week. An asterisk following a time will cause daily repetition at the given time; in this format no date can be given.

If * is used for the month but not for the year, the command will expire at the end of the specified year, or the current calendar year if none is specified.

When a wildcard date is used, the command is repeated until exhaustion of the expiration count. The expiration count can be set explicitly by using the /COUNT=n switch (global only) or by following a non-switch-format date/time with #n, where n is a decimal number. If no expiration count is given, the default of 50 is used. A count of 0 or ∞ will prevent the command from expiring unless it is explicitly cancelled by the user.

A4.2 Mail to Other ARPA Network Hosts

Mail can be sent to users of other computer systems connected to the ARPA network. To send network mail, you must tell the MAIL program the name of the recipient, as understood by the other host, and the name of the host. Each host has an official name, a string of letters, digits, and hyphens. For example, our host name is SU-AI. Hosts may also have unofficial *nicknames*. Either the official name or a nickname can be used to identify a host; you need only type enough characters to identify the host uniquely. For example, we have the nickname SAIL which can (as of this writing) be abbreviated SAI. Each host also has a number; you can type a decimal host number instead of a name. The format for a network destination is

name % host

where "name" is the recipient's name at the host and "host" is the host name or number. The recipient name can contain upper and lower case letters, digits, and hyphens; if any other characters are required the name must be enclosed in quotation marks ("...") or downarrows (↓...↓). The name must also be quoted if it does not start with a letter. Some hosts may consider the case of letters significant in their user names.

To send a message to several users at the same host, it is possible to specify a *sticky* host with a pseudo-destination of the form

% host

Thus, the command

```
MAIL FEINLER%BBNB,%AI KLH,TK,MINSKY,BH%SAIL,PAPERT
```

will mail the message to these recipients:

```
FEINLER at host BBN-TENEXB
KLH at host MIT-AI
TK at host MIT-AI
MINSKY at host MIT-AI
BH at host SU-AI
PAPERT at host MIT-AI
```

Of course, mail to SU-AI is not actually sent via the ARPA network! The sticky host specifier "%AI" was not followed by a comma in the example above; the comma is optional following sticky hosts since the use of a sticky host implies that at least one destination will follow.

Not all network hosts are servers; that is, some hosts can originate network connections but do not accept connections originated from another host. Mail can only be sent to servers, and the MAIL program will refuse to try to mail to a host which is not known to be a server. We are supposed to be notified promptly of host status changes, so our host tables should be up to date most of the time, but if MAIL does not recognize a host you think it should, consult a system programmer. MAIL will always accept hosts specified by number.

The process of delivering network mail is rather slow; therefore, the default action is to queue the message for delivery by the reminder phantom job rather than to mail it directly from your own job. The switch /-QUEUE will cause immediate mailing. This switch is automatically assumed for very long messages (more than about 20,000 characters). A network SEND command is also done directly from your own job. Note: normally the first attempt to deliver the mail will be made immediately by the reminder phantom, but if the system is very crowded there may be no job slots available for the phantom until later.

If you specify /-QUEUE, MAIL will notify you of the progress of your message. Network mail may not be delivered successfully on the first try, because the other host or the network communications equipment may be down. Therefore, the result of a network mail attempt from your own job is reported with one of three messages:

```
USER at HOST -- ok
USER at HOST -- refused
USER at HOST -- queued
```

The first message means that the mail was sent and acknowledged by the other host. The second means that the network connection was established but the other host rejected the mail, for example, because it did not recognize the recipient as a user of that system. Usually the other host will send back a message explaining why the mail was refused, which the MAIL program types out. The third message means that the network connection could not be made or was interrupted. The message is queued like a reminder and is automatically retried at three-hour intervals for three days. (The first repeat attempt is made 15 minutes after the original failure.) You can use the RETRY command (see the section on the RETRY command) to retry your queued mail manually. You will be notified via MAIL and SEND of an unfavorable disposition of queued mail; the possibilities are

```
Mail to USER at HOST -- ok
Mail to USER at HOST -- refused
Mail to USER at HOST -- expired
```

The last of these means that the three-day limit ran out without a successful connection. You will get the "ok" message only if the remote host sends back a nonstandard reply message; normally you should simply assume that the message will be delivered unless you are notified otherwise. You can use the CANCEL command (page 130) to see if a message is still in the queue, or to delete it if you no longer want it sent. If the message is not sent successfully, the notification you receive will contain the text of your message, so you can direct it elsewhere if necessary.

The header format for network mail is different from that used for local mail. The section of this

document on message formats explains the network header. In particular, the distribution list (list of recipients of the message) is by default included in network mail but not local mail. The /NODIST switch will eliminate the list from network mail.

If you often communicate with users at other hosts, you may want to send mail to a group of people listed in a file which was prepared elsewhere. To make this easier, the switch /ARPA applied to a distribution-file destination causes the file to be scanned using the syntax of the SNDMSG program on TENEX systems. Specifically:

The character “@” is used instead of “%” to indicate network host names.

Any character may be used in a recipient name except space, carriage return, linefeed, tab, quote, colon, comma, and at-sign without quoting the name. Note in particular that semicolon is a valid recipient name character.

A recipient name ending with colon (distribution list name) is ignored completely.

There is an ARPA network protocol for implementing the SEND command to a remote host. However, most hosts do not implement it. Most notably, it works to the ITS systems at MIT and the TENEX systems at SRI. Certain other hosts also accept this protocol, but not very many. The MAIL program will attempt it to any server host, but it generally won't work. When a SEND fails to a remote host for any reason, the message is never queued for later delivery—its delivery is simply aborted.

A message sent to a remote destination by SEND or SEND/NOMAIL, but not by SEND/YESMAIL or SEND/MAIL, will have a special short header instead of the normal ARPA network message header. The short header looks like this:

```
BHeSU-AI 7/12/78 16:19
```

(Note that the switches can apply to a single destination, rather than globally as shown here.) If you use /YESMAIL or /MAIL, and the remote host does not recognize the network SEND protocol, MAIL will try again using the standard MAIL protocol.

A4.3 The MAIL Command

The MAIL command is used to send a message to one or more users, but can also be used to write a message in an arbitrary disk file. This command takes a list of destinations which determines where the message is to go. See the section above on destination lists.

Mail sent to a user will be seen when he next reads his message file, but a note saying that mail has arrived is typed out on the terminal of each destination user who is logged in at the time the mail is delivered. You can edit your message file with E by giving the monitor command ETV d (or ETV \MAIL).

A notice can be mailed to the system message file NOTICE.TXT[2,2] via the MAIL * command. Users see the system messages when they log in. Since a system message may only be relevant for a short time, an expiration date can be specified for such a notice. To do this, use the /DATE=<date>

switch; the message will be deleted at the midnight that begins the given date, and therefore the notice will last be seen on the previous day. The date must not be *wild*; that is, it cannot contain the asterisk (*) wildcard specifier. System messages specified without an expiration date will be deleted after 14 days. (The expiration facility is implemented by including the expiration date as its octal DAYCNT number in the message header. A program run every midnight deletes expired notices.) See also the EVENT command for putting notices about events in the system message file.

Sometimes mail cannot be delivered to a given user right away. In this case, the mail system queues the message for later delivery. Attempts to deliver queued local mail will be made every ten minutes for four hours, with the first such attempt occurring about 2 minutes after the original failure. Queued network mail is retried every 3 hours for three days, starting 15 minutes after the original failure. When the message is finally delivered, or after 24 delivery failures, the sender is notified via the mail system of the eventual outcome of his mail attempt. Mail will have to be queued if the destination user is located at another ARPA network host and that host is down, or if the destination user is at Stanford and is currently reading his mail file. There is a limit on the size of a message to be queued (currently about 40,000 characters). Queued mail is handled automatically by the mail system and the sender need never take any further action. However, the sender can unqueue any of his outgoing messages that have been queued; the monitor command CANCEL allows selective deletion of these queued mail requests (see page 130). Also, the sender can cause MAIL to retry sending his outgoing queued mail immediately by giving the RETRY command (see page 123).

Mail for local users is normally kept in files in the [2,2] directory. These files must have 000 protection so that anyone can write into them, since the MAIL command is not specially privileged. However, it is possible to create a mail file which is protected against other users. If there is a file called MSG.MSG[1,PRG] then mail to PRG will go to that file instead of the usual [2,2] file; if the file is protected, so that the MAIL command cannot write the message, then the request will be queued for delivery by the reminder phantom, which does have the necessary privileges. The phantom will use its privilege to override file protection only for a file named MSG.MSG, so you cannot queue mail to an arbitrary protected file.

The LOGOUT program will type and delete a file in your directory called LOGOUT.MSG if you have no other jobs logged in. This mechanism can be used to send yourself a reminder to be seen when you leave:

```
MAIL #LOGOUT Get some ice cream on the way home tonight.
```

The MAIL program provides for *mail forwarding*, primarily intended for users whose primary mailbox is on another ARPAnet host. The feature also allows for "pseudo-mailboxes" which are forwarded to one or more real destinations.

First we consider the forwarding of mail for a programmer name. As an example, programmer name GFF forwards to GEOFF%SRI-KA.

```
MAIL GFF This is a test.
Forwarding GFF to Geoff%SRI-KA
Exit
↑C
```

In this case, the forwarding entry has been set up to notify the user that forwarding is taking place. This is optional; usually it will be done for real programmers and not for pseudo-mailboxes. The use of a user's human name rather than programmer name works with forwarding:

```
MAIL GOODFELLOW message text
GFF is the ID for user Geoff Goodfellow
Forwarding GFF to Geoff%SRI-KA
Exit
↑C
```

Forwarding happens only for MAIL, not SEND!

```
SEND GFF Test message
Exit
↑C
```

But if the SEND command causes mailing, the mail is forwarded:

```
SEND GFF Test message
GFF not logged in, type Y to mail. Y
Forwarding GFF to Geoff%SRI-KA
Exit
↑C
```

Another possible use for forwarding is to affect the processing of one's mail. For example, if user ABC always logs in remotely and reads his/her mail with RCV rather than E, ABC might be forwarded to ABC/-E to tell MAIL not to make an E directory when creating a new message file for that user. This brings up the question of switches applied to forwarders. Most such switches are applied to all of the corresponding real destinations, e.g., MAIL GFF/DIST will be treated like MAIL GEOFF%SRI-KA/DIST. Also, for forwarders only, /-DIST implies /NODIST, since the mailer may not know whether the actual recipient is local (in which case /-DIST is applicable) or at a remote host (/NODIST). A few switches, which affect the format of local files, are ignored for forwarders, e.g., MAIL GFF/-E ignores the switch, because the mailer doesn't really know what file s/he's writing. These switches are rarely used anyway; don't worry about it.

It is possible for a person to be known to the system only for the purpose of mail forwarding, i.e., s/he will have a forwarding entry and perhaps an entry in the personnel list so that his/her human name will be known, but will have no file directories. Such a person is a *mail-only* destination; suppose that GIO is forwarded to WIEDERHOLD%SUMEX and has no file directories at SAIL:

```
SEND GIO Hello.
Mail-only destination GIO will be ignored for SEND.
Type Y to continue, using valid destinations:
Exit
↑C
```

(In this case, nothing will happen even if you type Y because the only destination is a mail-only one, and mailing was not specified.)

```
MAIL GIO Hello
Forwarding GIO to Wiederhold%SUMEX
Exit
↑C
```

MAIL to this destination works straightforwardly, to coin a phrase. There are some other possibilities:

```

SEND/MAIL GIO text
Mail-only destination GIO will be ignored for SEND.
Type Y to continue, using valid destinations: Y
Forwarding GIO to Wiederhold%SUMEX
Exit
↑C

```

Since the command asks for both SEND and MAIL (/MAIL), it does have something to do despite nobody to send to. But you are given the chance to abort, since saying SEND/MAIL might mean that you expected it to be able to send also. The command SEND/YESMAIL does not act like SEND/MAIL, but like SEND:

```

SEND/Y GIO text of message
Mail-only destination GIO will be ignored for SEND.
Type Y to continue, using valid destinations: Y
Exit
↑C

```

The *pseudo-mailbox* capability is useful mainly for incoming network mail, since local users can easily use the simpler @FILE distribution list facility. For example, there is a large mailing list called HEADER-PEOPLE at MIT which includes several SAIL users. When anything is mailed to that list, MIT must set up several separate network connections to SAIL, one for each recipient, each of which starts up a separate MAIL job. This can be made more efficient by creating a pseudo-mailbox SAIL-HEADER-PEOPLE here, containing all the SAIL people in HEADER-PEOPLE, and replacing them with one entry for this new mailbox. Pseudo-mailboxes are always mail-only destinations, i.e., SEND SAIL-HEADER-PEOPLE won't work.

The pseudo-mailbox name must be entered completely; there is no partial name recognition as for human users. Case of letters is ignored, however.

Incoming network mail will be accepted for any destination listed in the forwarding file; the remote sender will be notified about the forwarding if a local mailer would be. The network SEND protocol does NOT accept forwarding, even in the case of SEND/MAIL (XMAS command in FTP). For the network SEND protocol, it is just as if forwarding didn't exist.

Reminders may not be sent to pseudo-mailboxes. If a reminder is sent to a programmer name for which forwarding is in effect, the programmer name is given to the REMIND phantom as is; when the reminder is actually delivered, the message will be sent to the actual programmer name and mailed to the specified forwarding addresses, as in the SEND/MAIL command.

If you want to establish forwarding for yourself, or set up a pseudo-mailbox, use the command MAIL FORWARD with a message describing what you want.

A4.4 The SEND Command

The SEND command is used to have a message typed out on one or more terminals. This is usually for the purpose of sending a brief message to the terminals of selected users so that they will see it immediately. The message is preceded by a header telling who sent the message and when; see the section above on message formats. This command takes a list of destinations which determines where the message is to go. See the section above on destination lists.

A message can be sent to all logged in users via the SEND * command. The message will be typed out on the terminal of every logged in user.

Also, the command SEND ARPA* will send a message to all users who are logged in at Stanford via the ARPA network.

The SEND command also accepts decimal job numbers as destinations, or terminal names in the form TTYn.

A user can prevent messages from being typed at his or her terminal by using the TTY GAG command. In this case, the person trying to send the message will be notified. Note that a user may be logged in at more than one terminal, and may have received the message even if one terminal was gagged; the /WHERE switch will notify the sender of all recipient terminals, successful as well as unsuccessful. A SEND can fail even if the terminal is not gagged, e.g., if it is holding output. This also causes notification. You will also be notified, even if the message is sent, if the recipient terminal is a display whose page printer geometry is nonstandard and might cause the message to be lost because the page printer area is too small to hold it. However, if the currently selected piece of paper is nonstandard, but piece of paper number 0 is standard in its geometry, the warning will not be given, since messages are sent to the display's piece of paper 0 as well as the currently selected piece of paper.

There is an ARPA network protocol for implementing the SEND command to a remote host. However, most hosts do not implement it. Most notably, it works to the ITS systems at MIT and the TENEX systems at SRI. Certain other hosts also accept this protocol, but not very many. The MAIL program will attempt it to any server host, but it generally won't work. When a SEND fails to a remote host for any reason, the message is never queued for later delivery—its delivery is simply aborted.

A4.5 The REMIND Command

The REMIND command allows the delivery of a message to be delayed until a particular date and time, and/or to be repeated periodically. This command requires a date and/or time, which are interpreted as explained above in the section on dates and times.

Reminders are often sent to oneself rather than to another user. The character "." (period) as a destination means your own login programmer name. Also, if the first thing after the REMIND command (and global switches) is a digit, it is taken as the beginning of a date or time, and your programmer name will be used as an implicit destination:

```
REMIND 21-JUN KINKS CONCERT TONIGHT
```

Reminders are normally both mailed and sent at the specified date and time. However, either of the switches /MAIL and /SEND can be used with the REMIND command to cause the reminder to be only mailed or only sent. Nevertheless, if a repeated reminder is being sent for the last time because its expiration count has run out, it is always mailed even if /SEND was used in the command.

When a repeated reminder's expiration count runs out, a warning message to that effect is included in the text of the last reminder.

To selectively delete reminder requests, use the monitor command CANCEL (see page 130).

A4.6 The GRIPE Command

Complaints, compliments, or criticisms about any aspects of the system or of the A.I. Lab in general can be entered by use of the GRIPE command. A gripe can be either “vectored” or sent to the general miscellaneous gripes file. A gripe is vectored by including a *single-word* gripe topic on the command line, or if nothing follows the GRIPE command, in response to a prompt for the topic. Note that this single-word topic is not the same as, and does not replace, the subject line which may also be given. If more than one word follows the GRIPE command on the command line, the line will be taken as the text of the gripe, which will go to the general gripe file. Also, if the topic you give is not recognized as a gripe topic, your gripe will go to the general file. The command GRIPE ? will list the known gripe topics. Topics are used as pseudo-mailboxes for the mail forwarding system, to direct your gripe to the specific person responsible for the program or device you are having trouble with. The general gripe file can be typed out with the system program COPY by giving the monitor command TYPE \GRIPES (which may currently be abbreviated TY\G). This file can be examined and edited with the E editor by giving the monitor command ETV \GRIPES (currently abbreviatable as ET\G).

A4.7 The EVENT Command

Messages of interest to everyone can be sent to the system message file by use of the command MAIL *. However, for a message about an event occurring on a particular day, a more useful facility is provided by the EVENT command, which also enters the message in the system message file NOTICE.TXT[2,2], but which does so several times: first on the day the EVENT command is given, then three days before the event, then again on the day before the event, and finally on the day of the event. The EVENT command takes an explicit date and optional time in the formats explained in the section on dates and times. EVENT will then list the message given prefixed by a special line of the form:

Event of DAY DD-MON-YY at HH:MMxm

where DAY is the day of the week the event will occur on, DD-MON-YY is the day, month, and year of the event’s date, HH:MM is the event’s time, if given, and xm is either “am” or “pm” for the time. Thus, the date and time need not be included in the message text itself.

Three days before the event, the day before the event, and again on the day of the event, the event notice will be deleted and re-entered in the notice file so that users will see it again on those days. It is finally deleted at the midnight that ends the day of the event. If the event is on a Monday, the notice is re-entered on the preceding Friday, Saturday, Sunday, and the Monday of the event. If the event is on a Sunday, the notice is re-entered on Friday, Saturday, and the Sunday of the event. Events on other days are re-entered three days before the event, the day before the event, and the day of the event. (The re-entering of upcoming events is implemented by including more than one expiration date in DAYCNT format in the message header.)

If you want to change the text in an event notice, you can do a RCV * and edit the message text or time of the event. However, if the date of the event is wrong, you should delete the notice (with RCV) and make a new one (with EVENT) because the octal dates in the header determine what days the message will be seen, and the header cannot be edited.

A4.8 The PLAN Command

The PLAN command is used to create or delete a file describing your whereabouts, office schedule, or whatever, to be read by users who give the FINGER command to find you when you are not logged in. The PLAN command writes a plan file called " PRG".PLN[2,2], where PRG is your programmer name. Any previous plan file is replaced with a new one containing only the new message you give. If you enter a null plan, your plan file is deleted. The plan applies to your login programmer name.

The PLAN command requires an expiration date (but no time) as explained in the section on dates and times. Your plan file will be deleted at the midnight that begins the expiration date. (Plan file deletion is implemented by entering a LATER request (see page 123) to run the program DELPLN, which will delete your plan file. You can selectively delete this LATER request by using the monitor command CANCEL—see page 130).

Your plan file can be referenced with the COPY or E programs by using the \PLN filehack. For instance, the monitor command TYPE \PLN will type out your plan file, and DELETE \PLN will delete it. Also, the COPY/E message-file specifier, partial-sign (@), can be used to reference your own or someone else's plan file: TYPE @.PLN will type out your own plan file and TYPE @ME.PLN will type out the plan file belonging to programmer name ME.

A4.9 The RETRY Command

The RETRY command is used to retry all queued mail immediately. It takes no arguments at all in the command line. RETRY searches the remind queue for queued mail (not reminders) sent by or to your login programmer name and tries to send the mail. (This will be done automatically without using the command, but the command is provided for impatient mailers.) Please do not type <call> or ↑C after giving this command, or you may lose a queued message.

A4.10 The LATER Command

The REMIND command, in essence, allows the execution of a MAIL or SEND command to be delayed to a specified time. The LATER command generalizes this delayed execution to allow an arbitrary program (dump file) to be run at a later time. The command format is:

```
LATER dumpfile core datetime count
```

The LATER command does *not* accept switches and requires all of its arguments to be on the command line. If the proper syntax is not followed, a message is typed explaining the proper syntax and nothing else is done.

The defaults for the program file to be executed are device DSK, extension .DMP, and your alias PPN. If a device is explicitly specified, it must be DSK or SYS. No check is made when the command is given to ensure that the specified file exists or is a valid program dump file.

The optional core argument can be used to control the core size and starting address used to run the program. If this argument is not used, the program will be run in as much core as is needed to fit it, and will be started at its normal starting address. The core argument can be in any of these formats:

<nP> <+m> <-m> <nP, +m> <nP, -m> <+m, nP> <-m, nP>

The angle brackets are actually to be typed. The nP subargument specifies the number of 512-word pages of core to be assigned to the program; n is a decimal integer. The +m or -m subargument is a starting address offset, that is, m will be added to or subtracted from the program's normal start address. Note: m is an OCTAL integer. The core size may also be given in 1024-word blocks by saying nK instead of nP. One K is equal to two pages.

The date and/or time must be given in the non-switch format as explained in the section on dates and times. If both a date and a time are given, the date must appear first.

The optional count argument specifies the number of times the program is to be run and is ignored unless a wildcard date is used. If no explicit count is given when a wildcard date is used, the default count of 50 will be assumed. The format for the count field in the LATER command is

#n

where n is a decimal integer. A count of #0 or #∞ will cause the count never to run out.

When a program specified by a LATER command is actually run, it will have your login PPN as its login and alias PPNs. The only privilege it will have will be the Local User Privilege (LUP). Also, it is run as a phantom job—if it is stopped because of an error, the job is killed.

Certain accumulators (ACs) are set up when the program is started to communicate information from the remind phantom to your program:

AC	Contents
1	filename of program dump file (same as job's name)
2	extension of program dump file
3	PPN of program dump file--[1,3] if device SYS was specified
4	-1 if this LATER request will not be run again, else 0

The program can only be run if a job slot is available at the time for which it is scheduled. If the remind phantom is unable to run the program because no job slots are available, it does not try again later (unless, of course, the date and time specify repetition). If the system is so crowded that job slots are in short supply, users who are present in the flesh get priority over LATER requests. This is rarely a problem except for weekday afternoons. (If there is no job slot for the remind phantom itself, it will try to run your request once, as soon as it is run itself.)

LATER requests can be selectively deleted by using the monitor command CANCEL (see page 130).

A4.11 The BATCH Command

Sometimes you may want to delay the execution of one or more monitor commands. To allow this, the BATCH command accepts a command string and enters a request to run a program which enters the commands through a pseudo-teletype (PTY). The *message* given with the BATCH command is taken to be the command or sequence of commands which is to be typed to the PTY. The only switches permitted are /DATE, /TIME, /COUNT, /TLIMIT, /NOW, and /DO. The /COUNT switch specifies the number of times the command sequence is to be executed and is ignored unless there is a wildcard date. The /TLIMIT switch (the name /LIMIT is equivalent) is used to specify the number of CPU-minutes the batch job will be allowed to run. The /NOW switch causes BATCH to execute the command sequence as soon as you have finished entering it instead of later; in this case *no date or time must be given*. The /DO switch causes certain character conversion to be done when entering the commands, as in the DO program and as explained below.

Character	Effect in /DO mode
RETURN	ignored
LINE	ignored
↔	translated to RETURN followed by LINE
↓	translated to LINE
≠	translated to ALT
λ	translated to deferred CALL (one ↑C)
VT	adds CONTROL bit to following character
α	adds CONTROL bit to following character
FORM	adds META bit to following character
β	adds META bit to following character
⊗	translated to ESC
⊗-	translated to BREAK
≡	quote the following character (no translation)

Note that the characters | and ? which have special meanings in the DO program are not treated specially by the BATCH/DO command.

If the /NOW switch is *not* used in the BATCH command, BATCH writes a file on your alias disk area called BATn.DMP (n is a number chosen to make the name unique) and enters a LATER request for the running of that program. The BATn program types the desired commands into a pseudo-teletype (PTY) when it is run. The LATER request starts the BATn program at one more than its normal start address; if it is started normally (e.g., RUN BATn), it simply types out the commands stored in it, so you can find out which BATCH program is which if you enter more than one. When the BATn program is started by the LATER request, it writes a file called BATn.LOG on your login disk area with a log of the commands and typed output. The BATn.DMP file is deleted when the request is being run for the last time.

If BATCH/NOW is used, no BATn.DMP file is written. The MAIL program itself executes the command sequence through a PTY as soon as you have entered the whole command sequence. In this case, the log file is called BATCH.LOG and is written in your alias directory.

BATCH requests can be deleted selectively by using the monitor command CANCEL (see page 130). If you delete a BATCH request with CANCEL, the BATn.DMP file will not have been deleted; you must delete this file separately.

The BATCH command has a time limit feature, to ameliorate the problem of runaway batch jobs. The feature is controlled by the /LIMIT or /TLIMIT switch (the two names are equivalent) in the form

/TLIMIT=mins or /TLIMIT=hrs:mins

The switch is given with other batch switches, right after the command name. The form /TLIMIT=∞ may be used for an infinite limit. *The default time limit is one hour* for jobs submitted for later execution, and is *infinite* for jobs run immediately with the /NOW switch. (Presumably the latter case implies that the user is watching the job run and can interrupt it himself if necessary.)

The limit applies to the job's runtime (CPU time), not realtime. The BATCH controller takes clock interrupts every two (real) minutes to check the controlled job's runtime, so the time limit may in fact be slightly exceeded before the job is stopped. If the job runs over its time limit, it is logged out immediately; the log file will end with a line saying

? BATCH: time limit exceeded.

The BATCH command has many deficiencies when considered as a full batch processing facility. There is no way to control the commands issued on the basis of past output, no provision for aborting a command sequence if an error occurs, no provision for mounting tapes or collecting listings. The command is not intended as a real batch processing system, but merely as a convenience for delaying the execution of simple commands.

A4.12 The REENTER Error Recovery Facility

The MAIL program remembers all of its input after execution, and can be made to repeat the command with or without modifications to the command, switches, destinations, and message. To invoke this facility, type REENTER to the monitor.

To use this feature while typing in the message text, you can type ↑C and REENTER, but any text still in the line editor buffer or the TTY buffer when you type ↑C will be lost. Unless the system is very heavily loaded, the problem of losing text from the TTY buffer should not come up, but be sure to type <CR> to activate any text in the line editor before typing ↑C.

Two different modes of REENTER operation are provided. If the command and destination list were all typed on one line of input from a Stanford display terminal, that line is loaded into the terminal's line editor and can be edited and re-activated. In this case, the program forgets all about the previous command and starts from scratch when the edited command is seen. The text of a single-line message will be included in the reloaded line; no message text will be loaded for a multi-line message, but the old text is remembered and if the new command does not contain a message or a pointer to a message file, the old text will be re-used. Note: the subject line is considered part of the text, and the state of the /SUBJECT switch may not be changed if the old text is re-used in this way.

The second mode of repeating a command is used if the destination list was given on more than one line or from a non-display. In this case, the command and destination list are typed and you are asked if you want to keep the old destinations. Whether you answer yes or no, the command name and global switches are loaded into your line editor (displays only) and new destinations can be added on the command line. Finally, if no message text appears in the re-edited command, the old message is used as in the first mode. (Note: in this mode, only valid destinations are remembered from the old destination list; in the first mode what is remembered is the actual string of characters you typed.)

If the old text is re-used, in either mode, before sending the message the MAIL program asks if the user wants to edit the command or the message text with E. (Editing with E is allowed whether or not the user is at a Stanford display, although of course it's easier to do from a display. SOS editing is not available because SOS does not use the same SNAIL-startup conventions as E.) In this case, a MAIL\$E.TXT file is written and E is run as in the <CONTROL><META>E case—return to the MAIL program is done via the <CONTROL>XRUN command.

A4.13 Hand-Holding Mode

If a command (other than LATER and RETRY) is given with no arguments other than optional global switches, the MAIL program will use an alternate syntax called *hand-holding mode* which is more verbose but less confusing to an inexperienced user. In this mode, MAIL will prompt separately for each piece of information appropriate to the given command; the possible prompts include

- "To" destinations
- "CC" destinations
- Date and time
- Expiration count
- Subject
- Message text

One possibly important difference between this and the normal syntax is that the subject line is always entered separately from the body of the message, even if the latter comes from a file.

Each prompt may be answered by a line containing just the character "?" to see a message explaining the format of the required information for that prompt.

A REENTER command following the use of this format will be treated the same as one following a multi-line destination list in the normal format.

A4.14 Interfacing to MAIL from Other Programs

The MAIL program can be run from another program using the SWAP UUC. A special facility is provided for providing MAIL with the required arguments via the ACs which are preserved by the SWAP UUC. To do this, start SYS:MAIL.DMP at one *less* than its normal starting address.

The message text (including subject line, if desired) must first be written in a disk file, or a TMPCOR file if the MAIL program is to be run in the same job as the swapping program. MAIL will, if desired, SWAP back to the originating program (or any other program). The ACs should be set up this way:

- 0-5 SWAP UUD arguments for return after MAIL if desired:
 - 0 device
 - 1 filename
 - 2 ext,,mode bits (see UUD manual)
 - 3 core,,starting address increment
 - 4 file PPN
 - 5 login PPN if starting new job
- 6-16 MAIL arguments:
 - 6 destination PPN or file (see below)
 - 7 file extension if required
 - 10 file PPN if required
 - 11 message text file name
 - 12 message text file extension
 - 13 unused
 - 14 message text file PPN
 - 15 date and time for REMIND (see below)
 - 16 flags (see below)
- 17 is the SWAP AC and is not preserved in return after MAIL.

The flags in 16 are:

- 400000,,0 not used
- 200000,,0 /-E
- 100000,,0 /NODIST for ARPAnet mail
- 40000,,0 /LIST
- 20000,,0 /SEND
- 10000,,0 /-HEADER
- 4000,,0 /-QUEUE
- 2000,,0 /APPEND
- 1000,,0 /DIST for local mail
- 400,,0 /WHERE
- 200,,0 /SUBJECT
- 100,,0 /MAIL
- 40,,0 /YESMAIL
- 20,,0 /NOMAIL
- 10,,0 /EXPAND
- 4,,0 command is REMIND
- 2,,0 command is MAIL
- 1,,0 command is SEND
- 770000 not used
- 4000 /-OUTGO
- 2000 read destinations from named file (ACs 6-10)
- 1000 return via SWAP when done
- 400 text input is from TMPCOR file
- 200 delete input text file when done
- 100 send to job (binary number in AC 6)
- 40 send to TTY (SIXBIT name in AC 6)
- 20 destination is * or ARPA* (AC 6 ignored)
- 10 mail to explicitly named file (ACs 6-10)
- 4 don't use this bit (/CC)
- 2 /ARPA (2000 bit must also be on)
- 1 if SEND, send to ARPA* (20 bit must also be on)
if MAIL, delete old contents of file
(for PLAN, see below)

Nothing is guaranteed about the results of setting inconsistent or irrelevant combinations of flags, e.g., /EXPAND for a command other than REMIND. Note that the /MAIL switch has two different meanings for the SEND command and the REMIND command, but is the same bit in either case. Bits which are currently unused should be zero.

If the 400 bit in AC 16 is on, the left half of AC 11 is taken as the TMPCOR file name; the TMPCOR file PPN is the alias PPN of the MAIL job. The 200 bit in AC 16 will delete either a DSK file or a TMPCOR file.

The usual case is sending a message to a programmer name. For this case, AC 6 should contain zero in the left half and the sixbit programmer name right-justified in the right half. Do not use "." for your own programmer name (use the programmer name explicitly) nor "*" for a wildcard destination (use 20 bit in AC 16).

Mail can be sent to any file (as in the #file destination format) by setting the 0,,10 bit in AC 16 and specifying the filename in ACs 6-10. More than one destination, or a recipient at another ARPA network host, must be specified by using an indirect file as in the @file destination format; set the 0,,2000 bit in AC 16 and specify the file with the destination list in ACs 6-10.

Any files used (except the TMPCOR message file) must be on device DSK. No default extensions will be used; specify the desired extension explicitly. (The right half of an extension word must be 0.) A zero file PPN uses the job's alias PPN as usual.

Exactly one of the 7,,0 bits should be on in AC 16 to specify the command to be carried out. The effect of the PLAN command is achieved by specifying a MAIL command with the delete-old-text bit and explicit file destination (2,,11 in AC 16) and " prg.PLN[2,2]" as the destination filename in ACs 6-10.

The date and time in AC 15 is in the format:

```
BYTE (4)MONTH (5)DAY (6)YEAR (3)DAY-OF-WEEK (6)HRS,MINS,FLAGS
```

The flags are used for special formats:

```
0,,1    absolute date, DAYCNT in left half
0,,2    date is tomorrow (lh is zero)
0,,4    every day reminder (lh is zero)
```

Otherwise, zero in the left half means today's date, a nonzero number in the 7,,0 bits is an every-week reminder (1=Wed, 7=Tues), and anything else is an every-month or every-year reminder with at least one of the month and year fields zero. Month 1 is January, day 1 is the first of the month, and year 1 is 1965. The HRS field is hours past midnight (0-23) and the MINS field minutes past the hour (0-59). An absolute date (no wild fields) must be in DAYCNT form.

Another facility is also provided for queuing MAIL requests when the SWAP technique is inappropriate, for example because another job slot might not be available and your program must keep itself going rather than be swapped back in later. This facility was designed for the FTP server but can be used by other programs. Write a file called <anything>.FTP[RMD,SYS] which contains, in text form, a MAIL command and destination list on the first page, and the message text on the second page. (This is the format of the MAIL\$E.TXT file used for editing messages with E.) When the remind phantom is next run, it will try to start a MAIL job which will execute the command and delete the file.

The normal SNAIL startup procedure (starting MAIL at one more than its normal start address) is used by E both to return to MAIL after you have used E to edit your input text file and in response to the MAIL command in E. This entry is not normally used except by E.

A4.15 The CANCEL Command

The monitor command CANCEL will run the CANCEL program which will list, and allow selective deletion of, REMIND requests from or to you, BATCH and LATER requests by you, and queued mail from you. This program can also be used to list these requests without asking whether you want to delete them: after any request is listed, type the letter L in answer to the deletion question and any remaining requests will be listed without interruption. Since the CANCEL program has to share the use of the reminder queue files with the system reminder phantom job, there will sometimes be a delay in the listing of reminders. An appropriate message is typed in this case.

A4.16 The RCV Command

The RCV program allows MAIL messages to be selectively deleted, listed, or saved as desired. The program is called by simply entering the monitor command RCV. All mail files which can be construed as being for you are offered for your perusal. It is also possible to edit other users' mail files with RCV by giving an argument, e.g., RCV BH. The command RCV * will allow editing of the system message file NOTICE.TXT[2,2], and RCV GRIPE will allow editing of messages sent by the GRIPE command. *Most mail files are maintained in E format, and E format is not preserved when such a file is edited with RCV—therefore:*

```
*** NOTE: IF YOU EDIT AN E-FORMATTED FILE, RCV WILL INVALIDATE ***
*** THE DIRECTORY PAGE AND ISSUE A WARNING. SUCH FILES SHOULD ***
*** USUALLY BE EDITED WITH E TO PRESERVE THE PAGES AND FORMAT. ***
```

RCV also allows editing of the files created by the news service automatic notification feature; the command RCV \ will present your notifications for editing, and RCV \PRG will allow you to edit the notifications sent to user PRG.

It is also possible to use RCV on files not in [2,2] (such as saved message files previously created with RCV) with a command like:

```
RCV #filename
```

with default of DSK:SAVED.MSG in your (alias) disk area. The conventions given below for output filenames also apply here.

Finally, the command RCV ?PRG will type the plan file for user PRG, if there is such a file. In this case, the file is merely typed, not edited.

*** NOTE: ONCE RCV HAS OPENED A MESSAGE FILE, YOU SHOULD EXIT ONLY ***
 *** BY TYPING "E" TO AN OPTION REQUEST, AS EXPLAINED BELOW. TYPING ***
 *** <CALL> TO EXIT MAY RESULT IN LOSING SOME OF YOUR MESSAGE TEXT! ***

If none of the messages in an input file is deleted or changed, RCV does not write out a new version of the file; thus the file's time and date written are unchanged, and, particularly, any ETV directory the file may have had is still intact.

RCV deletes without comment all formfeed characters in any file edited (that is, any file actually changed). Thus any page structure of such a file is destroyed by RCV.

When a mail file is opened, messages are typed out one at a time, and for each message you may select several processing options, in the format

```
<options> [ <space> <filename> ] <cr>
```

where the [...] represents an optional argument. Each option is represented by a letter; more than one may be used, except that a few options must be used alone, as noted below. The filename may be specified if you select the C (copy) or T (transfer) option. For help in specifying a file, use ? as the filename. Here are the option letters which select a final disposition for the message:

```
S  Save the message in the mail file.
D  Delete it from the mail file.
C  Copy it to a file of your choice.
T  Transfer it to a file. Like CD.
L  LPT spool it.
X  XGP spool it.
K  Transfer it to the file LOGOUT.MSG, which will be typed automatically when you log out.
```

If none of the above are used, the message will be Saved. Combinations may be used, e.g., LD will spool the message on the LPT and also delete it from the mail file. The message will be Saved unless D or T is explicitly specified. Along with any of the above (or alone, implying S) may be used at most one of the following editing options:

```
A  Append to the message from the terminal.
Z  (Stanford displays only) Edit the message line-by-line using the system line editor. If you need
    help with this facility, type <control><meta>? after entering the line editing mode.
M  (Stanford displays only) Modify the message by running the text editor, E, with that message as
    its input. Return from E by typing <control>X RUN.
```

Along with any of the above, or alone, can be used the following option letters to avoid seeing more messages:

```
E  Exit, and enter spooling requests from L or X options.
N  Go to the Next mail file, if any, else exit like E.
```

If the message was longer than the amount RCV types out before asking for option requests, the following option may be used along with any others specified:

```
Q  Quiet processing; do not type the remainder of the message while processing it as usual.
```

Note: E or N with no other options implies Q automatically. Alternatively, any of the following may be used as the only option, for special processing:

- P Postpone the decision for this message. This applies only if the message was longer than the amount RCV types before asking for options, and you wish to see the rest of the message before choosing options.
- ? Type this option list.

Spooling requested by L or X is not done until RCV is exited (by typing E to an option request, or by running out of input). The default filename for C and T is DSK:SAVED.MSG in your alias area. Filenames may only be given along with a C or T option; if C or T is used without a filename the previous file is continued. (The first time, you will be asked for a filename, which may be simply <cr> for SAVED.MSG.)

If you use an argument in the RCV command to edit another user's mail, the first time you specify any option which would remove a message from the mail file or alter its contents you are asked to confirm it. Once you have confirmed such an option, however, you are not asked again.

There are two options provided at Stanford displays for editing the text of a message. The M option writes the message in a disk file, QQRCV.TMP, and runs the E editor, which allows great flexibility because of E's powerful editing capability. However, it is rather slow, because all of RCV's internal information and all the message files must be saved on the disk. For minor corrections to a short message, you may prefer the weaker but faster editing capability provided in RCV itself by the Z option. If you select that option, the lines in your message will be presented to you one by one in your terminal's line editor. You may edit each line, using the normal line editor commands, and type <cr> when done with a line. You may also type the following special characters (α means control, β means meta):

α <cr>	Accept the current line as it now appears in your line editor buffer and stop line editing, accepting the rest of the message as is.
α β D	Delete the current line.
<alt>	Undo the changes in this line, loading a fresh copy into the line editor.
α β <cr>	Accept lines to be inserted before the current line, until an inserted line is terminated with α <cr> instead of <cr>, or <alt> is typed at a blank line.
α β I	Same as α β <cr>.
α D	(at the end of a line) Combine this and the next line and load the combined line into the line editor.
β <cr>	Break the line at the cursor, accepting the text to the left of the cursor as it stands and editing the remaining text as a new line.
α β A	Leave line edit mode, as for α <cr>, but accept text from the terminal to be appended after the existing text, as if the A option had been selected.
α β ?	Type this list.

Blank lines presented for editing in the Z option are indicated by the typing out of a space before loading the line editor with the empty line; thus originally empty lines will be indented in the typeout. In both M and Z options, the initial header line is not edited. In the Z option, the blank line at the end of the message is not edited. In the A option, appended text will be inserted in front of the blank line at the end of the message, unless the P option was previously used on the current message.

The first time you specify C or T, you are asked to specify a file name if one was not included in the option line. You can reply with either a standard filename, ?, or <cr>. Just <cr> uses the default file name, DSK:SAVED.MSG in your (alias) area. Just ? will print a helpful message and let you try again. Thereafter, whatever file you specified will be used for all C and T messages (the file is not closed until you exit or select another one) until you specify a new filename along with a C or T option as described above. Note: if the specified file already exists and is in E format, the new messages are added at the end of the file preceded by a single formfeed; if the file already exists but is not in E format, the new messages are added at the beginning of the file. If the file does not

already exist, a warning that you are creating a new file is issued; the file will be created in E format unless two conditions are met: if the output file is not in the [2,2] directory, and the input file is in non-E format, then the output file will also be in non-E format.

If all messages are removed from an input file, the file is deleted after all output files are closed, unless the input file is named OUTGO.MSG in which case the empty file (which may actually contain a few spaces, carriage returns, and linefeeds) will survive. If none of the messages in an input file is deleted or changed, RCV does not write out a new version of the file; thus the file's time and date written are unchanged, and, particularly, any ETV directory the file may have had is still intact.

The notation ↓chars↓ may be used to get non-alphameric characters in a filename. In addition, a shorthand notation is provided for entering names of mail files: if the character ∂ is the first character of the filename, then the device becomes DSK, the default extension .MSG, and the default ppn [2,2]. (The extension and ppn can be changed later in the filename.) If the next non-blank character is not alphameric, the filename used is your programmer name right justified, e.g., ↓ REG↓. (Note that this is your own login name, not the name whose mail you are editing.) Thus, the filename ∂ represents your own mail file. You can specify a different programmer name with the format ∂prg and either this or the format ∂ can also be optionally followed by an extension and/or a PPN to change the default values as described above, e.g., ∂PUR or ∂[1,ME].

RCV has a fairly small message buffer. If a message is longer than that buffer, seeing the first few lines of the message may tell you enough about it to decide how to process it without seeing the rest. Therefore, in that case, you will see as much as fits, followed by an overflow notice and an option request. After you enter the desired options, the remainder of the message will be typed as it is being processed, so you may safely use the Delete option and you will still see the rest of the message. If you want to avoid this continued typeout, type the letter Q (for Quiet) before your option choice, e.g., QD for Quiet Delete. Q is only recognized when a message overflows. If you select E or N with no other options, you get Q automatically. If this is not what you want, type, e.g., SE. If you wish, you may postpone the decision on how to process the message until after seeing the rest of it, by typing P to the option request. This will type the remainder of the message and ask again for options. QP is illegal, and P is only legal when a message overflows.

RCV may be used when not logged in; however, each user can control access to his own mail, by OPTION.TXT options. Three possibilities are available. The default is no access. To allow your mail to be read but not modified, put

```
RCV:READ
```

in your OPTION.TXT file. If the not-logged-in user types RCV PRG the file examined is OPTION.TXT[1,PRG]. You may also allow not-logged-in users to edit your mail file as if they were logged in by using

```
RCV:WRITE
```

as the option.

Not-logged-in users at Stanford display terminals (Data Disc or III) are always given at least READ access to mail, providing the message file is not protected against RCV.

It should be noted that the normal file protection mechanism still applies, so that most options

which involve copying the message to another file will not work when not logged in. This includes C, T, K, L, X, M, and P.

A not-logged-in user who is reading mail as permitted by the READ option may safely use <call> to quit RCV.

There is a special option available in RCV for reading the A.P. news digest. This option is provided when the RCV command is given without an argument, if you have an OPTION.TXT file with a line of the form

```
RCV:DIGEST;
```

You are asked "Read A.P. digest?" if the digest file exists. This option may be used along with READ or WRITE as described earlier.

Another special option available through the OPTION.TXT file is:

```
RCV:NOMAIL;
```

If you have this option and give the RCV command without an argument, RCV ignores your main message file and simply processes any other files which it would normally handle. This is provided for users of various special RCV features (such as DIGEST above) who use E to edit their mail files since mail files are now normally in E format. To override the NOMAIL option (i.e., to edit your mail file with RCV), give your programmer name explicitly in the command, e.g., RCV PRG.

For use at non-Stanford terminals, RCV has a mode in which it accepts tty input in SOS representation (see Appendix 13). The main reason for this is to allow ?* instead of ∂ for redirecting messages to another mail file, but it can also be used with the append option to allow appending upper/lower case text from an upper-case-only terminal. The following notes apply:

1. When RCV is started, it decides whether or not to use SOS representation on the basis of the FCS (full character set) bit maintained for each terminal by the monitor. This bit is set by the TTY FULL command (or ESC F at a Stanford display) and cleared by TTY NO FULL (or BREAK F).
2. Whenever RCV reads an altmode character from the tty, it will enter SOS mode and otherwise ignore the altmode, except that if you are already in SOS mode the sequence ?<alt> will leave that mode. (If you are not in SOS mode, of course, the ? will be taken as a real question mark and the <alt> will enter SOS mode!)
3. These comments apply even to the original command line, although the monitor will echo a crlf after the altmode to confuse you. Note that if your FCS bit is off and you want to invoke the RCV feature which types a user's plan file, you must say RCV ??prg instead of RCV ?prg as usual.
4. Under no circumstances is tty *output* done via SOS conversion. I hate seeing all those question marks!
5. If you are in SOS mode and type ? followed by a character which is not defined as an SOS code, the character will be treated, and echoed, as ASCII 7; this will ring your tty's bell if it has one, give an error message if you are responding to an option request, and insert a π in your message if you are appending.

APPENDIX 5

DART

The DART (Dump and Restore Technique) program is used to save disk files on magnetic tape. It also includes tape positioning commands. This appendix is excerpted from the file DART.REG[UP,DOC], which includes information on DART tape formats, complete disk dumps, etc., as well as the following facilities provided for individual users.

In the description below, braces, { and }, are used to denote optional items. Vertical bar, |, denotes an exclusive-or choice. Pointed brackets, < and >, are used to enclose syntactic items that are defined below.

DART accepts the following monitor commands:

```
DUMP      {<dest>+} {<source>}
RESTORE   {<dest>+} {<source>}
PUMPKIN   {<dest>+} {<source>}
REWIND     {<dev>{:}}
EOT        {<dev>{:}}
ADVANCE    {FILE|RECORD} {<dev>{:}} {<count>}
BACKSPACE  {FILE|RECORD} {<dev>{:}} {<count>}
LOCATE     {<dest>+} {<source>}
TLIST      {<dest>+} {<source>}
TURKEY     P<number>|T<number>

<dest> ::=      {<dev>:}{<file>{.<ext>}{[[<prj>,<prg>]]}
<source> ::=     {<dev>:}{[[<prj>,<prg>]]}@{
                  {<dev>:}{<file>{.<ext>}{[[<prj>,<prg>]]}{,<source>}}

<dev> is one of DSK, MTAn, or UDPh.
<file> is any file name or *
<ext> is any file extension or *
<prj> is any project code or *
<prg> is any programmer name or *
<count> is any string of decimal digits.
```

REWIND

This command will cause the device named to rewind to load point. MTA0 is the default if no device argument is used. The device named should be a magnetic tape unit.

EOT

This command will cause ADVANCE FILE to be repeated until either two adjacent file marks are seen (logical end of tape) or until physical end of tape is reached.

ADVANCE and ADVANCE FILE

This command will cause the tape to advance past the next file mark on the tape. If a repeat factor is given, then the command will be repeated that number of times.

Caution: DART often records more than one disk file on a mag tape file! Therefore, advance file will (sometimes) skip more than one disk file.

ADVANCE RECORD

Same as ADVANCE FILE except that instead of file marks, record marks are used. This command leaves you at the front of a record.

BACKSPACE and BACKSPACE FILE

This command will cause the tape to move backwards until a file mark is seen. DART then does one ADVANCE FILE operation to position the tape at the front of a file (immediately after the file mark just read). If a repeat argument is given, then that argument is used to repeat the backspace operation. Only one ADVANCE is used, after all backspace operations.

Caution: BACKSPACE or BACKSPACE 1 will position the tape at the front of the current mag tape file. BACKSPACE 2 will position one previous, etc.

BACKSPACE RECORD

Same as BACKSPACE FILE except that record marks are used to stop the operation instead of file marks. After all backspaces are completed, DART does an ADVANCE RECORD command.

LOCATE

For each file named in the argument list, this command prints the tape numbers where this file was dumped and the corresponding creation dates of the file.

TLIST

This command will list on the destination device the names of the files that are read from the source device.

TURKEY

This command takes as its argument the number of a DART system dump tape, and tells you who last used it. It is to find out whose thrug to squuk if you find a DART tape lying around not where it belongs.

DUMP

This command will write on the destination device, which should be a magtape (default is MTA0), those files that are specified by the source term. If there is no source argument, **[current area] is used.

RESTORE

This command will restore from tape to the destination the files that are described by the source term. A null source or destination means **[current area]. The command RESTORE [*,REG]←[*,REG] will restore all of REG's files to the areas that they were dumped from. Note that if any * terms are specified or implied in the source, the entire tape will be searched for matching terms.

PUMPKIN

This command is just like RESTORE except that instead of restoring files from tape immediately, it queues a request with the Great Pumpkin, who comes by night to restore your files. It will restore from the most recent system dump tape containing the file(s), unless you follow the command name with a switch in the form /P781 or /T31 to request a specific DART tape, or of the form /4-Oct-78 or 10/4/78 to request a version written on that date or earlier. (This would be useful if a file has been overwritten and dumped, and you want to restore an earlier version.) Don't specify a source device with this command.

APPENDIX 6

SERVICE LEVEL SYSTEM (RSL)

The RSL monitor command runs the RSL program to reserve service level. The monitor command takes no arguments; the RSL program itself accepts commands which control its operation. The first section of this appendix explains the concept of service level.

The timesharing scheduler gives different priorities to each of three user classes: interactive users, reserved users, and all others (scavengers). The first goal of the scheduler is to provide good service to anyone doing interactive work such as editing. The system decides whether a given job is currently interactive on the basis of keyboard input activity.

When you log in at a time for which you have made a service level reservation, you are assigned that service level. Whenever you are running, the scheduler will then attempt to give you a processing level (% of CPU time) given by

$$PL = SL - C * (B - 1) / 5$$

where SL is the service level, C is your current core size (in pages), and B is the price of service level. The system will not let more than a certain total service level to be allocated in this way (currently 80%).

If you are neither interactive nor reserved, you get some of what is left, which may be pitifully little. The system is supposedly rigged so that reserved users always get service at least as good as scavengers. Note that if you are editing and execute a long-running string search, the system may decide you are a scavenger and take forever to finish.

Each authorized user has an allowance of two kinds of money, called *whams* and *bams*. Whams may be used only to purchase reservations for peripheral devices, including III terminals, while bams are good only for CPU service. The allocations are as follows.

	Whams	Bams
Half time user	4	100
Full time user	8	200
Panic user	16	400

You are *half time* or *full time* in accordance with the portion of your time devoted to A.I. activities. In an emergency situation, you may be given a short term appointment as a Panic user.

Your "money" allocations are actually revolving funds in that if you reserve machine time and use it, you may then re-use the reservation money. In effect, then, there is a limitation on how much you can reserve at a given time. There is no conversion between whams and bams, nor can funds be transferred between persons.

The display costs W (whams/hour) vary with time of day as given in the table below. The price P (bams/hour) of buying any given service level SL (in percent) is

$$P = SL * B$$

where B is the rate (bams/hour) given in the same table.

Time	0000-0900	0900-1300	1300-1800	1800-2400
	W & B	W & B	W & B	W & B
Monday-Friday	1	2	3	2
Sat., Sun., holidays	1	1	2	1

You may reserve a display without a service level or vice versa, but reservations may be made only for integral hours beginning on the hour. Other peripheral devices may be reserved only if you have reserved a service level. The minimum service level purchase is 5%.

The time from 8AM to 9AM and from 5PM to 7PM on weekdays is reserved for maintenance. No service level or device reservations may be made in those time periods until 18 hours before the period. Maintenance reservations for the entire machine may be made, however.

Using the RSL Program

The monitor command RSL with no arguments runs the RSL program. You must be logged in first.

RSL will prompt you with an asterisk (“*”) when it’s ready to accept commands. Command format is discussed in detail below. Commands are RESERVE, CALENDAR, AVAILABLE, DISPLAY, MAINTENANCE, HELP, and EXIT.

For example, to reserve 15% SL at 1400 on the 31st of December, any of the following command strings will serve:

```
RESERVE 15 SL AT 1400 HOURS ON 31-DEC
r 15 on 31 decemb at 2 p.m.
RE 2 PM ON 31 DECEMBER 15
R 15 14 31 12
```

If you wish to change a service level reservation, just do another reservation for the new desired amount. Releasing a service level reservation may be done by “reserving” 0% SL.

IMPORTANT!

When you are through making reservations, you should exit from RSL by typing E. If you exit instead by typing CALL, then there is a possibility that your reservations may get lost. The E command causes reservations to be written out onto the disk.

How to Reserve Devices

The six III displays, the user disk pack, plotter, and MTA0 may also be reserved. RSL merely makes and retains device reservations; unlike service level, enforcement is left up to the users. To reserve III24, 5% service level, and the user disk pack, type RESERVE 5 SL III 24 UDP or R III4UDP5. A device may be unreserved by preceding it with a minus sign: R -III PLTR -UDP MTA will simultaneously reserve MTA0 and the plotter, and unreserve the UDP and any and all IIIs. If you don't care which III you get, you may say III instead of III4 or III23 etc. Note that the plotter is considered by the computer to be the same device as the paper tape punch, so reserving PLTR also makes the PTP unavailable.

Other Commands Available

To get a list of your current reservations, type DISPLAY or D for short. The command string D XYZ will get a list of programmer XYZ's current reservations.

The CALENDAR command will show you which programmers have what reservations on a given time and date. For example, C FOR 3 will show you the reservations for the current hour and the following 2 hours also.

AVAILABLE is a short form of CALENDAR. A 0000 HOURS FOR 24 ON 1-JAN-88 will yield 24 lines of output telling how much service level is still available at each time slot on the first day of 1988, should the system last so long.

HELP or H will type for you a summary of command string structure in condensed form, not necessarily comprehensible.

The MAINTENANCE or M command reserves the entire (bare) machine for hardware or software purposes, e.g., M 4 pm for 2 will request the machine for maintenance from 4 to 6 pm. This command can also be used to cancel a bare machine reservation. Just type a minus sign immediately in front of the date specification, e.g. M -25 nov 73 for 2. Do not use this command without prior administrative approval.

Proper use of the EXIT command can be very important, and requires an understanding of its function. The data base for all current and future reservations is kept internally in the *-SL-* program at all times. Many of the commands you give to RSL cause a change in this data base. However, these changes are not necessarily made in the permanent data base, which lives on the disk. The EXIT (or E) command causes the internal data base to be written out on the disk, and it is only this copy which can survive system crashes or system maintenance. *It is possible to lose reservations by leaving RSL with CALL instead of E.*

RSL Command Syntax

Anything in curly brackets is optional.

A slash ("/") between two terms represents one or more delimiters (space, slash, apostrophe, minus sign, comma).

A double arrow ("↔") between two terms indicates that the order of the terms is not always important. If the syntax of a term distinguishes it, then it may appear anywhere in the command line, provided no nondistinguishable term which might be confused with it precedes it. (It's simpler than it sounds.)

```

<RSL command> ≡ <Rcom>|<Acom>|<Ccom>|<Dcom>|<Hcom>|<Mcom>|<Ecom>
<Rcom> ≡ RESERVE {<sl-arg>} ↔ {<hr-arg>} ↔ {<date-arg>} ↔ {<for-arg>} ↔ {<device>}
<Acom> ≡ AVAILABLE {<hr-arg>} ↔ {<date-arg>} ↔ {<for-arg>}
<Ccom> ≡ CALENDAR {<hr-arg>} ↔ {<date-arg>} ↔ {<for-arg>}
<Dcom> ≡ DISPLAY {<programmer initials>}
<Hcom> ≡ HELP
<Mcom> ≡ MAINTENANCE {<hr-arg>} ↔ {<date-arg>} ↔ {<for-arg>}
<Ecom> ≡ EXIT
<sl-arg> ≡ <integer> | <integer> SL
<hr-arg> ≡ {AT} <military hour> | {AT} <civilian hour>
<civilian hour> ≡ [ 1 | 2 | ... | 12 ] { AM | A.M. | PM | P.M. }
<military hour> ≡ [ 0 | 1 | ... | 23 ] {00} {HRS|HOURS}
<date-arg> ≡ {ON} <people day> | {ON} <day-arg>/<month-arg>{/<year-arg>}
<people day> ≡ SUNDAY | MONDAY | ... | SATURDAY
<day-arg> ≡ 0 | 1 | 2 | ... | 31
<month-arg> ≡ <people month> | <computer month>
<people month> ≡ JANUARY | FEBRUARY | ... | DECEMBER
<computer month> ≡ 1 | 2 | ... | 12
<year arg> ≡ {19}71 | {19}72 | ... | {19}80
<for-arg> ≡ <integer> | FOR <integer>
<device> ≡ MTA | PLTR | UDP | <iii>
<iii> ≡ III | III0 | III1 | ... | III5 | III20 | III21 | ... | III25

```

Semantics

Names of commands, weekdays and months may be abbreviated by truncation, and may be capitalized or not. Device names may not be abbreviated.

The FOR term (e.g., FOR 3) specifies a consecutive number of hours, starting with the hour and date specified.

When the syntax of a command does not allow a left-to-right scanner to distinguish terms, then the ambiguous terms will be assumed to be in the order listed in the syntax.

If a term is missing from an argument, RSL will supply one it deems appropriate. RSL always has a time and date "in mind," called the default date. The default time and date are usually the last time and date you typed. If a command does not specify a time (date), then the default time (date) will be used.

Commands terminated with a line feed (altmode) will advance (backup) the default time and date before executing the command. The amount of advance (backup) is 1 hour, 1 day, 1 week, 1 month depending as plain, control, meta, or control-meta were used with the line feed (altmode).

A date specified by giving the name of a day, is the earliest future or present date falling on the given weekday. (Notice that at 0830 on Wednesday, "WEDNESDAY" specifies either the current date, or the date one week hence, depending on the hour specification.)

APPENDIX 7**FIXIMLAC**

The FIXIMLAC command is used from IMLAC terminals to reload the program which controls the terminal into its minicomputer. The command runs a program on the PDP-10, and may be used without logging in first. The use of the command is shown below as part of the startup procedure.

IMLAC RELOADING PROCEDURE**IF THE IMLAC IS DEAD:**

1. SHIFT LOCK off (up).
2. Start at 40 (Push STOP, then hold the AT 40 key while pushing START).
3. Type C (upper case) several times to send ↑C's.
4. Type fiximlM (upper case M goes out as a <CR>).
5. Wait about 10 seconds to give the system time to log you in.
6. Hit <CONTROL><TOP><SHIFT>S.

IF THE IMLAC IS ALIVE:

1. At monitor level, type FIXIML.
2. When the program puts out a *, type
 <CONTROL><TOP>B (Sends the IMLAC to its loader)
 <CONTROL><TOP><SHIFT>S (Starts the loader)

APPENDIX 8

ARPA NETWORK

The ARPA network is a facility organized by the Advanced Research Projects Agency of the Department of Defense to connect computers at various research centers funded by ARPA, allowing people at one host to use the resources of another host. The device which provides the interface between our computer and the network, called an IMP (*Interface Message Processor*), can be used by user programs like any other I/O device. Two main system programs are provided for connecting to other computers by console commands: the user TELNET program and the File Transfer Protocol program (FTP). The former allows you to use your terminal as if it were a terminal of the remote host computer; the latter provides high-speed transmission of data between hosts. This appendix explains the use of these programs. It contains excerpts from the file TELNET.MRC[UP,DOC]. The FTP command description here is taken from FTP.DCS[UP,DOC].

A network connection typically involves a program at both hosts. For example, to make a TELNET connection to another host, you run the user TELNET program here. This program establishes a connection with the server TELNET program at the other host. We provide user and server programs for several protocols besides the main ones described here; for example, the monitor commands WHO, TALK, and FINGER can function as network user programs for special protocols when directed to another host.

The TELNET or TN commands, which are identical in effect, are used to run TELNET. The FTP command runs FTP. There is also an OTN command, which runs a version of TELNET using the "old" protocol; this is now largely obsolete. The DTN command runs a special version of TELNET which simulates, to the remote host, a Datamedia 2500 terminal, so that you can use display programs on the other system. Finally, the SUPDUP command (abbreviated SD) runs another special version of TELNET which provides remote display support when talking to the ITS systems at MIT. Both DTN and SUPDUP can be used only from a Stanford display terminal. SUPDUP is documented later in this appendix. All of these commands require that you be logged in. They take a remote host name as argument. (The host names are listed in a later section of this appendix.) The host can be specified by number instead of by name, and a socket number can optionally be used. (The socket number determines the server program to which you are connected, e.g., socket 3 gets you the FTP server program, and socket 27 gets the new protocol TELNET server.)

A8.1 The User TELNET

Our user TELNET program is run by the TELNET and TN commands, which take a host name as argument. When it is run, it attempts to connect to the specified host. It will type Trying when it starts setting up the connection, then Open if successful, or an error message otherwise.

The monitor commands to access TELNET are TELNET, TN, and OTN. TELNET and TN are used to access the new TELNET protocol, which is the official standard today. The OTN command is used to access the archaic and obsolete old TELNET protocol. The syntax is:

TELNET socket,host

TELNET prompts for an argument if one isn't given. A "?" typed in the command line prints out a short help message.

The socket field is optional and must be a number followed by a comma or atsign if present. The host field is either a host name or a number. Numbers are octal unless followed by a decimal point or if an 8 or 9 occurs in the number, in which case it is assumed decimal.

The host name/number is the ARPAnet site you wish to connect to. There is a list of hosts in this appendix; an up-to-date list can be seen by giving the monitor command R HOSTAB.

The socket number argument is used to connect to some special server program at the remote host, rather than the normal TELNET server. Special servers are normally used by other programs, e.g., the WHO command when a host name is given as argument. Normally only the maintainers of network programs will need to specify socket numbers to TELNET.

Datamedia simulator TELNET is invoked by the DTN command. Read later on under Datamedia simulation.

The information which follows also applies to the DIAL program, a TELNET-like program which uses dial-up telephone connections rather than the ARPA network. The format of the DIAL command is explained on page 67. Another similar program is called PTYJOB, for communicating with a pseudo-teletype; it is not run with its own monitor command, but by R PTYJOB.

Foreign hosts do not use the Stanford character set. Instead, they accept standard ASCII codes. Letters, digits, and most of the punctuation characters available in ASCII are the same in the two codes, however. The main difference is that codes 1 to 37 (octal), used for special printing characters at Stanford, are control characters in ASCII. The precise use of these control characters is defined by the remote host. TENEX hosts, for example, use ASCII control-A to mean "delete one character," like our BS. For users at local terminals, TELNET handles this difference by interpreting the CONTROL key on a Stanford keyboard to mean ASCII control; i.e., CONTROL on a letter makes TELNET convert the letter to upper case and subtract 100 from the character code.

More precisely, TELNET has two modes: display and transparent. Display mode is legal for all terminals and is the default mode for all displays. Transparent mode is only legal for Datamedias and non-displays and is the default mode for non-displays.

In display mode, character set conversion between Stanford ASCII and standard ASCII is done. In addition, the CONTROL key converts the character to its ASCII control value; hence CONTROL-C sends 003 or 1C. TELNET uses META and CONTROL-META characters as commands to itself, as listed below. In some cases, META and CONTROL-META are equivalent. In others, a processing switch is set by META and cleared by CONTROL-META. In the list below, βX represents META-X, $\alpha\beta X$ is CONTROL-META-X, and $\otimes X$ means either one. Display mode is the only legal mode for DD's and III's.

There are certain differences between the two character sets in non-control characters. These are handled by automatic translation. Specifically, the following translations are done:

char	Stanford	↔	external
~	32	↔	176
}	176	↔	175
ALT	175	↔	33
≠	33	→	33

The Stanford not-equal has no external representation and cannot be entered from a remote host. All other codes are transmitted unchanged. Note that our BS (octal 177) is the ASCII delete; there is an ASCII backspace (10), which is our λ . Typing λ or CONTROL-H to TELNET will produce that code. The other possible ambiguities are our characters ↑ (136) and ← (137). These codes have two different meanings: the DEC PDP-10 version of ASCII agrees with our use, but the official version uses those codes for ^ and _ . Our network programs support the arrow version, and our ^ and _ characters are transmitted unmodified as 4 and 30.

In transparent mode, no character set conversion is done. All ASCII characters (including ↑C) are read and transmitted as is, with the exception of ↑↑ (control-uparrow, octal 036) which is the command escape. ↑↑ repeated sends ↑↑ to the foreign host (similar to TIP doubling of @). ↑↑ followed by - is the command complement escape. In other words, ↑↑E does the E command and ↑↑-E does the E complement command. Transparent mode is the only mode which works well on non-displays since it is the only mode where the full ASCII character set can be sent.

Transparent mode is occasionally useful for DM users if they wish to use a foreign host's DM service. For example, if you want to use TVEDIT on a Tenex you want to be in transparent mode (after telling the Tenex you're a DM). In transparent mode the <EDIT> and [NULL] keys are passed to the foreign host just as they are with no interference from SAIL's terminal service. On Datamedias, the escape character is <EDIT>[NULL] instead of ↑↑. Hence, <EDIT>[NULL] Q gets you out of TELNET, and <EDIT>[NULL] - D closes an input file.

Transparent mode is also useful for simulating the behavior you get when you use a TIP. In other words, your input gets passed to the remote host the way you expect it to without SAIL trapping certain characters. Of course, the escape character is trapped, but it can be changed to any character desired with the X command, so ↑↑X@ makes things even more like what a TIP does.

The proper way to flush a connection from TELNET is with the Q or C commands (their complements are the same). This gracefully closes the network connection and in the case of the C command prompts for another host to connect to. In transparent mode this is either ↑↑C or ↑↑Q. CALLing out of TELNET is never a good way to flush a connection and of course can't be done in transparent mode since ↑C is an ordinary character.

Commands are never echoed, although their effects may cause terminal output.

In the Datamedia simulator TELNET (DTN), commands are entered in a different way; neither transparent mode nor display mode is used. In the PTYJOB and DIAL programs, displays are always in display mode, and non-displays enter commands as in transparent mode, but ↑C (CALL) has its normal effect and is not passed on through the program.

A8.2 Control Commands to TELNET, PTYJOB, and DIAL

Note: α means CONTROL, β means META, $\otimes$ means either META or CONTROL-META. Filespecs may be aborted with altmode. There is no filename or extension defaulting and the PPN defaults to the ALIAS PPN in the usual way. The CONTROL and META bits apply to display mode; in transparent mode, β means $\uparrow\uparrow$ and $\alpha\beta$ means $\uparrow\uparrow-$.

- $\otimes A$ (TELNET only) Send an ATTN to the foreign host. This usually means something like "return to monitor".
- $\otimes B$ (TELNET only) Send a BREAK to the foreign host. This is distinct from ATTN; it appears to be something the Multics guys wanted and never used.
- $\otimes C$ (TELNET only) Close connection and ask for a new host name to connect to. This also exits transparent mode.
- βC (DIAL only) Turn on control character mode. In this mode all characters received from the dataphone are typed on your terminal.
- $\alpha\beta C$ (DIAL only) Turn off control character mode. In this mode, octal codes 1-10, 16-37, and 177 are not typed on your terminal when received from the dataphone.
- βD Open output file. This command asks for a file name, then proceeds to write everything that comes from the foreign host in this file. Type ALT when asked for the file name to abort the command. Note that characters you type, or characters from any input file that is opened, do not go into the output file unless the foreign host is duplexing them.
- $\alpha\beta D$ Close the output file. $\otimes Q$ implies this as well.
- βE Local echoing mode. Forces the foreign host to stop echoing. In the DIAL program, this makes you look like a half duplex terminal, and does not change the state of remote echoing.
- $\alpha\beta E$ Remote echo mode. Requests the foreign host to echo user typein. If the foreign host refuses to echo, the echo state is restored to local echoing. In the DIAL program, this turns off local echoing but does not attempt to change the state of remote echoing.
- βF Open append file. Everything sent from the foreign host goes in the file after the previous contents of the file. If there was a file previously opened with βD , it will append to that, otherwise it will ask for a file name.
- $\alpha\beta F$ Same as βF , but always ask for a file name.
- βI Open input file. Everything in the file is sent to the foreign host. Asks for a filespec. Type ALT as filespec to abort the command.
- $\alpha\beta I$ Close input file. $\otimes Q$ implies this as well.
- βJ (TELNET only) Echo mode. Turns on local echoing without consulting the foreign host. See βE .

- $\alpha\beta J$ (TELNET only) No echo mode. Turns off local echoing without consulting the foreign host. See $\alpha\beta E$.
- $\otimes K$ (TELNET only) Send protocol command to kill the remote job. This command is not guaranteed to work since many hosts do not implement the network logout protocol.
- βK (DIAL and PTYJOB only) Inhibit duplexing LF after CR. The system normally types a LF out when you type CR. This command inhibits that. This command is different in effect from βR below, because it alters only what appears on your screen, not what goes out over the line.
- $\alpha\beta K$ (DIAL and PTYJOB only) Enable duplexing LF after CR. The inverse of the above command.
- βL Enter line mode. This gives you the line editor on input. This is the default for displays unless TELNETing to an ITS system. The line editor is always echoed locally, but in remote or no echo mode the line editor will vanish after activation.
- $\alpha\beta L$ Enter character mode. Transmission is character-at-a-time. This is the default on non-displays and when TELNETing to an ITS system. Useful for running DDT at a remote host.
- βM (PTYJOB only) Enters a mode in which all 9-bit characters you type are sent unchanged to the PTY except βM , $\alpha\beta M$, βZ , and $\alpha\beta Z$. (Of course, if you are in line-at-a-time mode some control characters are still interpreted by the system line editor.)
- $\alpha\beta M$ (PTYJOB only) Leaves the above mode; meta-characters are again interpreted as commands to PTYJOB.
- $\otimes O$ (TELNET only) Send ABORT OUTPUT to the foreign host. This should have an effect like that of ESC O locally at SAIL.
- βO (DIAL and PTYJOB only) Turn on terminal typeout. See $\alpha\beta O$.
- $\alpha\beta O$ (DIAL and PTYJOB only) Turn off terminal typeout. Useful when output is already going to a file and you don't want to waste the time it takes to type out on your terminal as well.
- $\otimes Q$ Quit. Close network connection and exit.
- βR (TELNET only) Same as βI , but reads the file character-at-a-time instead of buffer-at-a-time. This is less efficient, but also results in much less messy output, as the characters will be sent synchronously. Asks for a filespec.
- $\alpha\beta R$ (TELNET only) Same as $\alpha\beta I$.
- βR (DIAL and PTYJOB only) Inhibit sending of LF after CR. Normally, when CR is typed, a LF is invented by the system and CRLF is sent to the foreign host. This command inhibits sending the LF after the CR.

$\alpha\beta R$	(DIAL and PTYJOB only) Enable sending LF after CR. The inverse of the above command.
$\otimes S$	(DIAL and PTYJOB only) In line mode, send the text in the line editor now, without waiting for an end-of-line character. If the $\otimes S$ is preceded by one or more octal digits with $\otimes$ (e.g., $\otimes 1\otimes 7\otimes 7\otimes S$) then the specified octal code is sent as a character. For example, $\otimes 0\otimes S$ will send a null.
βT	(TELNET only) Enter transparent mode. Character set conversion and local terminal handling are disabled.
$\alpha\beta T$	(TELNET only) Non-transparent mode. Character set conversion and local terminal handling are enabled.
$\otimes W$	(TELNET only) Get a status message from the host (sends an "are you there" query).
$\otimes X$	Change the command escape to the next typed-in character. This is only meaningful in transparent mode.
βY	(DIAL and PTYJOB only) Enter Datapoint simulation mode. Works only at a Data Disc terminal.
$\alpha\beta Y$	(DIAL and PTYJOB only) Leave Datapoint simulation mode.
$\otimes Z$	(PTYJOB only) Quotes the following 9-bit character, so it is sent to the PTY as is, regardless of the βM mode switch.
$\otimes <form>$	(DIAL and PTYJOB only) If there is an output file open, insert a form feed character in it.
$\beta @$	(TELNET only) Disable debug mode. This is the normal state.
$\alpha\beta @$	(TELNET only) Enter debug mode. Currently this reports the TELNET protocol negotiations between user and server.
$\alpha\beta ALT$	(TELNET only) Enter DDT if one is present. CPOPJ\$G returns to TELNET.

A8.3 Datamedia Simulation

A special version of TELNET, called DTN, uses a Datamedia simulator instead of the page printer. DTN only runs on displays, naturally. Additionally, Datamedia users are probably better off using TELNET in transparent mode, since besides the grossness of simulating a Datamedia *on* a Datamedia, it is also rather gross to use since the simulator's keyboard control has been oriented around the TV keyboards.

DTN is invoked by the DTN monitor command, and it takes the same format of commands as TN. When the connection is completed, the screen clears, and you are on a simulated Datamedia. The CONTROL key maps to ASCII CTRL, and the META key maps to the Datamedia EDIT key. NULL is sent the way you do it in ASCII, i.e., CTRL-@.

Commands are entered in as follows: to do a META command you use β FORM followed by the command letter; ie, β FORM Q gets you out of TELNET. To do a CONTROL-META command use β VT followed by the command letter; ie, β VT D closes an input file.

A8.4 SUPDUP

SUPDUP is the SAIL implementation of the ITS SUPDUP program, which is used for TELNETing between ITS sites. This program uses the internal ITS display codes for highly efficient communication and full use of ITS display programs over the ARPAnet. In addition, SUPDUP allows the user to send the full ITS ASCII character set (which includes some very hairy characters!) and maps both keyboard input and display output so that a Stanford display (DD, III, or DM) appears as a funny kind of ITS display.

The information in this section is excerpted from the file SUPDUP.MRC[UP,DOC], which contains further details on using SUPDUP.

The SUPDUP monitor command should be followed by a host name. The only hosts accepted are AI, DM, MC, ML, and SAIL; most of these can be abbreviated to a single letter. SUPDUP will then establish a connection with the specified ITS site, and after that you will effectively be a display on ITS, with full bucky bit capability. (SUPDUP to SAIL simulates a Datamedia. This is not very often useful for someone already at SAIL unless you are debugging the ARPAnet; in what follows it is assumed that you are SUPDUPing to an ITS system.)

Some mappings in input and output are made due to the differences between the SAIL and ITS ASCII character sets. Most of these mappings are transparent to the user. The design philosophy behind the mappings was to cause characters typed on the Stanford keyboard to be seen by ITS as the character appears on the key tops, and for characters received from ITS to be displayed as they would be at a console at MIT.

Visible output mappings: Caret is mapped to AND sign ($\wedge$) since there is no code in the SAIL character set for caret. There is no way to tell between AND sign and caret. In addition, centered-dot, gamma, delta, and circle-plus are not displayed (they generate an error message).

Visible input mappings: αz and αZ are mapped to [CALL], $\alpha \beta z$ and $\alpha \beta Z$ are mapped to α [CALL], and $\alpha _$ is mapped to [BACK NEXT]. $\uparrow$ is mapped to caret; to send $\uparrow$ (distinct from VT) see below. Some other more obscure characters must be sent by using a command; see below.

[ESCAPE]I is used to enter a SUPDUP command. The following commands are defined:

K or L	Kill the job on ITS and break network connections.
P	Temporarily restore the page printer. Typing any character will return your screen to the ITS display.
Q	Quit out, close network connections, detach the job on ITS if there is one.
R	Record contents of screen onto a file; asks you for name of file to write. REENTER does this too.
V	Re-draw the current screen.
?	Type a help text.

Command mode also allows some of the more esoteric ITS characters to be sent. For example, $\uparrow$,

$\alpha\uparrow$, $\beta\uparrow$, $\alpha\beta\uparrow$, αz , $\alpha\beta z$, αZ , $\alpha\beta Z$, and $\alpha_$ are sent in image mode without being mapped. Special mappings exist for keys not on the keyboard or trapped by the monitor. They may be sent with bucky bits, and they have no explicit graphic here or on ITS, as they are special function keys. These are:

.	centered-dot	[TAB]	gamma	[LF]	delta
[CR]	circle-plus	[BS]	integral	∂	[NULL]
λ	[BACK SPACE]	$\neq$	[CALL]	$\vee$	[BACK NEXT]
α	[ESCAPE]	β	[BREAK]	ϵ	[CLEAR]

A8.5 The File Transfer Protocol

The FTP command is used to transfer files between SAIL and another network host. FTP can be used in two-ways: for a simple transfer, a single FTP monitor command can be typed which contains all the information necessary for the transfer, much like a COPY command; if several files are to be transferred, or one of the less common protocol features must be used, the FTP command can establish a connection to a remote File Transfer Protocol server program, and you can then enter protocol commands to initiate transfers. FTP can also be used for file manipulation other than transfers at a remote host, e.g., directory listings, and file renaming and deleting.

Although the one-line FTP command is generally easier to use, this document will start with the more detailed use of the protocol commands. The reason for this choice is that to use even the one-line command effectively, you should understand some of the underlying protocol, which is more apparent in the multiple command mode of using FTP.

The FTP command is used to set up a connection to a remote File Transfer Protocol server. It takes a host argument like TELNET, except that if a host number is used it should be in decimal; to specify a nonstandard socket number, the syntax is `host#socket`. (The normal FTP connection is to socket 3.) Once the connection is set up, you can enter the following commands to the FTP program. Each command is a name followed by arguments as described below. Only the first four letters are significant (letters beyond four are ignored), and commands may be abbreviated by enough letters to make the abbreviation unique. Note that file specifications for remote hosts must follow the format conventions of that host; they are passed on verbatim by the FTP. In particular, the case of letters may be significant at some hosts.

Note: FTP command strings sent over the control link to the remote host are subject to character conversion to conform with standard ASCII, as described in the discussion of the TELNET program. The processing of file data depends on the transfer type; see the TYPE command below.

Some of the commands below are named the same as the corresponding command sent over the network in accordance with the actual File Transfer Protocol; in some cases alternate names are also defined. Complete information on the protocol is available in the network document with NIC number 14333, although there have been several modifications to the protocol since that document was written. There is also a "new" FTP which is not, so far at least, actually in use.

We do not implement all of the protocol options. In particular, we accept only MODE S (stream) and only STRU F (file). Our server accepts MODE and STRU commands but only with the above arguments; our user does not provide any way of specifying mode or structure.

When the connection is first established, our user FTP program tries to negotiate an appropriate type and byte size with the other end's server. First it sends TYPE I, and then BYTE 36. If the latter is accepted but the former was not, it sends TYPE I again. (These commands are explained below.) If these are not accepted, it sends BYTE 8, and if that is accepted, TYPE A. (All hosts are supposed to accept BYTE 8.) If nothing is accepted, the program tries again after you give the USER command, in case the problem is that their server was written by an obnoxious security freak as at a well-known system in Cambridge, Massachusetts.

The protocol provides an abort function which can be used to stop a transfer which is in progress. This function can be requested in our FTP user program by typing ESC I (from a non-display, type ↑← I). This will send the abort command only if there is a transfer in progress. Many servers do not process the abort sequence, but they should notice when we close the data connection. You may end up, if you are doing a STOR, writing an incomplete file on the other system.

In the file transfer commands below, the character = can be used instead of either ← or → as shown, for the convenience of users at non-Stanford terminals.

TYPE × × is A, I, L, P, or E, and specifies the *representation type* for the data. The uses of the various types are as follows:

TYPE A *ASCII* type means that the file to be transferred contains text. The characters in the file will be transferred to or from the SAIL character set, as explained for TELNET connections, and null characters will be ignored. This type should be used to transfer text files to or from a machine with a different text representation. Text files can be transferred to or from other PDP-10 systems more efficiently using image type (below) if the few characters which are translated by ASCII translation are unused in the file, since our character code is almost the same as that of other PDP-10s. ASCII transfers of local files will also eliminate the E directory or SOS line numbers, if any.

ASCII is an abbreviation for TYPE A.

TYPE I *Image* type means that the file is taken as a continuous stream of bits and sent accordingly. The number of bits sent at a time is determined by the byte size set with the BYTE command (below). The most efficient byte size for us is 36, the size of a PDP-10 word. Any byte size which is a factor of 36 can be used with no difficulty, if needed to accommodate some other system. Byte sizes of 8 or 32, however, cause extreme inefficiency because some bytes are split between two PDP-10 words. If you are retrieving a file from an 8-bit or 32-bit system, you might want to use TYPE L (local byte), which uses only the first 32 bits of each PDP-10 word, and can therefore be implemented using the normal ILDB/IDPB instructions. However, sending a file to such a system in local byte type will lose bits 32-35 of each word.

IMAGE is an abbreviation for TYPE I.

TYPE L *Local byte* type, as explained above, treats the file as normal PDP-10 bytes of the size specified in a BYTE command. For byte sizes which are a factor of 36, this is treated exactly the same as image type (above); for byte size 8 or 32 it uses only bits 0-31 of each PDP-10 word. Byte size 32 is more efficient than 8 in this type.

LOCAL is an abbreviation for TYPE L.

- TYPE P *ASCII print* type is treated by our system the same as ASCII type. This isn't right, but the protocol is not consistent about what to do. Some people interpret this type as indicating that the file contains FORTRAN style format control characters.
- TYPE E *EBCDIC* type is not implemented here. It is used for transferring text files between two IBM systems. To transfer a text file between here and an IBM system, use TYPE A; the IBM system will translate between ASCII and their local character code as needed.
- TYPE X This command to our FTP user program does not correspond to a real data type in the protocol. Instead, it is treated as TYPE L here but tells the other end TYPE I. Use this command if you are talking to an 8-bit machine and want to use local byte type, but the other machine does not accept the TYPE L command.
- BYTE n n is a (decimal) number indicating the *byte size* of the network data connection. Data will be sent from one host to another in bytes of this size, for image type and local byte type transfers. ASCII transfers always use a byte size of 8. (That is, each 7-bit local character code is translated into an 8-bit network ASCII code and vice versa.) The allowable byte sizes are 8, 32, and any factor of 36 (1, 2, 3, 4, 6, 9, 12, 18, 36). In general, the largest byte size which will work is the most efficient. (Note: the protocol specifies that it is the responsibility of the user FTP to ensure that 8-bit bytes are used for ASCII transfers. Accordingly, our user FTP program does not necessarily transmit a BYTE command as soon as the user types it; instead, the byte size in the command is remembered for use in binary transfers, and a BYTE command is sent to the FTP server whenever necessary as the transfer type changes.)
- USER × × is a string which the foreign host will recognize as a valid user description, user name, or ppn. Not all hosts require a user name. If the server requires a password to complete the login procedure, the user program will ask for the password and turn off echoing while you type it. Not all hosts require a password.
- LOGIN × is a synonym for USER.
- ACCT × × is an account number. Some hosts may require this for their billing/accounting purposes.
- XCWD × × is a user name, as in the USER command, which will be used as the default directory for foreign file specifiers; the effect is like the ALIAS monitor command here. The command name stands for *change working directory*.
- ALIAS × is a synonym for XCWD.
- RETR ×←y This command *retrieves* a file from the foreign host. × is a local file specifier, and y is a foreign file specifier. The foreign file y is copied to the local file ×. Current settings (or default values) are used for byte size and representation type. The ×← string is eaten locally, and the RETR y part is transmitted to the foreign host.
- GET ×←y is a synonym for RETR.
- TTY × is a version of RETR which uses × as the remote file specifier, and types the retrieved file on the user's terminal.

STOR $\times \rightarrow y$ STOR means *store*, and this command is the inverse of the RETR command. A local file is copied to a foreign host.

SEND $\times \rightarrow y$ is a synonym for STOR.

APPE $\times \rightarrow y$ This command will *append* the local file to the end of the foreign file.

PICKUP This command is used to resume an interrupted multiple file transfer and is explained further below.

LPPN This command sets the *local PPN flag*, which affects the interpretation of single-pathname transfer commands; see the section on multiple file transfer, below.

RPPN This command clears the *local PPN flag*, which affects the interpretation of single-pathname transfer commands; see the section on multiple file transfer, below. (The command name stands for *remote PPN*.)

MAIL $\times$ $\times$ is a user ID (like our PPN). Following this command you may type in a message which will be mailed to the specified user at the foreign host. The message must be terminated with a line containing only a period. The MAIL program (see Appendix 4) is a better way to send network mail.

MLFL $\times \rightarrow y$ In this case $\times$ is a local file specifier, and y is a foreign user ID. The text of the specified file will be mailed to the specified user.

XSEN $\times$ is like MAIL, but the message is sent to the remote user's terminal if s/he is logged in, like the SEND monitor command at SAIL. This command and the two following ones are only implemented at a few network hosts. The SEND monitor command uses this protocol if a network destination is specified.

XSEM $\times$ is like XSEN but mails the message instead if the user is not logged in, like SEND/YESMAIL here.

XMAS $\times$ is like XSEN but mails the message also, whether or not the recipient is logged in, like SEND/MAIL here.

LIST $\times \leftarrow y$ y is a foreign pathname, possibly including wildcard terms. The directory listing of the specified pathname is stored in the local file $\times$.

DIRECT is a synonym for LIST.

NLIST $\times \leftarrow y$ is like LIST, but the listing returned is guaranteed to have no extraneous information in it, simply one full pathname per line. This listing is intended to be read by programs rather than humans, but if you really want it...

HELP This command asks the foreign FTP server to send back a message indicating what commands it takes, etc.

STAT $\times$ If $\times$ is omitted, this command asks the FTP server to send back information regarding the FTP connection, e.g., socket numbers. If $\times$ is a directory name at their host (like our PPN), it lists that file directory. Note: contrary to protocol, most servers (including ours)

treat a STAT with no argument as a request for a listing of the directory specified in the USER or XCWD command.

- DELE × × is a foreign file specifier. The specified file is deleted at the foreign host.
- RNFR × × is a foreign file specifier. This command is short for *rename from*; it must be immediately followed by a RNTD.
- RNTD × × is a foreign file specifier. This command completes the file rename operation started by RNFR. (Some hosts accept * for wildcard specification in these commands.)
- QUOT × × is any character string, which should be an FTP command with arguments. The string is sent as is over the FTP control link (see protocol, NIC 10596, for terms). QUOT was included to allow you to execute commands which are as yet unimplemented here, or which are nonstandard, specific to some serving host. It will be of little use for those commands which require some special action by the FTP program at this end.
- BYE Terminate connections with the foreign host and quit.
- QUIT is a synonym for BYE.
- DISC is a synonym for BYE.
- XIND × × is a local file specifier. The file will be read and processed as FTP commands.

Multiple File Transfer

The RETR and STOR commands have some provisions for more convenient transfer of groups of files when similar names are used at both ends. The normal format of the commands requires the user to specify two file names, one for our local file system and the other for the remote host's file system. Our FTP knows enough about the filename syntax for certain systems (mainly the PDP-10 systems: ITS, TOPS-10, TOPS-20, and TENEX) to be able to extract a local filename from a remote one.

If only one filename is given in a RETR or STOR command, it is scanned to determine both the local and the remote file specification. Normally (in *local PPN mode*), the entire file specification is sent to the remote host except for anything inside [square brackets], which is taken to be a local SAIL PPN. In *remote PPN mode*, everything is sent to the remote host, and your own login or alias PPN is used for the local file. Here are the rules for determining the local file specification:

1. A "token" in these rules means a string of letters, digits, and the characters - and @. An asterisk (*) may be used alone as a token to indicate wildcard file groups, as explained below. If talking to one of the ITS systems at MIT (MIT-AI, MIT-ML, MIT-DMS, and MIT-MC) then a period (.) which follows a non-token character (space or punctuation) is taken as the first character of a token; such a token may also end with a period.
2. If talking to one of the ITS systems, then a single left or right bracket (< or >) may be used

alone as a token, which will be completely ignored. Otherwise, a token enclosed in brackets is ignored.

3. One or more tokens separated by commas, enclosed in square brackets ([and]), are ignored in remote PPN mode, as explained above; in local PPN mode, there must be only one or two such tokens, and they must form a valid SAIL PRJ or PPN.

4. Any characters following a semicolon (;) are ignored, except for the ITS systems, for which the token preceding the semicolon is ignored.

5. A token followed by a colon (:) is ignored.

6. A single non-ignored token is taken as the local filename. Two such tokens separated by a period (.) or separated only by one or more spaces are taken as the local filename and extension. The characters - and @ in a token are ignored, and the token is truncated to six or three characters for the local filename or extension. (The leftmost six or three characters are used.) A period which is taken as a token constituent, when talking to an ITS system, is ignored for local purposes. If two or more non-token periods are used in the filename, the token after the first period is taken as the extension, and tokens after the remaining periods are ignored.

7. No punctuation characters may appear in the filename string except the ones mentioned above. Spaces may be used anywhere except within a token.

The FTP program notifies the user whenever it determines the local file name from the remote one in this way, and, in the case of RETR, checks to see if that file already exists before writing it. If there is such a file, the user is told so and can choose from these options:

Type <cr> to overwrite the file.

Type <lf> to overwrite the file and to suppress further warnings of this kind for the remainder of the FTP connection.

Type <alt> to abort the transfer.

Type another filename to write a different local file.

If this syntax is used, the implied local device is DSK. (Note that device field in the remote file specification is ignored for local file purposes.) To use a different device the local=remote syntax must be used.

A local filename or extension of * either in an explicit local file specification or as determined from the remote specification is used to request the transfer of more than one file by a single command. Wildcard device or PPN is not allowed. If an explicit local file specification is used, it should agree with the remote file specification in which components are wild, e.g., the command

```
RETR *.EXT=<THEIRDIRECTORY>FILE.*
```

will store any file(s) retrieved with local filename FILE.EXT, probably not the desired effect. Note also that an explicit local file specification with no filename or extension implies *.* , so the command

```
RETR [PRJ,PRG]←REMOTE.FILE
```

will be treated unnecessarily as a multiple retrieve. This will work but will involve unnecessary operations. The desired effect can be obtained in local PPN mode simply by typing

```
RETR REMOTE.FILE [PRJ,PRG]
```

which will take the PPN as applying to the local output file, and will send only the filename and extension to the remote host. In remote PPN mode, it is necessary to repeat the filename:

```
RETR REMOTE.FIL [PRJ,PRG]=REMOTE.FILE
```

There is no corresponding local/remote switch for device names, which are always considered remote. Therefore, you must say, e.g.,

```
RETR UDP1:REMOTE.FIL←REMOTE.FILE
```

to avoid causing a multiple RETR. The TTY command can be used to avoid having to repeat the filename for the specific case of device TTY.

In the case of multiple STOR, the selected files are found in the user's local file directory and a STOR operation is performed for each file found which matches the template. The remote file specification is scanned for each operation and the character * is replaced wherever seen by the local filename or extension. (The first * is replaced by the local filename unless the character . (or, for ITS hosts only, space) precedes it, in which case the local extension is used. A second * is replaced by the local extension.) If the end of the specification line is seen before any replacement has been done, the local filename and extension are inserted before the carriage return as if *.* had been seen there, so that a command like

```
STOR *.SAI=<DIRECTORY>
```

will work. In this situation the filename and extension will be separated by a space for ITS hosts or a dot otherwise. No great effort is made to ensure the syntactic correctness of the resulting remote file specification; presumably the remote host will complain if necessary.

For a multiple RETR, the remote file specification is used exactly as typed in an NLST operation, the results of which are saved by the FTP program to be used in successive RETR operations. Note that the string used as the NLST argument starts immediately after the = or ←, if any, or after the space terminating the RETR or GET command name. In particular, any extra spaces will be sent to the remote host. The remote file specification in the typed command is not used to determine local file names; instead, the rules above are applied to each remote file specification received from the NLST operation. If an explicit local filename or extension is provided by the user, it will be used instead of the one derived from the remote filename, as in the command

```
GET *.TXT=<IGNORED>*.THIS-NAME-IS-REPLACED-BY-TXT-LOCALLY
```

Multiple RETR depends on the remote host implementing NLST. At present this command is implemented in both TENEX and ITS. TOPS-10 systems vary in which commands they implement. Other systems may have a filename syntax which does not follow the rules above, in which case none of the features in this section will work.

The PICKUP command can be used to continue a multiple transfer which is interrupted either by a transient network failure or by a user error (e.g., attempt to write into a protected file). The command permits the transfer to be re-entered along with the (local) name and extension of the first file actually to be transferred.

Suppose that you are trying to copy a complete directory from another system, let's say a TENEX, so you give the command

```
RETR <FOO>*.*
```

After several files have been retrieved, the connection is broken, while in the middle of copying <FOO>PARTIALLY.COPIED into PARTIA.COP in your directory. You re-establish the connection, and type

```
PICKUP PARTIA.COP
```

at which point you are invited to repeat the RETR command, which will ignore all files presented by the remote host until one comes along which matches the name PARTIA.COP, at which point it begins actually reading files.

If the multiple transfer is stopped because of an error which does not break the network connection, so you can continue using the same FTP, you can give the PICKUP command without a filename argument, in which case the last filename for which a transfer was attempted is used. Also, after the PICKUP command, you can just type <return> instead of repeating the STOR or RETR command, and the last such command given will be repeated. The last transfer must have been a multiple STOR or RETR.

If the PICKUP command line ends with <alt> instead of <return>, the first file transferred will be the one after the one in the argument (or the one after the one remembered from the aborted transfer). This is useful for the case of a transfer which cannot be done because of a protection violation, when you want to continue the multiple transfer, skipping the protected file. In a situation like that, you may not know the name of the first file you want to transfer, but only the name of the last file you don't want to transfer.

If a PICKUP command is given after a transfer which is not multiple but for which an implicit local filename was used, e.g.

```
RETR <FOO>FILENAME.EXTENSION
```

which implicitly specifies FILENA.EXT as the local name, then the PICKUP command with no argument can be used to repeat the transfer.

One-line FTP transfers

The format for using the FTP command to carry out a transfer all in one command line is this:

```
FTP local ← {host} remote
FTP {host} remote → local
FTP local → {host} remote
FTP {host} remote ← local
```

The braces are really to be typed; they're not meta-symbols. The direction of the arrow indicates the direction of the transfer; the first two examples retrieve files from the remote host, and the last two store files at the remote host. Therefore, the arrows may not be replaced with equal signs as in the multi-command syntax. The pathnames may include * for multiple transfers.

Although there must be an arrow, either of the pathnames may be null, in which case the other one will be used for both ends as in the RETR or STOR commands with only one pathname.

Normally FTP will attempt to do the transfer in IMAGE mode. If you want ASCII mode transfer, give the command as above, but with FTP/A instead of just FTP. If the remote host will not accept the specified mode, the transfer is aborted and you can try negotiating manually.

Other switches can also follow the FTP command: /R changes the mode for parsing a single pathname from LPPN to RPPN; /T is like the TTY command in multi-command FTP: it types the specified remote file on your terminal. (No local pathname or arrow is used with /T.)

After doing the transfer successfully, FTP will close the connection and exit. So if you want to do several transfer commands, it pays to use the multi-command syntax.

The host specification inside the braces can contain more than just a host name. Many hosts require a login sequence before files can be transferred; the parameters for this login are supplied in the host specification like this:

```
{hostname/user/acct}
```

This can be abbreviated to just {hostname/user} or {hostname}. If no acct is given, no ACCT command will be sent; if no user is given, the commands

```
USER ANONYMOUS
PASS SAIL
```

will be sent. These are the standard FTP guest login for Tenex systems. If you do include a user name, and the remote host asks for a password, you will be asked to supply it with echoing off.

If you often use a particular host-user-acct combination, you may want to store the information online instead of typing it in each time. To do this, include in your OPTION.TXT file one or more lines of the form

```
FTP: {host/user/acct}, {name:host/user/acct/pass}, {host/user//pass}
```

As you see in the example, the format allowed in OPTION.TXT has more fields allowed than the one typed in the command. One difference is that you can include a password, if you don't mind

taking the risk of storing your passwords online. (You can have a password but no acct as shown in the last host in the example.) Also, there is a “name” field which may begin the host specification; if no name is given, the first field serves both as name and as host. The way to refer to a stored host specification is to type {name↑} in the monitor command line. Note that this name must *exactly* match the one in OPTION.TXT, except for the case of letters. (Normally, a host name can be abbreviated to enough letters to determine it uniquely.) For example, suppose the host SRI-KL can be abbreviated SRI. Then these commands are equivalent:

```
FTP ← {SRI/NICGUEST} <NETINFO>LIAISON.TXT  
FTP ← {SRI-KL/NICGUEST} <NETINFO>LIAISON.TXT
```

but if your OPTION.TXT file contains the line

```
FTP: {SRI/NICGUEST}
```

your command must be {SRI↑} and not {SRI-KL↑}.

Table of Host Mnemonics

Hosts may be specified by number instead of by name in TELNET and FTP. TELNET takes host numbers in octal; FTP takes them in decimal. Here is the list of network hosts as we go to press; the HOSTAB program will list the current host table on your terminal. Normally you should not have to worry about host numbers, but if a new host joins the network, it may take a few days for its name to be added to our host tables.

Dec #	Oct #	Official Name	Dec #	Oct #	Official Name	Dec #	Oct #	Official Name
163	243	ACCAT-TIP	9	11	HARV-10	58	72	NYU
48	60	AFWL	79	117	I4-ELF	43	53	OFFICE-1
176	260	AFWL-TIP	15	17	I4-TENEX	89	131	OT-ITS
208	320	AMES-11	143	217	I4B-TENEX	96	140	PARC-GATEWAY
16	20	AMES-67	239	357	ILL-UNIX	32	40	PARC-MAXC
144	220	AMES-TIP	22	26	ISI-SPEECH11	160	240	PARC-MAXC2
55	67	ANL	52	64	ISI-XGP11	218	332	PENT-UNIX
28	34	ARPA-DMS	34	42	LBL	154	232	PENTAGON-TIP
156	234	ARPA-TIP	98	142	LBL-UNIX	18	22	RADC-MULTICS
220	334	ARPA-XGP11	10	12	LL	146	222	RADC-TIP
112	160	ASL	202	312	LL-11	210	322	RADC-TOPS20
232	350	BBN-GATEWAY	108	154	LL-ASG	82	122	RADC-XPER
69	105	BBN-INLAT	85	125	LLL-MFE	7	7	RAND-RCC
40	50	BBN-NCC	21	25	LLL-UNIX	135	207	RAND-TIP
133	205	BBN-PTIP	99	143	LOGICON	199	307	RAND-UNIX
177	261	BBN-SPEECH11	42	52	LONDON	46	56	RUTGERS
241	361	BBN-TENEX	234	352	LONDON-GATEWAY	174	256	RUTGERS-TIP
197	305	BBN-TENEXA	170	252	LONDON-TIP	54	66	SCRL-DEV
49	61	BBN-TENEXB	106	152	LONDON-VDH	118	166	SCRL-RSX
113	161	BBN-TENEXD	134	206	MIT-AI	231	347	SDAC-44
5	5	BBN-TENEXE	44	54	MIT-DEVMULTICS	39	47	SDAC-CCP
158	236	BBN-TESTIP	70	106	MIT-DMS	167	247	SDAC-NEP
63	77	BBN-UNIX	236	354	MIT-MC	103	147	SDAC-UNIX
27	33	BELVOIR	198	306	MIT-ML	243	363	SRI-C3P0
122	172	BNL	6	6	MIT-MULTICS	51	63	SRI-KA
29	35	BRL	172	254	MIT-TIP	66	102	SRI-KL
223	337	CCA-SIP	145	221	MITRE-TIP	194	302	SRI-NSC11
95	137	CCA-SPEECH	45	55	MOFFETT-ARC	115	163	SRI-R2D2
31	37	CCA-TENEX	109	155	MOFFETT-SUBNET	2	2	SRI-TSC
20	24	CCTC	200	310	NADC	179	263	SRI-UNIX
182	266	CHII	19	23	NBS-10	130	202	SRI-VIS11
164	244	CINCPAC-TIP	147	223	NBS-TIP	11	13	SU-AI
78	116	CMU-10A	211	323	NBS-UNIX	120	170	SU-ISL
14	16	CMU-10B	168	250	NCC-TIP	139	213	SU-TIP
142	216	CMU-10D	117	165	NCSC	56	70	SUMEX-AIM
206	316	CMU-CMMP	105	151	NDRE	1	1	UCLA-ATS
178	262	DARCOM-TIP	233	351	NDRE-GATEWAY	65	101	UCLA-CCN
212	324	DCEC	41	51	NORSAR-40A	129	201	UCLA-SECURITY
148	224	DCEC-TIP	169	251	NORSAR-TIP	215	327	USC-ECL
101	145	DEC-MARLBORO	3	3	NOSC-CC	86	126	USC-ISI
153	231	DOCB-TIP	131	203	NOSC-SDL	244	364	USC-ISIB
76	114	DTI	67	103	NOSC-SECURE1	150	226	USC-ISIC
72	110	DTNSRDC	35	43	NOSC-SECURE2	116	164	USC-ISIE
53	65	EGLIN	227	343	NOSC-SECURE3	151	227	USC-TIP
59	73	ETAC	1540	3004	NPRDC-UNIX	4	4	UTAH-11
61	75	ETAM	185	271	NPS-TIP	132	204	UTAH-TIP
33	41	FNWC	8	10	NRL	62	76	UTEXAS
60	74	GOONHILLY	4100	10004	NSWC-DL	110	156	WHARTON
141	215	GUNTER-TIP	136	210	NSWC-WO	47	57	WPAFB
13	15	GUNTER-UNIX	238	356	NUSC	111	157	WPAFB-AFAL
152	230	GWC-TIP	195	303	NWC	175	257	WPAFB-TIP

APPENDIX 9

CARE AND FEEDING OF DEVICES

A9.1 The Line Printer

On the front of the line printer (LPT) there are several buttons (STOP, START, TOP OF FORM, MANUAL PRINT, TEST PRINT, OFF, and ON) and several indicators. The normal state of the line printer is with the START button lit and all red indicators off. (TOP OF FORM is always lit.)

If the system or the spooler reports that the line printer is hung, check the following things. If the START button is not lit and there are no red indicators, push the START button. If there are red indicators, NO PAPER or PAPER LOW ALERT lit, then you have to put in more paper (or fix the paper that has gone astray).

To reload the paper, find someone who knows how to do it and watch him, or do it yourself. Reloading the paper is fairly obvious. The front cover (which includes a transparent section) is hinged at the top; lift it. The yoke is the assembly which carries the ribbon, the printing drum (which you can't see) and a rotating disk at the left side. Open the yoke by finding two toggle switches (one with each hand) and pushing them both down. These switches are located to the left and right of the yoke and below it. They are each labeled CLOSE - OPEN. The yoke will stop when it is fully open. Find four paper tractors, two above the printing hammers and two below. Each tractor has a sprocket wheel (which pokes through the holes in the edge of the paper) and a retainer which holds the paper against the sprocket. Open all the tractors. If the paper is perforated for 8.5 inch wide pages, be sure that the perforation is closer to the right side. Push the TOP OF FORM button before loading the new paper. Use the black arrows to align the "concave" fold. The concave fold is the one where the two adjacent sheets are front to front when folded. Close the four paper tractors so the sprockets fit into the holes in the paper. Close the yoke by finding those two switches and pressing up on both of them. Push START.

Always restart the fan-fold at the back of the LPT so the paper will stack properly.

The line printer logic can become hung. This condition is identified by the MANUAL PRINT light being on continuously. Push MANUAL PRINT and then START to clear this condition. If that fails, open the door on the right side and push the black button labeled RESET. If that fails, find a wizard.

If the ALARM STATUS light comes on or if the printer starts making a loud buzzing noise, shut off the printer (push OFF) and find a wizard. (This light may come on briefly when the printer is being turned on.)

The YOKE OPEN light comes on when the yoke is open while the paper is being changed. When the yoke is closed it should go off.

The printer has a ribbon that can be changed too. It is best to watch someone do it before trying it yourself.

A9.2 DECtapes

To mount a DECtape, first assign an available drive. Drives are identified to the system by a rotary switch labeled 1,2,...,8 corresponding to DTA1, DTA2, etc. Although there are eight numbers on the switches, there are only four drives, and the system will not recognize DTA5 and up. Generally you should have no reason to change the normal setting of these switches. Place your tape on the left reel of the drive and thread the tape over the top of the tape guide and head assembly. Wind several turns of the tape onto the takeup reel (turn the reel clockwise). Finally flip the OFF-WRITELOCK-WRITE switch to WRITE LOCK, or if you want to write, to the WRITE position.

To unload the tape, wind the tape off the takeup reel by the direction switch (push it to the left and hold it). Turn the drive off when all the tape is on the left reel. Grasp the reel firmly and pry it off the hub.

A9.3 Magnetic Tapes

Our magnetic tape drives are seven-track. They are capable of operating at densities of 200, 556, and 800 bits per inch; the default is 556.

First, decide whether you want to write on the tape or not. If you intend to write, place one of the plastic write-enable rings in your tape. Place your tape on the top hub and twist the handle in the hub clockwise to tighten the reel to the hub. (You can really lose if you don't tighten it all the way!)

Between the two reels there is a three-position switch labeled START-BRAKES. Move the switch to the right and hold it there to release the brakes on the hubs. Unwind the tape until the end of the tape is about a foot from the floor. Carefully thread the tape into the slot between the reels to the left of the brake release switch. Wind the tape onto the bottom reel (wind the reel clockwise). You must be holding the brake release switch to the right to wind the tape. Wind several feet of tape onto the takeup reel until you see the metal foil load point marker go by. Then push the brake release switch to the left (the START position). The tape will hiss and spin. Hold the switch in START until all motion stops.

There are some control buttons on the top panel. One is labeled (and lighted) REMOTE-LOCAL; this button toggles. In LOCAL mode, push the REWIND button. If you're not sure that you have the load point marker on the takeup reel, push the FORWARD button for several seconds to wind the tape forward onto the takeup reel until it passes the load point marker, then REWIND. REWIND positions the tape at load point. Set the drive in REMOTE and it is all set to use.

Unloading the tape is simple. In LOCAL mode, push the REWIND button. When the tape finished rewinding, push the brake release switch and manually wind the tape onto the top reel. Unscrew the hub and remove the reel.

The magnetic tapes are quite finicky and the system software is somewhat flaky. There are several things that you can do with mag tapes that will upset the timesharing system and require the intervention of a wizard who will be angry at you for disturbing his slumber. Never stop your job

when it is operating the tape by typing CALL. Instead, the job can be stopped by causing the magnetic tape to appear hung. This is done by switching back and forth between REMOTE and LOCAL until the system stops the job.

There is a reset button for each tape drive located behind the small front panel below the drive mechanism. This button clears all tape motion functions in a reasonable way.

A9.4 Xerox Graphics Printer

The normal condition of the XGP is indicated by the green ON light and the orange READY light being on. If all indicators are off, the XGP main power has been shut off inside the cabinet, possibly for some reason.

If the red OFF light is on, push the ON light. This will turn the XGP on, unless some abnormal condition obtains, such as a paper jam near the cutter.

If the orange STANDBY light is on, look inside the panel above these indicators. The following is the normal state of the indicators and switches within:

```
Knobs:  TEST PATTERN:      OFF
        VERTICAL POSITION:  leave this one alone

Lights that should be on:
    ALL POWER SUPPLIES lights
        (except 10kv and 2.5kv lights will
         be off any time there is some other
         problem)

    ALL PROCESS INTERLOCK lights

    ALL CONTROL LOGIC (except IN SYNC)

Lights that should be off:
    ALL SUPERVISORY SIGNALS
    FAULT
    IN SYNC
```

Certain conditions of the XGP are indicated by lights being OFF. (If any of the conditions listed below obtains, the 10kv and 2.5kv power supplies will shut off.)

- FWT Fixed wait timer has not run long enough after being turned on. Wait 2 minutes.
- FUSER The fuser is not hot enough.
 If the XGP has just been turned on, wait 5 minutes; otherwise, reset the over-temperature sensor. The over-temperature sensor is reset by a small black button located somewhere in the middle of the fuser wick assembly.
- WEB CLEAN The drum cleaning web must be replaced.
- PAPER OUT Load more paper.
- DRUM The drum assembly has been disengaged.
- SWEEP PRESENT The interface is not providing a sweep signal. This may mean the connector is unplugged or the interface turned off.

To load a new roll of paper, open the left door of the XGP. Unscrew the roll retainer and remove the old roll. Unwind a foot of paper from the old roll and then cut the paper at the roll. Mount the new roll and tape the front end of the new roll to paper separated from the old roll. Make the leading edge of the joint as smooth as possible. Using the MOTOR MANUAL DRIVE button, advance the splice through the machine until it has passed the cutter.

If any problems develop in this sequence, find a wizard. Beware: certain parts of the XGP are hot; you can be painfully burned if you touch the wrong things.

There is a drum cleaning web that runs out at inconvenient times. There is a small diagram inside the XGP explaining how to change the web.

Toner is icky black stuff that makes the marks on the paper. It gets used up and runs out too. Someone is supposed to check the toner every day and add more if it gets too low. If the XGP runs out of toner, the pages will start getting lighter, there will be occasional black splotches on the paper, and frequent sand-like particles will be found embedded in the paper. Toner is checked through the access panel in the back of the XGP. There is a lever there that adjusts the rate at which toner is used on the paper. This lever must be placed either at the lightest (inner) setting or one setting darker. Never, never set it anywhere else.

If you don't like the copy quality, complain to a wizard. Don't adjust things yourself.

A9.5 User Disk Pack

One or more of the drives in the Ampex disk system are usually reserved for private disk packs. The drives so designated will be labeled USER PACK. Use the command ASSIGN UDP_n to assign a particular drive before you mount your pack.

Assuming there is no pack mounted already, you can mount your pack by following this sequence. Make sure there is nothing on top of the clear door of the drive you are using. Then push the OPEN DOOR switch and the door will swing up. Hold your pack by the handle on its top cover and remove the bottom cover from the pack by squeezing the two handles on the bottom together. Place the bottom cover on top of the back part of the drive cabinet. Place the pack (still in the top cover) in the drive unit. Turn the handle clockwise. It will be somewhat hard to turn. Turn until it stops. Carefully remove the top cover from the pack and place it on the bottom cover. Make sure the two covers are aligned to keep dust out of the enclosed space. Turn the top cover so the label on it is visible from the front of the drive. Gently push the drive cover down and close it—it will lock. Push the START-STOP switch to START; the pack will begin to spin. When the green ready light comes on, the pack can be used. There is a READ ONLY-R/W switch on the drive; put this switch in the READ ONLY position if you do *not* intend to write anything on the pack and put it in R/W (Read/Write) if you *do* intend to write on the pack.

To unload the pack, push the START-STOP switch to STOP. When the pack comes to a stop, press the OPEN DOOR switch; the door will swing up. Take the top cover whose label corresponds to the label on the pack and place it carefully over the pack. Turn the handle counter-clockwise until it moves freely. Lift the pack out of the drive. Place the bottom cover under the pack and press it on until it latches. The covers help keep dust off the packs. Return the pack to the storage rack. Leave the label facing outwards. Gently close the cover on the drive until it locks.

Occasionally, there may be no drives available for user disk packs. If this happens, a wizard will explain why. Never touch a disk drive unit unless you have it assigned! Never touch anything but a drive which is marked USER PACK! If you are confused by something you see, ask about it before you touch!

A9.6 Core Storage

A hardware-detected parity error in core being used by a user program will stop that program but will generally not hurt the system otherwise. (There will be a brief period, while the system is finding the error, during which it will not respond to characters typed at terminals.) A parity error in the monitor itself will probably crash the system. It is possible for a memory error to go undetected if two bits are incorrect in the same word, but this should be uncommon. If a piece of hardware fails reliably, the parity error rate may become too high to allow operation of the system.

Another possible problem besides parity errors is a memory stop. This is usually a temporary problem. It will stop the timesharing system, but it may be possible to continue from a memory stop without reloading.

Memory errors have never, never been fixed by software means. In particular, reloading the system never solves the underlying problem. If the system becomes unusable because of these errors, a wizard should be found.

A hung memory may be indicated by a console terminal message saying NXM (non-existent memory). However, a hung memory may also result in the system stopping with no message at all. Therefore, whenever the system stops working with no apparent cause, look for a hung memory. There is a MEMORY STOP light on the KA-10 console, which also indicates this problem. In some circumstances, a PDP-6 memory stop may also interfere with the operation of the system.

When the memory has been reset, the system can sometimes be continued. If possible, find a wizard to fix it. Otherwise, read on. A hung memory can be identified by:

```
DEC MG10:  AW light off (there are two per cabinet, under "REQUESTS")
Suppes:    OK STATUS light off (one per cabinet, in the little box on top)
ARM10LX:   AWRQ light off (under "CONTROL STATUS"; try all positions of the "SECTOR DISPLAY"
            switch while you're looking)
new Ampex: UA light off (two per cabinet, one in each section)
PDP-6:     UA light off (one per cabinet)
```

Push STOP on the KA-10 and, if it's the PDP-6 memory, the PDP-6. At the KL-10's console terminal, type control-X followed by SP <return>. Reset the stopped memory by:

```
DEC MG10:  Push RESET toggle switch (inside the door, near the bottom)
Suppes:    Push the big green RESET button near the bottom, then push, in this order, the three
            buttons: STOP, CLEAR, CONT. These are located on the little box on top of the memory.
ARM10LX:   Push the four red RESET buttons, one per sector.
new Ampex: Push RESET (the big green-blue button) on Core 0 and Core 1 simultaneously, or Core 2 and
            Core 3 simultaneously.
PDP-6:     push the black CLEAR button inside the cabinet, near the bottom left, then push the white
            RESET button on the front of the small cabinet between the two memory cabinets. You may
            have to repeat this a few times to get UA lit.
```

Once the memory is reset, the computers have to be convinced that nothing bad happened. To do this on the KA-10, the USER MODE light on the console must be off. If it is on, first turn on the SINGLE INSTRUCTION switch on the console, then push CONTINUE. This should turn off the USER MODE light. Then turn off the SINGLE INSTRUCTION switch. When the USER MODE light is off, enter 700200 010003 in the data switches and push the EXECUTE key. If the MEMORY STOP light on the PDP-6 is on, follow the same procedure as on the KA-10 with these exceptions: Instead of a USER MODE light on the console, the PDP-6 has an EXEC MODE light at the top of the second bay to the left of the console; this light must be on for the execute. Also,

there is no SINGLE INSTRUCTION switch on the PDP-6; instead, while the INSTRUCTION STOP switch is down, push INSTRUCTION CONTINUE once. Finally, push CONTINUE on the KA-10 and on the PDP-6, and type RN <return> on the KL-10 console terminal. If the KL is in Executive DDT, you will also have to type CPOPJ<escape>G.

It is quite possible for a hung memory to leave the system in a state from which it can't recover. If the above procedure doesn't work, you'll have to reload, but even that won't work until you reset the hung memory.

APPENDIX 10

HOW TO START WAITS

FIND A WIZARD:

1. Before you fix anything yourself, you should try to find a wizard. Try paging, or if necessary call one at home.

Phone numbers:

BH (55) 751-1762
JBR (9) 494-3597
ME (9) 329-9081
MRC (9) 941-1096
Little Restaurant (9) 323-6540 (try here around dinnertime)

2. If that fails, you'll have to fix it yourself. Make a note in the log with the date and time of the failure, and sign it.

CONTINUING THE SYSTEM:

1. Most crashes will print a message followed by:

FIND A WIZARD OR TYPE "CPOPJ\$G". \$ MEANS ESC. YOU'RE IN DDT.

If it prints this, try typing CPOPJ<escape>G and a couple of returns. If you get monitor dots, you're winning. Type BEEP and return to tell everybody the good news, and record what the message was in the log.

2. If after you type CPOPJ\$G the same thing happens, try CPOPJ\$G again. If it repeatedly happens, you'll have to reload.
3. If no message at all was printed, the problem may be a memory stop. Look up the memory stop procedure just above in the Core Storage section of Appendix 9.
4. If it isn't a memory stop, or some message which doesn't look like the example in step one was printed, you will have to reload. Try to find a wizard.

RELOADING:

1. If there has been a power failure, go to 100.
2. If a message such as KL10 Halted ... appears followed by >. at the left margin go to step 4. The important part is that >. should appear to signify that KLDCP is listening.
3. Type ↑X (ie, control-X). The response should be KLDCP and a CRLF and a period. You may sometimes have to wait a few seconds for this response. If you don't get a period or >. at the left margin, go to 100.
4. Type DS and return. If you get the response DSKDMP go to step 5. If you don't get DSKDMP as a response to DS, then if the KA-10 is running, stop it and repeat this step. If you get DEPOSIT/EXAMINE ERROR IN DS then perform step 102 and repeat step 4. If the KA-10 is stopped and DS doesn't get you DSKDMP, go to 100.
5. Type WAITS and return to DSKDMP. If the system reloads and starts you are winning. Otherwise, if the KA-10 is running, stop it and repeat steps 4 and 5. If the KA-10 is stopped and DSKDMP still doesn't work, you need help.

Don't come here unless directed by the steps above.

100. If the PDP-11 is running and you are talking to KLDCP, go to step 101. Make sure a DECTape labeled "KL10 bootstrap" is mounted on a PDP-11 DECTape drive that is selected to unit 0 and is enabled for "remote" (ie, computer) operation. Press LOAD DECTAPE (located above and to the left of the red "Emergency Power Off" button) and hold it for at least a slow count to one. The DECTape should spin and eventually something like TCDP monitor should be typed. Type in KLDCP and return. KLDCP should load and type a message - go to step 101. If you don't get to TCDP you might try pressing the LOAD DECTAPE button again. If you get to TCDP and the KLDCP command doesn't work, call for help.

101. If there's been a power failure you will have to reload the KL-10's microcode. Type DT0 LR SU to KLDCP and return. If any error messages are printed, you need help.

102. [Only if there's been a power failure or in other exceptional circumstances] After loading the microcode, you must configure the memory adapter by typing I X4 and return.

103. If the command DS gives the message DSKDMP BOOT NOT LOADED, LD COMMAND, then load DSKDMP boot into the PDP-11 by LD BOOT and return.

104. You should now be able to perform steps 4 and 5, but if that doesn't work, you need help.

APPENDIX 11

MONITOR ERROR MESSAGES

This appendix lists the error messages typed by the monitor itself (not by other system programs) with explanations for some of them. Many have to do with errors induced by UUOs in your program, and these are explained in the *UUO Manual*, in the writeup of the UUO involved. The list is alphabetical; messages which start with a variable part are sorted under the first fixed word.

Note: Several of these messages are of the form XXX at user <address>. It is possible in some of these cases to get XXX at exec <address> instead, which means that the offending instruction was not in your program, but in the monitor, which was trying to do something on your behalf when it happened. If this happens repeatably, or if the message is shortly followed by a system crash, tell a system programmer.

Address check for Device <dev>

Your program was trying to use the indicated device, and supplied a buffer address, dump mode command address, or other address which is outside the bounds of your core image. For dump mode IO, the command list is too long or contains a loop, or the address of a word in the command list is illegal, or the address pointed to by a word in the command list is illegal (possibly because it points into a write-protected upper segment). In buffered mode IO, the buffer header address is illegal or the buffers themselves are improperly formed.

Address out of bounds, UUO at user <address>

Your program executed a UUO which takes as an argument an address in your core image, and the address was out of bounds. The address given is that of the offending UUO.

Already assigned to job <job number>

You typed an ASSIGN command, and the device you wanted is in use.

Already attached!

You gave an ATTACH command with your own job number as argument.

Already in use - can't change format

You gave an ASSIGN PUBLIC or ASSIGN PRIVATE command for a UDP which was already assigned as OLD, or vice versa. The UDP will be assigned to your job in the same mode as before.

Already in use - can't make private

You gave an ASSIGN PRIVATE or ASSIGN OLD command for a UDP which was already in use by another job. The UDP will be assigned to your job as PUBLIC.

? ambiguous job name

Your program gave a MAIL UUO which refers to another job by name, and there are two or more jobs with that name. If you type CONTINUE, the UUO will be tried again.

Attempt to initiate SPW job with one already running

Your program has a spacewar module active and is trying to start another one on the same processor.

Bad directory for Device <dev>

The device is a DECtape. This might mean that the tape is in a format not recognized by the monitor. (Our system uses a different directory format from the PDP-10 standard.) It can also happen if you try to write on a write-locked DECtape.

BAD RETRIEVAL

The pointers to your file on the disk are invalid. See a system programmer to try to fix it, or report the filename via GRIPE.

Buffer too large for Device <dev>; UUO at user <address>

Your program is trying to use an I/O device for which there is a limit on the allowable user buffer size, which you exceeded. The *UUO Manual* explains the rules for each device.

busy

You gave a TALK command addressed to a terminal which is in user mode or has characters in its input buffer.

Can't attach device

You gave the privileged ATTACH <device> command and don't have the DEV privilege.

Can't Attach job

You tried to attach a system phantom job (one which is not logged in). This requires the ATT privilege unless you are logged in with the PPN of the phantom, and its programmer name is not SYS.

<dev> can't be reassigned

You gave a REASSIGN command in which the device to be reassigned was your own terminal. This is illegal.

Can't continue

You typed CONTINUE after one of the other error messages came out, and you are not allowed to continue the job after that error. You also cannot continue a job after a SAVE or SSAVE.

Can't detach device

You gave the privileged DETACH <device> command and don't have the DEV privilege.

CAN'T ENTER-RENAME MFD

If you get this message, see a system programmer, who will eat you.

Can't FINISH phantom.

You gave a FINISH command with a job number argument, and the job is a system phantom job. This requires the ATT privilege.

Can't LOCK with segment

Your program, which has an upper segment, gave a LOCK UUO. This is illegal.

Cons trap at user <address>

You can only get this one from a program running on the PDP-6, which means it must be a spacewar module. The CONS instruction is a nonstandard addition to our PDP-6 installed for the benefit of LISP people. It sometimes gives this message. The address is that of the CONS which failed.

CORE DEADLOCK.

The monitor's free storage area has expanded enough so that user core is no longer big enough for your core image. This might fix itself if you wait a while, but if you must run immense programs your best bet is to come back at 4am.

Couldn't get you a segment. Will try to let you win with SETPR2.

You tried running a program with an upper segment. The program was loaded into core, but there is no room in the monitor's job tables to make an entry for the upper segment. The monitor will try to simulate an upper segment for you by leaving the upper segment code in your lower segment, but simulating upper-segment relocation by the SETPR2 UUO mechanism. This ought to work ok for most two-segment programs, but just in case it doesn't the monitor types this warning. If you are writing a two-segment program, consult the *UUO Manual* or a system programmer for advice.

Couldn't unpurify upper. Continue to try anyway

You typed a DDT command. JOBDDT in your core image points to an address in your upper segment as the DDT starting address, and your upper segment is write protected. DDT and RAID both contain instructions which modify themselves. The monitor tries to solve this by unprotecting your upper segment. If you are sharing the segment with other users, it has to load a new copy for you so the other users still have a protected one. This might fail, for example, because there are no job slots available. All highly unlikely.

<dev> deassigned, but still INITed

You gave a DEASSIGN command for a device which is in use by your program. This is all OK; the monitor is merely reminding you that you still control the device. The FINISH command will release it, if that is what you want.

Detached from system

You gave the PJOB <dev> command and the device you specified has been made unavailable to users, perhaps because it is not working.

Device <dev> isn't ready

The device is somehow disabled. If it's a mag tape, it may be in LOCAL mode. The line printer could be out of paper. You can fix the problem and type CONTINUE.

Device <dev> may be unloaded

You gave a DEASSIGN command for a UDP and no one is now using it so you can unload the pack.

? Device <dev> not available

The device specified in a RUN, R, GET, SAVE, or SSAVE command was unavailable to your job.

Device <dev> still in use - DON'T unload

You gave a DEASSIGN command for a UDP which is still being used by someone.

DISK IS FULL!

Your program is trying to write a file on the disk, and there is no room. If you delete some files (using another job!), you can type CONTINUE and the file will be written.

DISK TRANSMISSION ERROR

This is a disk failure. See a system programmer.

Enter failed.

The filename you typed to "Enter needed." (see below) didn't work, probably because of a protection failure.

? Enter failed

The file specified in a SAVE or SSAVE command cannot be written, probably because of a protection failure.

Enter needed. Please type file name:

Your program tried to write on the disk or a DECtape without doing an ENTER UUU to specify the filename. You are given the opportunity to specify a file.

ERROR IN DSKSER

Horrible error in the monitor disk service routine. See a system programmer.

? Error in job <job number>

This message will appear along with one of the others. The only reason you might need it is if you are running more than one job at once (through a PTY, for example).

Error in monitor

This message is usually followed closely by a system crash. Call a systems programmer.

File already exists. Is it OK to delete it?

The filename you typed in response to "Enter needed." (see above) already exists. If you type Y (and RETURN), it will be replaced with the new output; otherwise, you will be asked for another name.

Finish what?

You typed a FINISH command with an argument and weren't using that device (or there is no such device).

Gotcha!

This is not an error. You get it when you are in a device wait queue and your turn for the device comes.

Halt at user <address>

Your program executed a JRST 4, instruction. You are allowed to CONTINUE the job after this message. The address typed is the address containing the HALT instruction, not its effective address. If you type CONTINUE, the program will be continued at the address specified in the effective address of the JRST 4,.

Hung Device <dev>

The device did not respond properly to your attempt to use it in some predetermined time. This probably means that the device is unusable for some hardware reason.

I-level UUO when not at I-level, UUO at user <address>

Your program tried to execute a UUO which is only allowed in a user interrupt routine, when it was not servicing an interrupt.

Ill mem ref at user <address>

Your program tried to read or write an address greater than the size of its core image. The offending instruction is at the address typed, or possibly at one beyond that address (e.g., if the error occurred on a non-final write cycle of a BLT instruction).

Illegal attach loop

Your program is typing into a pseudo-teletype and gave an ATTACH command trying to attach its controlling job. In particular, people using the system via the ARPA network get this if they try to attach to their TELNET server job. (You also get it if you try to attach the job controlling the PTY controlling the job controlling you, etc.) Typed on the PTY.

Illegal data mode for Device <dev>

Your program has tried to use an I/O device in an undefined way, e.g., binary I/O to a terminal. The *UUO Manual* discusses data modes for each device.

ILLEGAL DD CHANNEL.

Your program is trying to use a Data Disc channel to which it is not allowed access.

ILLEGAL FORMAT DUMP MODE COMMAND LIST

Your program is trying to do dump mode I/O incorrectly. See the *UUO Manual*.

Illegal I-level call, UUO at user <address>

Your program executed at interrupt level a UUO which is not legal at interrupt level.

ILLEGAL INSTR. ADDRESS.

This message refers to instructions in a display program your job is running.

Illegal instruction at user <address>

Your program executed an undefined instruction or one which is not allowed in user mode. Could be a KL-only instruction on the KA-10.

Illegal old-style access to new format UDP. UUO at user <ADDRESS>

You have given an ASSIGN OLD command for a UDP, and mounted a pack which has been formatted for file-structured use, and your program tried to write on the pack (gave an ENTER UUO).

ILLEGAL PAGE SIZE SPEC.

You are trying to adjust the page printer geometry incorrectly on a display terminal.

ILLEGAL PIECE OF PAPER

Your program gave a PPIOT UUO which specified a piece of paper number greater than 17 (octal). See the *UUO Manual*.

Illegal protection code

You gave a SETUWP or CLRUWP command with an argument greater than 1000 octal.

Illegal UUO at user <address>

Your program executed a UUO which the monitor does not recognize. (Probably you are trying to execute data.)

in use

You gave the FLUSH command and the specified terminal is in use. You can flush it anyway if you first say ENABLE UPG.

Input Device <dev> cannot do output

Your program has done something silly like trying to write on the paper tape reader.

Input Device <dev> has no input buffer header, UUO at user <address>

Your program executed an input UUO in buffered mode, and had not provided an input buffer header when it opened the device.

Interrupt address out of bounds

Your program has enabled user interrupts and specified an interrupt address which is not in your core image.

Interrupt enablings conflict, UUO at user <address>

Your program tried to enable both old-style and new-style interrupts in a conflicting manner.

Intgen of non-enabled interrupt, UUO at user <address>

Your program tried to use the INTGEN UUO to send itself or another job an interrupt for which the job was not enabled.

I/O to unassigned channel at user <address>

Your program executed an I/O UUO specifying an I/O channel which was not opened. See the *UUO Manual* about channels and I/O.

<dev> is busy, will you wait?

Your program tried to open a device which another job is using. If you say Y (then RETURN), your job is placed in a queue of jobs waiting for the device, and will be continued automatically when it's your turn. If you say N, you are then asked "Would you do IO to the disk instead?" and can again say Y or N. If you say Y, you may be asked for a filename (if your program does input or output on that channel without specifying a name). If you say N again, your program gets a failure return on the INIT or OPEN UUO.

JMS NOT LEGAL; USE JSR.

Your program set up a III display program with the forbidden JMS instruction in it. See the *UUO Manual*.

Job capacity exceeded

The maximum allowable number of users are already logged in. Try again later.

LINKS STILL ACTIVE

Your program tried to re-initialize the system IMP tables in order to bring the IMP back up and someone else is still using the IMP.

Logical name already in use, <dev> Assigned

You typed an ASSIGN command with a logical device name argument which you were already using for another device. The device you requested is assigned to your job, but the logical name still refers to the old device.

Login please

You typed a command which requires that you log in first.

Lookup failed.

The filename you typed in response to "Lookup needed." (see below) could not be read. (It doesn't exist or is read protected against you.) You will be asked to type in another name.

Lookup needed. Please type file name:

Your program tried to read from the disk or a DECtape without specifying a file by executing a LOOKUP UUO. You are given the chance to specify the file to read.

Must assign Device <dev>; UUO at user <address>

Your program tried to INIT a UDP which hadn't been ASSIGNED. You must give the ASSIGN command to set the mode of pack to be used before you can use a UDP.

NEED ENTER BEFORE OUTPUT
NEED LOOKUP BEFORE INPUT

You get these messages if your program opens a directory device, closes a file, and then tries to do input or output on that channel without another LOOKUP or ENTER.

Need TTY or PRG

The argument to the TALK command must be either the name of a TTY or something which might be a programmer name, i.e., up to three letters or digits.

No <dev>'s available

You typed an ASSIGN command with a generic device name (e.g., MTA), and all of them are in use.

No core assigned

You typed a command which refers to your core image (e.g., DE or START), and you don't have a core image.

No DDT

You typed a DDT command, and your core image does not contain DDT or RAID (JOBDDT zero).

No other detached jobs with same PPN.

You typed the ATTACH command with no argument, trying to attach another job of yours, and there was no such job to attach.

No Reenter address

You gave a REENTER command and your program has no REENTER address (JOBREN is zero).

No starting address

You gave a START, R, or RUN command and the program has no starting address (JOBSA zero).

No such device

You typed something the monitor did not recognize when a device name was required.

No such job

You gave a command with a nonexistent job number as argument, e.g., ATTACH, KILL, or TTY WHO.

No such privilege

The privilege name specified in an ENABLE or DISABLE command does not exist.

No such TTY number

You gave a TALK command to a nonexistent TTY.

No upper segment

You typed a SETUWP or CLRUWP command and your core image does not include an upper segment.

? Non-ex job name or number

Your program gave a MAIL UJO addressed to a nonexistent job. If you type CONTINUE, the UJO will be tried again.

Non ex mem at user <address>

This can't happen, it says here. It means that your program tried to address a word of core storage which does not exist. However, we have the maximum possible amount of core, so there shouldn't be any nonexistent addresses. Possibly a hardware failure could cause this. (If you are running a spacewar module which resets its relocation and protection registers when for some reason we are running with less than the full amount of core, you can get this legitimately.)

? not a dump file

The file specified in a RUN, R, or GET command is not in dump file format.

NOT A NEW FORMAT UDP

You have typed an ASSIGN PUBLIC or ASSIGN PRIVATE command for a UDP, and mounted a pack which has not been formatted for file-structured use. The error is not detected until your program tries to perform some operation on the UDP. See a system programmer about formatting your pack.

Not a TTY

You gave the FLUSH command with an argument which is not the name of a terminal. (Might be a nonexistent TTY number.)

Not enough core

You typed a CORE command with an argument greater than the maximum core available to a user job.

? <filename> not found

The file specified in a RUN, R, or GET command was not found. If the problem is not a spelling error, make sure you are not aliased.

Not logged in at ungagged TTY

The programmer name you typed as argument to a TALK command was not logged in, or was logged in at a TTY which is gagged.

Out of bounds

Some numeric argument to a monitor command wasn't right. This could be an address not in your core image in a DE or E command.

Output Device <dev> cannot do input

Your program has tried to do something silly like read the line printer.

Output Device <dev> has no output buffer header, UUO at user <address>

Your program executed an output UUO in buffered mode and did not provide a buffer header when it opened the device.

? <n> pages of core needed

You tried to run a program and asked for less core than it needs to run or for more core than a single job can have. In either case, <n> is the minimum amount of core (in 512-word pages) needed to run the program, unless the program was on a mag tape, in which case <n> is the amount of core you asked for and is more than you can have.

Parity error in your core image at location <address>

Parity error in your upper segment at location <address>

A hardware failure has invalidated the contents of the indicated address in your core image. The safest thing to do in this case is to restart with a fresh copy of your program. If you were running for a long time and are willing to risk continuing, you can type CONTINUE. You can also try to fix the bad location with DDT or RAID, if present, or with the DE and E commands.

PC exceeds mem bounds at user <address>

Your program tried to jump to an address outside the range of your core image. The message, unfortunately, tells you the illegal address rather than the address of the jump instruction.

Pdl ov at user <address>

Your program had a pushdown list overflow. The address typed is the effective address of a PUSHJ, the return address for POPJ, or the address following a PUSH or POP.

PIECE OF GLASS TOO BIG

Your program is trying to run a display program which is too big for the monitor to handle. Sorry.

Please Kjob or Detach

You gave a LOGIN command when you were already logged in.

Please specify a TTY

The programmer name you typed as argument to the TALK command is logged in at more than one terminal. You must give another TALK command, using a particular TTY name as argument instead of the programmer name.

Please type ↑C first

You started your program with CSTART or CCONTINUE, leaving your terminal in monitor mode, and then typed a command which affects your core image. Such commands cannot be processed while the program is running. Type CALL and try again.

PPN mismatch

You typed an ATTACH, FINISH, RESET, or KILL command, and the job number and PPN arguments did not agree.

Protection failure

You typed a SETUWP or CLRUWP command, and are not permitted to change the write protection of your core image's upper segment.

? SAVE/GET IO error

You got an IO error from trying to save or get a core image. Try again. If it still fails, ask a wizard for help.

Spacewar lossage - <error>

The error indicated can be "Ill mem ref", "PC exceeds mem bounds", "Non ex mem", "Pdl ov", "Cons trap", or "Time out". It means that your job started a spacewar module (see the *UUO Manual*), which caused the error. "Time out" means that your spacewar module did not have the timeout-suppression bit set and used up the maximum amount of uninterrupted time allowed in a single run. For the other errors, see the particular error message in this list for an explanation.

Spacewar request for P2, but P2 is hung.

Your program is trying to run a spacewar module on P2 (the KA-10), which is not responding. Get a system programmer to fix it.

SWAP READ ERROR

Your job was swapped out of core, and when the monitor tried to read it back in, there was a hardware error on the swapping disk. Your core image is probably gone forever. If you really need it badly, you might be able to persuade a system programmer to try to recover it for you, but even then there's not much chance. Start over.

Too few arguments

This error message can be typed by several different monitor commands and should be self-explanatory.

TOTAL DPY BUFFER SPACE EXCEEDED.

Your program is trying to run a III display program, and there is no more room in your allocation of display buffer space.

TTY not in use

You gave a TALK command to a PTY which is not in use; you can establish a TALK link to an unused physical TTY but not to a PTY.

UDP OFFLINE OR WRITE LOCKED

Your program has tried to perform some operation on a UDP, and there is no pack mounted, or the drive is not turned on, or it does not have its ID plug inserted in the front panel, or it is switched to read-only and your program tried to write on it.

(Upper not saved)

You gave a SAVE command and you have an upper segment; SAVE does not include the upper segment in the dump file written. To have the upper saved also, use the SSAVE command. (You can give the SSAVE command after getting this message from SAVE if you decide you want the upper saved too.)

Use "KJOB" to log yourself out

You typed a KILL command with your own job number as the argument.

\$\$ USER DPY ERROR AT <address>

This message comes along with another which explains the error in your display program. The address is that of the offending instruction.

User Interrupt-Level <error>

The error can be “ill mem ref”, “NXM”, or “pdl ov”. See the associated message in this list for an explanation (see Non ex mem for NXM). The error happened in your program’s interrupt routine. If it says “CH3 error” instead of one of the above, see a system programmer.

User Interrupt-Level attempt to schedule, UUO at user <address>

Your program’s interrupt routine executed a UUO which would put it in a wait state. User interrupt routines aren’t allowed to do that. See the *UUO Manual*.

User Interrupt-Level timeout

Your program’s user interrupt routine has run longer than the maximum time allowed. (No other user can run while an interrupt routine is in progress, which is why they are limited in time. See the *UUO Manual* about interrupts.)

UUO at user <address>

This line appears with certain other messages. It tells you the location in your core image of the instruction which caused the error.

<dev> wasn’t assigned

You gave a DEASSIGN or REASSIGN command and hadn’t assigned the device.

wasn’t detached

You gave the privileged ATTACH <device> command and have the privilege, but the device wasn’t detached.

YOU ARE LOCKED OUT OF CORE!

The amount of user core available has been reduced by other jobs starting spacewar modules (thereby being locked in core). There is not enough room to fit your core image. This condition may fix itself quickly; the message does not return you to monitor mode, but will keep appearing every so often until you fit again. If you have a really huge program, run it late at night (e.g., 4am).

You can’t have it

You gave an ENABLE command asking for a privilege other than LIV, LUP, or UPG.

APPENDIX 12

BIBLIOGRAPHY

This appendix lists various other sources of information about system programs. Some of these are available on the disk, some are printed. Some programs, which were not written here but came from DEC, are documented in DEC manuals. Other programs, documented on the disk, are not included here; only the major processors are listed. The files AIMS[BIB,DOC], AIMS.OLD[BIB,DOC], SAILON[BIB,DOC], and PRUNE.DAT[UP,DOC] list other program documentation which may be of interest. A convention used in naming [S,DOC] and [UP,DOC] files is that the filename is the name of the program being documented, and the extension is the programmer name of the author. An update file for a program's documentation will have a file name matching the program name with the extension being .UPD. Programmers should have a *UVO Manual*, SAILON 55.5, which describes the monitor interface for user programs. This manual is online as UVO.ME[S,DOC].

SAILONs (Stanford Artificial Intelligence Laboratory Operating Notes) and AIMs (Artificial Intelligence Memos) are generally available in printed form from the project secretaries. Try this before making your own listing.

EDITORS:

E	E.ALS[UP,DOC] is the manual for the display editor.
SOS	SOS.LES[S,DOC], SAILON 50.3, is the reference manual for SOS.
TECO	There is a DEC manual which documents standard DEC TECO. The file TECO.MRC[UP,DOC] documents the version run here, but you should read the DEC manual first.

LANGUAGE PROCESSORS:

FAIL	FAIL.REG[AIM,DOC], AIM 226, is the reference manual.
SAIL	SAIL.KVL[AIM,DOC], AIM 204. This is a long manual; try to get the printed version instead of spooling a copy.
MACRO	This is a DEC program.
F40	This is the DEC FORTRAN.
LISP	Our main dialect, MACLSP, is described in LISP.RPG[S,DOC].
PUB	PUB.TES[S,DOC], SAILON 70, describes the PUB Document Compiler. See also PUB.UPD[S,DOC] for updates.

DEBUGGERS:

RAID	The display terminal debugger is described in RAID.PMP[S,DOC], SAILON 58.1.
DDT	The non-display debugger is a DEC program. See DDT.REG[UP,DOC] for a quick summary.
BAIL	The SAIL program debugger is described in BAIL.DOC[DOC,AIL], AIM 270.

APPENDIX 13

STANFORD CHARACTER SET

The Stanford ASCII character set is displayed in the following table. The three digit octal code for a character is composed of the number at the left of its row plus the digit at the top of its column. For example, the code for "A" is 100+1 or 101.

	ASCII	0	1	2	3	4	5	6	7
	↓↓↓								
	000	NUL	↓	α	β	^	¬	ε	π
	010	λ	TAB	LF	VT	FF	CR	∞	∂
SIXBIT	020	c	⌋	n	u	v	∃	⊗	↔
↓↓	030	_	→	~	≠	≤	≥	≡	√
00	040	SP	!	"	#	\$	%	&	'
10	050	(	)	*	+	,	-	.	/
20	060	0	1	2	3	4	5	6	7
30	070	8	9	:	;	<	=	>	?
40	100	@	A	B	C	D	E	F	G
50	110	H	I	J	K	L	M	N	O
60	120	P	Q	R	S	T	U	V	W
70	130	X	Y	Z	[	\	]	↑	←
	140	'	a	b	c	d	e	f	g
	150	h	i	j	k	l	m	n	o
	160	p	q	r	s	t	u	v	w
	170	x	y	z	{		ALT	}	BS

The tables below display the standard ASCII codes, and the *SOS representation* used at Stanford for entering the full Stanford ASCII character set from non-displays with restricted character sets. The obscure names for the ASCII codes below 40 are listed just for confusion. Notes: "DEL" (177) is the ASCII delete. "ESC" (33) is their alt mode. Codes 136 and 137 have two different interpretations, as shown below. The SOS representation is so called because it is provided by SOS, the non-display editor. Certain other programs also know about this representation, but it is not built into the monitor in any way. See also Section 3 for special system treatment of certain characters from non-displays.

Standard ASCII									SOS Representation								
0	1	2	3	4	5	6	7		0	1	2	3	4	5	6	7	
000	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	000	---	?!	?"	?#	?\$	?%	?&	?'
010	BS	TAB	LF	VT	FF	CR	SO	SI	010	? $\langle$	TAB	LF	VT	FF	CR	?)	?*
020	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	020	?+	?,	?-	?.	?/	?0	?1	?2
030	CAN	EM	SUB	ESC	FS	GS	RS	US	030	?9	?6	?4	?=	?<	?>	?7	?8
040	SP	!	"	#	\$	%	&	'	040	SP	!	"	#	\$	%	&	'
050	(	)	*	+	,	-	.	/	050	(	)	*	+	,	-	.	/
060	0	1	2	3	4	5	6	7	060	0	1	2	3	4	5	6	7
070	8	9	:	;	<	=	>	?	070	8	9	:	;	<	=	>	??
100	@	A	B	C	D	E	F	G	100	@	A	B	C	D	E	F	G
110	H	I	J	K	L	M	N	O	110	H	I	J	K	L	M	N	O
120	P	Q	R	S	T	U	V	W	120	P	Q	R	S	T	U	V	W
130	X	Y	Z	[	\	]	€↑	←	130	X	Y	Z	[	\	]	↑	←
140	'	a	b	c	d	e	f	g	140	?@	?A	?B	?C	?D	?E	?F	?G
150	h	i	j	k	l	m	n	o	150	?H	?I	?J	?K	?L	?M	?N	?O
160	p	q	r	s	t	u	v	w	160	?P	?Q	?R	?S	?T	?U	?V	?W
170	x	y	z	{		}	~	DEL	170	?X	?Y	?Z	?[	?:	?3	?]	BS

APPENDIX 14

MONITOR COMMAND SUMMARY

On the following pages, the monitor commands are listed first alphabetically and then by function. In the alphabetical lists, the command name is printed with the minimum necessary unique abbreviation capitalized and the rest of the name in lower case. One list provides a brief description of the purpose of each command, and another displays the arguments for some commands and gives a page reference to the complete discussion in this manual. If the command loads a program into your core image, its name is listed. The following abbreviations are used:

phydv	physical device name
dv	logical or physical device name
prj	a project name
prg	a programmer name
sa	octal starting address; if omitted use normal starting address
cor	decimal number of 512-word pages to assign to this program
lh	octal left half word
rh	octal right half word
adr	octal address
jn	decimal job number
fn	a file name (may include project-programmer name)
host	an ARPA network host name
prot	octal protection code
phone	telephone number, format {{ddd}}ddd{-}dddd
priv	privilege name
C...>	enclose privileged version of command
→	first program listed automatically loads the next
⊗	program to be loaded is specified as argument
{...}	enclose optional argument
	separates alternative arguments
-	no arguments for this command

NAME	DESCRIPTION	NAME	DESCRIPTION
@	make cross-reference listings with ATSIGN	LATER	run a program later
ADvance	mag tape positioning	LISP	run LISP
ALias	set disk PPN	LIST	list file on line printer
Assign	assign device to job	LOAD	compile and load programs
ATtach	attach job to TTY	LOCate	tell which dump tapes have file
ATtach	c attach phantom job and log it in >	Login	make a new job
ATtach	c attach device to system >	LOGOut	kill this job
BACKspace	mag tape positioning	MAIL	send mail to a user
BATch	scheduled delayed command execution	MAke	create file with TECO
BEep	c beep all terminals >	Ns	read AP or NYT stories
BOok	edit with E in book mode	Otn	old TELNET protocol
CAnceL	delete reminders	PJob	type job using device or this TTY
CContinue	continue job, TTY in monitor mode	PLan	create plan file
CDetach	continue job, detach TTY	POx	compile document with POX
CEtv	create file with E	PPpn	print PPN of job
CFork	continue job, detach TTY, make new job	PREpare	compile, load programs with debugger
CLrup	clear upper segment write protect	PRInt	list file on line printer
COMpile	compile programs	PROcess	execute command string
CONTinue	continue job, TTY in user mode	PTty	type TTY attached to job
COPy	copy files	PUB	compile document with PUB
Core	set or type core size	PUMpkin	ask for offline file restore
CREate	create file with SOS	Qspool	type spooler status and queues
CREF	make cross-reference listings with CREF	R	run program from [1,3] disk area
CStart	start job, TTY in monitor mode	RCv	receive mail for user
DAYtime	type time of day or job times	READ	edit system document with E
DDt	enter DDT or RAID	REASsign	assign device to another job
DE	deposit into core image	REEnter	start program at reenter address
Deassign	deassign device from job	REMind	create a reminder message
DEBug	compile and load programs, start debugger	REName	rename files or change protection
DECide	ask the computer for advice	RER	run program from SYS, give it text lines
DELeTe	delete files	RERUn	run program from DSK, give it text lines
DETach	detach job from TTY	RESEt	reset a job
DETach	c detach device from system >	RESources	type available system resources
DIAL	talk to computer by telephone	RESTore	restore files from dump tape
DIrectory	type file statistics	RETry	try again to send queued mail
DISable	c disable privileges >	REWind	mag tape positioning
DO	execute commands from a file	RSI	reserve service level or devices
DRd	run directory editor	RUUn	run a program from core image file
DSksiz	find out disk usage and quota	SAve	save core image in a file
DTn	DM simulator TELNET	SEND	send message to user's TTY
DUMp	dump disk files to mag tape	SETuwp	set upper segment write protect
E	examine core image	SLeveL	type service level assignment
EDDt	c stop system, enter Exec DDT >	SPool	request line printer listing
EDIt	edit file with SOS	SSave	save two-segment core image
EFind	find string in E-format file	Start	start job, TTY in user mode
ENable	c enable privileges >	SUPdup/SD	run SUPDUP display-tnet program
EOt	mag tape positioning	SYstat	type system status
ERead	edit file with E in readonly mode	TAlk	talk to another TTY
ETv	edit file with E	TECo	edit file with TECO
EVeNt	list an event in the system messages	TELnet/TN	talk to ARPA net computer
EXecute	compile, load, and run programs	TEST	run SYS:TEST.DMP if it exists
FIles	type status of files in use	TIme	type runtime for job or system
FIND	find character string in a file	TList	list files on a dump tape
FINGEr	type name and location of users	TRAnsfer	copy files, delete source
Finish	close and release device	TRY	compile, load with debugger, run
FIXimlac	initialize IMLAC terminal	TTy	set TTY parameters
FLush	clear TTY buffers	TURkey	see who last used a DART tape
FORk	detach, make new job	TYpe	type a file
FTp	ARPA net file transfer protocol	UDpufd	modify UDP file directory
Get	load core image from file	UFd	modify DSK file directory
GRipe	document system bugs	UNDelete	try to recover deleted file
HAIt	stop job (↑C)	UNProtect	reduce protection of a file
HELLO	type name of current monitor version	UNSpool	delete spooler request
HELP	explain system program	WHEN	see when a user last logged out
HOt	read AP or NYT news wire	WHere	type job info for PPN or job name
KATtach	kill this job, attach another	Who	display system status
KIll	kill another job	XGplisT	list file on XGP
Kjob	kill this job	XSpool	request XGP listing
KLog	kill this job, make another	Zero	clear DTA, UDP, or DSK directory

188 Monitor Command Summary

A14.

NAME	PROGRAM	ARGUMENTS	PAGE	NAME	PROGRAM	ARGUMENTS	PAGE
@	ATSIGN		68	LATer	MAIL		123
ADvance	DART		135	LISP	LISP	-	66
ALias		prj{,prg}	47	LIST	COPY		95
Assign		phydv {dv}	43	LOAD	SNAIL→		75
ATtach		jn{[prj,prg]}	42	LOCate	DART		136
ATtach		cjn{[prj,prg]}>	69	Login	LOGIN	prj,prg	48
ATtach		c dv >	69	LOGOut	LOGOUT	-	53
BACKspace	DART		136	MAIL	MAIL		117
BATch	MAIL		125	MAKe	SNAIL→TECO	fn	71
BEep < CTY only >		-	70	Ns	NS	-	67
BOok	E	{fn}	73	Otn	OTN	site	143
CAnceL	CANCEL	-	130	PJob		{dv}	45
CContinue		-	41	PLan	MAIL		123
CDetach		-	43	POx	SNAIL→POX	{fn}	73
CEtv	E	fn	71	PPpn		{jn}	46
CFork		-	43	PREpare	SNAIL→		75
CLrup		{prot}	41	PRInt	COPY		95
COMpile	SNAIL→		75	PROcess	SNAIL→		74
CONTinue		-	41	PTty		{jn}	46
COPy	COPY		87	PUB	SNAIL→PUB	{fn}	73
Core		{cor}	40	PUMpkin	DART		137
CREAtE	SNAIL→SOS	fn	71	Qspool	SPOOL	-	102
CREf	SNAIL→CREf	-	82	R		fn {cor}	39
CStArT		{sa}	41	RCv	RCV		130
DAytime		{jn}	46	READ	E	{fn}	72
DDt		-	41	REASsign		dv jn	45
DE (deposit)		lh rh {adr}	41	REEntEr		-	41
Deassign		{dv}	44	REMinD	MAIL		121
DEBUg	SNAIL→		75	REName	COPY		95
DECIde	ORACLE	-	68	RER	SNAIL→		74
DELeTe	COPY		95	RERUn	SNAIL→		74
DETach		-	42	RESEt		{jn{[prj,prg]}}	47
DETach		c dv >	69	RESources		-	46
DIAL	DIAL	dv phone	67	REStore	DART		136
DIRectory	COPY		95	RETry	MAIL	-	123
DISable		c {priv} >	69	REMinD	DART		135
DO	DO	{fn}	57	RSL		-	138
DRd	DIRED	{[prj,prg]}	68	RUn		{dv} fn {cor}	39
DSksiz	DSKSIZ	{prg}	66	SAve		{dv} fn {cor}	40
DTn	DTN	site	148	SENd	MAIL		120
DUMp	DART		136	SETuwp		{prot}	41
E (examine)		{adr}	41	SLeveL		-	47
EDDt < CTY only >		-	70	SPool	SPOOL		97
EDit	SNAIL→SOS	{fn}	71	SSave		{dv} fn {cor}	40
EFind	EFIND		64	StArT		{sa}	41
ENable		c {priv} >	69	SUPdup/SD	SUPDUP	site	149
EOt	DART		135	Systat	WHO	-	66
ERead	E	{fn}	72	TAlk		dv	47
ETv	E	{fn}	71	TECo	SNAIL→TECO	{fn}	71
EVent	MAIL		122	TELnet/TN	TELNET	host	143
EXecute	SNAIL→		75	TEST	TEST		68
FIles		{jn fn}	46	TIme		{jn}	46
FIND	FIND		62	TList	DART		136
FINGER	FINGER	{prg}	65	TRAnsfer	COPY		95
Finish		{dv}	45	TRY	SNAIL→		75
FIXimlac	FIXIML	-	142	TTy			37
FLush		dv	45	TURkey	DART		136
FORk		-	43	TYpe	COPY		95
FTp	FTP	host	150	UDpufd	UFD	prj,prg	67
Get		{dv} fn {cor}	39	UFd	UFD	prj,prg	67
GRipe	MAIL		122	UNDelete	UNDELE	fn	68
HAIt (or ^C)		-	42	UNProtect	UNPROT	fn	68
HELLO		-	47	UNSpool	SPOOL	-	103
HELP	HELP	{fn}	66	WHEN	WHEN	prg	66
HOt	HOT	-	67	WHERe	WHERE	{prj,} {prg}	65
KAttach	LOGOUT	jn{[prj,prg]}	53	Who	WHO	-	58
KIll		jn	47	XGplisT	COPY		95
KJob	LOGOUT		52	XSpool	SPOOL		100
KLog	LOGOUT→LOGIN	prj,prg	53	Zero	ZERO	dv	58

Commands that run programs:

SNAIL Class	COPY Class	DART Class	Information
BOOK	COPY	ADVANCE	EFIND
CETV	DELETE	BACKSPACE	FIND
COMPILE	DIRECTORY	DUMP	FINGER
CREATE	LIST	EOT	HELP
CREF	PRINT	LOCATE	SYSTAT
DEBUG	RENAME	RESTORE	WHEN
EDIT	TRANSFER	REWIND	WHERE
EREAD	TYPE	TLIST	WHO
ETV	XGPLIST	TURKEY	
EXECUTE			
LOAD	SPOOL Class	MAIL Class	Others
MAKE			
POX	QSPool	BATCH	@
PREPARE	SPOOL	CANCEL	DECIDE
PROCESS	UNSPool	EVENT	DIAL
PUB	XSPool	GRIPE	DO
READ		LATER	DRD
RER	TELNET Class	MAIL	DSKSIZ
RERUN		PLAN	FIXIMLAC
TECO	DTN	RCV	HOT
TRY	FTP	REMIND	LISP
	OTN	RETRY	NS
LOGIN Class	SD	SEND	RSL
	SUPDUP		TEST
KATTACH	TELNET		UDPUFD
KJOB	TN		UFD
KLOG			UNDELETE
LOGIN			UNPROTECT
LOGOUT			ZERO

Other system commands:

SAVEGET Class	START Class	Other	Information
CORE	CCONTINUE	ALIAS	DAYTIME
GET	CONTINUE	BEEP	FILES
R	CSTART	CLRUWP	HELLO
RUN	DDT	DE	PJOB
SAVE	HALT	DISABLE	PPPN
SSAVE	REENTER	E	PTTY
	START	EDDT	RESOURCES
Device Control		ENABLE	SLEVEL
	ATTACH Class	KILL	TIME
ASSIGN		SETUWP	
DDASSIGN	ATTACH	TALK	
FINISH	CDETACH	TTY	
FLUSH	CFORK		
REASSIGN	DETACH		
RESET	FORK		

- @ command 68
- activation 17
- activation character 14, 19
- activation characters 17
- ADC device 8
- add channel 28
- add piece of glass 29
- addresses, core 3, 10
- ADVANCE command 135
- AL 71
- AL switch in SNAIL 77, 83
- ALGOL 2
- alias 3, 21, 22, 43, 46, 47, 65
- ALIAS (in WHO display) 59
- ALIAS (in WHO line) 21
- ALIAS command 47
- ALL switch in COPY 91
- ALT 36
- ALT key 13
- alt mode 13, 185
- ALT MODE, non-display 36
- APPEND switch in MAIL 112
- arguments 6
- ARPA network 8, 35, 62, 65, 67, 115, 121, 143
- ARPA switch in MAIL 113
- ARROW (in TTY command) 38
- ARROW (LOGIN option) 51
- ASCII 13, 144, 185
- ASCII switch in COPY 91
- ASK switch in COPY 91
- ASK switch in SPOOL 98
- assembly language 2
- ASSIGN command 7, 43
- Associated Press 51, 67
- ATSIGN program 68
- ATT privilege 42, 69
- ATTACH command 42, 53, 69
- AUDIO (LOGIN option) 50
- audio switch 29, 50
- authorized users 9
- available channel 28
- backspace 13
- BACKSPACE command 135, 136
- backspace, deleting 15, 16
- backspace, non-deleting 15
- BAIL 71, 184
- BAIL switch in SNAIL 79, 83
- bams 138
- BATCH command 125
- BEEP 30
- BEEP (in TTY command) 38
- BEEP (LOGIN option) 50
- BEEP command 70
- beeping, automatic 30, 50
- bibliography 184
- BINARY switch in COPY 92
- binary, relocatable *see* relocatable binary
- BLINK (in TTY command) 32
- BLISS 71
- BLISS switch in SNAIL 77, 83
- BLOCKED switch in COPY 92
- BMAR switch in XSPOOL 100
- BOLD (in TTY command) 32
- BOLD (LOGIN option) 51
- BOOK command 73
- book mode 73
- BOW (LOGIN option) 51
- BREAK 18
- BREAK (in TTY command) 38
- BREAK I X 22, 38
- BREAK A 28
- BREAK B 30
- BREAK C 24
- BREAK CALL 26
- BREAK CLEAR 27
- BREAK D 28
- BREAK F 19, 38
- BREAK G 27
- BREAK H 25
- BREAK key 14
- BREAK L 27
- BREAK M 26
- BREAK N 24
- BREAK O 19
- BREAK P 24
- BREAK Q 20
- BREAK R 26
- BREAK S 28
- BREAK T 28
- BREAK U 30
- BREAK W 20
- BREAK X 22, 38
- BS 15
- BS (in TTY command) 32
- BS (LOGIN option) 51
- BS key 13
- bucky bit (non-display escape) 37
- bucky bits 14
- CALCOMP 8
- CALL 18, 22, 35, 41, 42, 47
- CALL key 14
- call, deferred 18
- cameras, TV 25
- CANCEL command 130

- CAR device 8
- carriage return 13, 17, 36, 38, 51
- CC switch in MAIL 112
- CCONTINUE command 42
- CDETACH command 43
- CETV command 10, 72
- CFORK command 43
- channel number 27
- channels, Data Disc 4, 21, 25, 45, 46, 69
- channels, private 25, 51
- channels, public 25
- CHAR (job queue) 22
- character conversion, non-display 35
- character set 4, 13, 185
- character set, full 19
- CLEAR 16
- CLEAR key 14
- clear line editor 16
- clear page printer 24
- CLRUWP command 41
- CMQ (job queue) 22
- code, character 13, 185
- command decoder 6, 14
- command files 57
- commands, monitor 6, 186
- commercial-free radio 30
- COMPILE command 11, 75
- COMPILE switch in SNAIL 76, 78, 83
- compiling programs 10
- complaints, compliments, criticisms 122
- ONLY switch in SNAIL 76, 78, 83
- continuable 41
- CONTINUE command 22, 41
- CONTROL 57
- CONTROL and META (non-display escape) 37
- CONTROL key 14, 144
- CONTROL key, non-display 35
- CONTROL-B 16
- CONTROL-B, non-display 36
- CONTROL-BREAK 19, 24
- CONTROL-BS 15
- CONTROL-C, non-display 35
- CONTROL-CALL 18
- CONTROL-CLEAR 19, 24
- CONTROL-D 15
- CONTROL-FORM 15
- CONTROL-I 16
- CONTROL-I, non-display 36
- CONTROL-K 16
- CONTROL-K, non-display 36
- CONTROL-L 16
- CONTROL-L, non-display 36
- CONTROL-META-BS 15
- CONTROL-META-LINE 19
- CONTROL-META-SPACE 15
- CONTROL-number 16
- CONTROL-O, non-display 36
- CONTROL-R 16
- CONTROL-RETURN 16, 17
- CONTROL-S 16
- CONTROL-SPACE 15
- CONTROL-T 16
- CONTROL-TAB 15
- CONTROL-U, non-display 35
- control-underscore 36
- CONTROL-Z, non-display 36
- CONVERT (in TTY command) 38
- CONVERT (LOGIN option) 51
- CONVERT switch in COPY 92
- COOKIE (LOGIN option) 51
- COPY 55, 87
- COPY command 55
- core *see* storage, core
- CORE command 40
- core image 3, 6, 10, 12, 21, 22, 39, 40, 48, 59
- COUNT switch in MAIL 112, 115
- crash 168
- CREATE command 71
- CREF command 82
- CREF switch in SNAIL 78, 82, 83
- CSTART command 41
- CTRL key, non-display 35
- CTY device 8
- cursors 5, 14
- DAC device 8
- DART 56, 135
- data 7
- Data Disc queue 26
- Data Discs 4, 13, 14, 20, 23, 24
- Datamedia 4
- DATAMEDIA (in TTY command) 32
- Datamedias 23, 31
- DATE (in WHO line) 21
- DATE switch in MAIL 112, 113
- DAY (in WHO line) 21
- DAY filehack in COPY 89
- DAYTIME command 6, 46
- DCQ (job queue) 22
- DD (in WHO line) 21
- DDT 40, 41, 71, 79, 184
- DDT command 41
- DDT switch in SNAIL 79, 83
- DDT, Executive 70

- DE (deposit) command 41
- DEASSIGN command 44
- DEBUG command 75
- debugging program *see* DDT or RAID
- DECIDE command 68
- DECtape 7, 8, 22, 58, 162
- default file protection, UFD's 94
- Defense, Department of 143
- deferred call 18
- delete 13, 185
- delete channel 28
- DELETE command 55, 95
- delete piece of glass 29
- DELETE switch in SPOOL 98
- deletion of REMINDers, LATER requests, and
queued MAIL 130
- DENSITY switch in COPY 92
- deposit in core *see* DE
- DET (in WHO display) 59
- DET (in WHO line) 21
- DETACH command 42, 69
- detached jobs 21, 42, 57, 59
- DEV privilege 69
- device 43, 45, 161
- device name 7, 69
- device name, logical 7, 43
- device name, physical 7, 8, 43
- device names, list of 8
- DIAL command 67, 144
- DIGEST (LOGIN option) 51, 52
- DIGEST filehack in COPY 89
- DIOW (job queue) 22
- directories 7
- DIRECTORY command 55, 95
- directory device 7, 45
- directory, telephone 62
- DIRED program 68
- DISABLE command 69
- disk 7, 21, 39, 44, 46, 47, 55, 56, 58
- disk PPN 3, *see* alias
- display *see* terminals, display
- DIST switch in MAIL 112
- DLN device 8
- DM (in TTY command) 32
- DM (in WHO line) 21
- DM (LOGIN option) 50
- DM128 (in TTY command) 32
- DM128 (LOGIN option) 50
- DM128=n (LOGIN option) 50
- DM=n (LOGIN option) 50
- DMP files 7, 10, 39, 83
- DO 74, 81
- DO command 57
- documentation, program 4, 184
- double bucky 14
- DOWN filehack in COPY 89
- DRD command 68
- DSK device *see* disk
- DSKF (in WHO line) 21
- DSKOPS (in WHO line) 21
- DSKQ (in WHO line) 21
- DSKSIZ command 66
- DSPOOL switch in COPY 92
- DTA device *see* DECTape
- DTN command 67, 143
- DTQ (job queue) 22
- DUMP command 135, 136
- dump file *see* DMP files
- DUMP switch in SPOOL 98
- DUMPED switch in COPY 92
- DWQ (job queue) 22
- E 9, 14, 19, 71, 184
- E (examine) command 41
- E switch in MAIL 112
- EBOX msec (in TIME command) 46
- ECHO (in TTY command) 38
- ECHO (LOGIN option) 51
- EDDT command 70
- EDIT command 71
- editing 9
- editor, display 9, 72
- editor, line *see* line editor
- EFIND command 64
- ELF device 8
- ENABLE command 69
- end of file 19, 36
- EOT command 135
- EREAD command 72
- error messages 170
- ESC 18
- ESC * 19
- ESC + 19
- ESC I X 22, 38
- ESC A 28, 29
- ESC B 30
- ESC C 24
- ESC CALL 26
- ESC CLEAR 27
- ESC D 28, 29
- ESC E 24
- ESC F 19, 38
- ESC FORM 24
- ESC G 24
- ESC H 19, 25

- ESC I 19
- ESC J 24
- ESC key 14
- ESC L 24
- ESC M 26
- ESC N 24
- ESC O 19
- ESC P 24
- ESC Q 20
- ESC R 24
- ESC S 28, 29
- ESC T 28
- ESC U 30
- ESC V 19
- ESC W 20, 42
- ESC X 22, 38
- ESC Y 23
- ESC Z 30
- ESCAPE (in TTY command) 38
- ESCAPE, non-display 36
- ETV command 10, 72
- ETV editor *see* E
- EVEN switch in COPY 92
- EVENT command 122
- examine core *see* E
- EXECUTE command 11, 75
- EXPAND switch in MAIL 113
- extension 7
- extensions, standard 7
- EXTRA switch in COPY 92
- EXTRA switch in SPOOL 98
- F4 switch in SNAIL 77, 83
- F40 71, 184
- FACT.TXT file 97
- FAIL 2, 71, 184
- FAIL switch in SNAIL 77, 83
- FASM 71
- FAST (LOGOUT option) 53
- FAST switch in COPY 92
- FDM (LOGIN option) 50
- FDM128 (LOGIN option) 50
- FDM128=n (LOGIN option) 50
- FDM=n (LOGIN option) 50
- FF switch in SPOOL 98
- file access groups 49
- file directory *see* directory
- file protection 48
- file storage 7
- File Transfer Protocol 67, 143, 150
- filenames 7
- FILES command 19, 46
- files, disk 7, 22
- FILL (in TTY command) 38
- FILL (LOGIN option) 51
- FIND command 62
- FINGER command 11, 53, 65
- FINISH command 45
- FIXIMLAC command 67, 142
- FLUSH command 45, 69
- FNOTABS (LOGIN option) 51
- FONT switch in COPY 92
- FONT switch in XSPOOL 100
- FOO switch in COPY 92
- FORK command 43
- form feed 13, 36, 89
- FORM key 13
- FORTTRAN 71, 184
- FORTTRAN switch in SNAIL 83
- FORTTRAN switch in SPOOL 98
- fortune cookie 51
- FORWARD filehack in COPY 89
- FORWARD switch in SNAIL 81, 83
- forwarding, mail 118
- FRM device 8
- FTABS (LOGIN option) 51
- FTP command 67, 143, 150
- FUDGE2 80
- FULL (in TTY command) 38
- FULL (LOGIN option) 51
- full character set mode 19, 51
- FULL switch in COPY 92
- gag 47
- GAG (in TTY command) 38
- GAG (LOGIN option) 50
- GET command 39
- GLASS (in TTY command) 38
- GLASS (LOGIN option) 51
- glass TTY 36
- glass, pieces of 23, 29
- glitch 23
- GOLD filehack in COPY 89
- graphics 4, 23
- GRIPE (LOGIN option) 50
- GRIPE command 122
- GRIPES filehack in COPY 89
- GTOTAL switch in COPY 92
- guest user 48
- HALT command 42
- HANG (in TTY command) 38
- HEADER switch in COPY 92
- HEADER switch in MAIL 112
- HEADING switch in SPOOL 98
- HELLO command 47
- HELP command 11, 66

- hide 19
- HIDE (LOGIN option) 51
- HOLD switch in SPOOL 98
- holding 19
- holding, automatic 24
- home terminal 26
- HOT command 67
- I/O *see* device
- IGNI switch in COPY 92
- IGNO switch in COPY 92
- III (in WHO line) 21
- IIIs 4, 13, 14, 23, 29
- IMLAC 67
- IMP device 8, 143
- Increment (in TIME command) 46
- information, system 4, 11, 45, 58
- INIT (LOGIN option) 50
- input buffer 6, 17, 18, 57
- Interface Message Processor *see* IMP
- interrupt 14
- Introduction for New Users 1
- Introduction to Terminals 4
- INTW (job queue) 22
- IOWQ 22, 42
- IOWQ (job queue) 22
- JBS (in WHO line) 21
- JLOG 42, 58, 69
- JOB (in WHO display) 59
- JOB (in WHO line) 21
- Job Data Area 12, 40, 41
- job name 39, 59, 65
- job number 3, 6, 9, 42, 45, 46, 47, 59, 65
- JOBDDT 40, 41
- JOBFF 40
- JOBNAM (in WHO display) 59
- JOBNAM (in WHO line) 21
- JOBPC 41
- JOBRD 22
- JOBREN 41
- jobs 21
- JOBSA 12, 41
- KA-10 22, 59
- KATTACH command 53
- keyboard mapping 25
- keyboard scanner 14, 18
- keyboard, SUMEX 31
- keyboards 13
- KILL command 47
- KILL switch in COPY 92
- KIM device 8
- KJOB command 9, 52
- KL-10 22, 59
- KL-10 console 70
- KLOG command 53
- L switch in E 72
- languages, programming 2, 184
- LATER command 123
- LATER requests, deleting 130
- letters, lower case 6, 7, 19, 35, 38
- LIBRARY switch in SNAIL 76, 80, 83
- LINE *see* line feed
- LINE (in WHO display) 59
- LINE (job queue) 22
- line editor 5, 13, 14, 17, 19, 24
- line feed 13, 36
- LINE key 13
- line number, terminal 27, 46, 59
- line numbers, SOS 9, 91, 93, 98
- line printer *see* printer, line
- LINK 71
- LINK switch in SNAIL 79, 83
- LISP 2, 57, 184
- LISP command 66
- LIST command 55, 95
- LIST switch in COPY 93
- LIST switch in MAIL 112
- LIST switch in SNAIL 78, 83
- LIV privilege 3, 69
- LMAR switch in XSPOOL 100
- LOAD (in WHO line) 21
- LOAD command 11, 75
- LOADER 40, 41, 71, 83
- LOADER switch in SNAIL 79, 83
- loader switches 82, 84
- loading programs 10
- local user 48, 67, 69
- LOCATE command 135, 136
- LOCK 22
- logical device name *see* device name, logical
- LOGIN command 3, 6, 8, 48
- login, automatic 6
- LOGOUT 9, 10, 52
- LOGOUT command 53
- logout, automatic 3
- LOGOUT.MSG file 53
- LOGRUN 52
- LOGRUN (LOGIN option) 50
- LONLY switch in SNAIL 76, 78, 83
- lower segment 12
- LPT device *see* printer, line
- LPT0 switch in SPOOL 98
- LPT100 switch in SPOOL 98
- LUP privilege 69
- MACRO 71, 184

- MACRO switch in SNAIL 77, 83
- MAGIC 29
- magnetic tape 7, 8, 22, 39, 43, 56, 162
- MAIL 50, 56, 104
- MAIL command 48, 117
- MAIL error recovery 126
- MAIL filehack in COPY 89
- MAIL switch in MAIL 111
- MAIL\$E.TXT file 106, 127
- mail, ARPA network 115
- mail, deleting queued 130
- MAIL, interfacing with, from other programs 127
- maintenance mode 50
- MAKE command 71
- MAP switch in SNAIL 79, 83
- mapping 26
- mapping, keyboard *see* keyboard mapping
- MAS access group 49
- ME (LOGIN option) 51
- ME (LOGOUT option) 53
- memory stops 166
- MESSAG (LOGIN option) 50, 52
- messages, sending and mailing 104
- messages, system 9, 48, 117, 122
- META 15, 16, 57
- META key 14
- META-BS 15
- META-CALL 18
- MICRO-PLANNER 2
- MIDAS 71
- MIDAS switch in SNAIL 77, 83
- MLENGTH switch in COPY 93
- MODE switch in SPOOL 98
- monitor mode 6, 9, 14, 18, 39, 41, 42
- MSG filehack in COPY 89
- MTA device *see* magnetic tape
- MTQ (job queue) 22
- Multics 151
- N switch in E 72
- NAP filehack in COPY 89
- NARROW switch in SPOOL 98
- New York Times 67
- news service *see* NS
- NL (in WHO line) 21
- NO switch in MAIL 112
- NOARROW (LOGIN option) 51
- NOBOLD (LOGIN option) 51
- NOBOW (LOGIN option) 51
- NOBS (LOGIN option) 51
- NOCMFI switch in SNAIL 76, 83
- NOCONVERT (LOGIN option) 51
- NOCOPY switch in SPOOL 98
- NODELETE switch in SPOOL 98
- NODIST switch in MAIL 112
- NODMP switch in SNAIL 76, 80, 83
- NODUMP switch in SNAIL 80
- NODUMP switch in SPOOL 98
- NOECHO (LOGIN option) 51
- NOFF switch in COPY 93
- NOFF switch in SPOOL 98
- NOFILL (LOGIN option) 51
- NOFORTRAN switch in SPOOL 98
- NOFULL (LOGIN option) 51
- NOGAG (LOGIN option) 50
- NOGLASS (LOGIN option) 51
- NOHEADING switch in SPOOL 98
- NOLOAD switch in SNAIL 76, 81, 83
- NOMAIL (LOGIN option) 50, 52
- NOMAIL switch in MAIL 111
- non-displays 51
- NONARROW switch in SPOOL 98
- NONOTE (LOGIN option) 50
- NONUMBER switch in SPOOL 98
- NONUMBERS switch in COPY 93
- normal activation mode 17
- normalize page printer 24
- NOSAISEG switch in SNAIL 79, 83
- NOSPACES switch in COPY 93
- NOTABS (LOGIN option) 51
- NOTICE filehack in COPY 89
- NOTICE.TXT file 48, 117, 122, 130
- NOTITLE switch in SPOOL 98
- NOWARN switch in SPOOL 98
- NOXGP switch in XSPOOL 100
- NS 52, 130
- NS command 67
- NS filehack in COPY 89
- NTNODE switch in XSPOOL 100
- NUL device 8
- null job 46, 59
- null-time 21
- NULQ (job queue) 22
- NUMBER switch in SPOOL 98
- number, line *see* line numbers
- NVNODE switch in XSPOOL 100
- OCTAL switch in SPOOL 98
- ODD switch in COPY 93
- OFFSET switch in COPY 93
- old mode (UDP) 44
- operator 47
- OPTIMIZE switch in COPY 93
- OPTION filehack in COPY 89
- OPTION.TXT file 48, 50, 52, 134

- OTN command 67, 143
- OUTGO switch in MAIL 113
- OUTGO.MSG file 108, 133
- output buffer 19
- P switch in E 72
- page printer 23, 24, 29
- Page-Seconds 46
- pages 12
- paging, telephone 30
- PALX 71
- PALX switch in SNAIL 77, 83
- paper tape 8
- paper, pieces of 23
- Partial font loading 100
- PASCAL 71
- PASCAL switch in SNAIL 77, 83
- password 48, 58
- PAUSE switch in COPY 93
- PDP-10 1, 2, 13
- PDP-10 console 8
- PDP-11 device 8
- PDP-6 8
- PGX pseudo-device (COPY) 91
- phantom 42, 58, 69
- physical device name *see* device name, physical
- pieces of glass 23
- pieces of paper 23
- PJOB command 45
- PL (in WHO display) 59
- PLAN command 123
- plan file 65, 89, 123, 130
- PLAN filehack in COPY 89, 123
- PLN filehack in COPY 89, 123
- plotter 8
- PLT device 8
- PMAR switch in XSPool 100
- PORNO (LOGIN option) 51
- POX 71
- POX command 73
- PPN 3, 7, 8, 22, 42, 46, 47, 48, 53, 65, 69
- PPN (in WHO display) 59
- PPN (in WHO line) 21
- PPN, disk *see* alias
- PPPN command 46
- PREFIX, non-display 36
- PREPARE command 75
- PRINT command 55, 95
- printer, line 7, 8, 43, 55, 95, 97, 161
- private device 44
- privilege, file access 49
- privileged commands 69
- PROCESS command 74
- PROCESSOR switch in SNAIL 80, 83
- programmer name 3, 20, 47, 56, 65
- programs, display 23, 29
- programs, system 4, 39, 48
- programs, system information 6, 11
- project name 3
- project-programmer name *see* PPN
- protected mail files 118
- protection 48
- PROTECTION switch in COPY 93
- PS (in TIME command) 46
- pseudo-mailbox 120
- pseudo-teletype *see* PTY
- PTP device 8
- PTR device 8
- PTTY command 46
- PTY 8, 54, 58
- PTYJOB 144
- PUB 71, 184
- PUB command 73
- public device 44
- PUMPKIN command 135, 137
- QMAINT (LOGIN option) 51
- QSPool 102
- QSPool command 55
- QSPool switch in SPOOL 98
- queue 65
- QUEUE (in WHO display) 59
- QUEUE (in WHO line) 21
- queue names 22
- QUEUE switch in MAIL 112
- queued local mail 118
- queued mail 112, 116, 118, 123
- queued mail, deleting 130
- QUIET switch in COPY 93
- R (in WHO line) 21
- R command 39, 48
- R switch for editors 72
- RAID 40, 41, 71, 79, 184
- RAID switch in SNAIL 79, 83
- RCOR (in WHO line) 21
- RCV 130
- READ command 11, 72
- REASSIGN command 45
- RECOPY switch in SPOOL 98
- records 7
- REENTER command 41
- REFERENCED switch in COPY 93
- refresh 24
- REL files 7
- REL switch in SNAIL 76, 83

- reloading 168
- relocatable binary 7, 10
- REMINd command 121
- reminders, deleting 130
- RENAME command 55, 95
- RENAME switch in COPY 93
- repeat escape command 19
- REPEAT switch in SPOOL 98
- RER command 74
- RERUN command 74
- RESET 24, 40
- RESET command 47
- RESOURCES command 46
- responsible keyboard 25
- RESTORE command 135, 136
- retrieve last line 16
- RETRY command 123
- RETURN *see* carriage return
- RETURN key 13, 14
- REWIND command 135
- RMAR switch in XSPOOL 100
- RP (in WHO line) 21
- RPG 52, 53
- RPG filehack in COPY 89
- RPGSAV (LOGIN option) 50
- RPGSAV (LOGOUT option) 53
- RSL command 66, 138
- RTIME 22
- RTJ device 8
- RUBOUT, non-display 35
- RUN (LOGIN option) 50
- RUN command 39
- run time 46, 59, 65
- running programs 10, 39
- RUNQ (job queue) 22
- RUNTIME (in WHO line) 21
- runtime, incremental 21
- S1 switch in SNAIL 77, 83
- SAI files 7
- SAIL 2, 71, 184
- SAIL switch in SNAIL 77, 83
- SAM device 8
- SAVE command 40
- SAVE switch in COPY 93
- SAVE switch in SNAIL 79, 83
- SAVED.MSG file 132
- scanner, keyboard *see* keyboard scanner
- scrolling 23
- SD command 67, 143
- SEARCH switch in COPY 94
- SEG (in WHO display) 59
- segment, lower *see* lower segment
- segment, upper *see* upper segment
- segments 12
- SEGNAM (in WHO line) 21
- select audio channel 30
- select channel 28
- select piece of glass 29
- SEND command 120
- SEND switch in MAIL 111
- service level 47, 59, 66
- SETPR2 172
- SETUWP command 41
- SHIFT key, non-display 35
- SHIFT keys 4, 13
- SHIFT LOCK key 4, 13, 19
- SIX device 8
- SIZE (in WHO display) 59
- SIZE (in WHO line) 21
- SL (in WHO display) 59
- SLEEP 22
- SLEVEL command 47
- SNAIL 10, 54, 71
- SNDMSG *see* MAIL program
- SNK (job queue) 22
- SOS 9, 71, 184
- SOS representation 35
- Spacewar 3
- spacewar module 22, 59
- special activation mode 17
- SPEED (in TTY command) 38
- SPOOL command 55, 97
- SPOOL switch in COPY 94
- spoolers 55, 91
- spying 28
- SSAVE command 40
- Stanford ASCII 13
- START command 41
- starting programs 41
- status, system 20
- STOP (job queue) 22
- stop timeout 19, 36
- storage, core 2, 11, 21, 166
- SUBJECT switch in MAIL 112
- SUPDUP 149
- SUPDUP command 67, 143
- suspend timeout 19, 36
- Suspended XGP listings 101
- SW1 (in WHO display) 59
- SW2 (in WHO display) 59
- switch, audio *see* audio switch
- switch, video *see* video switch
- SYS device 8, 39, 44
- SYSTAT (LOGIN option) 51

SYSTAT command 66
 system messages 117, 122
 system programs 39
 system WHO line *see* WHO line
 TAB 16, 36
 TAB key 13
 TABS (in TTY command) 38
 TABS (LOGIN option) 51
 tabs, hardware 38
 TALK command 47
 talk ring 47
 tape, magnetic *see* magnetic tape
 tape, paper 7
 TCOR (in WHO line) 21
 TECO 9, 71, 184
 TECO command 71
 Teletypes 4
 TELNET 67, 143
 TELNET command 67, 143
 TEMPC (in WHO line) 21
 TEMPF (in WHO line) 21
 terminal 6, 8, 37
 terminals, display 4, 13, 23, 58
 terminals, non-display 9
 TEST command 68
 text nodes 100
 TGAG (in TTY command) 38
 tick 22, 59
 TIME (in TTY command) 38
 TIME (in WHO display) 59
 TIME (in WHO line) 21
 TIME command 46
 TIME switch in COPY 94
 TIME switch in MAIL 112, 113
 timesharing 1
 TITLE switch in COPY 94
 TITLE switch in SPOOL 98
 TLIST command 135, 136
 TMAR switch in XSPool 100
 TMP 52
 TMPCOR 52, 57
 TN command 67, 143
 TOP key 5, 13
 TPL pseudo-device (COPY) 91
 TQ (job queue) 22
 TRANSFER command 55, 95
 TRY command 75
 TTY (in WHO line) 21
 TTY command 37
 TTY device *see* terminals
 TTYUO 36
 TURKEY command 135, 136

TV device 8
 TV, lounge 27
 TYPE command 55, 95
 type WHO line 37
 typeahead 6
 UCOR (in WHO line) 21
 UDP 8, 44, 45, 46, 47, 58, 90, 165
 UDPFD command 67
 UFD 7
 UFD command 67
 UFD protection 94
 UFDPRO switch in COPY 94
 UNDELETE command 68
 UNHIDE (LOGIN option) 51
 UNPROTECT command 68
 UNSPOOL 103
 UNSPOOL command 55
 UPDATE (in TTY command) 38
 UPG privilege 69
 upper segment 12, 21, 22, 40, 59
 user disk pack *see* UDP
 User File Directory 7
 user mode 6, 41
 user, authorized 9
 USETI switch in XSPool 101
 UTPCLR 58
 UUOs 2, 4
 vector 4
 vector nodes 100
 vertical position 23
 vertical tab 13
 video switch 25
 virtual terminal 24
 volleyball players 108
 VT 36
 VT key 13
 Wait (in TIME command) 46
 WAIT switch in COPY 94
 wait time 21
 WARN switch in SPOOL 98
 whams 138
 WHEN command 66
 WHERE command 11, 65
 WHERE switch in MAIL 112
 WHO (in TTY command) 38
 WHO (LOGIN option) 50
 WHO command 6, 11, 58
 WHO line 4, 20, 38, 42, 45, 50, 59
 WIDTH (in TTY command) 38
 WIDTH (LOGIN option) 51
 word delete (non-display escape) 37
 WRITER switch in COPY 94

200 INDEX

XDIGEST (LOGIN option) 51
Xerox Graphics Printer *see* XGP
XFACT.TXT file 97
XGP 8, 55, 97, 100, **163**
XGP switch in XSPool 100
XGPLIST command 55, 95
XLINE switch in XSPool 100
XMAIL (LOGIN option) 50
XP (in WHO line) 21
XSPool command 55, 97, **100**
XTIME 22, 38
XTIME (in WHO line) 21
YESMAIL switch in MAIL 111
ZERO command 58
↑← C (non-display escape) 37
↑← D (non-display escape) 37
↑← F (non-display escape) 37
↑← H (non-display escape) 37
↑← I (non-display escape) 37
↑← L (non-display escape) 37
↑← M (non-display escape) 37
↑← N (non-display escape) 37
↑← n V (non-display escape) 37
↑← R (non-display escape) 37
↑← W (non-display escape) 37
↑← X (non-display escape) 37
↑← Z (non-display escape) 37
↑← ↑W (non-display escape) 37